Dylan

The Language, the Interpreter and the Compiler

A User Manual

Blake McBride

Manual revised November 5, 2026

This document was typeset with LaTeX on Linux.

Copyright © 2026 Blake McBride
All rights reserved.
`https://github.com/blakemcbride/Dylan`
Manual revised November 5, 2026

The language this manual describes is the one defined in *Dylan(TM): an object-oriented dynamic language*, Apple Computer Eastern Research and Technology, April 1992, cited here as *DIRM*.
Dylan is a trademark of Apple Computer, Inc.

Contents

I Getting Started **1**

1 Introduction **3**
1.1 What Dylan is . . . 3
1.2 The two implementations . . . 4
1.3 A short history . . . 5
1.4 How this book is organized . . . 6
1.5 Conventions . . . 6
1.6 License . . . 7

2 Getting Started **9**
2.1 Requirements . . . 9
2.2 Getting the source . . . 10
2.3 The interactive implementation . . . 10
2.4 Compiling a file to Lisp . . . 12
2.5 A first program with dylanc . . . 13
2.6 The examples and the test suites . . . 15
2.7 Where to go next . . . 16

3 A Tutorial **17**
3.1 How to follow along . . . 17
3.2 Expressions and prefix syntax . . . 18
3.3 Numbers and arithmetic . . . 19
3.4 Variables . . . 20
3.5 Defining functions . . . 21
3.6 Making decisions . . . 22
3.7 Output and loops . . . 23
3.8 Lists, vectors and strings . . . 25
3.9 Tables . . . 27
3.10 Classes and objects . . . 28
3.11 Generic functions and multiple dispatch . . . 30
3.12 Calling the next method . . . 31
3.13 Collections and functional operations . . . 32
3.14 Leaving early with bind-exit . . . 33
3.15 Conditions and errors . . . 33
3.16 A complete program . . . 35
3.17 Compiling the program . . . 38
3.18 Where to go from here . . . 38

II The Language **41**

4 Syntax and Program Structure **43**
4.1 How the examples are shown . . . 43
4.2 The reader . . . 44
4.2.1 White space and delimiters . . . 44
4.2.2 Comments . . . 44
4.2.3 Numbers . . . 44
4.2.4 Characters . . . 45
4.2.5 Strings . . . 45
4.2.6 Symbols . . . 46
4.2.7 Setter names . . . 46
4.2.8 Keywords . . . 47
4.2.9 True, false and the empty list . . . 47
4.2.10 Lists and vectors . . . 47
4.2.11 Quote and backquote . . . 47
4.2.12 #key, #rest and #next . . . 48
4.2.13 Unreadable objects . . . 48
4.3 Expressions and evaluation . . . 48
4.3.1 Literal constants . . . 48
4.3.2 Variable references . . . 49
4.3.3 Function calls . . . 49
4.3.4 Syntax forms . . . 49
4.3.5 Multiple values . . . 50
4.4 Program structure . . . 50
4.4.1 Top-level forms and definitions . . . 50
4.4.2 When things happen . . . 51
4.4.3 Source files and the module: header . . . 51

5 Variables and Control **53**
5.1 Module variables: define . . . 53
5.2 Lexical variables: bind . . . 54
5.2.1 Receiving multiple values . . . 54
5.2.2 Type declarations . . . 55
5.3 Assignment: set! . . . 55
5.3.1 The extended form . . . 55
5.4 Truth and if . . . 56
5.5 Sequencing: begin . . . 56
5.6 Conditionals . . . 57
5.6.1 when and unless . . . 57
5.6.2 and and or . . . 57
5.6.3 cond . . . 57
5.6.4 case . . . 58
5.6.5 select . . . 58
5.7 Iteration . . . 58
5.7.1 for . . . 59
5.7.2 for-each . . . 59
5.7.3 dotimes . . . 60

5.7.4 while and until . . . 60
5.8 Local methods: bind-methods . . . 61
5.9 Multiple values . . . 61
5.10 Non-local exits . . . 62
5.10.1 bind-exit . . . 62
5.10.2 unwind-protect . . . 63
5.11 quote . . . 63

6 Functions and Methods 65
6.1 Methods and generic functions . . . 65
6.2 Making methods: method . . . 66
6.2.1 Closures . . . 66
6.3 Parameter lists . . . 67
6.3.1 Required parameters . . . 67
6.3.2 Rest parameters . . . 67
6.3.3 Keyword parameters . . . 68
6.3.4 Combining #rest and #key . . . 68
6.3.5 The next-method parameter . . . 69
6.3.6 Keyword argument checking . . . 69
6.4 Defining methods: define-method . . . 70
6.4.1 Congruent parameter lists . . . 70
6.5 Declaring generic functions: define-generic-function . . . 71
6.6 Specializers and singletons . . . 72
6.7 Method dispatch . . . 72
6.7.1 Specificity . . . 72
6.7.2 The class precedence list . . . 73
6.7.3 Calling the next method . . . 74
6.8 Calling functions: apply . . . 74
6.9 Setters as generic functions . . . 75
6.10 Reflective operations . . . 75
6.10.1 Changing generic functions . . . 76
6.11 Proper tail calls . . . 77

7 Classes and Objects 79
7.1 Defining classes: define-class . . . 79
7.1.1 Multiple inheritance . . . 79
7.2 Slots . . . 80
7.2.1 Slot specifications . . . 80
7.2.2 Initial values . . . 81
7.2.3 Allocation . . . 82
7.2.4 Inherited slots . . . 83
7.3 Making instances: make and initialize . . . 83
7.4 Adding slots: define-slot . . . 84
7.5 Examining classes and objects . . . 85
7.5.1 Slot descriptors . . . 85
7.5.2 Copying and converting . . . 86
7.6 Singletons . . . 87
7.7 Sealing . . . 87

7.8 Changing classes at run time . . . 88
7.9 The built-in classes . . . 88
7.9.1 Which classes can be subclassed . . . 89

8 Numbers, Characters and Symbols 91
8.1 The number classes . . . 91
8.1.1 Integers . . . 92
8.1.2 Ratios . . . 92
8.1.3 Floats . . . 92
8.1.4 Complex numbers . . . 93
8.2 Writing numbers . . . 93
8.3 Arithmetic . . . 94
8.3.1 Contagion . . . 95
8.4 Comparing numbers . . . 95
8.5 Predicates . . . 96
8.6 The division family . . . 96
8.7 gcd, lcm, abs, min and max . . . 97
8.8 Exponentials and trigonometry . . . 98
8.9 Integers as bits . . . 99
8.10 Parts of numbers . . . 100
8.11 Converting numbers . . . 101
8.12 How floats print . . . 101
8.13 Characters . . . 101
8.14 Symbols and keywords . . . 102

9 Collections 105
9.1 The collection classes . . . 105
9.2 Keys, elements and size . . . 106
9.3 The iteration protocol . . . 107
9.4 Defining your own collection . . . 108
9.4.1 A sequence . . . 109
9.4.2 A mutable explicit-key collection . . . 109
9.5 Functions on all collections . . . 110
9.5.1 do, map, map-as and map-into . . . 110
9.5.2 any?, every?, reduce and reduce1 . . . 111
9.5.3 member? and find-key . . . 112
9.5.4 replace-elements! and fill! . . . 112
9.6 Functions on sequences . . . 112
9.6.1 Adding and removing . . . 113
9.6.2 Sets . . . 113
9.6.3 Copying and joining . . . 114
9.6.4 Reversing and sorting . . . 114
9.6.5 Particular elements and searching . . . 115
9.7 Lists and pairs . . . 115
9.8 Vectors and arrays . . . 116
9.9 Strings . . . 117
9.10 Stretchy vectors . . . 118
9.11 Deques . . . 118

9.12 Ranges . . . 118
9.13 Tables . . . 119
9.14 Equality: = and id? . . . 120

10 Functional Operations **121**
10.1 compose . . . 121
10.2 complement, disjoin and conjoin . . . 122
10.3 curry and rcurry . . . 122
10.4 always and identity . . . 123
10.5 apply . . . 123
10.6 Idioms . . . 123

11 Conditions **125**
11.1 How the condition system works . . . 125
11.2 The condition classes . . . 125
11.3 Signalling . . . 127
11.4 Handling with handler-case . . . 128
11.5 Handling with handler-bind . . . 129
11.6 Default handlers . . . 129
11.7 Restarts . . . 130
11.8 Introspection . . . 131
11.9 Defining your own conditions . . . 131
11.10 Unhandled conditions . . . 132
11.10.1 In the Common Lisp implementation . . . 132
11.10.2 In the compiler to C . . . 133
11.11 Errors the implementations signal . . . 133

12 Modules **135**
12.1 What a module is . . . 135
12.2 The predefined modules . . . 136
12.3 Declaring a module . . . 136
12.3.1 `use` clauses . . . 136
12.3.2 `export` and `create` clauses . . . 137
12.3.3 Rules . . . 137
12.4 An example of each option . . . 137
12.5 Which module code belongs to . . . 138
12.6 Modules in the compiler to C . . . 139
12.7 Modules in the interactive implementation . . . 139
12.7.1 At the read-eval-print loop . . . 140
12.7.2 Files . . . 140
12.8 Read-only variables . . . 141
12.9 A complete multi-file program . . . 141

13 Printing and Streams **143**
13.1 Two ways to print an object . . . 143
13.2 The printing functions . . . 143
13.2.1 Pretty-printing . . . 144
13.3 How objects print . . . 144

13.4 Customizing printing with `print-object` . . . 146
13.5 Streams . . . 147
13.6 `format` . . . 147
13.7 Common Lisp's format . . . 148

III The Implementations 149

14 The Interactive Implementation 151
14.1 How it works . . . 151
14.2 Loading it . . . 153
14.3 The read-eval-print loop . . . 153
14.3.1 Errors and the Lisp debugger . . . 155
14.3.2 Running a file at the loop . . . 156
14.4 Compiling Dylan to Lisp files . . . 156
14.5 Dylan values as Lisp objects . . . 158
14.6 Calling Common Lisp from Dylan . . . 158
14.7 How it is made fast . . . 160
14.8 The book's examples and the regression test . . . 160
14.9 Differences from the compiler to C . . . 161
14.10Limitations . . . 162

15 The Compiler to C 163
15.1 A first build . . . 163
15.2 The options . . . 164
15.3 The files the compiler writes . . . 166
15.4 The library's cache . . . 167
15.5 Size . . . 168
15.6 Debug builds . . . 168
15.7 Testing builds . . . 169
15.8 The garbage collector . . . 169
15.9 Calling the compiler from Lisp . . . 170
15.10Errors when compiling . . . 171
15.11Errors when running . . . 172
15.12What a compiled program cannot do . . . 172

16 Macros and Threads 175
16.1 Macros . . . 175
16.1.1 `define-macro` . . . 175
16.1.2 Rules . . . 176
16.1.3 Names in expansions: `gensym` and capture . . . 177
16.1.4 The compile-time environment and `define-for-macros` . . . 178
16.1.5 Macros and modules . . . 179
16.1.6 Seeing expansions: `dylanc -E` . . . 179
16.2 Threads . . . 180
16.2.1 Making and joining threads . . . 180
16.2.2 When the program ends . . . 181
16.2.3 Unhandled conditions in threads . . . 181

16.2.4 Locks . . . 182
16.2.5 Condition variables . . . 183
16.2.6 Atomic integers . . . 184
16.2.7 Thread variables and `dynamic-bind` . . . 184
16.2.8 What threads may share . . . 185

17 Inside the Compiler **187**
17.1 The stages . . . 187
17.2 How values are represented . . . 189
17.3 The calling convention . . . 189
17.4 Proper tail calls . . . 190
17.5 Closures . . . 191
17.6 Generic dispatch . . . 191
17.7 Exits, cleanups and conditions . . . 192
17.8 The garbage collector . . . 193
17.9 Threads . . . 193
17.10 Separate compilation of the library . . . 194
17.11 Tree shaking . . . 194
17.12 Type inference and inlining . . . 194
17.13 Reading the generated C . . . 196

18 Performance **199**
18.1 The benchmarks . . . 199
18.1.1 The compiler to C . . . 200
18.1.2 The interactive implementation . . . 200
18.2 Writing fast Dylan for the compiler . . . 201
18.2.1 Prefer `define-method` to `define` of a `method` . . . 201
18.2.2 Let the compiler know classes: declarations in `bind` . . . 202
18.2.3 Keep floats unboxed . . . 202
18.2.4 Typed slots . . . 203
18.2.5 Local exits are free; escaping ones are not . . . 203
18.2.6 First-class calls . . . 203
18.2.7 `#rest` and keywords . . . 203
18.2.8 What the static program gives . . . 204
18.3 Writing fast Dylan for the interactive implementation . . . 204
18.4 Size . . . 204
18.5 Measuring . . . 205
18.6 Tuning the garbage collector . . . 205

A Differences from the 1992 Manual **207**
A.1 Different behavior, in both implementations . . . 207
A.2 Additions in both implementations . . . 208
A.3 The interactive implementation . . . 208
A.4 The compiler to C . . . 208
A.4.1 What a static program cannot do . . . 208
A.4.2 Numbers . . . 209
A.4.3 Conditions without a debugger . . . 209
A.4.4 Representation . . . 209

A.4.5 Additions . . . 209
A.5 Where the two implementations disagree . . . 210

B Library Reference **211**
B.1 Objects, equality and comparison . . . 211
B.2 Classes, instances and slots . . . 212
B.3 Functions and generic functions . . . 213
B.4 Functional operations . . . 214
B.5 Numbers . . . 214
B.6 Characters, strings and symbols . . . 216
B.7 Collection classes . . . 217
B.8 Collection operations . . . 217
B.9 Sequence operations . . . 218
B.10 Lists . . . 219
B.11 Vectors, arrays, deques, ranges and tables . . . 220
B.12 Conditions . . . 220
B.13 Printing and streams . . . 221
B.14 Threads . . . 222
B.15 Other extensions . . . 223
B.16 Special forms and macros . . . 223

C Testing **225**
C.1 The suites at a glance . . . 225
C.2 The front end and the IR . . . 225
C.3 Programs compiled and run . . . 226
C.3.1 The case format . . . 226
C.3.2 Modes . . . 227
C.3.3 Running some of the cases . . . 228
C.3.4 How long they take . . . 228
C.4 The command line . . . 228
C.5 The runtime . . . 229
C.6 The interactive implementation . . . 229
C.7 Adding a test . . . 229

Part I

Getting Started

Chapter 1

Introduction

This book is a user manual for Dylan, the object-oriented dynamic language that Apple Computer described in 1992, and for the two implementations of it that this project provides: an interactive implementation written in Common Lisp and a compiler that translates Dylan programs into C. It teaches the language from the beginning, explains how to use each implementation, and documents where the implementations depart from the 1992 definition.

This chapter describes the language in broad terms, introduces the two implementations and when to choose each, gives a short history of the project, and explains how the rest of the book is organized and what its typographic conventions mean.

1.1 What Dylan is

Dylan was designed at Apple Computer's Eastern Research and Technology laboratory and published in April 1992 as *Dylan: an object-oriented dynamic language*. That book, which this manual calls the *DIRM*, is the standard both implementations follow. The language it defines is the original Dylan, written in a fully parenthesized *prefix syntax* like that of Scheme and Common Lisp. Later versions of Dylan moved to an infix syntax that looks more like Pascal or C; that later language is not the subject of this book. Everything here is the 1992 prefix language.

A short piece of Dylan shows its flavor:

```
(define-class <point> (<object>)
  (x init-value: 0 init-keyword: x:)
  (y init-value: 0 init-keyword: y:))

(define-method norm ((p <point>))
  (sqrt (+ (* (x p) (x p)) (* (y p) (y p)))))

(norm (make <point> x: 3 y: 4))   ; => 5.0
```

The language's main ideas are few, and they fit together closely.

Objects all the way down. Every value in Dylan is an object, and every object is an instance of a class. Integers, characters, strings, lists, functions and classes themselves are all objects, with classes such as `<integer>`, `<string>` and `<class>`. There are no "primitive" values that stand outside the object system, so any value can be passed to any function, stored in any collection, and used to select a method.

Generic functions and multiple dispatch. In most object-oriented languages a method belongs to a class and is chosen by the class of a single receiver object. In Dylan, methods belong to *generic functions*. A generic function is an ordinary function that holds a set of methods; when it is called, it looks at the classes of *all* of its required arguments and runs the most specific method that applies. This is called *multiple dispatch*. A method may also be specialized on one particular object (a *singleton*) rather than on a class.

Classes with multiple inheritance. A class names one or more superclasses and describes *slots*, the named fields each instance holds. Dylan defines the getter and setter functions for each slot automatically, and those functions are themselves generic functions, so slot access and method calls look exactly alike. A class may inherit from several superclasses at once.

First-class functions. Functions are objects. They can be created anonymously with `method`, passed as arguments, returned as results, and stored in data structures. A function created inside another function remembers the variables around it (it is a *closure*). The library includes a set of functional operations, such as `curry`, `compose` and `complement`, that build new functions from old ones.

Conditions. Errors and other exceptional situations are represented by *condition* objects, which are instances of classes such as `<error>` and `<warning>`. A program signals a condition and a dynamically established handler decides what to do with it: return a value, take a non-local exit, or decline and let an outer handler try. Because conditions are objects, handlers select them by class, just as methods do.

Collections. Dylan has a rich, uniform library of collections: lists, vectors, strings, stretchy vectors, deques, ranges, arrays and tables. All of them share one iteration protocol, so the same operations (`map`, `reduce`, `do`, `choose`, `member?`, `for-each` and the rest) work on every kind of collection, and new collection classes can join in by implementing the protocol.

A dynamic language that can be compiled well. Dylan programs need no type declarations, and in the 1992 definition classes and methods can be added and changed while a program runs. The language also offers ways to say that parts of a program will not change, such as sealing a class, so that a compiler can produce fast code. The two implementations described here sit at the two ends of this range.

1.2 The two implementations

This project provides two separate implementations of the same language. They share the reader (the code that turns program text into data), and they are tested against the same examples from the DIRM, but otherwise they are independent programs.

The interactive implementation. The first is written in Common Lisp and runs inside SBCL. Its source is in `src/`, and this book calls it *the interactive implementation* or *the Common Lisp implementation*. It is not an interpreter: it compiles each Dylan form into Common Lisp as the form is entered and lets the Lisp compiler run it. You use it in three ways: at an interactive read-eval-print loop, the command `dylan`; by compiling a file of Dylan into a file of Common Lisp with `dylan:compile-file`; and by loading such compiled files into a Lisp that runs them.

It implements the whole language of the DIRM, including the parts that let a program change its own classes and generic functions as it runs (`add-method`, `remove-method`, `add-slot` and their relatives), and it has arbitrarily large integers, because Common Lisp does.

The compiler to C. The second is a compiler, `bin/dylanc`, that translates a complete Dylan program into C and then runs a C compiler to produce a native executable. Its source is in `compiler/`, and its run-time support is in `runtime/`: a small runtime written in C and the Dylan library, itself written in Dylan. The compiled programs have proper tail calls, generic dispatch compiled to decision trees, type inference, inlining, unboxed floating-point arithmetic, a generational garbage collector, and real operating-system threads. The system also adds two things the 1992 language does not have: macros, in both implementations, and threads, in the compiler.

The price of this speed is that the compiler treats a program as *static*. Everything the program defines must be visible when it is compiled, and the program cannot change its classes or generic functions while it runs, cannot evaluate or load new code, and has no bignums or complex numbers: its integers are 63 bits wide, and overflow is an error. Chapter 15 and Appendix A describe these restrictions in full.

Which to use. The two are meant to be used together.

- Use the **interactive implementation** to learn the language, to try out ideas, to explore the library, and to develop code a piece at a time. You type a form and see its value at once. It is also the right choice for programs that really do need to redefine classes or add methods while they run, or that need integers larger than 63 bits.
- Use the **compiler** when you want a standalone program: a fast executable that starts instantly, needs no Lisp to run, and can use threads. A program developed interactively can normally be compiled unchanged, provided it stays within the static-program restrictions.

Throughout this book, examples work in both implementations unless the text says otherwise. Where the two differ, for instance in how they print an object that has no readable printed form, the text points it out.

1.3 A short history

In 1992, shortly after the DIRM appeared, Matt Birkholz, Jim Miller and Ron Weiss of Digital Equipment Corporation's Cambridge Research Laboratory wrote *Thomas*, an implementation of the language in Scheme. They wrote it in four weeks, and their stated goals were to learn the language by implementing it from the book, to help others learn from readable source, and to run small programs; performance was explicitly not a goal. They named it Thomas rather than Dylan because they had not received permission to use the trademark.

This project began as a direct translation of Thomas from Scheme into Common Lisp. Since then it has moved a long way from that starting point. The Common Lisp implementation was checked against the DIRM chapter by chapter and corrected wherever Thomas and the book disagreed (for example, `#f` and the empty list are now distinct objects, as the book requires), and it gained modules, keyword checking, sealing and slot options that Thomas lacked. It was then rebuilt for speed: generic dispatch goes through caches, known calls are made directly, and common arithmetic, element access, iteration and slot access are compiled in line.

The compiler to C is new. It was written for this project and shares only the reader with the Common Lisp implementation (which also runs its macros while compiling). Thomas's own sources are no longer part of the tree; each file in `src/` still records which Thomas file it grew from, and Thomas's original 1992 README and release notes are kept as `README.thomas` and `RELEASE`.

1.4 How this book is organized

The book has three parts and three appendices.

Part I, Getting Started, is for reading in order. This chapter is the first. Chapter 2 explains what you need to install, how to start the interactive implementation, and how to compile and run a first program with `dylanc`. Chapter 3 is a hands-on tour of the language, from arithmetic to classes, generic functions and conditions, ending with a small complete program built step by step and then compiled.

Part II, The Language, is a reference to the language, in the order a reader usually needs it: syntax (Chapter 4), control flow (Chapter 5), functions (Chapter 6), classes (Chapter 7), numbers (Chapter 8), collections (Chapter 9), functional operations (Chapter 10), conditions (Chapter 11), modules (Chapter 12), and printing and output (Chapter 13). These chapters follow the DIRM and note, section by section, where either implementation differs from it.

Part III, The Implementations, describes each implementation as a tool: the Common Lisp implementation (Chapter 14), the `dylanc` compiler and its options (Chapter 15), the extensions—macros and threads (Chapter 16), how the compiler works inside (Chapter 17), and how to write fast programs and measure them (Chapter 18).

The appendices collect the differences between this system and the 1992 manual (Appendix A), a summary of the library (Appendix B), and how to run the test suites and benchmarks (Appendix C).

1.5 Conventions

Code. Dylan code, file names, commands and other text you type or the computer prints are set in a typewriter face, as in `(+ 1 2)` or `src/load-dylan.lisp`. In a syntax description, italic words such as *name* or *body* stand for something you supply.

Examples and their results. Most examples are written as Dylan source. When an example shows what an expression evaluates to, the value follows the expression in a Dylan comment that begins with `; =>`:

```
(+ 1 2)          ; => 3
(list 1 'a "b")  ; => (1 a "b")
```

The comment is not part of the value; you can type the whole line, comment and all, and Dylan will ignore everything from the semicolon to the end of the line. When an expression returns more than one value, the values are separated by commas:

```
(truncate/ 17 5)  ; => 3, 2
```

When an example writes output, the text it writes is shown in a comment beginning `; prints`:

```
(display "Hello")  ; prints: Hello
```

Terminal sessions. A session at the shell shows a `$` prompt before each command you type, followed by the command's output:

```
$ bin/dylanc hello.dyl
$ ./hello
Hello, world!
```

Interactive sessions. A session at the interactive implementation's read-eval-print loop shows its `?` prompt before each form you type, and the `;Value:` line it prints in reply:

```
? (* 6 7)

;Value: 42
```

References to the 1992 manual. A citation such as *DIRM* chapter 12 refers to a chapter of the 1992 Dylan manual. An annotated copy of it, in HTML, is in the repository under `manual/Dylan-1992/`; open `START-HERE.html` there. When this book and the DIRM seem to disagree about the language, the DIRM is the standard, and any deliberate difference is listed in Appendix A.

Notes, tips and warnings. Three kinds of aside appear in the text. A **Note** adds a detail or a cross-reference that you can skip on a first reading. A **Tip** suggests an easier or better way to do something. An **Important** box warns about a mistake that is easy to make or a difference between the implementations that can surprise you.

1.6 License

This software is *source-available*, not open source. The original Thomas system is Copyright 1992 Digital Equipment Corporation, and the Common Lisp implementation, which began as a translation of it, is a derivative work governed by Digital's 1992 notice. That notice allows you to use, copy, modify and redistribute the software at no charge to recipients who accept the same terms, but it forbids selling it or transferring it for compensation. Because of that restriction, the terms meet neither the Open Source Definition nor the Free Software Definition and are not compatible with the GPL, MIT, BSD or Apache licenses.

The notice must appear in all copies, derivative works and modified versions. It is reproduced in full in the file `LICENSE` at the top of the repository, together with an explanation of what it permits; read that file before redistributing any part of this system.

The home of the project is `https://github.com/blakemcbride/Dylan`.

Chapter 2

Getting Started

This chapter takes you from an empty directory to running Dylan in both implementations. It lists what you need to install, shows how to get the source, walks through a first session with the interactive implementation, and compiles a first program with `dylanc`. By the end of it you will have everything you need for the tutorial in Chapter 3.

2.1 Requirements

Both implementations are written in Common Lisp and need **SBCL**, Steel Bank Common Lisp. The system was developed on SBCL 2.6.3, and any recent release should work. The interactive implementation needs nothing else but `make` to build it, and even that can be done without.

The compiler to C needs two more things, and can use a third:

- **A C compiler that supports the `musttail` attribute**, which guarantees that a call in tail position is compiled as a jump. That means **GCC 15 or later** or **Clang 13 or later**. Any other C11 compiler can be used with the `--trampoline` option, which implements tail calls with trampolines instead, at some cost in speed. `dylanc` uses the compiler named by the `CC` environment variable, or `cc` if `CC` is not set; the `--cc` option names one explicitly.
- `make` and a POSIX shell. `bin/dylanc` is a shell script, and it builds the C runtime library with `make` the first time it is needed.
- Optionally, **the Boehm-Demers-Weiser garbage collector**, `libgc`, *built with thread support*, together with its header files, for programs built with `--gc boehm` (Section 15.8); the runtime then defines `GC_THREADS` and links with `-lgc -lm -lpthread`. The default collector, the Memory Pool System, is part of the source tree and needs nothing installed.

Checking what you have. These commands report the versions installed:

```
$ sbcl --version
SBCL 2.6.3
$ gcc --version | head -1
gcc (GCC) 16.2.1 20260819 (Red Hat 16.2.1-2)
$ clang --version | head -1
clang version 22.1.8 (Fedora 22.1.8-4.fc44)
```

Your version numbers will differ; what matters is that SBCL runs and that one of the C compilers is new enough (or that you plan to use `--trampoline`). For the Boehm collector, the

header `gc.h` must be present too (`ls /usr/include/gc.h`). On Fedora and related systems its development package is called `gc-devel`; on Debian and Ubuntu it is `libgc-dev`. The packaged collectors on these systems are normally built with thread support. If you build the collector yourself, configure it with threads enabled.

Note: The C compiler is needed only to *build* programs. An executable made by `dylanc` needs only the C library at run time (and the shared `libgc`, if built with `--gc boehm`); it does not need SBCL.

2.2 Getting the source

The project lives at `https://github.com/blakemcbride/Dylan`. Clone it with Git:

```
$ git clone https://github.com/blakemcbride/Dylan.git
$ cd Dylan
```

There is nothing to configure and nothing to install. Every command in this book is run from the top directory of the clone unless the text says otherwise. The directories you will meet first are these:

`src/`	the interactive implementation, in Common Lisp
`Makefile`	builds `bin/dylan`
`bin/dylan`	the interactive implementation, once built
`bin/dylanc`	the command that runs the compiler to C
`compiler/`	the compiler to C, in Common Lisp
`runtime/`	the compiler's runtime: `runtime/c` in C, `runtime/dylan` in Dylan
`manual/Dylan-1992/`	the annotated 1992 Dylan manual (the DIRM)
`dylan-examples.dyl`	the examples from the 1992 manual
`tests/`, `bench/`	test suites and benchmarks

You can run `dylan` and `dylanc` from anywhere by putting `bin` on your `PATH`, or by making symbolic links to them in a directory that is already on it. `dylan` is self-contained, and the `dylanc` script finds the rest of the system relative to its own real location, so a link works for both.

2.3 The interactive implementation

Build the interactive implementation once, in the top directory of the clone:

```
$ make
sbcl --script src/build-dylan.lisp
;Writing /home/you/Dylan/bin/dylan
```

This loads the whole system into SBCL and saves it as one executable, `bin/dylan`. (On Windows, without `make`, run `sbcl --script src/build-dylan.lisp` yourself; it writes `bin\dylan.exe`.) Run `make` again whenever `src/` changes; it does nothing when `bin/dylan` is up to date.

The first build compiles every Lisp file of the implementation and writes the compiled files (*fasl* files) into a directory under `src/fasl/` named for the Lisp and its version, such as `src/fasl/SBCL-2-6-3/`. This takes a little while. Later builds recompile only a changed source file and every file loaded after it, since later files may depend on its macros; you never need to

delete the fasl directory by hand. Because the fasl files are kept apart for each Lisp version, upgrading SBCL simply makes a new directory.

Starting the loop. Run `dylan`. It starts a read-eval-print loop that reads Dylan rather than Lisp. Its prompt is a question mark:

```
$ bin/dylan

Entering Dylan read-eval-print-loop.
Leave by typing "(exit)"

? (define greeting "Hello")

;Value: greeting

? (concatenate greeting ", world")

;Value: "Hello, world"
```

Type a form and press Enter. The loop reads the form, compiles it into Lisp, runs it, and prints its value after `;Value:`. A form may span several lines; the loop keeps reading until the parentheses balance. A definition such as `define` answers the name it defined. A form that returns several values shows each one on its own line, numbered:

```
? (truncate/ 17 5)

;Value[1]: 3
;Value[2]: 2
```

Note: In a real terminal the loop prints an extra blank line after each form you type. The transcripts in this book leave it out.

Leaving the loop. Type `dylan:done` or `(exit)`, or end the input with Ctrl-D, to leave `dylan`.

Loading it into SBCL yourself. Instead of `dylan`, you can start a plain SBCL in the top directory of the clone, load `src/load-dylan.lisp`, and call the function `dylan:rep`, which starts the same loop. This needs no build step, since the load compiles what has changed, and it leaves you in a Lisp where you can mix Lisp and Dylan:

```
$ sbcl
* (load "src/load-dylan.lisp")

Call DYLAN:REP to start a Dylan read-eval-print loop.
;Dylan loaded.  (dylan:rep) starts a read-eval-print loop.
T
* (dylan:rep)

Entering Dylan read-eval-print-loop.
Leave by typing "(exit)"

?
```

There `dylan:done` returns to the Lisp prompt. What you defined is not lost: if you call `(dylan:rep)` again, the variables, classes and methods you defined earlier are still there. To leave SBCL altogether, type `(exit)` at the `?` prompt, or `(sb-ext:exit)` at the Lisp prompt.

When something goes wrong. An error that your Dylan code does not handle stops the loop and enters SBCL's debugger. The debugger offers restarts that return you to the Dylan prompt; choose one by typing its number:

```
? (error "Account ~a is overdrawn" 42)

debugger invoked on a DYLAN::DYLAN-CONDITION in thread
#<THREAD tid=701926 "main thread" RUNNING {1200030003}>:
  Error: Account 42 is overdrawn

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [DYLAN-RESTART] Return to the read-eval-print loop.
  1: [DYLAN-REP    ] Return to the Dylan read-eval-print loop.
  2: [ABORT        ] Return to the Dylan read-eval-print loop.
...
0] 0
;Aborted

? (+ 3 4)

;Value: 7
```

Note: A few errors are detected by Common Lisp itself rather than by Dylan; for example, `(car 5)` is reported as a Lisp `TYPE-ERROR`. For such an error the debugger still offers `DYLAN-REP`, "Return to the Dylan read-eval-print loop", which gives you a new `?` prompt. Dylan's own handlers see these errors as instances of `<error>`, so `handler-case` catches them normally. In a Lisp you loaded the system into, the debugger's `ABORT` leaves the loop for the Lisp `*` prompt; nothing you defined is lost; type `(dylan:rep)` to carry on.

Running a file at the loop. The loop reads from standard input, so you can feed it a whole file of Dylan from the shell. This runs every form in `program.dyl` and then leaves:

```
$ bin/dylan < program.dyl
```

Each form's value is printed after `;Value:`, as if you had typed it. Since no one is there to use the debugger, an unhandled error ends the run as it ends a compiled program: it prints `Dylan error:` and its message on the standard error, and the status is 1. `break` prints `Dylan break:` and its message, and the run carries on.

2.4 Compiling a file to Lisp

The interactive implementation can also compile a whole file of Dylan into a file of Common Lisp, which can then be loaded and run without reading or compiling any Dylan. Suppose `greet.dyl` contains this:

```
(define-method greet ((name <string>))
  (display "Hello, ")
  (display name)
  (display "!")
  (newline))

(greet "world")
```

Load `src/load-compiler.lisp` and call `dylan:compile-file` with the names of the input and output files:

```
$ sbcl --non-interactive --eval '(load "src/load-compiler.lisp")' \
       --eval '(dylan:compile-file "greet.dyl" "greet.lisp")'
```

Compiling does not run the program; it only writes `greet.lisp`. That file is ordinary Common Lisp. To run it, load `src/load-runtime.lisp` and then load the compiled file:

```
$ sbcl --non-interactive --eval '(load "src/load-runtime.lisp")' \
       --eval '(load "greet.lisp")'
...
Hello, world!
```

You can also load a compiled file into a session and then start the loop; the file's definitions are then available at the `?` prompt, and you can call them, redefine them, or add methods to them.

Note: In this version, `load-compiler.lisp` and `load-runtime.lisp` both load the whole system, exactly as `load-dylan.lisp` does; they are separate entry points so that a program can say what it needs. Chapter 14 describes `dylan:compile-file`, `dylan:compile-to-file` and modules in detail.

2.5 A first program with dylanc

The compiler to C takes one or more files of Dylan and produces a native executable. Write this into `hello.dyl`:

```
(define-method greet ((name <string>))
  (format #t "Hello, ~a!~%" name))

(greet "world")
```

Then compile it and run the result:

```
$ bin/dylanc hello.dyl
$ ./hello
Hello, world!
```

A program is simply a sequence of top-level forms, run in order from the top of the file when the executable starts. There is no `main` function; here the last form calls `greet`.

Note: The 1992 manual uses `format` in examples but does not define it; both implementations provide it (Chapter 13).

The first build takes longer. The first time you run `dylanc`, it compiles the compiler itself into fasl files, builds the C runtime library with `make`, and compiles the Dylan library's C code, all inside the clone (under `compiler/fasl/` and `runtime/build/`). Each of these is kept and reused, so later builds of a small program take only a second or two. The clone therefore needs to be writable by the user who runs `dylanc`.

The executable's name. By default the executable is named after the first file, without the `.dyl`. The `-o` option names it yourself:

```
$ bin/dylanc -o greeter hello.dyl
$ ./greeter
Hello, world!
```

Several files. A program can be split across several files. Name them all on the command line; they are compiled, in the order given, into one program, exactly as if they were one long file. Put definitions before the code that uses them at top level. With `shapes.dyl`

```
(define-class <square> (<object>)
  (side init-keyword: side:))

(define-method area ((s <square>))
  (* (side s) (side s)))
```

and `main.dyl`

```
(print (area (make <square> side: 7)))
```

the program is built and run like this:

```
$ bin/dylanc -o app shapes.dyl main.dyl
$ ./app
49
```

What files appear. Normally the only file `dylanc` leaves behind is the executable. The C it generates is written beside the executable and deleted once the C compiler has finished with it. The `-k` (`--keep-c`) option keeps it:

```
$ bin/dylanc -k hello.dyl
$ ls
hello  hello.c  hello.dyl  hello-dylan.c  hello-dylan-user.c  hello.h
```

`hello.c` holds the program's data, its generic-function dispatch and the C `main` function; `hello.h` is a header the other files share; and there is one file for each module's functions: `hello-dylan-user.c` for your code, which is in the module `dylan-user`, and `hello-dylan.c` for the library's module `dylan`. The generated C contains `#line` directives that point back to your Dylan source, so a C debugger or a C compiler's message names the line of Dylan it came from. Reading this C is a good way to see what the compiler did with your program; Chapter 17 explains it.

Errors when compiling. A syntax error stops the compilation with a message giving the file, line and column, and a non-zero exit status:

```
$ bin/dylanc bad.dyl
bad.dyl:1:15: error: End of file inside a Dylan list.
```

A name that is used but never defined is only a warning; the program is still built, and the error comes when the program runs and reaches that name:

```
$ bin/dylanc -o bad main.dyl
main.dyl:1:8: warning: area is not defined (module dylan-user)
main.dyl:1:14: warning: <square> is not defined (module dylan-user)
$ ./bad
Dylan error: the variable area@dylan-user has no value
```

Take these warnings seriously: they almost always mean a misspelling or a missing file on the command line. A program that ends with an unhandled error prints `Dylan error:` and a message to standard error and exits with status 1.

Other options. `bin/dylanc -h` lists every option. The ones you are most likely to want soon are `-O0` to `-O3` (how hard the C compiler optimizes; the default is `-O2`), `-Os` for a small executable, `-g` for a debugging build that checks every inline operation, and `--trampoline` and `--cc` for other C compilers. Chapter 15 describes them all.

2.6 The examples and the test suites

The repository includes the examples from the 1992 manual and a large set of tests. You do not need to run any of them to use the system, but they are a useful check that an installation works, and the examples are good reading.

The interactive implementation runs the manual's examples, in `dylan-examples.dyl`, and compares the results with the transcripts in `src/reference/`. It must be run from the `src` directory:

```
$ cd src
$ sbcl --eval '(load "load-dylan.lisp")' \
       --eval '(load "full-test.lisp")' \
       --eval '(dylan:test-examples)'
```

The compiler to C has its own suites, among them the manual's examples chapter by chapter, tests of the code generator, stress tests, tests of the `dylanc` command, and tests of the C runtime:

```
$ sbcl --script tests/conformance/run.lisp
$ sbcl --script tests/codegen/run.lisp
$ sbcl --script tests/stress/run.lisp
$ sh tests/driver/run.sh
$ make -C runtime test
```

Important: By default each `run.lisp` builds every test with both GCC and Clang, with `musttail` and with trampolines, which takes a long time. Give the mode `quick` as an argument, as in `sbcl --script tests/conformance/run.lisp quick`, for a faster run with one configuration. Appendix C describes the modes and the benchmarks.

2.7 Where to go next

Chapter 3 is a hands-on tour of the language. Start the interactive loop, keep this book open beside it, and type the examples as you read; at the end of the tutorial you will build a small program and compile it with `dylanc`.

If you already know Lisp or Scheme and want the details first, the language reference begins with Chapter 4. The interactive implementation is described fully in Chapter 14, and the compiler in Chapter 15.

Chapter 3

A Tutorial

This chapter is a hands-on tour of Dylan. It starts with arithmetic and works up, a step at a time, through variables, functions, control flow, collections, classes, generic functions and conditions, and it ends by building a small complete program and compiling it into an executable. Each section introduces a few ideas with short examples that you can type and run, and each points to the chapter in Part II that covers the same ground in full.

The tutorial assumes you know some other programming language, but not Lisp or Scheme. If you do know a Lisp, you will find much that is familiar, and you can move quickly through the first sections.

3.1 How to follow along

The easiest way to follow the tutorial is at the interactive implementation's read-eval-print loop. Build `dylan` and start it, as Chapter 2 explains:

```
$ make
$ bin/dylan

Entering Dylan read-eval-print-loop.
Leave by typing "(exit)"

?
```

Then type the examples as you come to them. The examples are written as Dylan source, with the value of each expression shown in a comment beginning `; =>`. At the loop, the same value appears after `;Value:`:

```
(+ 1 2)   ; => 3
```

When an example prints something, the text it prints is shown in a comment beginning `; prints`. Every example in this chapter has been run in both implementations: at the loop, and compiled with `dylanc`. You can also put the examples of a section into a file and compile it, as Section 3.17 shows; in a compiled program, nothing is printed unless the program prints it, so wrap an expression in `print` to see its value.

Note: The examples build on one another. A function defined in one section is often used in the next, so type them in order, in one session.

3.2 Expressions and prefix syntax

A Dylan program is made of *expressions*, also called *forms*. The simplest are literal constants, which evaluate to themselves:

```
42          ; => 42
-7          ; => -7
3.5         ; => 3.5
"a string"  ; => "a string"
#\a         ; => #\a
#t          ; => #t
#f          ; => #f
```

`#t` and `#f` are the true and false values, and `#\a` is the character `a`. Everything else is written as a parenthesized list. A list whose first element names a function is a *call*: the function comes first, and its arguments follow, separated by white space.

```
(+ 1 2)               ; => 3
(max 3 9 4)           ; => 9
(* (+ 1 2) (- 10 4))  ; => 18
```

This is *prefix syntax*: the operator comes before its operands, and the parentheses mark exactly where each expression begins and ends. There is no operator precedence to remember, because nesting says everything. The last example multiplies the value of `(+ 1 2)` by the value of `(- 10 4)`. Operators such as `+` and `max` accept any number of arguments:

```
(+ 1 2 3 4)   ; => 10
(< 1 2 3)     ; => #t
```

Arithmetic operators are ordinary functions, not special syntax, which is why they are written the same way as every other call. A few lists are not calls but *special forms* with rules of their own, such as `if`, `define` and `method`; you will meet them in the sections that follow.

Names. Names in Dylan may contain letters, digits and many punctuation characters, so `even?`, `set!`, `<point>`, `*count*` and `remove-key!` are all single names. Dylan uses punctuation by convention to suggest what a name is for:

- a name in angle brackets, such as `<integer>`, names a class;
- a name ending in `?`, such as `even?`, names a predicate, a function that answers true or false;
- a name ending in `!`, such as `add!`, names a function that may modify one of its arguments;
- a name between asterisks, such as `*count*`, names a module variable whose value changes.

Names are not case-sensitive: `Max`, `MAX` and `max` are the same name. Because spaces separate the parts of a form, you must put spaces around operators: `(- x 1)` subtracts, but `x-1` is a single name.

Symbols, keywords and quoting. A name preceded by a quote mark is a *symbol*, a value in its own right rather than a reference to a variable. A name ending in a colon is a *keyword*, which is used to label arguments. Both evaluate to themselves:

```
'apple       ; => apple
size:        ; => size:
```

The quote mark also stops a list from being evaluated, which is one way to write a constant list. A hash sign before a list makes a constant vector:

```
'(1 2 3)     ; => (1 2 3)
#(1 2 3)     ; => #(1 2 3)
```

Comments. A semicolon starts a comment that runs to the end of the line. Text between `#|` and `|#` is a comment that can span several lines.

Chapter 4 describes the syntax completely (*DIRM* chapter 3).

3.3 Numbers and arithmetic

Dylan has integers, exact ratios and floating-point numbers. Division of integers that do not divide evenly gives an exact ratio, not a rounded result:

```
(/ 10 5)       ; => 2
(/ 10 4)       ; => 5/2
(+ 1/3 1/6)    ; => 1/2
(/ 10.0 4)     ; => 2.5
(as <float> 1/4)  ; => 0.25
```

Mixing a float with an exact number gives a float, and `as` converts a number to another class. Integer division that yields an integer is done by the functions whose names end in a slash. Each returns two values, the quotient and the remainder:

```
(truncate/ 17 5)    ; => 3, 2
(floor/ -7 2)       ; => -4, 1
(modulo -7 2)       ; => 1
(remainder -7 2)    ; => -1
```

`truncate/` rounds the quotient toward zero and `floor/` rounds it down; `modulo` and `remainder` return just the remainder that `floor/` and `truncate/` would. Functions that return several values are common in Dylan, and Section 3.4 shows how to receive them.

There are the usual comparisons and predicates:

```
(= 2 2.0)      ; => #t
(< 1 2 3)      ; => #t
(even? 4)      ; => #t
(zero? 0)      ; => #t
(abs -3)       ; => 3
(min 3 9 4)    ; => 3
```

`=` compares numbers by value, whatever their class; Dylan also has `id?`, which answers whether two values are the very same object. Powers, roots and the rest of the mathematical library are there too:

```
(expt 2 10)    ; => 1024
(sqrt 16)      ; => 4.0
(sqrt 2)       ; => 1.4142135623730951
```

Notice that `(sqrt 16)` answers the float `4.0`, not the integer `4`. The 1992 manual defines `sqrt`, `exp` and the trigonometric functions as in Common Lisp, and its own examples show them answering floats, so both implementations do the same.

Important: The two implementations differ in the size of integers. The interactive implementation has integers of any size, as Common Lisp does. A program compiled with `dylanc` has 63-bit integers, and an operation whose result does not fit signals an error:

```
(expt 2 62)   ; => 4611686018427387904 in the interactive implementation
              ; error: integer overflow (integers are 63 bits) when compiled
```

Neither implementation has complex numbers.

Chapter 8 covers numbers in full (*DIRM* chapter 14).

3.4 Variables

`define` creates a *module variable*, one that the whole program can see, and gives it a value. `set!` changes the value of a variable that already exists:

```
(define *count* 0)            ; => *count*
(set! *count* (+ *count* 1))  ; => 1
*count*                       ; => 1
```

At the loop, a `define` answers the name it defined, and `set!` answers the new value. Writing a variable's name by itself evaluates to its value.

`bind` creates *local variables* that exist only inside its body. Its first argument is a list of bindings, each a name and an initial value; the remaining forms are the body, and the value of the last one is the value of the whole `bind`:

```
(bind ((x 3)
       (y 4))
  (sqrt (+ (* x x) (* y y))))   ; => 5.0
```

The bindings are made in order, so a later initial value can use an earlier variable:

```
(bind ((a 1)
       (b (+ a 1)))
  (list a b))                   ; => (1 2)
```

A binding with more than one name receives multiple values, one in each name. This is how you use the quotient and remainder from `truncate/`:

```
(bind ((quotient rem (truncate/ 17 5)))
  (list quotient rem))          ; => (3 2)
```

Local variables are the usual way to name intermediate results. Module variables are best kept for things that really belong to the whole program; by convention, a module variable whose value changes is named between asterisks, as `*count*` is. Chapter 4 covers variables, and Chapter 12 covers modules, the namespaces module variables live in (*DIRM* chapters 4, 5 and 7).

3.5 Defining functions

The usual way to define a function is `define-method`. It takes the function's name, a list of parameters, and a body:

```
(define-method square ((n <number>))
  (* n n))                        ; => square

(square 12)                       ; => 144
(square 1/2)                      ; => 1/4
```

The parameter `(n <number>)` says that `n` must be a number. The class is called a *specializer*; if you leave it out and write just `n`, the parameter accepts any object. A function returns the value of the last form in its body; there is no `return` statement. Functions can of course call other functions:

```
(define-method hypotenuse ((a <real>) (b <real>))
  (sqrt (+ (square a) (square b))))

(hypotenuse 5 12)                 ; => 13.0
```

`define-method` does a little more than define a function: it adds a method to a *generic function*, which can hold several methods for different classes of argument. Section 3.11 returns to this, and it is the heart of Dylan. For now, think of `define-method` as "define a function".

Anonymous functions. The special form `method` makes a function without naming it. Such a function is a value like any other: it can be called directly, stored in a variable, or passed to another function.

```
((method (x y) (* x y)) 6 7)      ; => 42

(define add1 (method (x) (+ x 1)))
(add1 41)                         ; => 42
```

Keyword and rest parameters. After its required parameters, a parameter list may name *keyword parameters* after `#key`. The caller passes them by name, in any order, and each may have a default value. A parameter after `#rest` collects any remaining arguments in a sequence.

```
(define-method greet (name #key (greeting "Hello"))
  (concatenate greeting ", " name "!"))

(greet "Ada")                         ; => "Hello, Ada!"
(greet "Ada" greeting: "Goodbye")     ; => "Goodbye, Ada!"

(define-method sum-all (#rest numbers)
  (reduce + 0 numbers))

(sum-all 1 2 3 4 5)                   ; => 15
```

Returning several values. `values` returns more than one value at once, and `bind` receives them:

```
(define-method smallest-and-largest ((s <sequence>))
  (values (reduce min (first s) s)
          (reduce max (first s) s)))

(smallest-and-largest '(3 1 4 1 5 9 2 6))   ; => 1, 9

(bind ((lo hi (smallest-and-largest #(10 -2 7))))
  (- hi lo))                                ; => 12
```

Closures. A function made inside another function can use the local variables around it, and it keeps them for as long as it exists, even after the function that made it has returned. Here `make-counter` returns a new function each time it is called, and each of those functions has its own private count:

```
(define-method make-counter ()
  (bind ((n 0))
    (method ()
      (set! n (+ n 1))
      n)))

(define tick (make-counter))
(tick)                          ; => 1
(tick)                          ; => 2
(tick)                          ; => 3
```

Chapter 6 describes functions and parameter lists in detail (*DIRM* chapters 6 and 10).

3.6 Making decisions

`if` takes a test, a form to evaluate when the test is true, and a form to evaluate when it is false:

```
(define-method sign-of ((n <real>))
  (if (negative? n)
      "negative"
      (if (zero? n) "zero" "positive")))

(sign-of -4)                    ; => "negative"
(sign-of 0)                     ; => "zero"
```

Important: In Dylan, `#f` is the *only* false value. Zero, the empty list and the empty string all count as true:

```
(if 0 'true 'false)             ; => true
(if '() 'true 'false)           ; => true
```

Each branch of an `if` is a single form. To do several things in a branch, group them with `begin`, which evaluates its forms in order and returns the value of the last.

`when` and `unless` are handy when there is only one branch. `when` evaluates its body if the test is true, `unless` if it is false; otherwise each returns `#f`. Their bodies may contain several forms.

```
(when (> 3 2) "yes")            ; => "yes"
```

```
(when (> 2 3) "yes")            ; => #f
(unless (> 2 3) "no")           ; => "no"
```

`cond` chooses among several cases. Each clause starts with a test; the first clause whose test is true is evaluated, and the rest are skipped. A final clause whose test is `else:` catches everything else.

```
(define-method classify ((n <integer>))
  (cond ((< n 0)  "negative")
        ((= n 0)  "zero")
        ((< n 10) "small")
        (else:    "large")))

(classify 7)                    ; => "small"
(classify 42)                   ; => "large"
```

`case` compares one value against lists of constants, using `id?`. The constants are not evaluated, so symbols in them need no quote marks.

```
(define-method day-kind (day)
  (case day
    ((saturday sunday) "weekend")
    ((monday tuesday wednesday thursday friday) "weekday")
    (else: "unknown")))

(day-kind 'sunday)              ; => "weekend"
(day-kind 'friday)              ; => "weekday"
(day-kind 'someday)             ; => "unknown"
```

Without an `else:` clause, a `case` with no matching clause signals an error. Its relative `select` lets you choose the test that compares the values; with `instance?` as the test it chooses by class. Chapter 5 describes it.

`and` and `or` evaluate their arguments from left to right and stop as soon as the answer is known. `and` returns `#f` or the value of its last argument; `or` returns the first true value. `not` turns `#f` into `#t` and every other value into `#f`.

```
(and 1 2 3)                     ; => 3
(or #f 2 3)                     ; => 2
(not 0)                         ; => #f
```

Chapter 5 covers all of these (*DIRM* chapter 8).

3.7 Output and loops

Printing. Before writing loops it helps to be able to print. Both implementations have `display`, which writes a string or character as it is, and any other object in its printed form; `newline`, which ends the line; and `print`, which writes an object the way you would type it, strings in quotes, followed by a newline.

```
(begin
  (display "The answer is ")
  (display 42)
  (newline))
; prints: The answer is 42
```

`format` does the same in one call, `(format #t "The answer is ~a~%" 42)`; Chapter 13 describes it and the other printing functions.

for. `for` is the general loop. Its first argument lists the loop variables, each with an initial value and a form that computes its next value. The second is a list of an end test and the forms whose value the loop returns when the test becomes true. Any further forms are a body, evaluated on each pass. This loop computes a factorial without any body at all:

```
(define-method factorial ((n <integer>))
  (for ((i n (- i 1))
        (product 1 (* product i)))
       ((<= i 0) product)))

(factorial 10)                  ; => 3628800
```

On each pass, all the variables are stepped together, using the values from the previous pass.

dotimes. `dotimes` runs its body a fixed number of times, with a variable counting from zero:

```
(define-method sum-to ((n <integer>))
  (bind ((total 0))
    (dotimes (i (+ n 1))
      (set! total (+ total i)))
    total))

(sum-to 100)                    ; => 5050
```

for-each. `for-each` steps through the elements of one or more collections. Like `for`, it takes a list of variables, an end test with result forms, and a body. The loop also ends, returning `#f`, when any of the collections runs out.

```
(define-method first-even ((s <sequence>))
  (for-each ((x s))
            ((even? x) x)))

(first-even '(1 3 8 5 6))       ; => 8
(first-even '(1 3 5))           ; => #f
```

When there is no end test, write an empty list in its place. Here two collections are stepped together, one of them an unbounded range of integers:

```
(for-each ((name '(ann bob cy))
           (i (range from: 1)))
          ()
  (display i) (display ". ") (display name) (newline))
; prints: 1. ann
;         2. bob
;         3. cy
```

while and until. `while` repeats its body as long as a test is true, and `until` repeats it until a test becomes true. Both return `#f`, so they are used for their effects:

```
(define-method collatz-steps ((n <integer>))
  (bind ((steps 0))
    (until (= n 1)
      (set! n (if (even? n) (/ n 2) (+ (* 3 n) 1)))
      (set! steps (+ steps 1)))
    steps))

(collatz-steps 27)              ; => 111
```

A method's parameters are ordinary local variables, so `set!` can change them, as this one changes `n`.

In practice, many loops in Dylan are written not with these forms but with the collection functions of Section 3.13. Chapter 5 covers the loops (*DIRM* chapter 8).

3.8 Lists, vectors and strings

Lists, vectors and strings are all *sequences*: collections whose elements are numbered from zero. They share most of their operations.

Lists. `list` makes a list of its arguments. `first`, `second` and `third` take elements from the front; `element` takes any element by its index; `size` counts them.

```
(define primes (list 2 3 5 7 11))

(first primes)                  ; => 2
(second primes)                 ; => 3
(element primes 4)              ; => 11
(size primes)                   ; => 5
(last primes)                   ; => 11
```

A list is a chain of *pairs*. `cons` makes a new pair, putting an element on the front of a list; `car` is a list's first element and `cdr` the rest of it.

```
(cons 1 (list 2 3))             ; => (1 2 3)
(car primes)                    ; => 2
(cdr primes)                    ; => (3 5 7 11)
(empty? '())                    ; => #t
```

Many operations make new sequences and leave their arguments alone:

```
(reverse primes)                ; => (11 7 5 3 2)
(append primes '(13 17))        ; => (2 3 5 7 11 13 17)
(add primes 1)                  ; => (1 2 3 5 7 11)
primes                          ; => (2 3 5 7 11)
```

`add` adds an element wherever is most natural for the kind of sequence, which for a list is the front.

Vectors. A vector holds its elements in a block, so any element can be reached in the same time. `vector` makes one from its arguments; `make` makes one of a given size. `set!` with `element` changes an element.

```
(define scores (vector 10 20 30))
```

```
(element scores 1)              ; => 20
(set! (element scores 1) 99)    ; => 99
scores                          ; => #(10 99 30)

(make <vector> size: 3 fill: 0) ; => #(0 0 0)
```

The form `(set! (element scores 1) 99)` works because `set!` can change more than variables: given a call such as `(element scores 1)`, it calls the matching *setter* function instead. You will see the same thing with the slots of objects in Section 3.10.

A vector's size is fixed. A `<stretchy-vector>` grows as you add to it with `add!`:

```
(define names (make <stretchy-vector>))
(add! names 'ada)
(add! names 'grace)
(size names)                    ; => 2
(element names 1)               ; => grace
```

Strings. A string is a sequence of characters, so the sequence operations work on strings too:

```
(define lang "Dylan")

(size lang)                     ; => 5
(element lang 0)                ; => #\D
(concatenate lang " 1992")      ; => "Dylan 1992"
(as-uppercase lang)             ; => "DYLAN"
(reverse "abc")                 ; => "cba"
(copy-sequence "Hello, world" start: 7)   ; => "world"
```

`=` compares sequences element by element, while `id?` asks whether two values are the same object. `as` converts between kinds of sequence:

```
(= "abc" "abc")                     ; => #t
(id? "abc" (copy-sequence "abc"))   ; => #f
(as <list> "abc")                   ; => (#\a #\b #\c)
(as <string> '(#\x #\y))            ; => "xy"
```

Ranges. A `<range>` is a sequence of numbers in arithmetic progression. It does not store its elements but computes them, so a range can be very large or even unbounded, as the one in the `for-each` example of Section 3.7 was. `range` makes one: `from:` is the first number, `up-to:` an upper bound that is not included, `through:` a bound that is included, `by:` the step, and `size:` the number of elements.

```
(as <list> (range from: 1 up-to: 5))            ; => (1 2 3 4)
(as <list> (range from: 1 through: 5))          ; => (1 2 3 4 5)
(as <list> (range from: 10 through: 0 by: -5))  ; => (10 5 0)
(as <list> (range from: 0 size: 4))             ; => (0 1 2 3)
(reduce + 0 (range from: 1 through: 100))       ; => 5050
(member? 50 (range from: 0 by: 10))             ; => #t
```

Deques. A `<deque>`, a double-ended queue, grows and shrinks efficiently at both ends. `push` and `pop` work at the front, and `push-last` and `pop-last` at the back:

```
(define line-up (make <deque>))
(push line-up 1)
(push-last line-up 2)
(push-last line-up 3)
(as <list> line-up)             ; => (1 2 3)
(pop line-up)                   ; => 1
(pop-last line-up)              ; => 3
(as <list> line-up)             ; => (2)
```

Sorting. `sort` returns a sorted copy of a sequence. Its `test:` keyword argument is the function that compares two elements:

```
(sort '(3 1 2) test: <)                     ; => (1 2 3)
(sort (vector "pear" "apple" "fig") test: <) ; => #("apple" "fig" "pear")
(sort '(3 1 2) test: >)                     ; => (3 2 1)
```

Chapter 9 describes every kind of sequence and its operations (*DIRM* chapter 12).

3.9 Tables

A `<table>` maps keys to values; other languages call it a hash table, dictionary or map. Make one with `make`, store values with `set!` and `element`, and read them back with `element`:

```
(define ages (make <table>))

(set! (element ages 'alice) 34)
(set! (element ages 'bob) 27)
(set! (element ages 'carol) 41)

(element ages 'bob)                     ; => 27
(size ages)                             ; => 3
(element ages 'dave default: 'unknown)  ; => unknown
```

Asking for a key that is not in the table is an error unless you supply `default:`, whose value is returned instead. `remove-key!` removes a key. A table is a collection whose elements are its values, so the collection operations apply to it; `key-sequence` gives its keys.

```
(reduce + 0 ages)                       ; => 102
(remove-key! ages 'bob)
(size ages)                             ; => 2
```

Here is a common use: counting how often each word occurs.

```
(define word-counts (make <table>))

(for-each ((w '(the cat saw the dog and the bird)))
          ()
  (set! (element word-counts w)
        (+ 1 (element word-counts w default: 0))))

(element word-counts 'the)              ; => 3
(element word-counts 'cat)              ; => 1
```

The order of a table's elements and keys is not defined, so sort them when the order matters. Tables are described with the other collections in Chapter 9.

3.10 Classes and objects

`define-class` defines a class. It takes the class's name, a list of superclasses, and a description of each *slot*. Every class ultimately inherits from `<object>`.

```
(define-class <point> (<object>)
  (x init-value: 0 init-keyword: x:)
  (y init-value: 0 init-keyword: y:))
```

Each slot here has a name, a default value given by `init-value:`, and a keyword given by `init-keyword:` that sets the slot when an object is made. `make` makes an *instance* of a class:

```
(define pt (make <point> x: 3 y: 4))

(x pt)                          ; => 3
(y pt)                          ; => 4
(y (make <point>))              ; => 0
```

Defining the class also defined a *getter* function for each slot, named after the slot, which is why `(x pt)` reads the slot `x`. It defined a setter too, which `set!` uses:

```
(set! (x pt) 10)                ; => 10
(x pt)                          ; => 10
```

`instance?` asks whether an object belongs to a class, and `object-class` answers the object's class:

```
(instance? pt <point>)          ; => #t
(instance? 5 <point>)           ; => #f
(object-class pt)
```

`(object-class pt)` prints as `{the class <point>}`, and `pt` itself, like any object without a readable printed form, as `{instance of <point>}`. Chapter 13 explains how to give your own classes a printed form.

Inheritance. A class inherits the slots of its superclasses and adds its own. Here is a small family of shapes, which the following sections use:

```
(define-class <shape> (<object>)
  (name init-value: "shape" init-keyword: name:))

(define-class <circle> (<shape>)
  (radius required-init-keyword: radius:))

(define-class <rect> (<shape>)
  (width  init-keyword: width:)
  (height init-keyword: height:))
```

`required-init-keyword:` means that `make` must be given that keyword; making a `<circle>` without a `radius:` is an error. Instances of the subclasses have the inherited `name` slot as well as their own, and they are instances of `<shape>` as well as of their own classes:

```
(define wheel (make <circle> radius: 2 name: "wheel"))

(name wheel)                    ; => "wheel"
(radius wheel)                  ; => 2
(instance? wheel <shape>)       ; => #t
(subclass? <circle> <shape>)    ; => #t
```

Multiple inheritance. A class may have more than one superclass, and it then inherits from all of them:

```
(define-class <colored> (<object>)
  (color init-value: 'black init-keyword: color:))

(define-class <colored-circle> (<circle> <colored>))

(define dot (make <colored-circle> radius: 1 color: 'red))
(list (color dot) (radius dot) (name dot))   ; => (red 1 "shape")
```

Initializing objects. A slot's initial value need not be a constant. `init-function:` gives a function of no arguments that is called to compute the value each time an object is made. This matters when each object needs its own mutable value, such as a fresh collection, or a value computed afresh, such as a serial number:

```
(define *next-id* 0)

(define-class <ticket> (<object>)
  (id    init-function: (method ()
                          (set! *next-id* (+ *next-id* 1))
                          *next-id*))
  (title init-value: "untitled" init-keyword: title:)
  (notes init-function: (method () (make <stretchy-vector>))))

(id (make <ticket>))                        ; => 1
(id (make <ticket> title: "printer jam"))   ; => 2
```

Had `notes` used `init-value:` instead, every ticket would share one and the same stretchy vector.

For initialization that slot options cannot express, add a method to the generic function `initialize`, which `make` calls on each new object with the keyword arguments it was given. Call `next-method` first, so that the slots are filled in the usual way, and then do the rest. Here a rectangle made without a height becomes a square; `slot-initialized?` asks whether a slot has a value yet:

```
(define-class <rectangle> (<object>)
  (width  init-keyword: width:)
  (height init-keyword: height:))

(define-method initialize ((r <rectangle>) #key)
  (next-method)
  (unless (slot-initialized? r height)
    (set! (height r) (width r))))
```

```
(height (make <rectangle> width: 4))              ; => 4
(height (make <rectangle> width: 4 height: 3))    ; => 3
```

Important: A slot's getter is an ordinary function in the module, so a local variable or parameter with the same name hides it. Inside a method with a parameter called `name`, the call `(name s)` would try to call the parameter rather than the getter. Give parameters names that differ from your slots.

Chapter 7 describes classes, slot options and the initialization protocol (*DIRM* chapters 6 and 11).

3.11 Generic functions and multiple dispatch

Every `define-method` you have written so far has added a *method* to a *generic function*. A generic function can hold many methods, each for a different class of argument. When the generic function is called, it chooses the most specific method whose specializers match the arguments and runs it.

`define-generic-function` declares a generic function and its parameters. It is optional—the first `define-method` creates the generic function if there is none—but it documents the protocol in one place:

```
(define-generic-function area (shape))

(define-method area ((c <circle>))
  (* 3.14159 (radius c) (radius c)))

(define-method area ((r <rect>))
  (* (width r) (height r)))

(area (make <rect> width: 3 height: 5))      ; => 15
(area wheel)                                 ; => 12.56636
```

Because `area` works on any shape, functions that use it work on any shape too:

```
(define some-shapes
  (list (make <circle> radius: 1) (make <rect> width: 2 height: 2)))

(map area some-shapes)          ; => (3.14159 4)
```

The most specific method wins. Methods may overlap. When more than one applies, the one whose specializers are more specific is chosen. A method on `<shape>` serves every shape that has no method of its own:

```
(define-method describe ((s <shape>))
  (concatenate "a shape called " (name s)))

(describe (make <rect> width: 1 height: 1))   ; => "a shape called shape"
```

Dispatch on several arguments. A method can be specialized on any or all of its required arguments, and the choice takes all of them into account. This is *multiple dispatch*, and it is what makes generic functions more than methods attached to a single class:

```
(define-method collide ((a <circle>) (b <circle>))
  "two circles")

(define-method collide ((a <circle>) (b <rect>))
  "a circle and a rectangle")

(define-method collide ((a <shape>) (b <shape>))
  "two shapes")

(collide wheel (make <rect> width: 1 height: 1))  ; => "a circle and a rectangle"
(collide (make <rect> width: 1 height: 1) wheel)  ; => "two shapes"
(collide wheel wheel)                             ; => "two circles"
```

Specializing on a single object. A specializer can also be one particular object, written with `singleton`. Such a method applies only when the argument is that object, and it is more specific than any method on a class. This gives a neat way to write the base case of a recursion:

```
(define-method fact ((n <integer>))
  (* n (fact (- n 1))))

(define-method fact ((n (singleton 0)))
  1)

(fact 5)                          ; => 120
```

Singletons also suit symbols that act as commands or names:

```
(define-method speak ((animal (singleton 'dog))) "woof")
(define-method speak ((animal (singleton 'cat))) "meow")
(define-method speak ((animal <symbol>)) "...")

(speak 'dog)                      ; => "woof"
(speak 'fish)                     ; => "..."
```

Built-in classes such as `<integer>`, `<string>` and `<symbol>` are classes like any other, so you can add methods for them to your own generic functions. Chapter 6 describes generic functions and how methods are ordered (*DIRM* chapters 6 and 10).

3.12 Calling the next method

Often a method for a subclass wants to do what the method for the superclass does, and a little more. Inside a method, `next-method` calls the next most specific method, the one that would have run if this one did not exist. Called with no arguments, it passes along the original arguments:

```
(define-method describe ((c <circle>))
  (concatenate "a circle, " (next-method)))

(describe wheel)                  ; => "a circle, a shape called wheel"
```

Here the method on `<circle>` runs first and calls the method on `<shape>` for the rest of the description. `next-method` may also be given arguments, which replace the original ones;

the bank example in Section 3.16 uses this to change an amount before passing it on. Chapter 6 has the details.

3.13 Collections and functional operations

Because functions are values, much of Dylan's power comes from functions that take other functions as arguments. These work on every kind of collection.

`map` applies a function to each element and collects the results in a new collection of the same kind as the first argument. Given several collections, it takes one element from each. `map-as` names the kind of collection to return.

```
(map (method (x) (* x x)) '(1 2 3 4))   ; => (1 4 9 16)
(map + '(1 2 3) '(10 20 30))            ; => (11 22 33)
(map-as <vector> square '(1 2 3))       ; => #(1 4 9)
```

`reduce` combines the elements with a function of two arguments, starting from an initial value; `reduce1` uses the first element as the initial value.

```
(reduce + 0 '(1 2 3 4 5))       ; => 15
(reduce1 max '(3 9 2))          ; => 9
```

`choose` keeps the elements that satisfy a predicate. `any?` and `every?` ask whether some or all of them do. `find-key` answers the key, or index, of the first element that satisfies a predicate.

```
(choose even? '(1 2 3 4 5 6))   ; => (2 4 6)
(any? negative? '(3 -1 4))      ; => #t
(every? positive? '(3 -1 4))    ; => #f
(find-key #(5 8 13) even?)      ; => 1
```

`do` calls a function on each element just for its effect, and returns `#f`:

```
(do (method (s) (display (area s)) (newline)) some-shapes)
; prints: 3.14159
;         4
```

Making functions from functions. `curry` fixes the first arguments of a function. `compose` chains functions, the rightmost applied first. `complement` turns a predicate into its opposite. `apply` calls a function with arguments taken from a sequence.

```
(define add10 (curry + 10))
(add10 5)                                ; => 15
(map (curry * 2) '(1 2 3))               ; => (2 4 6)

(define sqrt-of-abs (compose sqrt abs))
(sqrt-of-abs -16)                        ; => 4.0

(choose (complement even?) '(1 2 3 4 5)) ; => (1 3 5)

(apply max '(4 7 1))                     ; => 7
```

Together these replace many explicit loops. The total area of a list of shapes, for instance, is one line:

```
(reduce + 0 (map area some-shapes))      ; => 7.14159
```

Chapter 9 lists the collection operations and Chapter 10 the functional operations (*DIRM* chapters 12 and 17).

3.14 Leaving early with bind-exit

Sometimes you want to stop a computation partway through and return a value at once, for example when a search finds what it was looking for. `bind-exit` binds a name to an *exit procedure*. Calling that procedure, from anywhere inside the `bind-exit`, however deeply nested, ends the `bind-exit` immediately and makes its argument the value of the whole form.

```
(define-method find-first ((pred <function>) (s <sequence>))
  (bind-exit (return)
    (do (method (x)
          (when (pred x) (return x)))
        s)
    #f))

(find-first even? '(1 3 4 5 6))     ; => 4
(find-first even? '(1 3 5))         ; => #f
```

The exit procedure is called from inside the function passed to `do`, yet it leaves `find-first` entirely. If it is never called, `bind-exit` returns the value of its last form, here `#f`.

`unwind-protect` guarantees that some cleanup forms run, however its first form is left, whether normally or by an exit:

```
(bind-exit (leave)
  (unwind-protect
    (leave 'escaped)
    (display "cleanup runs anyway")
    (newline)))
; prints: cleanup runs anyway
; => escaped
```

Chapter 5 describes non-local exits (*DIRM* chapter 8).

3.15 Conditions and errors

When something goes wrong, a Dylan program *signals* a *condition*, an object that describes what happened. `error` signals an error. In its simplest form it takes a message, which may contain `~a` where each further argument is to be inserted:

```
(define-method safe-divide ((a <number>) (b <number>))
  (if (zero? b)
      (error "cannot divide ~a by zero" a)
      (/ a b)))

(safe-divide 10 4)              ; => 5/2
```

If nothing handles the error, the interactive implementation enters the debugger, as Section 2.3 showed, and a compiled program prints the message and exits.

Handling errors. `handler-case` evaluates a form and, if a condition of a given class is signaled while it runs, abandons the form and evaluates the matching clause instead. Each clause begins with a list of the class and, after `condition:`, an optional name for the condition object:

```
(handler-case (safe-divide 1 0)
  ((<error> condition: c)
   (list 'caught (condition-format-string c))))
; => (caught "cannot divide ~a by zero")
```

An error signaled by `error` with a message is an instance of `<simple-error>`. The function `condition-format-string` returns its message, and `condition-format-arguments` returns the other arguments in a list. Errors that the system itself signals are conditions too, and `handler-case` catches them the same way:

```
(handler-case (element (vector 1 2) 5)
  ((<error>) "no such element"))    ; => "no such element"

(handler-case (+ 1 'a)
  ((<error>) "not a number"))       ; => "not a number"
```

Your own conditions. Conditions are objects, and their classes are ordinary classes. To signal something more specific than a message, define a subclass of `<error>` with slots that describe the problem, and pass an instance of it to `error`:

```
(define-class <out-of-stock> (<error>)
  (item  required-init-keyword: item:)
  (short required-init-keyword: short:))

(define-method take-from-stock ((on-hand <integer>) (wanted <integer>))
  (if (> wanted on-hand)
      (error (make <out-of-stock> item: 'widget short: (- wanted on-hand)))
      (- on-hand wanted)))

(take-from-stock 10 4)          ; => 6

(handler-case (take-from-stock 10 15)
  ((<out-of-stock> condition: e)
   (list 'short-by (short e))))  ; => (short-by 5)
```

A `handler-case` may have several clauses; they are tried in order, so put the more specific classes first.

Signals that are not errors. `signal` signals a condition that a program may ignore. Given a message, it signals a `<simple-warning>`. If no handler takes it, `signal` prints the warning and returns `#f`, and the program carries on:

```
(handler-case (signal "low on ~a" 'paper)
  ((<warning> condition: w)
   (condition-format-arguments w)))   ; => (paper)

(signal "nobody handles this")    ; => #f
; prints: WARNING: nobody handles this          (interactive)
```

```
;         Dylan warning: nobody handles this    (compiled)
```

Dylan's condition system can do much more: `handler-bind` establishes a handler that runs *without* first leaving the signaling code, so it can return a value to it, decline, or choose among *restarts*. Chapter 11 explains it all (*DIRM* chapter 13).

3.16 A complete program

This section builds a small but complete program, a model of a bank, one piece at a time. It brings together classes, generic functions, `next-method`, tables, the collection functions and conditions. Type the pieces at the loop as you go, or put them all in a file called `bank.dyl`; the next section compiles that file.

Accounts. Every account has an owner and a balance. A savings account also earns interest at some rate, and a checking account charges a fee on each withdrawal. Balances are whole numbers of dollars, and the interest rate is an exact ratio, so no money is lost to rounding until we choose to round.

```
(define-class <account> (<object>)
  (owner   required-init-keyword: owner:)
  (balance init-value: 0 init-keyword: balance:))

(define-class <savings-account> (<account>)
  (rate init-value: 1/50 init-keyword: rate:))

(define-class <checking-account> (<account>)
  (fee init-value: 1 init-keyword: fee:))
```

An error class. Withdrawing more than the balance is an error, and the condition records which account and how much was asked for. The slot names are chosen so that they do not clash with the names of parameters used elsewhere (see the warning in Section 3.10).

```
(define-class <insufficient-funds> (<error>)
  (refused-account required-init-keyword: account:)
  (refused-amount  required-init-keyword: amount:))
```

The operations. Three generic functions make up the protocol of an account. The basic methods are on `<account>`:

```
(define-generic-function deposit! (account amount))
(define-generic-function withdraw! (account amount))
(define-generic-function month-end! (account))

(define-method deposit! ((a <account>) (amount <integer>))
  (set! (balance a) (+ (balance a) amount)))

(define-method withdraw! ((a <account>) (amount <integer>))
  (when (> amount (balance a))
    (error (make <insufficient-funds> account: a amount: amount)))
  (set! (balance a) (- (balance a) amount)))
```

```
(define-method month-end! ((a <account>))
  (balance a))
```

Try them on a plain account:

```
(define acct (make <account> owner: 'dave balance: 100))

(deposit! acct 25)              ; => 125
(withdraw! acct 40)             ; => 85
(balance acct)                  ; => 85
```

Specializing with next-method. The other kinds of account differ in small ways, which is exactly what `next-method` is for. A checking account's withdrawal is an ordinary withdrawal of the amount *plus the fee*, so its method calls the next method with a changed argument. A savings account adds its interest at the end of the month and then does whatever any account does.

```
(define-method withdraw! ((a <checking-account>) (amount <integer>))
  (next-method a (+ amount (fee a))))

(define-method month-end! ((a <savings-account>))
  (deposit! a (round (* (balance a) (rate a))))
  (next-method))
```

`round` returns two values, the nearest integer and the remainder, but `deposit!` uses only the first.

```
(define chk (make <checking-account> owner: 'erin balance: 100 fee: 2))
(withdraw! chk 10)              ; => 88

(define sav (make <savings-account> owner: 'fred balance: 500))
(month-end! sav)                ; => 510
```

Because the withdrawal for a checking account goes through the method on `<account>`, it also gets that method's check for insufficient funds, fee included:

```
(handler-case (withdraw! chk 87)
  ((<insufficient-funds> condition: c)
   (refused-amount c)))         ; => 89
```

The bank. The bank keeps its accounts in a table, keyed by the owner's name. `open-account` takes the class of account to open, so one function opens every kind. It passes any further keyword arguments on to `make` with `apply`.

```
(define *accounts* (make <table>))

(define-method open-account ((class <class>) (who <symbol>) #rest options)
  (bind ((a (apply make class owner: who options)))
    (set! (element *accounts* who) a)
    a))

(define-method lookup ((who <symbol>))
```

```
  (element *accounts* who default: #f))
```

Handling a refusal. `try-withdraw` attempts a withdrawal and reports a refusal instead of letting the error escape. It answers whether the withdrawal was made.

```
(define-method try-withdraw ((who <symbol>) (amount <integer>))
  (handler-case (begin (withdraw! (lookup who) amount) #t)
    ((<insufficient-funds> condition: c)
     (display "refused: ")
     (display (owner (refused-account c)))
     (display " cannot withdraw ")
     (display (refused-amount c))
     (newline)
     #f)))
```

Reports. The total of all deposits maps `balance` over the table and adds up the results. The report lists the accounts in alphabetical order of owner; since the table's keys come in no particular order, it sorts them, comparing the symbols as strings.

```
(define-method total-deposits ()
  (reduce + 0 (map balance *accounts*)))

(define-method report ()
  (for-each ((who (sort (key-sequence *accounts*)
                        test: (method (a b)
                                (< (as <string> a) (as <string> b))))))
            ()
    (display who)
    (display ": ")
    (display (balance (lookup who)))
    (newline)))
```

A month at the bank. Finally, the program itself: open three accounts, make some transactions, close the month, and report. `(do month-end! *accounts*)` runs the month's end for every account, each with its own method.

```
(open-account <savings-account> 'alice balance: 1000)
(open-account <checking-account> 'bob balance: 200 fee: 2)
(open-account <account> 'carol)

(deposit! (lookup 'carol) 50)
(withdraw! (lookup 'bob) 48)
(try-withdraw 'carol 80)
(do month-end! *accounts*)
(report)
(display "total: ")
(display (total-deposits))
(newline)
```

At the loop, each form's value appears after it as usual, mixed with the output. The output itself is this:

```
refused: carol cannot withdraw 80
alice: 1020
bob: 150
carol: 50
total: 1220
```

Alice earned two percent interest on 1000; Bob's withdrawal of 48 cost him 50 with the fee; and Carol's attempt to withdraw 80 from her 50 was refused and reported, without stopping the program.

3.17 Compiling the program

The bank is a complete program, and `dylanc` can turn it into an executable. Put the definitions and the final forms of the previous section—everything except the small experiments with `acct`, `chk` and `sav`—into `bank.dyl`, in the order they were given, and compile it:

```
$ bin/dylanc bank.dyl
$ ./bank
refused: carol cannot withdraw 80
alice: 1020
bob: 150
carol: 50
total: 1220
```

The output is the same as at the loop, because the program uses only `display` and `newline` for its output, which both implementations have. Nothing in the program changes a class or adds a method while it runs, so it is a static program, and the compiler accepts it unchanged.

You can also run the same file in the interactive implementation without typing it, either by feeding it to the loop or by compiling it to Lisp, as Chapter 2 showed:

```
$ bin/dylan < bank.dyl
```

This is the usual way of working with the two implementations: develop and test a piece at a time at the loop, where every definition can be changed and tried at once, then compile the finished program into a fast executable that needs neither SBCL nor Dylan to run.

Tip: The compiler warns about any name a program uses but never defines. Building with `dylanc` from time to time, even while you are still working at the loop, is a quick way to catch misspelled names.

3.18 Where to go from here

You have now seen most of the language at least once. Part II covers each topic thoroughly, in the order of this tutorial: syntax and variables (Chapter 4), control flow (Chapter 5), functions and generic functions (Chapter 6), classes (Chapter 7), numbers (Chapter 8), collections (Chapter 9), functional operations (Chapter 10), conditions (Chapter 11), modules (Chapter 12), and printing (Chapter 13).

Part III describes the two implementations as tools. Read Chapter 15 before building larger programs with the compiler, and Chapter 16 for its macros and threads. Appendix A lists everything in which this system differs from the 1992 manual.

The 1992 manual itself is worth reading alongside this book. It is short, clearly written, and full of examples, and its annotated copy in `manual/Dylan-1992/` links each chapter to the design notes that later changed it.

Part II

The Language

Chapter 4

Syntax and Program Structure

Dylan, as the 1992 manual defines it, is written in a parenthesized prefix notation of the kind Lisp and Scheme use. A program is a sequence of *forms*; each form is either an atom, such as a number, a string or a name, or a list whose first element says what the form does. There are no statements, no operator precedence and no semicolons ending lines: a semicolon starts a comment.

This chapter describes the notation itself: how the reader turns characters into Dylan objects (Section 4.2), how those objects are evaluated as expressions (Section 4.3), and how a program is put together from top-level forms and source files (Section 4.4). The standard is *DIRM* chapters 2 and 3; symbols and keywords are also covered in *DIRM* chapter 15. Both implementations read programs with the same reader, so everything in Section 4.2 holds for both.

4.1 How the examples are shown

The examples in this part of the book are Dylan source. Each has been run with both implementations, compiled with `bin/dylanc` and typed at the read-eval-print loop of the Common Lisp implementation, except where the text says that an example applies to only one of them. The value of an expression is shown in a comment after it, introduced by `=>`:

```
(+ 3 4)                 ; => 7
```

When an expression returns several values, the comment lists them in order, separated by commas; `(values 98 102)` is shown as `; => 98, 102`. When it returns no values at all, the comment says `(no values)`. A long value continues on the comment lines below. Output that a form prints is shown in comments introduced by `prints:`, one line of output to each comment:

```
(print "hello")         ; prints: "hello"
```

A form whose evaluation signals an error is marked `; => error`, usually followed by a short description of the error. The exact wording of error messages differs between the two implementations, and the comment does not try to reproduce either.

Values that cannot be read back in, such as classes and functions, are printed inside braces: `{the class <integer>}`, as the 1992 manual writes them.

4.2 The reader

The *reader* turns the characters of a program into Dylan objects: numbers, strings, symbols, lists and so on. A program is just those objects; evaluation (Section 4.3) gives them meaning. The reader is the same in both implementations: the compiler to C uses the Common Lisp implementation's reader, so a file that one reads, the other reads identically.

4.2.1 White space and delimiters

Spaces, tabs, newlines, returns and page breaks are *white space*. White space separates tokens, and the amount of it does not matter, so a program can be laid out freely. A token also ends at a parenthesis, a double quote, a semicolon, a quote mark, a backquote or a comma. Everything else is part of a token, which is why names such as `set!`, `even?`, `<integer>` and `*standard-output*` are single names.

4.2.2 Comments

A semicolon starts a comment that runs to the end of the line. The characters `#|` start a block comment, which runs to the matching `|#`. Block comments may span lines and may be nested, so a block comment can comment out code that already contains one:

```
(+ 1 #| an inline comment |# 2)   ; => 3
#| (print "never run")
   #| a nested comment |#
   (print "still inside the outer comment") |#
```

A comment is white space as far as the reader is concerned. Note that `#;`, which comments out one form in some Lisps, does not exist in Dylan.

4.2.3 Numbers

Numbers are written in the usual way. Chapter 8 describes the number classes and arithmetic; here is only how numbers are written.

- An *integer* is an optional sign followed by decimal digits: `0`, `42`, `-17`, `+57`. A trailing decimal point is allowed and still denotes an integer: `1.` is the integer `1`.
- A *ratio* is two integers separated by a slash, with no spaces: `1/3`, `-6/4`. Ratios are always reduced, and a ratio that reduces to a whole number is an integer: `4/2` reads as `2`.
- A *floating-point number* has a decimal point with digits after it, an exponent, or both: `1.5`, `-4.0`, `.5`, `1e10`, `2.5d2`. An exponent is introduced by `e` or `d` for a double float, the default, or by `f` or `s` for a single float; `1.5s0` is a single float.
- An integer or ratio in another base is written `#x` (hexadecimal), `#o` (octal) or `#b` (binary) followed by the digits, which may have a sign: `#x1f4e`, `#xFF`, `#o-17`, `#b101`. Letters may be in either case.

```
(list 123 +57 -4.0 1.5 #x1f4e)     ; => (123 57 -4.0 1.5 8014)
(list 1. 6/4 4/2 .5)               ; => (1 3/2 2 0.5)
(list #o-17 #b101 #xff/2 1.0e-3)   ; => (-15 5 255/2 0.001)
```

A single float prints with its `s` marker, `1.5s0`, and a double float without one, `10000000000.0` (Section 8.12).

Important: The compiler to C has no bignums: an integer is 63 bits, and an integer literal that does not fit is an error when compiling. It has no complex numbers either. The Common Lisp implementation has both, and reads a complex number in Common Lisp's notation, `#c(1 2)`. See Chapter 8.

4.2.4 Characters

A *character* is written `#\` followed by the character itself, or by the character's name. The names of the 1992 manual are `newline`, `space`, `rubout`, `page`, `tab`, `backspace`, `return` and `linefeed`; their case does not matter, so `#\Space` and `#\space` are the same character. `#\linefeed` is the same character as `#\newline`. A single letter after the backslash is that letter, in its case: `#\a` and `#\A` are different characters. Any other single character may follow the backslash too, parentheses, semicolons and quotes included.

```
(list #\a #\A #\( #\; #\")   ; => (#\a #\A #\( #\; #\")
(list #\Space #\TAB)         ; => (#\space #\tab)
(map (method (c) (as <integer> c))
     (list #\newline #\linefeed #\space #\tab #\rubout))
                             ; => (10 10 32 9 127)
```

Because the reader is built on Common Lisp's, it also accepts the other character names Common Lisp knows, such as `#\Nul` and `#\Escape`. A program that is to be portable to other Dylan implementations should keep to the manual's names.

4.2.5 Strings

A *string* is written between double quotes. Every character between the quotes is part of the string, white space and newlines included. A backslash quotes the character after it, which is how a string includes a double quote or a backslash:

```
"abc"              ; => "abc"
(size "a\"b")      ; => 3
(size "a\\b")      ; => 3
(element "a\"b" 1) ; => #\"
```

A backslash does nothing more than quote the next character, as *DIRM* chapter 3 says. In particular there are no escape sequences: `"a\nb"` is the three characters `a`, `n` and `b`, not a string with a newline in it.

```
(size "a\nb")      ; => 3
(element "a\nb" 1) ; => #\n
```

To put a newline into a string, write the newline itself between the quotes, or build the string from characters at run time:

```
(size (as <string> (list #\a #\newline #\b)))   ; => 3
```

4.2.6 Symbols

Any token that is not a number, and does not end in a colon, is a *symbol*. Symbols name variables, and they are also objects in their own right, of class `<symbol>`. A symbol may contain letters, digits and most punctuation: `+`, `<=`, `set!`, `even?`, `*x*`, `$pi`, `1+`, `-` and `...` are all symbols.

```
(list '+ '<= 'set! 'even? '*x* '$pi '1+ '- '...)
                        ; => (+ <= set! even? *x* $pi 1+ - ...)
(instance? 'foo <symbol>)   ; => #t
```

Symbols are not case-sensitive: `Foo`, `foo` and `FOO` are the same symbol. But a symbol remembers the case it was first written in, and that is how it prints (*DIRM* chapter 15). In the example below the symbol is written `Hello` first, so it prints that way however it is written later:

```
(id? 'Hello 'hello)        ; => #t
'HELLO                     ; => Hello
(as <string> 'hello)       ; => "Hello"
```

"First written" means first read. The compiler to C reads a whole program before running any of it, so a symbol's spelling is the one in the source text that comes first. The Common Lisp implementation reads a form at a time, so a symbol first made at run time by `(as <symbol> "Abc")` keeps the spelling `Abc` if no form read earlier wrote it.

Dylan programs follow naming conventions (*DIRM* chapter 3). Except for setter variables, described below, the conventions mean nothing to the language; they only help a reader.

- Names of classes begin and end with angle brackets: `<window>`, `<integer>`.
- Module variables meant to change while the program runs begin and end with asterisks: `*window-count*`.
- Constants begin with a dollar sign: `$pi`.
- Predicates, functions that answer true or false, end with a question mark: `even?`, `instance?`.
- Operations that destructively modify their arguments end with an exclamation point: `reverse!`, `sort!`.

4.2.7 Setter names

A variable name is either a symbol or a list of the form `(setter` *`symbol`*`)`. The second kind names a *setter variable*, the variable that holds the function which stores into the place the function in *symbol* reads. The value of `element` reads an element of a collection; the value of `(setter element)` stores one. A setter name can be used wherever a symbol can name a variable:

```
(define (setter my-variable) 20)
(setter my-variable)            ; => 20
```

Because `setter` has this special meaning, the symbol `setter` itself cannot be the name of a variable. The design note that later replaced `(setter foo)` with `foo-setter` is not part of this language. Setters are described in Section 5.3 and Section 6.9.

4.2.8 Keywords

A token ending in a colon is a *keyword*: `foo:`, `size:`, `init-value:`. Keywords look like symbols but are a separate class, `<keyword>`, and are used as symbolic constants, chiefly to name keyword arguments. The colon is not part of a keyword's name. Keywords, like symbols, are not case-sensitive and remember the case they were first written in.

```
foo:                          ; => foo:
(id? hello: HELLO:)           ; => #t
(instance? foo: <keyword>)    ; => #t
(instance? foo: <symbol>)     ; => #f
(as <keyword> "size")         ; => size:
(as <string> size:)           ; => "size"
```

Two keywords, or two symbols, with the same name are always the same object: `id?` is true of them.

4.2.9 True, false and the empty list

`#t` is the canonical true value and `#f` is the false value; either may be written in upper case. `#f` is the *only* false value: every other object counts as true when tested, including `0`, the empty string and the empty list.

The empty list, written `()`, is an object of its own, of class `<empty-list>`. Unlike in older Lisps it is not the same object as `#f`, and it is true:

```
(id? '() #f)                  ; => #f
(if '() 'true 'false)         ; => true
(instance? '() <list>)        ; => #t
```

4.2.10 Lists and vectors

A list is written as its elements between parentheses. A dot between the last two elements writes a dotted list, whose final tail is not the empty list. A vector, a one-dimensional array, is written like a list with a `#` before the opening parenthesis.

```
'(a (b c) ())         ; => (a (b c) ())
'(a . b)              ; => (a . b)
'#(1 2 3)             ; => #(1 2 3)
(instance? '#(1 2) <vector>)   ; => #t
```

A list in a program is normally a form to be evaluated, so a list meant as data must be quoted. A vector need not be: `#(1 2 3)` evaluates to itself. The elements of a literal list or vector are not evaluated, and literal data should not be modified.

4.2.11 Quote and backquote

A single quote before a form, *`'form`*, is an abbreviation for `(quote` *`form`*`)`, which evaluates to the form itself, unevaluated. Quoting is needed for symbols and lists used as data; other literals evaluate to themselves and need no quote, though quoting them does no harm.

```
(quote foo)           ; => foo
'foo                  ; => foo
'(+ 10 10)            ; => (+ 10 10)
```

```
(+ 10 10)            ; => 20
'"s"                 ; => "s"
```

A *backquote* quotes a template in which parts are filled in when it is evaluated. Within a backquoted form, a comma, `,`*form*, evaluates *form* and puts its value in that place; `,@`*form* evaluates *form*, which must answer a list, and splices its elements in. *DIRM* chapter 3 defers backquote to a section the manual does not contain, so both implementations follow the usual Lisp rules, including backquotes nested inside backquotes and backquoted vectors.

```
(define x 5)
(define xs '(a b))
`(x ,x)              ; => (x 5)
`(1 ,@xs 2)          ; => (1 a b 2)
`(1 . ,x)            ; => (1 . 5)
`#(1 ,x)             ; => #(1 5)
```

The reader turns `'x`, `` `x ``, `,x` and `,@x` into the lists `(quote x)`, `(quasiquote x)`, `(unquote x)` and `(unquote-splicing x)`. The compiler to C prints these lists in that long form; the Common Lisp implementation prints them abbreviated, as `'x` and so on. A comma outside any backquote is an error. Backquote is most useful in macros, an extension of the compiler to C described in Chapter 16.

4.2.12 #key, #rest and #next

The tokens `#key`, `#rest` and `#next` mark the parts of a parameter list (Section 6.3); `#rest` also appears in `bind` (Section 5.2). Like `#t` and `#f` they are case-insensitive.

Both implementations also read `!key`, `!rest` and `!next` as these same tokens, because Thomas, from which the Common Lisp implementation grew, wrote them that way, and so do its programs, the manual's examples among them. As a consequence those three names cannot name variables. Any other name may.

4.2.13 Unreadable objects

Some objects, such as functions and classes, have no notation that reads back as the same object. They print inside braces, `{the generic function print}`, and there is no notation that reads them back. Any other `#` syntax than those above (`#t`, `#f`, `#\`, `#(`, `#x`, `#o`, `#b`, `#|`, `#key`, `#rest`, `#next`, and `#c` in the Common Lisp implementation) is an error.

4.3 Expressions and evaluation

A program is made of expressions. Evaluating an expression yields zero or more values, may have side effects, and may evaluate other expressions inside it. There are four kinds of expression (*DIRM* chapter 3): literal constants, variable references, function calls and syntax forms.

4.3.1 Literal constants

Numbers, characters, strings, keywords, vectors, `#t`, `#f` and the empty list `()` evaluate to themselves. Symbols and non-empty lists must be quoted to be used as constants, because unquoted they are variable references and calls.

```
"abc"      ; => "abc"
123        ; => 123
foo:       ; => foo:
#\a        ; => #\a
#t         ; => #t
()         ; => ()
'(1 2 3)   ; => (1 2 3)
```

4.3.2 Variable references

A symbol, or a setter name `(setter` *`symbol`*`)`, evaluates to the value of the variable it names. There are two kinds of variable. *Lexical variables* are created by `bind`, by a method's parameter list and by several other forms, and are visible only in the text of the form that creates them. *Module variables* are created by the defining forms (Section 4.4) and are visible throughout their module (Chapter 12). An inner variable *shadows* any outer one of the same name, lexical or module.

```
(define my-variable 25)
my-variable                      ; => 25
(bind ((my-variable 50))
  (+ my-variable my-variable))   ; => 100
```

Referring to a variable that has no value is an error. The compiler to C also warns, when compiling, of a module variable that is used but defined nowhere in the program.

4.3.3 Function calls

A list whose first element is not the name of a syntax operator is a *function call*. Its first element, the *operator*, is evaluated to give the function; the rest are evaluated to give the arguments, and the function is called with them. The values of the call are the values the function returns.

The operator may be any expression that evaluates to a function, not only a name. In this Dylan is like Scheme and unlike Common Lisp:

```
(+ 3 4)              ; => 7
(* (+ 3 4) 5)        ; => 35
((if #t + *) 4 5)    ; => 9
((method (x) (* x x)) 6)   ; => 36
```

The arguments are evaluated from left to right. The manual allows the operator to be evaluated at any time before, during or after the arguments; both implementations evaluate it first, so a call is evaluated from left to right, operator and all. A program meant for other Dylans should not depend on it.

```
(list (note 1) (note 2) (note 3))   ; => (1 2 3)
(reverse trail)                     ; => (1 2 3)
```

(Here `note` records each argument in a list as it is evaluated.)

4.3.4 Syntax forms

A list whose first element names a *syntax operator* is a *syntax form*. A syntax form looks like a call, but it is not one: each syntax operator has its own rule for which of its parts are evaluated,

when, and what the others mean. In `(bind ((x 10)) (+ x 1))`, for example, `((x 10))` is not evaluated at all; `bind` reads it as a list of bindings.

There are three kinds of syntax operator.

Special forms are built into the language: `begin`, `bind`, `bind-exit`, `bind-methods`, `if`, `method`, `quote`, `set!`, `setter` and `unwind-protect`.

Macros are syntax forms defined in terms of others. The language's macros are `and`, `or`, `when`, `unless`, `cond`, `case`, `select`, `for`, `for-each`, `dotimes`, `while`, `until`, and the condition forms `handler-bind` and `handler-case` (Chapter 11). Backquote, `quasiquote`, is one too.

Definitions create module variables: `define`, `define-class`, `define-generic-function`, `define-method`, `define-slot` and `define-module`.

`not`, by contrast, is an ordinary function. A syntax operator is not a function and has no value: `(map and ...)` is meaningless, and so is passing `if` as an argument.

The 1992 manual has no way for a program to define its own syntax forms. The compiler to C adds Common Lisp–style macros, defined with `define-macro` and expanded when compiling (Chapter 16); the Common Lisp implementation does not have them.

4.3.5 Multiple values

An expression can return one value, several, or none. `values` returns its arguments as multiple values, and `bind` receives them (Section 5.9). Where only one value is wanted, as an argument to a call, for instance, the first value is used, or `#f` if there are none.

```
(values 1 2 3)              ; => 1, 2, 3
(print (list (values 1 2) 3))   ; prints: (1 3)
(print (list (values)))         ; prints: (#f)
```

4.4 Program structure

4.4.1 Top-level forms and definitions

A form that is not part of any other form is a *top-level form*; so are the forms directly inside a `begin` that is itself at top level. A program is a sequence of top-level forms. Some are definitions, which create module variables; the rest are expressions, evaluated for their effects.

Definition	Creates
`define`	a variable with an initial value (Section 5.1)
`define-method`	a method, added to a generic function (Section 6.4)
`define-generic-function`	a generic function (Section 6.5)
`define-class`	a class, its slots and their accessors (Section 7.1)
`define-slot`	a slot of an existing class or object (Section 7.4)
`define-module`	a module (Chapter 12)

The compiler to C adds three more definitions, which are described in Chapter 16: `define-macro`, `define-for-macros` and `define-thread-variable`.

The 1992 manual describes definitions as if they were ordinary forms that could appear anywhere, and both implementations accept `define` inside a method, where it still defines a module variable. Later design notes ("Definitions are Declarative") restrict definitions to top level, and that is the only place they belong: a definition inside a method happens again each time the method runs, and in the compiler to C a `define-method` that refers to local variables cannot be compiled: it stops with `define-method: the method refers to local variables (k)`, naming the variables. Put definitions at top level.

4.4.2 When things happen

The two implementations agree on what a program means, but not on when a definition takes effect, because one runs a static program and the other is interactive.

The **Common Lisp implementation** evaluates each top-level form when it reaches it, at the prompt or in a file. A definition takes effect then, and not before; a later form may redefine a variable, which only draws a warning, or redefine a class or add methods to a generic function that is already in use.

The **compiler to C** compiles a whole program at once. Every class, generic function and method the program defines exists before any of its expressions run, wherever in the program it is defined. The top-level expressions, and the initial values of `define`, are then evaluated in order, from the first file to the last. A method may therefore be called by an expression that comes before its definition in the source, but a variable defined with `define` has no value until its definition has been evaluated.

```
(later 2)                    ; => 20
(define-method later (x) (* x 10))
later-var                    ; => error: later-var has no value yet
(define later-var 5)
```

In the Common Lisp implementation the first expression is an error too, since `later` is not yet defined when it is evaluated.

A static program is also stricter about definitions. In the compiler to C a class or generic function may be defined only once, a name cannot be defined both by `define` and as a generic function, and a variable that holds a class or a generic function cannot be assigned. These are errors when compiling. Chapter 15 lists the full set of restrictions.

4.4.3 Source files and the module: header

A Dylan source file conventionally has the extension `.dyl`. It may begin with a *header* (design note 33): lines of the form *keyword*`:` *value*, each starting at the beginning of a line, ended by a blank line. The keyword `module:` names the module the file's forms belong to; any other keywords, such as `author:`, `copyright:` and `version:`, are accepted and ignored. A file without a header belongs to the module `dylan-user`.

```
module: shapes
author: A. Programmer
version: 1.0

(define-method greet ()
  (print "hello from shapes"))
(greet)
```

The header is not Dylan syntax: it is read before the reader starts, and design note 33 does not allow comments in it. The module it names must be defined by a `define-module` form, which can be in an earlier file of the program; Chapter 12 describes modules. Both implementations honor the header when they compile a file: the compiler to C for each file on its command line, and the Common Lisp implementation in `dylan:compile-file`. At the interactive prompt, `(dylan:in-module` *`name`*`)` sets the module instead (Chapter 14).

A program for the compiler to C may be spread over several files, given on the command line in order: `bin/dylanc -o app a.dyl b.dyl`. They form one program, and their top-level expressions run in that order (Chapter 15).

Chapter 5

Variables and Control

This chapter covers the forms a program uses to name values and to direct the flow of evaluation: defining and binding variables, assigning them, choosing between alternatives, repeating, returning several values, and leaving a computation early. They are the subject of *DIRM* chapters 4, 7 and 8. Most of them are syntax forms, special forms or macros (Section 4.3.4); for each, this chapter says which of its parts are evaluated, in what order, and what it returns.

5.1 Module variables: define

```
(define variable-name initial-value)
```

`define` creates a module variable named *variable-name* in the current module and stores in it the value of *initial-value*. The name is not evaluated; it must be a symbol or a setter name `(setter symbol)`. The initial value is evaluated, and if it returns several values, the first is used. At the interactive prompt `define` returns the variable's name.

```
(define foo 10)
(define bar foo)
(+ foo bar)                ; => 20
(define (setter my-place) 0)
(setter my-place)          ; => 0
(define v (values 1 2))
v                          ; => 1
```

A module variable can be read from anywhere in its module, by code before or after its definition, though it has no value until the definition has been evaluated (Section 4.4.2). A variable created by `define` may be assigned with `set!` (Section 5.3).

Defining a variable that already exists in the module is allowed by the Common Lisp implementation, which warns `redefining variable` and stores the new value; in the compiler to C, defining a variable twice with `define` simply stores twice, but defining a name both with `define` and as a class or generic function is an error when compiling.

Note: Later design notes changed `define`: one renamed it `define-constant` and added `define-variable`, another let it declare types and receive multiple values, as `bind` does. None of these is part of this language. There is no `define-variable` or `define-constant`, and `define` takes exactly one plain name and one expression; `(define (x <integer>) 3)` and `(define a b (values 1 2))` are errors in both implementations. The compiler to C

adds `define-thread-variable`, which defines a variable each thread has its own value of (Chapter 16).

5.2 Lexical variables: bind

```
(bind (binding ...)  form ...)
```

`bind` creates lexical variables and evaluates the *form*s, its *body*, with them in scope. It returns the values of the last form, or `#f` if there are no forms. Each *binding* is a list of one or more variables followed by an expression, the *init*:

```
(variable ...  [#rest rest-variable] init)
```

where each *variable* is a name, or a list `(name type)` that declares the variable's type.

The bindings are made one after another. Each init is evaluated in an environment that includes the variables of the bindings before it, so a later init can use an earlier variable:

```
(bind ((x 20)
       (y (+ x x)))
  (+ y y))                ; => 80
(bind ((x 1)))            ; => #f
```

A variable made by `bind` shadows any module variable or outer lexical variable of the same name, for the extent of the body:

```
(define foo 10)
(bind ((foo 35))
  (+ foo foo))            ; => 70
(bind ((foo 20))
  (bind ((foo 50))
    (+ foo foo)))         ; => 100
foo                       ; => 10
```

5.2.1 Receiving multiple values

When a binding has several variables, they receive the values of its init in order. Missing values are `#f`, and values left over are ignored, unless the variables end with `#rest` and a name, in which case that variable receives a sequence of the remaining values.

```
(bind ((a b c (values 1 2 3)))
  (list a b c))           ; => (1 2 3)
(bind ((a b (values 1)))
  (list a b))             ; => (1 #f)
(bind ((first #rest others (values 1 2 3)))
  others)                 ; => (2 3)
(bind ((#rest all (values)))
  (empty? all))           ; => #t
```

Multiple values also give parallel binding, like `let` in Lisp, since the init is evaluated before any of its variables exist:

```
(bind ((x 10) (y 20))
  (bind ((x y (values y x)))
    (list x y)))          ; => (20 10)
```

5.2.2 Type declarations

A variable written (*name type*) must receive an instance of *type*, or an error is signaled. The type is an expression, evaluated before the binding's init; it must evaluate to a class or a singleton (Section 6.6).

```
(bind (((n <integer>) (+ 1 2)))
  n)                      ; => 3
(bind (((x <integer>) (sqrt 2)))
  x)                      ; => error: 1.4142135623730951 is not
                          ;    an instance of <integer>
```

The type is checked when the variable is bound. It is not checked again when the variable is assigned: in both implementations `(bind (((x <integer>) 1)) (set! x 'a) x)` returns `a`. (A later design note made the restriction apply to assignment too; the 1992 manual does not.) The compiler to C uses declared types to choose faster code (Chapter 18).

5.3 Assignment: set!

(set! *place new-value*)

`set!` stores a new value in a place and returns the new value (*DIRM* chapter 7). The place is either a variable name or a call-like list.

When *place* is a variable name, lexical or module, the variable is given the value of *new-value*. It is an error if there is no such variable.

```
(define counter 0)
(set! counter (+ counter 1))   ; => 1
counter                        ; => 1
(bind ((x 1))
  (set! x (* x 10))
  x)                           ; => 10
```

Some module variables cannot be assigned. A variable made by `define-class` is read-only in both implementations. In the compiler to C a variable holding a generic function cannot be assigned either, nor can any variable of the `dylan` module; these are errors when compiling. The Common Lisp implementation allows assigning a generic function's variable, as the 1992 manual does.

5.3.1 The extended form

When *place* is a list (*name arg ...*), it stands for a location that the call (*name arg ...*) would read, and `set!` stores into it by calling the setter function:

(set! (*name arg ...*) *new-value*)
means
((setter *name*) *arg ...* *new-value*)

The new value comes *last* among the setter's arguments. Whatever the setter itself returns, `set!` returns the new value.

```
(define v (vector 'a 'b 'c))
(set! (element v 1) 'sea)       ; => sea
v                               ; => #(a sea c)
((setter element) v 0 'x)       ; => x
v                               ; => #(x sea c)
```

The arguments and the new value are each evaluated once. The compiler to C evaluates the arguments first, left to right, and the new value last, the order of the call `set!` stands for; the Common Lisp implementation evaluates the new value first. Slot accessors (Chapter 7) and `element` (Chapter 9) are the setters used most, and a program can define its own (Section 6.9).

5.4 Truth and if

`#f` is the only false value; every other object is true (Section 4.2.9). A function described as "returning true" may return any object other than `#f`, not necessarily `#t`.

`(if` *test consequent alternate*`)`

`if` evaluates *test*. If its value is true, `if` evaluates *consequent* and returns its values; otherwise it evaluates *alternate* and returns its values. Exactly three operands are required: an `if` without an alternate, or with more than one form in either branch, is an error in both implementations. Use `when` or `unless` for a one-armed test, and `begin` to put several forms in a branch.

```
(if (< -11 0) 'negative 'non-negative)   ; => negative
(if 0 'yes 'no)                          ; => yes
(if '() 'yes 'no)                        ; => yes
(if #f 1 (values 4 5))                   ; => 4, 5
```

`(not` *thing*`)` is a function: it returns `#t` if *thing* is `#f`, and `#f` otherwise.

```
(not #f)                  ; => #t
(not 3)                   ; => #f
```

5.5 Sequencing: begin

`(begin` *form ...*`)` evaluates the forms in order and returns the values of the last one, or `#f` if there are none. Many forms, such as `bind`, `method` and `when`, already contain an implicit `begin` around their bodies; an explicit one is needed where a single form is expected, as in a branch of `if`.

```
(if #t
    (begin (print "it was true")
           #t)
    #f)                   ; prints: "it was true"
                          ; => #t
(begin)                   ; => #f
```

A `begin` at top level makes its forms top-level forms too (Section 4.4).

5.6 Conditionals

5.6.1 when and unless

```
(when test form ...)
(unless test form ...)
```

`when` evaluates *test*; if it is true, it evaluates the forms and returns the values of the last. `unless` evaluates the forms if *test* is false. When the forms are not evaluated, or there are none, the result is `#f`.

```
(when (> 3 2) 'big)      ; => big
(when (> 2 3) 'big)      ; => #f
(unless (> 2 3) 'a 'b)   ; => b
(when #t)                ; => #f
```

5.6.2 and and or

```
(and form ...)
(or form ...)
```

`and` evaluates the forms from left to right until one returns `#f` as its first value; then it returns `#f` and evaluates no more. If none does, it returns all the values of the last form. `or` evaluates the forms from left to right until one returns a true first value, and returns that value alone; if every form but the last is false, it returns all the values of the last. With no forms, `and` returns `#t` and `or` returns `#f`.

```
(and 1 2 3)              ; => 3
(and 1 #f (error "never evaluated"))   ; => #f
(and 1 (values 2 3))     ; => 2, 3
(or #f 'x 'y)            ; => x
(or (values #f 3) 4)     ; => 4
(or #f (values 1 2))     ; => 1, 2
(and)                    ; => #t
(or)                     ; => #f
```

5.6.3 cond

```
(cond (test consequent ...)  ...)
```

`cond` evaluates the tests in order. At the first that is true, it evaluates that clause's consequents and returns the values of the last; later tests are not evaluated. A clause with no consequents returns the value of its test. If no test is true, `cond` returns `#f`. By convention the last clause's test is `else:` or `#t`; both are simply true values.

```
(define-method compare (new old)
  (cond ((< new old) "the new position is less")
        ((= new old) "the positions are equal")
        (else: "the new position is greater")))
(compare 1 2)            ; => "the new position is less"
(compare 3 2)            ; => "the new position is greater"
(cond (#f 'no) ((+ 1 2)))     ; => 3
(cond (#f 1))            ; => #f
```

5.6.4 case

```
(case target (match-list consequent ...)  ...)
```

`case` evaluates *target* and looks for its value in the match lists, comparing with `id?`. The match lists are lists of objects and are not evaluated. At the first clause whose list contains the value, `case` evaluates that clause's consequents and returns the values of the last, or `#f` if it has none. A clause whose match list is `else:` or `#t` matches when no other does; it must be the last clause. If nothing matches and there is no such clause, an error is signaled.

```
(define-method size-name (n)
  (case n
    ((0) 'none)
    ((1 2 3) 'few)
    (else: 'many)))
(size-name 2)            ; => few
(size-name 10)           ; => many
(case 'Red ((red green) 'warm-or-cool) (else: 'other))   ; => warm-or-cool
(case 9 ((3) 'three))    ; => error: no clause matches 9
```

Because the comparison is `id?`, matches are normally numbers, characters, symbols, keywords, `#t`, `#f` or `()`. Strings and lists are not suitable: two strings with the same characters need not be the same object. The same match may not appear in two clauses; the compiler to C reports that when compiling, and the Common Lisp implementation uses the first.

5.6.5 select

```
(select target test (match-list consequent ...)  ...)
```

`select` is like `case`, but the matches are expressions, evaluated, and the comparison is the function *test*, called as `(test target-value match)`. The clauses are tried in order, and within a clause the matches from left to right; the matches are evaluated only as they are reached, and once one matches, no more are evaluated or tested (design note 34). `else:` and `#t` work as in `case`, and no match without them is an error.

```
(define-method kind-of (x)
  (select x instance?
    ((<integer> <ratio>) "exact")
    ((<float>) "inexact")
    (else: "not a number")))
(kind-of 3)              ; => "exact"
(kind-of 2.5)            ; => "inexact"
(kind-of 'x)             ; => "not a number"
(select "b" = (("a") 1) (("b") 2))   ; => 2
(select 5 < ((3 4) 'small) ((6) 'medium) (else: 'large))   ; => medium
```

With a test of `instance?`, as in `kind-of`, `select` does what `typecase` does in Common Lisp.

5.7 Iteration

The loops below are the language's iteration forms. Collections can also be walked with functions such as `do` and `map` (Chapters 9 and 10), and any loop can be written as a recursive method, since tail calls do not use stack (Section 6.11).

5.7.1 for

```
(for ((var init step) ...)
     (end-test result ...)
     body-form ...)
```

`for` is the general loop. Each `(var init step)` is a *variable clause*, and it is the only kind of clause: each must have all three parts. The loop runs like this:

1. The inits are evaluated, in a scope that does not include any of the loop variables, and the variables are bound to their values.
2. The end test is evaluated. If it is true, the results are evaluated in order and `for` returns the values of the last, or `#f` if there are none.
3. Otherwise the body forms are evaluated, for their effects.
4. The steps are evaluated, in a scope that includes the variables' current values, and the variables are bound afresh to the new values. The loop continues at step 2.

The end-test clause may be the empty list `()`, meaning no end test: the loop then runs until something exits from it (Section 5.10). Because all the steps are evaluated before any variable is rebound, the variables are stepped in parallel.

```
(define-method factorial ((n <integer>))
  (for ((i n (- i 1))
        (v 1 (* v i)))
       ((<= i 0) v)))
(factorial 5)            ; => 120
(for ((i 0 (+ i 1))
      (acc '() (cons i acc)))
     ((= i 4) acc))      ; => (3 2 1 0)
(for ((a 0 b) (b 1 (+ a b)) (i 0 (+ i 1)))
     ((= i 10) a))       ; => 55
```

Each pass binds the variables anew; it does not assign them. A method made in the body therefore keeps the values of that pass:

```
(define thunks
  (for ((i 0 (+ i 1))
        (acc '() (cons (method () i) acc)))
       ((= i 3) acc)))
(map (method (f) (f)) thunks)   ; => (2 1 0)
```

5.7.2 for-each

```
(for-each ((var collection) ...)
          (end-test result ...)
          body-form ...)
```

`for-each` iterates over the elements of one or more collections at once. The collection expressions are evaluated first. Then, on each pass, each *var* is bound to the next element of its collection; if any collection has no more elements, the loop ends and returns `#f`. Otherwise the end test

is evaluated, with the variables in scope; if it is true, the results are evaluated and the values of the last returned (`#f` if none). Otherwise the body forms are evaluated and the next pass begins. As with `for`, the end-test clause may be `()`, and each pass binds the variables anew.

```
(define-method first-even ((s <sequence>))
  (for-each ((number s))
            ((even? number) number)))
(first-even '(1 3 5 4 7 9 10))   ; => 4
(first-even '(1 3))              ; => #f
(for-each ((c "abc")
           (i (range from: 0)))
          ()
  (print (list c i)))            ; prints: (#\a 0)
                                 ; prints: (#\b 1)
                                 ; prints: (#\c 2)
                                 ; => #f
```

The second example stops after three passes because the string runs out, although the range `(range from: 0)` is unbounded. Any collection may be used: lists, vectors, strings, ranges, tables (whose elements are its values) and the program's own collection classes. `for-each` uses the iteration protocol of Chapter 9.

5.7.3 dotimes

```
(dotimes (var count [result]) form ...)
```

`dotimes` evaluates *count*, which should be an integer, then evaluates the forms once for each integer from 0 up to but not including the count, with *var* bound to it. If the count is zero or negative, the forms are not evaluated. Then *result* is evaluated and its values returned; without a result, `dotimes` returns `#f`. In both implementations the result form can see *var*, bound to the number of passes made (as in Common Lisp).

```
(dotimes (i 3)
  (print i))             ; prints: 0
                         ; prints: 1
                         ; prints: 2
                         ; => #f
(dotimes (i 4 i))        ; => 4
(dotimes (i -2 'done))   ; => done
```

5.7.4 while and until

```
(while test form ...)
(until test form ...)
```

`while` evaluates *test* before each pass and evaluates the forms as long as it is true. `until` evaluates the forms as long as *test* is false. Both always return `#f`.

```
(define n 0)
(while (< n 3)
  (set! n (+ n 1)))      ; => #f
n                        ; => 3
(until (= n 0)
```

```
  (set! n (- n 1)))      ; => #f
n                        ; => 0
```

5.8 Local methods: bind-methods

```
(bind-methods ((name parameter-list form ...)  ...)
              body-form ...)
```

`bind-methods` binds each *name* to a method, made as `method` makes one (Section 6.2), and evaluates the body forms, returning the values of the last. The scope of the names includes the methods themselves, so the methods can call themselves and each other. (`bind` cannot do that, since an init cannot see its own variable.)

```
(define-method root-mean-square ((s <sequence>))
  (bind-methods ((average (numbers)
                   (/ (reduce1 + numbers) (size numbers)))
                 (square (n) (* n n)))
    (sqrt (average (map square s)))))
(root-mean-square '(5 6 6 7 4))   ; => 5.692099788303083
```

The next example, Newton's method for square roots, has a method that calls itself and two helpers:

```
(define-method newtons-sqrt (x)
  (bind-methods ((sqrt1 (guess)
                   (if (close? guess)
                       guess
                       (sqrt1 (improve guess))))
                 (close? (guess)
                   (< (abs (- (* guess guess) x)) .0001))
                 (improve (guess)
                   (/ (+ guess (/ x guess)) 2)))
    (sqrt1 1)))
(newtons-sqrt 25.0)      ; => 5.000000000053722
```

The 1992 manual calls `(newtons-sqrt 25)`. With an integer argument the arithmetic stays exact, and the guesses become ratios with ever larger numerators and denominators: the Common Lisp implementation returns an enormous ratio, and in the compiler to C, which has no bignums, the integers overflow and an error is signaled. A float argument keeps the computation in floating point.

Later versions of Dylan have a `local` form for this purpose; this language has only `bind-methods`.

5.9 Multiple values

```
(values object ...)
```

An expression may return any number of values. The function `values` returns its arguments as multiple values; `(values)` returns none. `bind` (Section 5.2) receives them.

```
(define-method edges ((center <number>) (radius <number>))
```

```
  (values (- center radius) (+ center radius)))
(edges 100 2)            ; => 98, 102
(define-method opposite-edges ((center <number>) (radius <number>))
  (bind ((min max (edges center radius)))
    (values max min)))
(opposite-edges 100 2)   ; => 102, 98
(bind ((#rest nums (edges 100 2)))
  nums)                  ; => (98 102)
```

Multiple values pass through the forms that return the values of a subform: `begin`, `bind`, `if`, `when`, `unless`, `cond`, `case`, `select`, the last form of `and` and `or` as described above, the results of `for`, `for-each` and `dotimes`, `bind-exit`, `unwind-protect`'s protected form, and calls of methods whose last form returns them. Anywhere else, as an argument to a function, as the init of a one-variable binding, as the value in a `set!` or a `define`, only the first value is used, and an expression that returns no values supplies `#f`.

```
(begin 1 (values 2 3))   ; => 2, 3
(print (list (values 1 2) 3))   ; prints: (1 3)
(bind ((x (values))) x)  ; => #f
(values)                 ; => (no values)
```

5.10 Non-local exits

5.10.1 bind-exit

`(bind-exit (`*exit-variable*`)` *form* `...)`

`bind-exit` binds *exit-variable* to an *exit procedure* and evaluates the forms. If the exit procedure is not called, `bind-exit` returns the values of the last form, or `#f` if there are none. Calling the exit procedure, with any number of arguments, leaves the `bind-exit` at once, and the arguments become its values. This is a *non-local exit*.

```
(bind-exit (return)
  1
  (return 2 3)
  4)                     ; => 2, 3
(bind-exit (return))     ; => #f
```

The exit procedure is an ordinary function: it can be passed to other functions and stored. But it is valid only during the *dynamic extent* of the `bind-exit` that made it, that is, until that `bind-exit` returns or is itself exited. Calling it afterwards is an error.

```
(define-method first-even ((seq <sequence>))
  (bind-exit (exit)
    (do (method (item)
          (when (even? item)
            (exit item)))
        seq)
    #f))
(first-even '(1 3 5 4 7 9 10))   ; => 4
```

In the compiler to C a non-local exit is a jump and costs little; `bind-exit` is the normal way to return early from a loop or a method.

5.10.2 unwind-protect

`(unwind-protect` *protected-form cleanup-form ...*`)`

`unwind-protect` evaluates *protected-form*, then the cleanup forms. The cleanup forms are evaluated however the protected form is left: normally, by a non-local exit through it, or by an error that a handler outside it handles by exiting (Chapter 11). When the protected form returns normally, `unwind-protect` returns its values; the cleanup forms' values are discarded.

```
(unwind-protect (values 1 2)
  (print 'cleanup))      ; prints: cleanup
                         ; => 1, 2
(bind-exit (k)
  (unwind-protect (k 5)
    (print 'cleanup)))   ; prints: cleanup
                         ; => 5
(handler-case
    (unwind-protect (error "boom")
      (print 'cleanup))
  ((<error>) 'caught))   ; prints: cleanup
                         ; => caught
```

A cleanup form may itself exit. If it does, the new exit replaces the one in progress, and the remaining cleanup forms of that `unwind-protect` are skipped:

```
(bind-exit (k)
  (unwind-protect (k 1)
    (k 2)))              ; => 2
(bind-exit (one)
  (bind-exit (two)
    (unwind-protect (two 2)
      (one 1)))
  3)                     ; => 1
```

These are the rules design note 25 later gave for its `block` form, which replaced `bind-exit` and `unwind-protect` in later Dylan; this language keeps the two forms of the 1992 manual.

5.11 quote

`(quote` *object*`)`, usually written `'`*object*, returns *object* without evaluating it (Section 4.2.11). It is a special form of its own, listed here for completeness with the other evaluation forms of *DIRM* chapter 8. `apply`, the last function of that chapter, calls a function on a list of arguments and is described with functions (Section 6.8).

```
'+                       ; => +
(quote (+ 10 10))        ; => (+ 10 10)
```

Chapter 6

Functions and Methods

Functions are Dylan's means of doing anything. This chapter describes the two kinds of function, methods and generic functions; how a method's parameter list says what arguments it takes; how a generic function chooses which of its methods to run; and the operations that call functions and examine them. The standard is *DIRM* chapters 6 and 10.

6.1 Methods and generic functions

Every function is an instance of the class `<function>`, and there are two kinds.

A *method* (class `<method>`) is a piece of code with a parameter list. The parameter list may give a *specializer* for each required parameter, a class or a singleton that the corresponding argument must match. A method can be called directly, like a procedure in Scheme or a closure in Lisp; called with arguments that do not match its specializers, it signals an error.

A *generic function* (class `<generic-function>`) holds a set of methods. When it is called, it finds the methods whose specializers match the arguments, the *applicable* methods, and runs the most specific of them. This choice, *method dispatch*, is how Dylan does object-oriented programming: behavior is attached to classes by defining methods on generic functions, not by putting functions inside classes.

A method made with `method` (Section 6.2) belongs to no generic function, and none is made for it. `define-method` (Section 6.4) is what joins the two: it makes a method and adds it to the generic function held in a module variable, creating that generic function if there is none. The generic function holds its methods, and one method can be in more than one generic function (Section 6.10.1).

```
(define-method double ((thing <number>))
  (+ thing thing))
(define-method double ((thing <sequence>))
  (concatenate thing thing))
(double 10)                    ; => 20
(double "the rain ")           ; => "the rain the rain "
(double '(a b c))              ; => (a b c a b c)
(double 'cup)                  ; => error: no applicable method
(instance? double <generic-function>)          ; => #t
(instance? (method (x) x) <method>)            ; => #t
(instance? double <function>)                  ; => #t
```

Functions are ordinary objects. They can be stored in variables and data structures, passed as arguments and returned as results.

6.2 Making methods: method

```
(method parameter-list form ...)
```

The special form `method` makes and returns a method. When the method is called, its parameters are bound to the arguments as lexical variables and the forms, its *body*, are evaluated in order; the method returns the values of the last form, or `#f` if there are none.

```
((method (a b) (+ a b)) 3 4)             ; => 7
(bind ((square (method (n) (* n n))))
  (square (square 3)))                   ; => 81
(sort '(3 1 2) test: (method (a b) (> a b)))   ; => (3 2 1)
```

A method made by `method` is an *anonymous method*. It can be given to a function, as the test to `sort` above, or kept in a variable. To give a group of local methods names that can call one another, use `bind-methods` (Section 5.8).

6.2.1 Closures

A method can refer to the variables in scope where it was made, and it keeps them for as long as it exists, even after the form that bound them has returned. Such a method is a *closure*. The variables are shared, not copied: assignments made through one closure are seen by every other closure over the same variable.

```
(define-method make-counter ()
  (bind ((count 0))
    (values (method () (set! count (+ count 1)))
            (method () count))))
(define-method demo ()
  (bind ((next current (make-counter)))
    (next)
    (next)
    (list (next) (current))))
(demo)                          ; => (3 3)
```

A method that returns a method is a common idiom. The method below takes a function and returns one that calls it twice:

```
(define-method twice ((f <function>))
  (method (#rest args)
    (apply f args)
    (apply f args)))
(define print-twice (twice (method (n) (print n))))
(print-twice 55)                  ; prints: 55
                                ; prints: 55
                                ; => 55
```

6.3 Parameter lists

The complete syntax of a parameter list (*DIRM* chapter 6) is

```
({required | (required specializer)}*
 [#next next]
 [#rest rest]
 [#key {keyparam | ([keyword] keyparam [default])}*])
```

The parts must appear in this order. `#next`, `#rest` and `#key` may also be written `!next`, `!rest` and `!key` (Section 4.2.12).

Parameter list	Accepts
`(a b c)`	three required arguments
`((a <list>) b #rest c)`	two required (the first a list), then any number
`(#rest a)`	any number of arguments
`(a b #key foo bar)`	two required, then keywords `foo:` and `bar:`
`(#key bar baz bim)`	keywords only
`(#rest all #key fee fi)`	keyword arguments, all kept in `all`

6.3.1 Required parameters

Required parameters come first, and a call must supply an argument for each, in order. A required parameter is a name, or a list of a name and a specializer: `(x <integer>)`. A parameter without a specializer accepts any object, as if specialized on `<object>`. Specializers are described in Section 6.6.

Calling a method with too few or too many arguments, or with an argument that does not match its specializer, is an error:

```
((method ((s <string>)) s) "x")   ; => "x"
((method ((s <string>)) s) 3)     ; => error: 3 does not match <string>
((method (a #rest r) a))          ; => error: too few arguments
```

6.3.2 Rest parameters

After `#rest` comes one name. When the method is called, the arguments left after the required ones are put into a sequence, which becomes the value of the rest parameter; it is empty if there are none.

```
(define-method show-rest (a #rest b)
  (list a b))
(show-rest 10 20 30 40)        ; => (10 (20 30 40))
(show-rest 10)                 ; => (10 ())
```

In both implementations the sequence is a list, but a program should rely only on its being a sequence.

6.3.3 Keyword parameters

After `#key` come the *keyword parameters*. Keyword arguments are optional, are given after the required arguments, and may come in any order; each is a keyword followed by a value. A keyword parameter is written in one of four ways:

Written	Meaning
name	keyword *name*`:`, parameter *name*, default `#f`
(*name default*)	keyword *name*`:`, parameter *name*
(*keyword param*)	the keyword and the parameter named separately
(*keyword param default*)	the same, with a default

The *default* is an expression evaluated each time the method is called without that keyword. It is evaluated in a scope that includes all the parameters before it: the required parameters, the next-method and rest parameters, and the earlier keyword parameters. Without a default, an unsupplied keyword parameter is `#f`.

```
(define-method percolate (#key (brand 'maxwell-house)
                               (cups 4)
                               (strength (if (> cups 6) 'strong 'mild)))
  (list brand cups strength))
(percolate)                          ; => (maxwell-house 4 mild)
(percolate cups: 10 brand: 'folgers) ; => (folgers 10 strong)
```

Naming the keyword and the parameter separately lets a method take a keyword such as `position:` without calling its parameter `position`, which would shadow the module variable of that name, often a generic function the method needs:

```
(define-method place (widget #key (position: the-pos 0) (size: the-size))
  (list widget the-pos the-size))
(place 'w size: 3)                   ; => (w 0 3)
```

All the required arguments are taken before any keyword argument. If a call supplies too few, keywords and values are taken as required arguments:

```
(define-method show-keys (req1 req2 #key foo)
  (list req1 req2 foo))
(show-keys 'one 'two foo: 'three)    ; => (one two three)
(show-keys foo: 'three)              ; => (foo: three #f)
```

The arguments after the required ones must be keyword/value pairs; anything else is an error. If the same keyword is given more than once, the leftmost occurrence is used.

```
(define-method grow ((x <number>) #key (by 1)) (+ x by))
(grow 5 by: 2 by: 100)               ; => 7
(grow 5 'by 2)                       ; => error: not keyword/value pairs
```

6.3.4 Combining #rest and #key

When a parameter list has both, `#rest` comes first. The rest parameter then receives all the keyword arguments, keywords and values, as a sequence, and the method accepts any keyword (Section 6.3.6); the arguments must still be keyword/value pairs.

```
(define-method test (the-req #rest the-rest #key a b)
  (list the-req the-rest a b))
(test 1 a: 2 b: 3 c: 4)              ; => (1 (a: 2 b: 3 c: 4) 2 3)
```

6.3.5 The next-method parameter

After the required parameters may come `#next` and a name. That parameter receives the *next method*, used to call less specific methods (Section 6.7.3). `define-method` gives every method a next-method parameter named `next-method` unless the parameter list names one with `#next`, so it is normally written only to choose another name, or in a method made with `method` that will be added to a generic function.

6.3.6 Keyword argument checking

A method accepts only the keywords its parameter list names after `#key`, unless it has a rest parameter, in which case it accepts any keyword. A generic function accepts, for a given call, every keyword that any of the methods applicable to that call accepts; so if any applicable method has `#rest`, any keyword is accepted. The check is made by the generic function before it runs any method, and a method called through a generic function or through `next-method` does not check again. Calling a method or generic function with a keyword it does not accept is an error.

```
(define-method label ((x <object>) #key price)
  (list price x))
(define-method label ((x <sequence>) #key unit-price)
  (add x (* unit-price (size x))))
(define-method label ((x <list>) #rest info #key calories)
  (add x calories))
(label 'grape price: 189)                       ; => (189 grape)
(label 'grape price: 189 unit-price: 2)         ; => error
(label (vector 3 4 5) price: 189 unit-price: 2) ; => #(6 3 4 5)
(label (vector 3 4 5) calories: 9)              ; => error
(label (list 3 4 5) protein: 7 calories: 9)     ; => (9 3 4 5)
```

The first `label` method is the only one applicable to a symbol, so only `price:` is accepted. A vector is a sequence, so both `price:` and `unit-price:` are. For a list the third method applies, and it has `#rest`, so anything goes.

The applicable methods also decide whether the arguments after the required ones must be keyword/value pairs. They must be if any applicable method has `#key`, even when another has only `#rest`; if none has `#key`, they may be anything. (A generic function declared with `#key` always requires pairs; Section 6.5.)

```
(define-method mx ((a <object>) #rest r) r)
(define-method mx ((a <integer>) #key x) (list x (next-method)))
(mx 'sym 1 2)                   ; => (1 2)
(mx 5 x: 3)                     ; => (3 (x: 3))
(mx 5 1 2)                      ; => error: not keyword/value pairs
```

A method whose parameter list has `#key` with no keywords after it accepts no keywords, as the rule says, though its arguments after the required ones must still be keyword/value pairs; both implementations check this.

6.4 Defining methods: define-method

```
(define-method name parameter-list form ...)
```

`define-method` makes a method, as `method` would, and adds it to the generic function in the module variable *name*. If the variable does not exist, it is created, holding a new generic function. If the generic function already has a method with exactly the same specializers, the new method replaces it. *name* may be a setter name, `(setter symbol)` (Section 6.9). At the interactive prompt `define-method` returns the name.

```
(define-method area ((r <integer>)) (* r r))
(area 3)                        ; => 9
(define-method area ((r <integer>)) (* 3 r r))
(area 3)                        ; => 27
```

That is the Common Lisp implementation, where each definition takes effect when it is evaluated. In the compiler to C all of a program's methods exist before it starts (Section 4.4.2), so the later definition simply wins: both calls of `area` above return 27.
It is an error for *name* to hold something other than a generic function. The compiler to C also refuses to add methods to the functions the manual calls methods rather than generic functions, such as `id?`, `not`, `list` and `apply`, and to the reflective functions it implements in C.

6.4.1 Congruent parameter lists

All the methods of a generic function, and its `define-generic-function` if it has one, must have *congruent* parameter lists: the same number of required parameters, and either all of them have `#rest` or `#key` (or both), or none has either. Adding a method that is not congruent is an error, reported when compiling by the compiler to C.

Nothing else has to agree. Each method has its own specializers, and they are what dispatch chooses among. The parameter names, the keywords named after `#key`, and the next-method parameter may all differ from one method to another, except that a `define-generic-function` that names keys requires every method to accept them (Section 6.5).

Groups of congruent parameter lists
`(a b)  (x y)`
`(a b #key)  (x y #key foo)  (a b #key bar)` `(a b #rest l)  (a b #rest l #key quux)`
`(a b c)  (d e f)`

When `define-method` creates the generic function, the generic function takes the same number of required arguments as the method, and, if the method has `#rest` or `#key`, a rest argument. `function-arguments` (Section 6.10) shows this:

```
(define-method foo (a #key x) x)
(function-arguments foo)        ; => 1, #t, #f
```

A generic function with no required parameters can hold only one method, since all its methods have the same (empty) specializers; defining another replaces the first.

Note: Design note 21 later added result-type declarations: `#values` at the end of a parameter list, with congruency rules for them between a generic function and its methods. Neither implementation supports them; `#values` is a syntax error.

6.5 Declaring generic functions: define-generic-function

```
(define-generic-function name parameter-list keyword option ...)
```

`define-generic-function` creates a generic function with no methods and stores it in the module variable *name*, which is made read-only. It is never required, since `define-method` creates generic functions as needed, but it documents the protocol in one place and fixes the parameter list that every method must be congruent with.

The parameter list has the form

```
(required ...  [#rest rest] [#key keyword ...])
```

Required parameters are names only, without specializers. The keys may be written as symbols or keywords, and `#key` may appear with no keys at all. `#key` means that the arguments after the required ones must be keyword/value pairs; `#rest` means any number of further arguments. Either way, every method must have `#rest` or `#key`. The meaning of giving both is unspecified.

```
(define-generic-function resize (shape #key scale))
(define-method resize ((n <integer>) #key scale)
  (* n (or scale 1)))
(resize 4 scale: 3)            ; => 12
(function-arguments resize)    ; => 1, #f, (scale:)
```

When keys are listed, every method must accept them, by naming them after `#key` or by having `#rest`.

The methods may be defined before or after the declaration. If the variable already holds a generic function that `define-method` made, the declaration replaces the parameter list `define-method` gave it (Section 6.4) and keeps its methods. Either way, every method must be congruent with the declaration and accept its keys, or it is an error. It is also an error if the variable holds something other than a generic function.

```
(define-method tag (x #key color) (list x color))
(define-generic-function tag (x #key color))
(tag 'box color: 'red)         ; => (box red)
(function-arguments tag)       ; => 1, #f, (color:)
(define-method tint (x #key shade) shade)
(define-generic-function tint (x #key hue))
                               ; => error: the method does not accept hue:
```

The options are keyword/value pairs reserved by the manual for future use. Both implementations accept them and ignore them. In the Common Lisp implementation a generic function may be declared again, as when a file is loaded a second time, if the new declaration has the same number of required parameters, the same `#rest` and the same keys; in the compiler to C it may be declared only once.

6.6 Specializers and singletons

A specializer is an expression, evaluated once when the method is made, not on each call. It must evaluate to a class or to a singleton.

- A *class* specializer matches any *general instance* of the class: a direct instance, or an instance of any subclass. The specializer `<number>` matches integers, ratios and floats.
- A *singleton* specializer matches one object, compared with `id?`. `(singleton` *`object`*`)` makes one; so does `(make <singleton> object:` *`object`*`)`.

```
(define-method factorial ((n <integer>))
  (* n (factorial (- n 1))))
(define-method factorial ((n (singleton 0)))
  1)
(factorial 5)                   ; => 120
(define-method measure ((x (singleton 'cup))) 'pint)
(measure 'cup)                  ; => pint
(instance? 3 (singleton 3))     ; => #t
```

`singleton` is a function; `<singleton>` is the class of singletons, and a singleton prints as `{singleton 3}`. Because singletons compare their objects with `id?`, they suit symbols, keywords, characters, small integers, classes and other objects whose identity is what matters; a singleton of a string literal matches only that very string object.

Singletons can also give an individual object slots of its own (Section 7.4).

Important: In the compiler to C every specializer of a method added to a generic function must be known when compiling: a class or a variable whose value is a class, or `(singleton` *`x`*`)` or `(make <singleton> object:` *`x`*`)` where x is a literal, a class, or a variable whose `define` gives it a literal value. `(singleton (+ 1 2))`, and a singleton of an object made at run time, are errors there, though the Common Lisp implementation accepts them.

6.7 Method dispatch

When a generic function is called, it

1. finds the *applicable* methods: those whose every specializer matches the corresponding argument;
2. sorts them from most to least specific;
3. calls the most specific, passing it the next one as its next method.

If no method is applicable, an error is signaled.

6.7.1 Specificity

For one argument, a singleton specializer is more specific than any class, and a class is more specific than its superclasses. So a method on `<integer>` overrides one on `<number>` for integer arguments, and a method on `(singleton 5)` overrides both for the argument 5.

```
(define-method describe ((x <number>)) 'number)
(define-method describe ((x <integer>)) 'integer)
(define-method describe ((x (singleton 5))) 'five)
(describe 2.5)                  ; => number
(describe 2)                    ; => integer
(describe 5)                    ; => five
```

Methods that specialize more than one parameter are *multimethods*, and dispatch considers all the specialized arguments. Following *DIRM* chapter 10, both implementations sort the applicable methods by comparing their specializer lists *lexicographically*: the first parameter decides, and a later parameter is looked at only when the earlier specializers are the same.

```
(define-method meet ((a <number>) (b <integer>)) 'number-integer)
(define-method meet ((a <integer>) (b <number>)) 'integer-number)
(meet 1 2)                      ; => integer-number
```

Both methods apply to `(meet 1 2)`. The first parameter's specializers differ, and `<integer>` is more specific than `<number>`, so the second method wins, though the first is more specific in the second parameter.

6.7.2 The class precedence list

When a class has several direct superclasses, two specializers can both be superclasses of an argument's class without either being a subclass of the other. They are then ordered by the argument's class's *class precedence list*: the class, then all its superclasses, in an order that keeps each class before its superclasses, keeps the direct superclasses in the order the class's definition lists them, and keeps each simple chain of superclasses together. `all-superclasses` returns this list (Section 7.5). The algorithm is the one CLOS uses, given in design note 31.

```
(define-class <life-form> (<object>))
(define-class <sentient> (<life-form>))
(define-class <bipedal> (<life-form>))
(define-class <intelligent> (<sentient>))
(define-class <humanoid> (<bipedal>))
(define-class <vulcan> (<intelligent> <humanoid>))
(define-class <human> (<humanoid> <intelligent>))
(all-superclasses <vulcan>)
  ; => ({the class <vulcan>} {the class <intelligent>}
  ;     {the class <sentient>} {the class <humanoid>}
  ;     {the class <bipedal>} {the class <life-form>}
  ;     {the class <object>})
(define-method psychoanalyze ((being <intelligent>)) 'intelligent)
(define-method psychoanalyze ((being <humanoid>)) 'humanoid)
(psychoanalyze (make <vulcan>)) ; => intelligent
(psychoanalyze (make <human>))  ; => humanoid
```

A `<vulcan>` lists `<intelligent>` first among its superclasses, so the `<intelligent>` method is more specific for it; a `<human>` lists `<humanoid>` first. With multimethods the same rule is applied lexicographically:

```
(define-method superior ((a <intelligent>) (b <intelligent>)) 'smarter)
(define-method superior ((a <humanoid>) (b <humanoid>)) 'better-looking)
(superior (make <vulcan>) (make <human>))   ; => smarter
(superior (make <human>) (make <vulcan>))   ; => better-looking
```

Note: Design note 31 later replaced the lexicographic rule with one that compares every argument position and calls the methods *ambiguous*, an error, when the positions disagree, as they do in the last two calls above. Neither implementation follows that note: both use the 1992 rule, under which a call whose applicable methods all have class or singleton specializers is never ambiguous.

6.7.3 Calling the next method

A method often wants to add to the behavior it overrides rather than replace it. Its next-method parameter, normally named `next-method`, holds a function that calls the next most specific applicable method. Called with no arguments, it passes along the arguments the current method received; called with arguments, it passes those instead.

```
(define-method greet ((x <object>)) (list 'object x))
(define-method greet ((x <integer>)) (cons 'integer (next-method)))
(define-method greet ((x (singleton 5))) (cons 'five (next-method 7)))
(greet "hi")                    ; => (object "hi")
(greet 3)                       ; => (integer object 3)
(greet 5)                       ; => (five integer object 7)
```

When new arguments are given, they must lead to the same applicable methods, in the same order, as the original ones; otherwise the result is undefined. (In `greet` above, 7 is still an integer and an object.) Keyword arguments passed to `next-method` with changed values are not checked.

When there is no next method, the parameter is `#f`, so a method can test it before calling it; calling `#f` is an error.

```
(define-method last-one ((x <object>)) next-method)
(last-one 1)                    ; => #f
(define-method chain ((x <number>) #next more) (list 'number more))
(define-method chain ((x <integer>) #next more) (list 'integer (more)))
(chain 1)                       ; => (integer (number #f))
```

When methods take different keyword arguments, giving each a rest parameter lets a method pass all of them on, changed or not, with `apply` (Section 6.8):

```
(define-method grow ((x <number>) #rest options #key (by 1))
  (+ x by))
(define-method grow ((x <integer>) #rest options #key by)
  (apply next-method x by: 10 options))
(grow 5)                        ; => 15
(grow 5 by: 2)                  ; => 15
```

The second call shows the leftmost-keyword rule at work: the `by: 10` the integer method puts first wins over the caller's `by: 2`.

6.8 Calling functions: apply

```
(apply function argument ... sequence)
```

`apply` calls *function* with the given arguments followed by the elements of *sequence*, the last argument, and returns its values.

```
(apply + 1 '(2 3))              ; => 6
(apply max (vector 3 9 2))      ; => 9
(define math-functions (list + * -))
(apply (first math-functions) 1 2 '(3 4))   ; => 10
(apply list '())                ; => ()
```

6.9 Setters as generic functions

A setter is an ordinary function, normally a generic function, stored in a setter variable `(setter` *`name`*`)`. Its arguments are those of the getter followed by the new value. `define-class` defines setter methods for slots, and a program can define its own with `define-method`, so that `set!` works on a computed property as it does on a slot:

```
(define-class <temperature> (<object>)
  (kelvin init-value: 0 init-keyword: kelvin:))
(define-method celsius ((t <temperature>))
  (- (kelvin t) 273))
(define-method (setter celsius) ((t <temperature>) c)
  (set! (kelvin t) (+ c 273)))
(define t1 (make <temperature>))
(set! (celsius t1) 25)          ; => 25
(kelvin t1)                     ; => 298
(celsius t1)                    ; => 25
```

Section 5.3.1 describes how `set!` calls a setter. Later Dylan changed both the name, to `celsius-setter`, and the argument order, putting the new value first; neither change is part of this language.

6.10 Reflective operations

These functions examine functions and generic functions (*DIRM* chapter 10). Ordinary programs seldom need them, but they are useful in tools and for understanding dispatch.

`(function-arguments` *`function`*`)` returns three values: the number of required arguments, whether the function takes a rest argument, and a sequence of the keywords it accepts, or `#f` if it has no `#key`. The sequence is empty for `#key` with no keywords.

`(applicable-method?` *`function argument ...`*`)` returns true if *function* is a method applicable to the arguments, or a generic function with such a method.

`(sorted-applicable-methods` *`generic-function argument ...`*`)` returns a sequence of the methods applicable to the arguments, most specific first.

`(find-method` *`generic-function specializers`*`)` returns the method whose specializers are exactly the classes and singletons in the sequence *specializers*, or `#f`.

`(generic-function-methods` *`generic-function`*`)` returns a sequence of all its methods, in no particular order.

`(method-specializers` *`method`*`)` returns a sequence of a method's specializers, one for each required parameter.

None of the sequences these functions return should be modified.

```
(function-arguments (method (a b #rest r) a))     ; => 2, #t, #f
(function-arguments (method (a #key x (y: z 3)) a)) ; => 1, #f, (x: y:)
(define-method kind ((x <number>)) 'number)
(define-method kind ((x <integer>)) 'integer)
(define-method kind ((x (singleton 5))) 'five)
(applicable-method? kind 1/2)                     ; => #t
(applicable-method? kind "x")                     ; => #f
(map (method (m) (m 5))
     (sorted-applicable-methods kind 5))          ; => (five integer number)
((find-method kind (list <integer>)) 7)           ; => integer
(find-method kind (list <string>))                ; => #f
(size (generic-function-methods kind))            ; => 3
(method-specializers (find-method kind (list <integer>)))
                                                  ; => ({the class <integer>})
```

6.10.1 Changing generic functions

The 1992 manual also has functions that change a generic function while the program runs. `add-method`, `remove-method` and `freeze-methods` exist only in the Common Lisp implementation; in the compiler to C, whose generic functions are fixed when the program is compiled, using them is an error when compiling. `make-read-only` exists in both, but in the compiler to C it changes nothing, since no generic function can change there anyway; it returns its argument.

(add-method *generic-function method*) adds the method, replacing any with the same specializers, and returns two values: the method, and the method it replaced or `#f`.

(remove-method *generic-function method*) removes the method and returns it.

(make-read-only *generic-function*) forbids adding or removing methods from then on.

(freeze-methods *generic-function specializer ...*) forbids adding, replacing or removing methods whose specializers are the given ones or more specific.

```
(define-method double ((x <number>)) (+ x x))
(define m (method ((s <string>)) (concatenate s s)))
(add-method double m)          ; => {a method on double}, #f
(double "ab")                  ; => "abab"
(remove-method double m)       ; => {an anonymous method}
(double "ab")                  ; => error: no applicable method
```

One method can be added to several generic functions. Each holds it independently, so removing it from one leaves it in the others:

```
(define dbl (method ((n <integer>)) (* n 2)))
(define-generic-function scale (x))
(define-generic-function enlarge (x))
(add-method scale dbl)
(add-method enlarge dbl)
(scale 3)                      ; => 6
(enlarge 4)                    ; => 8
(remove-method scale dbl)
```

```
(enlarge 5)                     ; => 10
(scale 5)                       ; => error: no applicable method
```

A generic function can also be made with `make`, with the init-keywords `required:` (the number of required arguments, which must be given), `rest?:`, `key?:` and `debug-name:`. It starts with no methods. In the compiler to C such a generic function can never have any, since methods cannot be added at run time, so it is of little use there; that implementation also ignores `rest?:` and `key?:`.

```
(define expand
  (make <generic-function> required: 1 debug-name: 'expand))
(add-method expand (method ((n <integer>)) (* n 10)))
(expand 5)                      ; => 50
```

6.11 Proper tail calls

A call is in *tail position* when its value is the value of the enclosing method: the last form of a method's body, either branch of an `if` in tail position, the last form of a `begin` or `bind` in tail position, and so on. The compiler to C guarantees *proper tail calls*: every call in tail position, to a known method, a generic function, a closure or a function in a variable, runs in constant stack space. A recursive method whose recursive calls are all tail calls is therefore a loop, and two methods can call each other in tail position indefinitely. The `for`, `while` and other loops of Chapter 5 are themselves compiled this way.

```
(define-method my-even? ((n <integer>))
  (if (= n 0) #t (my-odd? (- n 1))))
(define-method my-odd? ((n <integer>))
  (if (= n 0) #f (my-even? (- n 1))))
(my-even? 10000000)             ; => #t
```

The Common Lisp implementation compiles Dylan to Common Lisp, and SBCL eliminates tail calls in that code too, so the example above runs there as well; but the Common Lisp standard does not require it, and the Common Lisp implementation makes no guarantee. Chapter 15 describes how the compiler to C implements tail calls, with the C compiler's `musttail` or with trampolines.

Chapter 7

Classes and Objects

Every Dylan object is a *direct instance* of exactly one class, its class, and an *indirect instance* of all that class's superclasses; a *general instance* of a class is a direct or an indirect instance. Classes describe the structure of their instances, the *slots* that hold an object's state, and together with generic functions they determine the instances' behavior. Classes are themselves objects, instances of `<class>`, and can be passed around and examined like any other.

This chapter describes defining classes and their slots, making and initializing instances, the functions that examine classes, singletons and sealing, and the classes built into the language. The standard is *DIRM* chapters 2, 6 and 11. Dispatch on classes is described in Chapter 6.

7.1 Defining classes: define-class

```
(define-class name (superclass ...)  slot-spec ...)
```

`define-class` creates a class and stores it in the module variable *name*, which is made read-only. The superclass expressions are evaluated and must give at least one class, with no class twice; the new class inherits from all of them. Each *slot-spec* adds a slot (Section 7.2). At the interactive prompt `define-class` returns the name.

```
(define-class <point> (<object>)
  (x init-value: 0 init-keyword: x:)
  (y init-value: 0 init-keyword: y:))
(define p (make <point> x: 3))
(x p)                           ; => 3
(y p)                           ; => 0
(set! (y p) 4)                  ; => 4
(instance? p <point>)           ; => #t
```

By convention class names begin and end with angle brackets. A class that needs no other superclass names `<object>`, the root of the class hierarchy.

In the Common Lisp implementation, defining a class whose name already holds a class modifies that class in place, as the 1992 manual says, so that everything referring to it sees the new definition. In the compiler to C a class may be defined only once.

7.1.1 Multiple inheritance

A class may have several direct superclasses. It inherits the slots of all of them, and methods specialized on any of them apply to its instances. The order of the superclasses matters: it

determines the class's *class precedence list*, which orders the class and all its superclasses from most to least specific, and so decides which method runs when methods on two unrelated superclasses both apply (Section 6.7.2).

```
(define-class <named> (<object>)
  (name init-keyword: name: init-value: "anonymous"))
(define-class <located> (<object>)
  (place init-keyword: place: init-value: 'nowhere))
(define-class <landmark> (<named> <located>))
(define l (make <landmark> name: "Big Ben" place: 'london))
(list (name l) (place l))      ; => ("Big Ben" london)
(all-superclasses <landmark>)
  ; => ({the class <landmark>} {the class <named>}
  ;     {the class <located>} {the class <object>})
```

A class cannot be its own superclass, directly or indirectly. If the superclasses' orderings cannot all be satisfied at once, no class precedence list exists and defining the class is an error.

7.2 Slots

A slot is a place in an object that holds a value. A slot is not read by name; it is read and written by *getter* and *setter* generic functions, to which `define-class` adds methods. A slot named `x` gets a getter method on the generic function `x` and a setter method on `(setter x)`, and `(set! (x p) 10)` calls the setter. Since accessors are generic functions, a slot looks to the rest of the program exactly like a computed property, and a subclass can replace a stored slot with a computed one without changing any caller.

7.2.1 Slot specifications

A slot specification takes one of three forms:

slot-name
`(`*`slot-name keyword value ...`*`)`
`(`*`keyword value ...`*`)`

The slot name is a symbol and is not evaluated. It supplies the default getter name, *slot-name*, and the default setter name, `(setter `*`slot-name`*`)`. In the third form there is no slot name, and `getter:` must be given. Each option may be given at most once, and two slots of one `define-class` may not have the same getter: `(define-class <a> (<object>) (x) (x))` is an error, `define-class slots: x appears twice`.

`getter:` ***name*** The variable to which the getter method is added. Not evaluated.

`setter:` ***name*** The variable to which the setter method is added. Not evaluated. `setter: #f` gives the slot no setter.

`type:` ***type*** A class or singleton that the slot's values must be instances of; storing anything else is an error. Evaluated once. The default is `<object>`.

`init-value:` ***value*** The slot's initial value, used for every new instance. Evaluated once, when the class is defined.

init-function: ***function*** A function of no arguments, called to compute the initial value for each new instance. Evaluated once, to give the function. Cannot be combined with `init-value:`.

init-keyword: ***keyword*** A keyword with which `make` may be given the initial value. Not evaluated.

required-init-keyword: ***keyword*** Like `init-keyword:`, but `make` must be given it. Cannot be combined with `init-keyword:`, `init-value:` or `init-function:`.

allocation: ***kind*** Where the slot's value is kept: `instance`, `class`, `each-subclass`, `constant` or `virtual` (Section 7.2.3). Not evaluated. The default is `instance`.

```
(define-class <rectangle> (<object>)
  (top type: <integer> init-value: 0 init-keyword: top:)
  (left type: <integer> init-value: 0 init-keyword: left:)
  (bottom type: <integer> init-value: 100 init-keyword: bottom:)
  (right type: <integer> init-value: 100 init-keyword: right:))
(define r (make <rectangle> top: 50 left: 50))
(top r)                         ; => 50
(bottom r)                      ; => 100
(set! (bottom r) 55)            ; => 55
(set! (bottom r) 'foo)          ; => error: foo is not an instance of <integer>
```

The getter and setter options let a slot have accessor names that differ from its slot name, or no setter at all:

```
(define-class <account> (<object>)
  (number required-init-keyword: number: setter: #f)
  (balance init-value: 0
           getter: current-balance setter: (setter current-balance))
  (getter: owner-name init-keyword: owner:))
(define a (make <account> number: 1001 owner: "Ann"))
(number a)                      ; => 1001
(current-balance a)             ; => 0
(owner-name a)                  ; => "Ann"
(make <account>)                ; => error: number: is required
```

7.2.2 Initial values

When `make` creates an instance, each slot is initialized from the first of these that applies: the init-keyword's argument to `make`, if given; the init-value; the result of calling the init-function. The init-function is not called when the keyword is given. A slot with none of these is left *uninitialized*, and reading it is an error until it has been set. The initial value is stored directly into the slot; the setter generic function is not called.

An init-value is evaluated once, when the class is defined, and the same object becomes the initial value of every instance. To give each instance its own fresh object, such as an empty collection, use an init-function:

```
(define-class <bag> (<object>)
  (shared init-value: (make <stretchy-vector>))
  (own init-function: (method () (make <stretchy-vector>))))
```

```
(define b1 (make <bag>))
(define b2 (make <bag>))
(id? (shared b1) (shared b2))  ; => #t
(id? (own b1) (own b2))        ; => #f
```

`(slot-initialized?` *`instance getter`*`)` tells whether the slot that *getter* reads in *instance* has a value. It is an error if *getter* reads no slot of the instance. There is no way to make a slot uninitialized again.

```
(define-class <memo> (<object>) (note))
(define m (make <memo>))
(slot-initialized? m note)     ; => #f
(note m)                       ; => error: the slot note is not initialized
(set! (note m) "remember")     ; => "remember"
(slot-initialized? m note)     ; => #t
```

7.2.3 Allocation

`instance` Each instance has its own value. This is the default.

`class` There is one value, shared by all the general instances of the class, its subclasses' instances included. Setting it through one instance changes it for all.

`each-subclass` The class has one value, shared by its direct instances, and each subclass has a value of its own, shared by the subclass's direct instances.

`constant` The value is the init-value, which must be given, and it cannot be changed: the slot has no setter.

`virtual` No storage is allocated. The program defines methods on the getter, and on the setter if it wants one, that compute the value; `define-class` only makes sure the generic functions exist.

A `class` or `each-subclass` slot can be initialized only with `init-value:`; without one it starts uninitialized, like any other slot, until it is set; a `virtual` slot takes no initialization options at all, and `slot-initialized?` is always true of it.

```
(define-class <vehicle> (<object>)
  (count allocation: class init-value: 0)
  (kind allocation: each-subclass init-value: 'vehicle)
  (wheels allocation: constant init-value: 4)
  (description allocation: virtual))
(define-class <truck> (<vehicle>))
(define-method description ((v <vehicle>))
  (list (kind v) (wheels v)))
(define bike (make <vehicle>))
(define truck (make <truck>))
(set! (count bike) 2)          ; => 2
(count truck)                  ; => 2
(set! (kind truck) 'truck)     ; => truck
(kind bike)                    ; => vehicle
(kind (make <truck>))          ; => truck
(description truck)            ; => (truck 4)
(slot-initialized? bike description)   ; => #t
```

A virtual slot can also give an inherited stored slot a new behavior in a subclass, a *filtered slot*: the subclass declares the slot virtual for documentation and defines getter and setter methods that call `next-method` to reach the stored value.

```
(define-class <view> (<object>)
  (position init-value: 0 init-keyword: position:))
(define-class <displaced-view> (<view>)
  (offset init-value: 10))
(define-method position ((v <displaced-view>))
  (+ (next-method) (offset v)))
(define-method (setter position) ((v <displaced-view>) new)
  (next-method v (- new (offset v))))
(define dv (make <displaced-view> position: 5))
(position dv)                   ; => 15
(set! (position dv) 100)        ; => 100
(position dv)                   ; => 100
```

7.2.4 Inherited slots

Two slots are the same slot if they have the same getter. A subclass may respecify a slot it inherits, to give it, for example, a different initial value. The 1992 manual asks that the new specification repeat every feature the inherited one specified (its type, an initial value, its init-keyword, a non-instance allocation and a setter), with a type no wider than the inherited one, and both implementations report a respecification that leaves one out ("redefining an inherited slot must respecify its init-keyword:"). The exception is a slot redeclared `virtual`, a filtered slot (below), which respecifies nothing: the inherited slot keeps its storage, its type and its initialization, so `make` still takes its init-keyword. A class that inherits the same slot from two superclasses with different specifications must respecify it.

```
(define-class <base> (<object>)
  (color init-value: 'red init-keyword: color:))
(define-class <derived> (<base>)
  (color init-value: 'blue init-keyword: color:))
(color (make <base>))                 ; => red
(color (make <derived>))              ; => blue
(color (make <derived> color: 'green)) ; => green
```

7.3 Making instances: make and initialize

`(make` *`class keyword value ...`*`)`

`make` returns a general instance of *class*. For a class the program defines, it allocates a new instance, initializes its slots (Section 7.2.2), then calls the generic function `initialize` with the instance and all the keyword arguments, and returns the instance; what `initialize` returns is ignored. Making an instance of an abstract class, such as `<number>` or `<collection>`, is an error. Built-in instantiable classes have `make` methods of their own, described with them (Chapter 9). Those methods are for the classes themselves: an instance of a program's subclass of `<table>`, `<deque>`, `<stretchy-vector>` or `<range>` is made by the default `make`, which takes only the init-keywords of the subclass's own slots. The new instance starts empty; a range starts at 0, by 1, without end.

The keywords `make` accepts are the init-keywords of the class's slots, inherited slots included, and the keywords accepted by the applicable `initialize` methods. An `initialize` method with `#rest` accepts any keyword. Any other keyword is an error, as the manual specifies: `keyword argument not accepted: y:`.

Programs customize instance creation by adding methods to `initialize`, not to `make`. An `initialize` method normally calls `next-method` so that the superclasses' methods run too; the method on `<object>` does nothing. When `initialize` runs, the slots already have their initial values.

```
(define-class <shape> (<object>)
  (name init-keyword: name: init-value: "shape")
  (area))
(define-class <square> (<shape>)
  (side init-keyword: side: init-value: 1))
(define-method initialize ((s <shape>) #rest options)
  (next-method)
  (print (list 'initializing (name s))))
(define-method initialize ((s <square>) #rest options)
  (next-method)
  (set! (area s) (* (side s) (side s))))
(define sq (make <square> side: 3 name: "sq"))
                              ; prints: (initializing "sq")
(area sq)                     ; => 9
```

The `<square>` method above takes `#rest` rather than `#key side`: a parameter named `side` would shadow the getter `side` within the method. To receive a keyword under another name, write `#key (side: the-side)` (Section 6.3.3).

7.4 Adding slots: define-slot

```
(define-slot owner [slot-name] keyword value ...)
```

`define-slot` adds a slot to an existing class or to a single object. *owner* is evaluated and must be a class or a singleton; the slot name and options are those of a slot specification in `define-class`. If the owner is a class, all its instances get the slot. If it is a singleton, only that one object does, and its allocation must be `instance`, `constant` or `virtual`, without `init-keyword:` or `required-init-keyword:`.

```
(define-class <pt> (<object>) (px init-value: 0))
(define p (make <pt>))
(define-slot (singleton p) note init-value: "only me")
(note p)                      ; => "only me"
(note (make <pt>))            ; => error: no applicable method
(define x (list 1 2))
(define-slot (singleton x) tag init-value: 'special)
(set! (tag x) 'changed)       ; => changed
(tag x)                       ; => changed
(define-slot (singleton 'cup) volume allocation: constant init-value: 8)
(volume 'cup)                 ; => 8
(define-slot <pt> weight init-value: 10)
(weight p)                    ; => 10
```

As the examples show, the object may be of any class, built-in or not, and the slot's accessors are generic functions with a method specialized on the singleton.

In the Common Lisp implementation `define-slot` takes effect when it is evaluated, and instances that already exist gain the slot. In the compiler to C the slot is part of the program from the start: a slot defined on a class is in every instance ever made, and is listed by `slot-descriptors` even before the `define-slot` form is reached. The compiler to C also requires a class owner to be a class the program itself defines, known when compiling, rather than a built-in class or a value computed at run time; a singleton owner may be any object.

7.5 Examining classes and objects

`(object-class` *`object`*`)` returns the class of which *object* is a direct instance.

`(instance?` *`object type`*`)` returns true if *object* is a general instance of the class *type*, or is the object of the singleton *type*.

`(subclass?` *`class1 class2`*`)` returns true if *class1* is *class2* or a direct or indirect subclass of it.

`(all-superclasses` *`class`*`)` returns the class precedence list: *class* first, `<object>` last.

`(direct-superclasses` *`class`*`)` returns the superclasses named in the class's definition, in order.

`(direct-subclasses` *`class`*`)` returns the classes that name *class* as a direct superclass.

```
(define-class <animal> (<object>))
(define-class <dog> (<animal>))
(define rex (make <dog>))
(object-class rex)              ; => {the class <dog>}
(instance? rex <animal>)        ; => #t
(instance? 3 <real>)            ; => #t
(instance? 3 (singleton 3))     ; => #t
(subclass? <dog> <animal>)      ; => #t
(subclass? <dog> <dog>)         ; => #t
(direct-superclasses <dog>)     ; => ({the class <animal>})
(direct-subclasses <animal>)    ; => ({the class <dog>})
(all-superclasses <dog>)
  ; => ({the class <dog>} {the class <animal>} {the class <object>})
```

None of the sequences these functions return should be modified. `#t` and `#f` are instances of an internal class that prints as `{the class <boolean>}` but is not the value of any variable a program can name; the manual names no class for them. The built-in classes that cannot be subclassed, such as `<integer>` and `<pair>`, are sealed, so `all-superclasses` and `direct-superclasses` of them return empty sequences (Section 7.7). `<object>` is abstract: `(make <object>)` is an error.

7.5.1 Slot descriptors

A *slot descriptor*, of class `<slot-descriptor>`, describes one slot. Programs seldom need them, but debuggers and inspectors do. Both implementations provide the slot-descriptor functions of *DIRM* chapter 11. (Design note 9 later removed them from the language.)

(slot-descriptors *class*) all the slots of the class, inherited ones included: those of each direct superclass in turn, each slot where it first appears, and then the class's own. A slot redefined more specifically keeps its place. The built-in classes' own slots, which no program can name, are not among them.

(slot-descriptor *instance getter*) the slot that *getter* reads in *instance*, or `#f`.

(slot-getter *sd*), (slot-setter *sd*) the getter and setter generic functions; `slot-setter` returns `#f` for a slot without a setter.

(slot-type *sd*) the type, `<object>` if none was given.

(slot-allocation *sd*) the allocation, a symbol.

(init-value *sd*) two values: the init-value and `#t`, or `#f` and `#f` if there is none.

(init-function *sd*), (init-keyword *sd*) the init-function or init-keyword, or `#f`.

(slot-value *instance sd*) the value of the slot in the instance. Its setter, used as `(set! (slot-value *instance sd*) *value*)`, sets it, checking the slot's type. Neither works on a virtual slot.

```
(define-class <cell> (<object>)
  (contents type: <integer> init-value: 1 init-keyword: contents:))
(define c (make <cell> contents: 3))
(size (slot-descriptors <cell>))       ; => 1
(define sd (slot-descriptor c contents))
(id? (slot-getter sd) contents)        ; => #t
(id? (slot-setter sd) (setter contents)) ; => #t
(slot-type sd)                         ; => {the class <integer>}
(slot-allocation sd)                   ; => instance
(init-value sd)                        ; => 1, #t
(init-keyword sd)                      ; => contents:
(init-function sd)                     ; => #f
(slot-value c sd)                      ; => 3
(set! (slot-value c sd) 4)             ; => 4
(contents c)                           ; => 4
(slot-descriptor c object-class)       ; => #f
```

7.5.2 Copying and converting

`(shallow-copy *object*)` returns a new object with the same contents, made with `make` on `(class-for-copy *object*)`, which by default is the object's class. The library has methods for collections. For other objects the 1992 manual leaves the method to the programmer, and that is so in the Common Lisp implementation; the compiler to C has a default method that copies any instance's slots.

`(as *class object*)` converts *object* to *class*, returning it unchanged if it is already an instance. Its methods for numbers, characters, symbols and collections are described in Chapters 8 and 9.

7.6 Singletons

A singleton is a specializer for one object (Section 6.6). `(singleton` *`object`*`)` and `(make <singleton> object:` *`object`*`)` both make one. With methods on singletons a program can give an individual object behavior of its own, and with `define-slot` (Section 7.4) state of its own, without defining a class for it. A method on a singleton is more specific than any method on the object's classes.

```
(define-class <window> (<object>))
(define main-window (make <window>))
(define-method title ((w <window>)) "untitled")
(define-method title ((w (singleton main-window))) "Main")
(title (make <window>))        ; => "untitled"
(title main-window)            ; => "Main"
```

The compiler to C does not accept that program: there, a singleton used as a method's specializer must be of a literal, a class, or a variable whose definition gives it a literal value, because the specializer must be known when compiling (Section 6.6). A singleton of an instance made at run time can still own slots, through `define-slot`, and singletons of symbols, keywords, numbers and characters work everywhere:

```
(define-method plural ((word <symbol>))
  (as <symbol> (concatenate (as <string> word) "s")))
(define-method plural ((word (singleton 'mouse))) 'mice)
(plural 'cat)                  ; => cats
(plural 'mouse)                ; => mice
(make <singleton> object: 4)   ; => {singleton 4}
```

7.7 Sealing

`(seal` *`class`*`)` seals a class: no subclass of it can be created any more, and `all-superclasses`, `direct-superclasses` and `slot-descriptors` return empty sequences for it. Sealing changes nothing about existing instances or methods. `(make-read-only` *`class`*`)` forbids changing the class's slots and superclasses without forbidding subclasses. Both return the class.

```
(define-class <a> (<object>) (a-slot init-value: 1))
(define-class <b> (<a>))
(seal <b>)                     ; => {the class <b>}
(all-superclasses <b>)         ; => ()
(direct-superclasses <b>)      ; => ()
(a-slot (make <b>))            ; => 1
(instance? (make <b>) <a>)     ; => #t
```

In the Common Lisp implementation, defining a subclass of a sealed class is then an error. In the compiler to C all of a program's classes are created when it is compiled, before `seal` runs, so `seal` affects only what the reflective functions return afterwards; and there `make-read-only` on a class does exactly what `seal` does. A built-in class that cannot be subclassed, such as `<integer>` or `<list>`, is reported when compiling, if a program tries.

7.8 Changing classes at run time

The 1992 manual lets a running program change its classes: `add-slot` and `remove-slot` add and remove slots, returning slot descriptors; `(set! (direct-superclasses` *`class`*`) `*`list`*`)` gives a class new superclasses; and `(make <class> superclasses:` *`classes`* `debug-name:` *`name`*`)` creates a new class. Existing instances are updated to the new definition before they are next used.

All of this works in the Common Lisp implementation. A program for the compiler to C is static: its classes and their slots are fixed when it is compiled. There `add-slot` and `remove-slot` are errors when compiling, `make` of `<class>` is an error when run, and `define-slot` on a class, which the compiler handles as part of the class's definition, is the way to add a slot (Section 7.4).

```
(define-class <pt> (<object>) (px init-value: 0))
(define p (make <pt>))
(define-generic-function weight (x))
(define-generic-function (setter weight) (x v))
(define sd (add-slot <pt> getter: weight setter: (setter weight)
                          init-value: 3))
(weight p)                      ; => 3
(remove-slot <pt> sd)
(weight p)                      ; => error: no applicable method
(define c (make <class> superclasses: (list <object>) debug-name: '<dyn>))
(instance? (make c) c)          ; => #t
```

7.9 The built-in classes

The classes below are those of the 1992 manual, as the compiler to C arranges them. Each class is shown under its first direct superclass; a class with more than one is marked with the others. Where the Common Lisp implementation differs, a note says so.

```
<object>
  <boolean>                          (internal)
  <number>
    <complex>
      <real>
        <rational>
          <integer>
          <ratio>
        <float>
          <single-float>
          <double-float>
          <extended-float>
      <rectangular-complex>          (Common Lisp only)
  <collection>
    <explicit-key-collection>
      <mutable-explicit-key-collection>   (also <mutable-collection>)
        <array>
          <vector>                   (also <mutable-sequence>)
            <simple-object-vector>
            <stretchy-vector>
            <byte-string>            (also <string>)
            <unicode-string>         (also <string>)
```

```
      <table>
    <mutable-collection>
      <mutable-sequence>            (also <sequence>)
        <string>
        <list>
          <pair>
          <empty-list>
        <deque>
    <sequence>
      <range>
  <function>
    <generic-function>
    <method>
  <class>
  <singleton>
  <slot-descriptor>
  <symbol>
  <keyword>
  <character>
  <condition>
    <serious-condition>
      <error>
        <simple-error>
        <type-error>
        <thread-error>              (compiler to C only)
    <warning>
      <simple-warning>
    <restart>
      <simple-restart>
      <abort>
```

Both implementations add the stream class `<stream>`, a direct subclass of `<object>` with the subclass `<string-stream>` (Chapter 13). The compiler to C has further classes for its threads (Chapter 16), also direct subclasses of `<object>`: `<thread>`, `<lock>`, `<condition-variable>` and `<atomic-integer>`. `<lock>` has the subclass `<recursive-lock>`.

Numbers are described in Chapter 8, collections in Chapter 9 and conditions in Chapter 11.

7.9.1 Which classes can be subclassed

A program may define subclasses of `<object>` and of the abstract classes meant to be extended: `<number>`, the collection classes `<collection>`, `<sequence>`, `<mutable-sequence>`, `<explicit-key-collection>` and the others above them, the instantiable collection classes `<array>`, `<vector>`, `<string>`, `<table>`, `<deque>`, `<stretchy-vector>` and `<range>`, and `<function>`, `<condition>`, `<error>` and the other condition classes. These classes are *sealed*, and subclassing them is an error in both implementations: the number classes below `<number>`, from `<complex>` down; the list classes `<list>`, `<pair>` and `<empty-list>`; and the concrete vector and string classes `<simple-object-vector>`, `<byte-string>` and `<unicode-string>`.

The compiler to C also refuses, when compiling, to use as a superclass the classes whose instances its runtime makes itself: `<method>`, `<generic-function>`, `<class>`, `<singleton>`, `<symbol>`, `<keyword>` and `<character>`. The Common Lisp implementation accepts these.

Chapter 8

Numbers, Characters and Symbols

This chapter describes Dylan's simplest objects: numbers, characters, symbols and keywords. It follows chapters 14, 15 and 16 of the 1992 manual (*DIRM* chapters 14–16) and, for comparison, chapter 9 (*DIRM* chapter 9). The two implementations agree on almost everything here, but numbers are where they differ most: the Common Lisp implementation has the unbounded integers and the complex numbers the manual describes, and the compiler to C has neither. Each difference is pointed out where it arises; appendix A collects them.

The examples show their results as comments, `; => value`. Where the two implementations print a result differently, both are shown.

8.1 The number classes

Numbers are arranged in a hierarchy of classes that follows the mathematical kinds of number. The abstract classes have no direct instances; they exist so that methods can specialize on them.

Class	Direct superclass	Kind
`<number>`	`<object>`	abstract
`<complex>`	`<number>`	abstract
`<real>`	`<complex>`	abstract
`<rational>`	`<real>`	abstract
`<integer>`	`<rational>`	concrete
`<ratio>`	`<rational>`	concrete
`<float>`	`<real>`	abstract
`<single-float>`	`<float>`	concrete
`<double-float>`	`<float>`	concrete
`<extended-float>`	`<float>`	concrete
`<rectangular-complex>`	`<complex>`	concrete; Common Lisp only

All of these classes but `<number>` are sealed: a program may not define new subclasses of them, though it may define methods on them. Every number belongs to exactly one of the concrete classes, which `object-class` reports:

```
(object-class 3)        ; => <integer>
(object-class 1/2)      ; => <ratio>
(object-class (/ 4 2))  ; => <integer>
(object-class 1.5)      ; => <double-float>
```

```
(object-class 1.5s0)    ; => <single-float>
```

(A class prints as `{the class <integer>}`; this chapter writes just the name.)

8.1.1 Integers

An `<integer>` is a whole number. The two implementations differ in how large one may be:

- In the **Common Lisp implementation**, integers are unbounded, as the manual requires: `(expt 2 100)` is `1267650600228229401496703205376`, and an integer of any size is still an `<integer>`.
- In the **compiler to C**, an integer is 63 bits, from -2^{62} to $2^{62} - 1$, that is, from `-4611686018427387904` to `4611686018427387903`. Every integer operation checks for overflow, and a result outside that range signals an error rather than giving a wrong answer:

```
(+ 4611686018427387903 1)
; C:    Dylan error: integer overflow (integers are 63 bits)
; Lisp: => 4611686018427387904
```

The error is an ordinary `<simple-error>` (chapter 11), so a program can handle it:

```
(handler-case (* 4611686018427387903 2)
  ((<error>) 'overflow))
; C:    => overflow
; Lisp: => 9223372036854775806
```

The check applies to every operation that makes an integer: `+`, `-`, `*`, `expt`, `ash`, `lcm`, and the numerators and denominators of ratios.

8.1.2 Ratios

A `<ratio>` is an exact fraction, kept in lowest terms with a positive denominator. Dividing integers that do not divide evenly makes one, and a ratio whose denominator would be 1 is made an integer instead (*rational canonicalization*):

```
(/ 1 3)         ; => 1/3
(/ 6 3)         ; => 2
(/ -6 4)        ; => -3/2
(+ 1/3 1/6)     ; => 1/2
(+ 1/2 1/2)     ; => 1
(* 1/2 4)       ; => 2
```

In the compiler to C the numerator and the denominator are each limited to 63 bits, with the same overflow error as integers; intermediate results are computed exactly in 128 bits, so only a final result that does not fit is an error. In the Common Lisp implementation ratios are unbounded.

8.1.3 Floats

The three float classes are the IEEE formats the manual names. Their representations are:

Class	Compiler to C	Common Lisp implementation
`<single-float>`	C `float` (32 bits)	SBCL `single-float`
`<double-float>`	C `double` (64 bits)	SBCL `double-float`
`<extended-float>`	C `double`, a class of its own	SBCL `double-float`, a class of its own

`<double-float>` is the default: a float literal without an exponent marker, or with `e` or `d`, is a double. An `<extended-float>` is a separate class with a double's precision, the widest of the three for contagion; the only way to make one is `as` (see section 8.11). `(+ (as <extended-float> 1/8) 1)` is the `<extended-float>` `1.125l0`.

8.1.4 Complex numbers

The manual's `<complex>` numbers exist only in the Common Lisp implementation, where they are Common Lisp's complex numbers and belong to the class `<rectangular-complex>`:

```
(make-rectangular 1 2)                  ; Lisp: => #c(1 2)
(make <complex> real: 1 imag: 2)        ; Lisp: => #c(1 2)
(* (make-rectangular 0 1)
   (make-rectangular 0 1))              ; Lisp: => -1
(abs (make-rectangular 3 4))            ; Lisp: => 5.0
(sqrt (make-rectangular 0 2))           ; Lisp: => #c(1.0 1.0)
```

The init-keywords of `<complex>` are `real:`, `imag:`, `magnitude:` and `angle:`, as in *DIRM* chapter 14; `make-rectangular` and `make-polar` make complex numbers from parts, and `real-part`, `imag-part` and `angle` take them apart. As the manual requires, a complex number whose imaginary part is an exact zero is canonicalized to a real: `(make-rectangular 3 0)` is `3`. A real argument never gives a complex result, in either implementation: `(sqrt -4)` is an error, as it is in a compiled program (section 8.8).

In the compiler to C, `<complex>` is kept, abstract, as the superclass of `<real>`, but it has no instances of its own. `make-rectangular` and `make-polar` accept only an exact zero for the imaginary part or the angle, and answer the real; anything else is an error, as is `make` of `<complex>`:

```
(make-rectangular 3 0)   ; => 3
(make-rectangular 1 2)   ; C: error: complex numbers are not supported: 1 2
```

8.2 Writing numbers

Numbers are written as in Common Lisp:

Written	Value	Class
`42`, `-7`	42, −7	`<integer>`
`1.`	1	`<integer>` (a trailing point means decimal)
`#xFF`, `#b101`, `#o17`	255, 5, 15	`<integer>`
`6/4`	`3/2`	`<ratio>`
`1.5`, `.5`, `-0.0`		`<double-float>`
`1e3`, `2d0`	`1000.0`, `2.0`	`<double-float>`
`1.5s2`, `1.5f2`	`150.0s0`	`<single-float>`
`1.5l0`	`1.5`	`<double-float>`

The exponent markers `e` and `d` make a double, `s` and `f` a single. The marker `l` also makes a double, in both implementations; there is no way to write an `<extended-float>` literal. A ratio is written with no spaces, `numerator/denominator`, and is reduced as it is read.

8.3 Arithmetic

The four arithmetic functions take any number of arguments, as in *DIRM* chapter 14:

```
(+)             ; => 0
(*)             ; => 1
(+ 1 2 3)       ; => 6
(- 5)           ; => -5         the additive inverse
(- 10 1 2 3)    ; => 4          10 - 1 - 2 - 3
(/ 2)           ; => 1/2        the multiplicative inverse
(/ 0.5)         ; => 2.0
(/ 60 2 3)      ; => 10         60 / 2 / 3
(/ 7 2)         ; => 7/2
(/ 7.0 2)       ; => 3.5
```

`-` and `/` need at least one argument. With several arguments, each function combines them from left to right by calling *binary+*, `binary-`, `binary*` or `binary/`; with one, `-` calls `unary-` and `/` calls `unary/`. The variadic functions themselves are methods and cannot be extended, but the binary and unary ones are generic functions: to make `+` work on a class of your own, add a method to `binary+`.

```
(define-class <money> (<object>)
  (cents init-keyword: cents:))

(define-method binary+ ((a <money>) (b <money>))
  (make <money> cents: (+ (cents a) (cents b))))

(cents (+ (make <money> cents: 150)
          (make <money> cents: 275)
          (make <money> cents: 5)))      ; => 430
```

Dividing by an exact or a floating zero is an error in both implementations, as design note 24 requires; so is `floor/`, `modulo` or any of the division family with a zero divisor:

```
(handler-case (/ 1 0)   ((<error>) 'caught))   ; => caught
(handler-case (/ 1.0 0) ((<error>) 'caught))   ; => caught
```

8.3.1 Contagion

When the arguments of an arithmetic function are of different classes, the rules are Common Lisp's (*DIRM* chapter 14): an operation between a rational and a float answers a float of the float's format, and an operation between two floats answers the wider format (single is narrower than double, double than extended). Rational results are canonicalized. Because a call with several arguments is a chain of binary calls, the conversions happen from left to right.

```
(+ 1 2.0)          ; => 3.0
(+ 1/2 0.5)        ; => 1.0
(* 2 0.5s0)        ; => 1.0s0
(+ 1.0s0 1.0d0)    ; => 2.0         a <double-float>
(* 1.5s0 1.5d0)    ; => 2.25        a <double-float>
```

Note: The compiler to C converts a rational to the float's format by way of a C `double`, so a single-float result can differ from the Common Lisp implementation's in the last place: `(+ 1/3 0.5s0)` is `0.8333333s0` in compiled code and `0.8333334s0` in the Common Lisp implementation.

8.4 Comparing numbers

The comparison functions of *DIRM* chapter 9 take two or more arguments and compare them pairwise from left to right:

Function	True when
`(= a b c ...)`	every argument is `=` to the next
`(/= a b)`	the two arguments are not `=` (exactly two)
`(< a b c ...)`	each argument is less than the next
`(> a b c ...)`	each argument is greater than the next
`(<= a b c ...)`	each argument is less than or equal to the next
`(>= a b c ...)`	each argument is greater than or equal to the next

```
(< 1 2 3)        ; => #t
(< 1 3 2)        ; => #f
(<= 1 1 2)       ; => #t
(>= 3 3 1)       ; => #t
(= 3 3 3)        ; => #t
(/= 1 2)         ; => #t
```

Numbers are compared by mathematical value, whatever their classes. A float is compared with a rational exactly (*rational contagion for comparisons*): the float's exact value is used, not a rounded one. Since the double nearest 0.1 is not exactly 1/10, they are not `=`:

```
(= 1 1.0)                      ; => #t
(= 1/2 0.5)                    ; => #t
(= 0.1 1/10)                   ; => #f
(< 1/3 0.3333333333333333)     ; => #f
```

`=` is not `id?`. Two numbers of the same class and value are `id?`; numbers of different classes never are, even when they are `=`:

```
(id? 1 1)       ; => #t
(id? 1 1.0)     ; => #f
```

The comparison functions are methods; to extend them to a class of your own, add methods to the generic functions `binary=` and `binary<`, on which all six are built (section 9.14 shows an example).

8.5 Predicates

These generic functions test a number and answer `#t` or `#f`:

Function	Argument	True when
`zero?`	a number	it is zero (`0`, `0.0`, `-0.0`)
`positive?`	a real	it is greater than zero
`negative?`	a real	it is less than zero
`odd?`	an integer	it is odd
`even?`	an integer	it is even
`integral?`	a number	it has an integer value

```
(zero? 0.0)         ; => #t
(positive? -1)      ; => #f
(negative? -1/2)    ; => #t
(odd? 3)            ; => #t
(even? 3)           ; => #f
(integral? 3)       ; => #t
(integral? 2.0)     ; => #t
(integral? 2.5)     ; => #f
(integral? 1/2)     ; => #f
```

To ask which kind of number an object is, use `instance?` with the classes of section 8.1:

```
(instance? 3 <rational>)     ; => #t
(instance? 1/2 <rational>)   ; => #t
(instance? 1.5 <real>)       ; => #t
(instance? 1.5 <integer>)    ; => #f
```

8.6 The division family

Dylan has the rounding and division functions of Common Lisp. Each of the one-argument functions rounds a real to an integer and answers two values: the integer and the remainder, which is the difference between the argument and the integer. They differ in which way they round:

Function	Rounds toward
`floor`	negative infinity
`ceiling`	positive infinity
`truncate`	zero
`round`	the nearest integer; a tie goes to the even one

```
(floor 7/2)      ; => 3 and 1/2
(floor -7/2)     ; => -4 and 1/2
(ceiling 2.5)    ; => 3 and -0.5
(truncate -2.7)  ; => -2 and -0.7000000000000002
(round 2.5)      ; => 2 and 0.5       the tie goes to 2, which is even
(round 3.5)      ; => 4 and -0.5
(round -2.5)     ; => -2 and -0.5
```

Use `bind` to receive both values (chapter 5); where one value is wanted, the first is used:

```
(bind ((q r (floor 7/2)))
  (list q r))                ; => (3 1/2)
(+ (round 7/2) 1)            ; => 5
```

The two-argument functions `floor/`, `ceiling/`, `round/` and `truncate/` divide their first argument by their second, round the quotient as the one-argument function of the same name does, and answer the rounded quotient and the remainder:

```
(floor/ 7 2)         ; => 3 and 1
(floor/ -7 2)        ; => -4 and 1
(ceiling/ 7 2)       ; => 4 and -1
(truncate/ -7 2)     ; => -3 and -1
(round/ 7 2)         ; => 4 and -1
(round/ 5 2)         ; => 2 and 1
(floor/ 7.5 2)       ; => 3 and 1.5
```

`modulo` answers the remainder of `floor/`, which has the sign of the divisor; `remainder` answers the remainder of `truncate/`, which has the sign of the dividend:

```
(modulo 7 2)         ; => 1
(modulo -7 2)        ; => 1
(modulo 7 -2)        ; => -1
(remainder 7 2)      ; => 1
(remainder -7 2)     ; => -1
(remainder 7 -2)     ; => 1
```

8.7 gcd, lcm, abs, min and max

`gcd` answers the greatest common divisor of its integer arguments and `lcm` their least common multiple. With no arguments they answer `0` and `1`; with one, the argument. They are built on the generic functions `binary-gcd` and `binary-lcm`, which programs can extend.

```
(gcd)            ; => 0
(gcd 12 18)      ; => 6
(gcd 12 18 8)    ; => 2
(gcd -12 18)     ; => 6
(lcm)            ; => 1
(lcm 4 6)        ; => 12
(lcm 4 6 10)     ; => 60
(lcm 0 5)        ; => 0
```

`abs` answers the absolute value of a number (for a complex number, in the Common Lisp implementation, its magnitude); `magnitude` is the same for a real.

```
(abs -5)       ; => 5
(abs -5/3)     ; => 5/3
(abs -2.5)     ; => 2.5
(magnitude -3) ; => 3
```

`min` and `max` take one or more reals and answer the least or the greatest of them, comparing with `binary<`. The answer is one of the arguments, not converted:

```
(min 3 1 2)       ; => 1
(max 3 1 2)       ; => 3
(max 1/2 0.4)     ; => 1/2
(max 1 2.0)       ; => 2.0
(min 3 1.5s0)     ; => 1.5s0
(min 2)           ; => 2
```

Of equal arguments the first is the answer: `(min 1 1.0)` is `1` and `(min 1.0 1)` is `1.0`. Because `min` and `max` use `binary<`, they work on any objects `<` can compare, characters among them: `(max #\a #\z)` is `#\z`.

8.8 Exponentials and trigonometry

The irrational functions of *DIRM* chapter 14 are defined as in Common Lisp:

Function	Answers
`(sqrt x)`	the square root of x
`(exp x)`	e^x
`(log x)`	the natural logarithm of x
`(expt x y)`	x^y
`(sin x)`, `(cos x)`, `(tan x)`	the trigonometric functions, x in radians
`(asin x)`, `(acos x)`, `(atan x)`	their inverses
`(atan2 y x)`	the angle of the point (x, y), from $-\pi$ to π
`(sinh x)`, `(cosh x)`, `(tanh x)`	the hyperbolic functions
`(asinh x)`, `(acosh x)`, `(atanh x)`	their inverses

For a rational argument they answer a `<double-float>`; for a float argument, a float of the same format. So `(sqrt 25)` is `5.0`, not `5`: the manual's examples show these functions answering floats, and so do both implementations.

```
(sqrt 25)        ; => 5.0
(sqrt 2)         ; => 1.4142135623730951
(sqrt 2.0s0)     ; => 1.4142135s0
(exp 1)          ; => 2.718281828459045
(log 1)          ; => 0.0
(log 100)        ; => 4.605170185988092
(sin 0)          ; => 0.0
(cos 0)          ; => 1.0
(atan 1)         ; => 0.7853981633974483
(atan2 1 1)      ; => 0.7853981633974483
(atan2 -1 -1)    ; => -2.356194490192345
(asin 1)         ; => 1.5707963267948966
(acosh 1)        ; => 0.0
```

`expt` is exact when its base is rational and its power is an integer, and answers a float otherwise:

```
(expt 2 10)      ; => 1024
(expt 2 -2)      ; => 1/4
(expt 2/3 3)     ; => 8/27
(expt 10 0)      ; => 1
(expt 2.0 3)     ; => 8.0
(expt 4 1/2)     ; => 2.0
(expt 2.0 0.5)   ; => 1.4142135623730951
```

A float result too large for its format is an error in both implementations, as Common Lisp signals floating-point overflow — `(exp 1000)`, `(* 1.0e308 10.0)` — and so is an operation with no real result, `(/ 0.0 0.0)`, an invalid operation; a program never has an infinity or a NaN. A real argument whose true result is not a real number is an error in both implementations, with the same message:

```
(sqrt -4)        ; error: sqrt: the result would be complex: -4
(log 0)          ; error: log: the argument must be positive: 0
(asin 2)         ; error: asin: the result would be complex: 2
(acosh 0.5)      ; error: acosh: the result would be complex: 0.5
(atanh 1)        ; error: atanh: the argument is out of range: 1
(expt -8 1/3)    ; error: expt: the result would be complex: -8
```

Only an integer too large for a compiled program behaves differently: `(expt 2 100)` is an error, "integer overflow", in the compiler to C, and `1267650600228229401496703205376` in the Common Lisp implementation.
An error from Common Lisp's arithmetic reaches a Dylan handler as a `<simple-error>` whose message begins `Lisp condition:` (chapter 11).

8.9 Integers as bits

These functions treat an integer as an infinite two's-complement string of bits, as Common Lisp does. `logior`, `logxor`, `logand` and `logeqv` take any number of integers; with none, they answer their identities, `0`, `0`, `-1` and `-1`. The rest take exactly the arguments shown.

Function	Answers
`(logior i ...)`	inclusive or
`(logxor i ...)`	exclusive or
`(logand i ...)`	and
`(logeqv i ...)`	equivalence (exclusive nor)
`(lognot i)`	the complement
`(lognand i j)`	`(lognot (logand i j))`
`(lognor i j)`	`(lognot (logior i j))`
`(logandc1 i j)`	`(logand (lognot i) j)`
`(logandc2 i j)`	`(logand i (lognot j))`
`(logorc1 i j)`	`(logior (lognot i) j)`
`(logorc2 i j)`	`(logior i (lognot j))`
`(logbit? index i)`	whether bit *index* of *i* is 1
`(ash i count)`	*i* shifted left *count* bits (right if negative)

```
(logior 1 2 4)      ; => 7
(logand 12 10)      ; => 8
(logxor 12 10)      ; => 6
(logeqv 12 10)      ; => -7
(lognot 5)          ; => -6
(lognand 12 10)     ; => -9
(lognor 12 10)      ; => -15
(logandc1 12 10)    ; => 2
(logandc2 12 10)    ; => 4
(logorc1 12 10)     ; => -5
(logorc2 12 10)     ; => -3
(logbit? 0 5)       ; => #t
(logbit? 1 5)       ; => #f
(ash 1 10)          ; => 1024
(ash 1024 -3)       ; => 128
(ash -8 -1)         ; => -4
```

In the compiler to C a shift whose result does not fit in 63 bits is an overflow error: `(ash 1 70)` is an error there and `1180591620717411303424` in the Common Lisp implementation.

8.10 Parts of numbers

`numerator` and `denominator` answer the parts of a rational in lowest terms; an integer's denominator is 1. For a float, both implementations answer the parts of the fraction the float holds exactly, as floats:

```
(numerator 6/4)       ; => 3
(denominator 6/4)     ; => 2
(denominator 5)       ; => 1
(numerator 1.5)       ; => 3.0
(denominator 1.5)     ; => 2.0
(denominator 0.1)     ; => 3.602879701896397e16
```

The last line shows that the double nearest 0.1 is a fraction with a large power of two below it. `rationalize` answers the simplest rational that is within the float's precision of it, which is usually the fraction you meant:

```
(rationalize 0.5)     ; => 1/2
(rationalize 0.1)     ; => 1/10
(rationalize 0.333)   ; => 333/1000
(rationalize 1/3)     ; => 1/3
```

`real-part`, `imag-part` and `angle` work on reals in both implementations (and on complex numbers in the Common Lisp implementation): a real is its own real part, its imaginary part is `0`, and its angle is zero for a non-negative number and π for a negative one.

```
(real-part 5)     ; => 5
(imag-part 5)     ; => 0
(angle 5)         ; => 0 (C)   0.0 (Lisp)
(angle -5)        ; => 3.141592653589793
```

8.11 Converting numbers

`as` converts a real to a float of a given format:

```
(as <float> 1/2)           ; => 0.5        a <double-float>
(as <double-float> 1)      ; => 1.0
(as <single-float> 1/3)    ; => 0.33333334s0
(as <extended-float> 1)    ; => 1.0l0
```

`as` also converts a float to the exact number it holds, as *DIRM* chapter 11 has `as` answer an instance with the same contents: `(as <integer> 3.0)` is `3`, `(as <rational> 0.5)` is `1/2` and `(as <ratio> 0.5)` is `1/2`, but `(as <integer> 2.5)` and `(as <ratio> 2.0)` are errors. To go from any float to an integer, round it with `floor`, `ceiling`, `round` or `truncate`; to the simplest nearby rational, use `rationalize`.

8.12 How floats print

Both implementations print a float with the fewest digits that read back as the same float, so a float printed and read again is the same float. A float prints in fixed notation when its magnitude is at least 10^{-5} and less than 10^{16}, or it is zero, and otherwise with an exponent. A double-float, the default format, is marked only by its exponent, written with `e`; a single-float ends in `s0` (or `s`n with an exponent), an extended-float in `l0`.

Value	Prints as
`0.1`	`0.1`
`(/ 1.0 3)`	`0.3333333333333333`
`100.0`	`100.0`
`123456789.0`	`123456789.0`
`1e15`	`1000000000000000.0`
`1e16`	`1.0e16`
`1e-4`	`0.0001`
`1e-7`	`1.0e-7`
`(/ 2.0s0 3)`	`0.6666667s0`
`(* 1.5s0 2)`	`3.0s0`

Integers print in decimal and ratios as `numerator/denominator`. Printing in general is the subject of chapter 13.

8.13 Characters

A `<character>` is written `#\` followed by the character, or by one of the names `space`, `newline`, `tab`, `page`, `rubout` and `backspace`:

```
(list #\a #\A #\( #\space #\newline #\tab)
; => (#\a #\A #\( #\space #\newline #\tab)
```

A character's integer is its Unicode code point, in both implementations. `as` converts in either direction (*DIRM* chapter 16):

```
(as <integer> #\A)        ; => 65
(as <integer> #\newline)  ; => 10
(as <character> 97)       ; => #\a
```

`as-uppercase` and `as-lowercase` answer the other case of a character, or the character itself if it has no other case. Both implementations follow Unicode, not only ASCII: the uppercase of Greek small lambda, code point 955, is capital lambda, 923.

```
(as-uppercase #\a)    ; => #\A
(as-lowercase #\Z)    ; => #\z
(as-uppercase #\1)    ; => #\1
(as <integer> (as-uppercase (as <character> 955)))   ; => 923
```

Characters compare by code point, so case is significant and every capital ASCII letter is less than every small one:

```
(< #\a #\b)       ; => #t
(< #\A #\a)       ; => #t
(> #\a #\B)       ; => #t
(= #\a #\A)       ; => #f
(id? #\a #\a)     ; => #t
```

`as-uppercase` and `as-lowercase` also work on whole strings (section 9.9), which is how to compare strings without regard to case. A string of characters that do not all fit in eight bits is a `<unicode-string>` (section 9.9).

8.14 Symbols and keywords

Symbols and keywords are objects identified by their names (*DIRM* chapter 15). A symbol is written as a name, quoted when it is to be a value rather than a variable: `'apple`. A keyword is a name followed by a colon, `apple:`, and evaluates to itself; the colon is not part of its name. Symbols name variables; keywords name keyword arguments and slot options.

```
(object-class 'apple)     ; => <symbol>
(object-class apple:)     ; => <keyword>
```

Names are not case sensitive: `Foo`, `foo` and `FOO` are one symbol, and two symbols with the same name are always `id?`. A symbol does remember a spelling, the one it was first written with, and prints that way. Converting between symbols, keywords and strings is done with `as`:

```
(list 'Hello 'hello 'HELLO)           ; => (Hello Hello Hello)
(id? 'Hello 'HELLO)                   ; => #t
(as <symbol> "hello")                 ; => Hello
(as <string> 'hello)                  ; => "Hello"
(as <keyword> "colour")               ; => colour:
(as <string> 'colour:)                ; => "colour"
(as <symbol> "two words")             ; => two words
```

The spelling remembered is the first anywhere, and the implementation's own names come first. In the compiler to C that includes every name the library uses, so `'Size` prints as `size` and `Test:` as `test:` there, while the Common Lisp implementation, which predefines fewer names, prints them `Size` and `Test:`. Either way `(id? 'size 'Size)` is `#t`; only the printing differs.

Symbols are compared only by identity: `(= 'abc 'ABC)` and `(id? 'abc 'ABC)` are both `#t`, and there is no ordering, so `(< 'a 'b)` is an error (no applicable method for `binary<`). To

sort symbols, sort their names:

```
(sort '(pear apple fig)
      test: (method (a b) (< (as <string> a) (as <string> b))))
; => (apple fig pear)
```

Chapter 9

Collections

A *collection* is an object that holds other objects, its *elements*, each under a *key*: lists, vectors, strings, tables, ranges and the rest. Dylan defines collections by a protocol of generic functions rather than by their representations, so that one function, `map` or `sort` or `reduce`, works on all of them, and a class you define joins them by providing a handful of methods. This chapter follows chapter 12 of the 1992 manual (*DIRM* chapter 12) and the parts of chapter 9 that concern collections.

The chapter first describes the classes and the protocol every collection shares, then the functions on all collections and on sequences, and then each built-in class in turn. Examples show their results as comments. A list, a vector and a string print the same in both implementations; the other collections print differently, and where that matters the examples convert them to lists first. A class in a result is written by its name alone, as in chapter 8.

9.1 The collection classes

Collections are classified along two independent lines. A collection either finds its elements by integer position, as a *sequence* does, or by keys of any kind, as an *explicit-key collection* does; and it either can be changed after it is made, as a *mutable collection* can, or cannot. The abstract classes that express this, and the concrete classes below them, are:

- `<collection>`
  - `<explicit-key-collection>`
  - `<mutable-collection>`
  - `<sequence>`
    - `<range>`
  - `<mutable-explicit-key-collection>` (an explicit-key and a mutable collection)
    - `<table>`
    - `<array>`
  - `<mutable-sequence>` (a mutable collection and a sequence)
    - `<list>`, with subclasses `<pair>` and `<empty-list>`
    - `<deque>`
    - `<string>`
    - `<vector>` (also an `<array>`)
      - `<simple-object-vector>`
      - `<stretchy-vector>`
      - `<byte-string>` (also a `<string>`)

- `<unicode-string>` (also a `<string>`)

The concrete classes, the ones a program makes instances of, are these:

Class	What it is
`<list>`, `<pair>`, `<empty-list>`	linked lists of pairs (sealed)
`<simple-object-vector>`	a fixed-size vector of any objects (sealed)
`<byte-string>`	a fixed-size string of 8-bit characters (sealed)
`<unicode-string>`	a fixed-size string of any characters (sealed)
`<stretchy-vector>`	a vector that grows and shrinks
`<deque>`	a double-ended queue
`<range>`	an arithmetic progression, possibly unbounded; not mutable
`<table>`	a hash table from any keys to any elements
`<array>`	a multi-dimensional array

`<vector>`, `<string>` and `<array>` are instantiable too, but `make` of them answers an instance of a concrete subclass: `(make <vector> size: 3)` is a `<simple-object-vector>`, `(make <string>)` a `<byte-string>`, and an `<array>` of one dimension is a `<simple-object-vector>`.

9.2 Keys, elements and size

Every collection is a mapping from keys to elements. The keys of a sequence are the integers from 0 up to its size; the keys of a table are whatever objects were stored in it; the keys of an array are sequences of indices. `element` answers the element under a key, and `(setter element)` replaces it:

```
(element '(a b c) 1)                        ; => b
(element "abc" 0)                           ; => #\a
(element #(a b c) 5 default: 'none)         ; => none
(define v (vector 1 2 3))
(set! (element v 0) 'first)
v                                           ; => #(first 2 3)
```

Asking for a key the collection does not have is an error, unless a `default:` is given, which is then the answer. Setting the element of a key a collection does not have is an error too, except for a table, which adds the key. `element` finds a key with `=` (as the manual's default method does), so a float equal to an integer, such as `1.0`, is that index of a sequence: `(element '(a b c) 1.0)` answers `b`. Any other key, such as `1.5`, is not among a sequence's keys.

`size` answers the number of elements, or `#f` for a collection of unbounded size; `empty?` answers whether there are none; and `key-sequence` answers a sequence of the keys:

```
(size '(1 2 3))           ; => 3
(size "abc")              ; => 3
(size (range from: 1))    ; => #f
(empty? '())              ; => #t
(empty? #(1))             ; => #f
(key-sequence "ab")       ; => a sequence of 0 and 1
```

The key sequence of a sequence holds the integers from 0 up to its size (a range, except that the Common Lisp implementation answers a list for a vector or a string); that of a table is a list of its keys, in no particular order.

`class-for-copy` answers the class used when a function such as `map` or `reverse` makes a new collection like an existing one. For most collections it is the collection's own class; a list, of either subclass, and a range, which is not mutable, answer `<list>`, and a string answers `<byte-string>`. A collection class that is not mutable must define a method for it (section 9.4).

9.3 The iteration protocol

All iteration over collections, by `for-each`, `map`, `reduce` and every other function in this chapter, is done with four generic functions. A collection chooses its own representation for an *iteration state*, an object recording where an iteration has got to; for every collection, the state `#f` means that the iteration is finished.

`(initial-state` ***`collection`***`)` answers the state for the first element, or `#f` if the collection is empty.

`(next-state` ***`collection state`***`)` answers the state for the next element, or `#f` if there is none. It may change the state it is given, which must not be used again.

`(current-element` ***`collection state`***`)` answers the element at a state.

`(copy-state` ***`collection state`***`)` answers a state that `next-state` will not change, for a program that wants to return to a place in an iteration.

Explicit-key collections add one more, and mutable collections another:

`(current-key` ***`collection state`***`)` answers the key of the element at a state.

`((setter current-element)` ***`collection state value`***`)` replaces the element at a state; it is written `(set! (current-element` *`c s`*`)` *`value`*`)`.

And collections that can be walked backward, the vectors, the strings, the deques and the bounded ranges, also have:

`(final-state` ***`collection`***`)` answers the state for the last element, or `#f` if the collection is empty.

`(previous-state` ***`collection state`***`)` answers the state for the element before, or `#f`.

Here is the protocol used directly. A sequence's keys are counted alongside its states, since a sequence is walked in key order:

```
(define v (vector 10 20 30))
(for ((state (initial-state v) (next-state v state))
      (key 0 (+ key 1)))
     ((not state))
  (print (list key (current-element v state))))
; prints (0 10), (1 20), (2 30)

(for ((state (final-state v) (previous-state v state)))
```

```
      ((not state))
    (print (current-element v state)))
; prints 30, 20, 10
```

A table supplies its keys through `current-key`, and any mutable collection can be changed in place as it is walked:

```
(define t (make <table>))
(set! (element t 'a) 1)
(for ((state (initial-state t) (next-state t state)))
     ((not state))
  (print (list (current-key t state) (current-element t state))))
; prints (a 1)

(define l (list 1 2 3))
(for ((state (initial-state l) (next-state l state)))
     ((not state))
  (set! (current-element l state) (* 10 (current-element l state))))
l    ; => (10 20 30)
```

`copy-state` keeps a place while the iteration goes on:

```
(bind ((s (initial-state l)))
  (bind ((saved (copy-state l s)))
    (set! s (next-state l s))
    (list (current-element l s) (current-element l saved))))
; => (20 10)
```

The protocol assumes that a collection does not change while it is iterated over, except through `(setter current-element)`. A *sequence* must also be *stable*: every iteration visits its elements in the same order, the order of their keys, which is its *natural order*. A table need not be: two iterations over the same table may visit its elements in different orders.

Note: A 1994 design note, number 26, later replaced this protocol with two functions, `forward-iteration-protocol` and `backward-iteration-protocol`. Both implementations follow the 1992 manual: the protocol is the one described here, and the newer functions do not exist.

Most programs never call these functions; they use `for-each` and the other loops of chapter 5, or the functions in the rest of this chapter, all of which are built on them.

9.4 Defining your own collection

To make a new kind of collection, define a subclass of one of the abstract classes of section 9.1: a sequence class for a collection keyed by position, an explicit-key class for one with keys of its own. Then give it methods for the iteration protocol:

- every collection: `initial-state`, `next-state`, `current-element` and `copy-state`;
- an explicit-key collection: also `current-key`;
- a mutable collection: also `(setter current-element)`;
- a collection that is not mutable: also `class-for-copy`, naming a mutable class for copies.

Nothing else is required: `element`, `size`, `map`, `reduce`, `sort` and the rest all have default methods that work through the protocol. They may be slow (the default `element` on a sequence walks from the start), so a class that can do better should also define methods for `element`, `(setter element)`, `size` or `member?`.

9.4.1 A sequence

This class is an immutable sequence of the first *count* squares, computed when asked for. Its iteration state is simply the index:

```
(define-class <squares> (<sequence>)
  (count init-keyword: count: init-value: 0))

(define-method initial-state ((s <squares>))
  (if (> (count s) 0) 0 #f))

(define-method next-state ((s <squares>) (state <integer>))
  (bind ((next (+ state 1)))
    (if (< next (count s)) next #f)))

(define-method current-element ((s <squares>) (state <integer>))
  (* state state))

(define-method copy-state ((s <squares>) (state <integer>))
  state)

(define-method class-for-copy ((s <squares>)) <list>)
```

With only these, it is a full collection:

```
(define sq (make <squares> count: 6))
(size sq)                          ; => 6
(element sq 4)                     ; => 16
(map (method (x) (+ x 1)) sq)      ; => (1 2 5 10 17 26)
(reduce + 0 sq)                    ; => 55
(member? 9 sq)                     ; => #t
(choose odd? sq)                   ; => (1 9 25)
(reverse sq)                       ; => (25 16 9 4 1 0)
(as <vector> sq)                   ; => #(0 1 4 9 16 25)
(= sq '(0 1 4 9 16 25))            ; => #t
(for-each ((x sq)) ()
  (display x) (display " "))       ; prints 0 1 4 9 16 25
```

`map`, `choose` and `reverse` answer lists, because `class-for-copy` says so.

9.4.2 A mutable explicit-key collection

This class keeps its keys and elements in an association list, a list of pairs. Its iteration state is the rest of that list, from the current pair on; it supplies `current-key`, and, being mutable, `(setter current-element)`:

```
(define-class <alist> (<mutable-explicit-key-collection>)
  (entries init-value: '()))
```

```
(define-method initial-state ((a <alist>))
  (if (empty? (entries a)) #f (entries a)))
(define-method next-state ((a <alist>) (state <pair>))
  (if (empty? (cdr state)) #f (cdr state)))
(define-method current-element ((a <alist>) (state <pair>))
  (cdr (car state)))
(define-method current-key ((a <alist>) (state <pair>))
  (car (car state)))
(define-method (setter current-element) ((a <alist>) (state <pair>) value)
  (set! (cdr (car state)) value))
```

The default `element` and `(setter element)` find a key by comparing with `=`; `key-sequence`, `size`, `do` and the others work as for any collection:

```
(define a (make <alist>))
(set! (entries a) (list (cons 'red 1) (cons 'green 2) (cons 'blue 3)))
(element a 'green)                ; => 2
(element a 'pink default: 0)      ; => 0
(set! (element a 'blue) 30)
(element a 'blue)                 ; => 30
(size a)                          ; => 3
(reduce + 0 a)                    ; => 33
(any? (method (v) (> v 10)) a)    ; => #t
(do print a)                      ; prints 1, 2 and 30
```

(The class need not define `copy-state`: the default method answers the state itself, which is right for a state that `next-state` never changes, as here.)

9.5 Functions on all collections

The functions in this section work on every collection. Those that take several collections process corresponding elements together: elements with the same key. For sequences that means elements at the same position, and processing stops at the end of the shortest. Other collections are *aligned*: only the keys all of them have are processed (*DIRM* chapter 12, "Collection Alignment").

9.5.1 do, map, map-as and map-into

`(do` *function collection ...*`)` calls the function on each group of corresponding elements, for effect, and answers `#f`:

```
(do (method (a b) (print (+ a b)))
    '(100 100 200 200)
    '(1 2 3 4))
; prints 101, 102, 203 and 204, then answers #f
```

`map` answers a new collection of the results, of the `class-for-copy` of the first collection; `map-as` takes the class of the result as its first argument:

```
(map + '(100 100 200 200) '(1 2 3 4))              ; => (101 102 203 204)
(map + '(1 2 3) #(10 20))                          ; => (11 22)
(map + (range from: 1) '(10 20 30))                ; => (11 22 33)
(map as-uppercase "abc")                           ; => "ABC"
(map-as <vector> + '(100 100 200 200) '(1 2 3 4))  ; => #(101 102 203 204)
```

Mapping over a table makes a table with the same keys:

```
(define prices (make <table>))
(set! (element prices 'tea) 3)
(set! (element prices 'cake) 5)
(define doubled (map (curry * 2) prices))
(element doubled 'cake)                          ; => 10
```

The class of the result decides what it can hold. `map` over a string makes a string, so a function that answers something other than a character is an error there; ask for another class with `map-as`:

```
(map-as <list> (method (c) 1) "ab")              ; => (1 1)
(map (method (c) 'x) "ab")                       ; => error
```

The compiler to C reports the error as `a <byte-string> holds only 8-bit characters: x`; the Common Lisp implementation as `as -- not specialized for this class type`. (The Common Lisp implementation turns an integer result into the character with that code instead, so `(map (method (c) 1) "ab")` answers a string of two characters there and is an error in the compiler to C.)

`(map-into` *target function collection ...*`)` stores the results into the mutable collection *target*, replacing its elements, and answers it. The target may be one of the sources.

```
(define x (list 100 100 200 200))
(map-into x + '(1 2 3 4) '(10 20 30 40))   ; => (11 22 33 44)
x                                          ; => (11 22 33 44)
(map-into x - x)                           ; => (-11 -22 -33 -44)
```

The function receives only the elements of the source collections. The 1992 manual's own example, `(map-into x + '(1 2 3 4))` answering `(101 102 203 204)`, also passes the target's elements, contrary to its prose; in both implementations it answers `(1 2 3 4)` (appendix A). Mapping into a table stores each result under its key, adding keys the table lacks.

9.5.2 any?, every?, reduce and reduce1

`any?` answers the first true value the function returns for a group of elements, or `#f`; `every?` answers `#t` if the function is true for every group and `#f` otherwise. Both stop as soon as the answer is known.

```
(any? > '(1 2 3 4) '(5 4 3 2))       ; => #t
(any? even? '(1 3 5 7))              ; => #f
(any? (method (x) (and (even? x) (* x 10))) '(1 2 3))   ; => 20
(every? > '(1 2 3 4) '(5 4 3 2))     ; => #f
(every? odd? '(1 3 5 7))             ; => #t
```

`(reduce` *function initial collection*`)` combines the elements from the first to the last: it calls the function on the initial value and the first element, then on that result and the second element, and so on, and answers the last result, or the initial value if the collection is empty. `reduce1` uses the first element as the initial value; it is an error on an empty collection.

```
(reduce max 10 '(3 1 4 1 5 9))    ; => 10
(reduce max 10 '(3 12 9 8 8 6))   ; => 12
(reduce - 0 '(1 2 3))             ; => -6      ((0 - 1) - 2) - 3
(reduce + 0 '())                  ; => 0
```

```
(reduce1 + '(1 2 3 4 5))          ; => 15
```

9.5.3 member? and find-key

`(member? value collection test: test)` answers whether the collection has an element that matches the value. The test defaults to `id?` and is called with the value first and an element second.

```
(define flavors #(chocolate pistachio pumpkin))
(member? 'chocolate flavors)         ; => #t
(member? 'banana flavors)            ; => #f
(member? 2.0 #(1 2 3))               ; => #f      not id? to 2
(member? 2.0 #(1 2 3) test: =)       ; => #t
```

On a list, as the 1992 manual says, the true value is the rest of the list from the matching element on: `(member? 2 '(1 2 3))` answers `(2 3)`. (A 1993 design note made it a boolean; neither implementation follows that.)

`(find-key collection predicate)` answers the key of the first element for which the predicate is true, or the `failure:` value (default `#f`). `skip:` *n* skips the first *n* matches.

```
(find-key flavors (method (f) (id? f 'pistachio)))   ; => 1
(find-key '(1 2 3 4) even?)                          ; => 1
(find-key '(1 2 3 4) even? skip: 1)                  ; => 3
(find-key '(1 2 3 4) even? skip: 2)                  ; => #f
(find-key '(1 3) even? failure: 'none)               ; => none
```

9.5.4 replace-elements! and fill!

`(replace-elements! collection predicate new-value-function)` replaces each element for which the predicate is true by the result of calling the function on it, at most `count:` of them if that is given, and answers the collection:

```
(replace-elements! (list 10 13 16 19) odd? (method (x) (* 2 x)))
; => (10 26 16 38)
(replace-elements! (list 1 3 5 7) odd? (method (x) 0) count: 2)
; => (0 0 5 7)
```

`(fill! collection value)` stores the value under every key. For a sequence, `start:` (inclusive, default 0) and `end:` (exclusive, default the size) limit it to a part. A `start:` or `end:` that is not an index of the sequence, or an `end:` before `start:`, is an error, here and for `copy-sequence`:

```
(fill! (list 'a 'b 'c 'd 'e 'f) 3 start: 2)      ; => (a b 3 3 3 3)
(fill! (vector 1 2 3 4 5) 0 start: 1 end: 3)     ; => #(1 0 0 4 5)
```

9.6 Functions on sequences

These functions work on any sequence. Unless a description says otherwise, a function that answers a new sequence answers one of the `class-for-copy` of its (first) argument and leaves the argument unchanged; a function whose name ends in `!` may change its argument and reuse

its storage, and its result may or may not be the argument itself, so always use the result. Functions that take a `test:` default it to `id?`.

9.6.1 Adding and removing

`(add` *sequence element*`)` answers a new sequence with the element added; `add!` may add it to the sequence itself. Where the element goes depends on the class: at the front of a list, a vector, a string and a deque. `add` and `add!` on a stretchy vector add at the end. `add-new` and `add-new!` add the element only if the sequence does not already have it (by `test:`).

```
(define numbers '(3 4 5))
(add numbers 1)             ; => (1 3 4 5)
numbers                     ; => (3 4 5)
(add #(3 4 5) 1)            ; => #(1 3 4 5)
(add "bc" #\a)              ; => "abc"
(add! (list 3 4 5) 1)       ; => (1 3 4 5)
(add-new '(3 4 5) 1)        ; => (1 3 4 5)
(add-new '(3 4 5) 4)        ; => (3 4 5)
```

`(remove` *sequence value*`)` answers the sequence without the elements that match the value, or without the first `count:` of them; `remove!` may change the sequence. `choose` keeps the elements for which a predicate is true, and `choose-by` keeps the elements of one sequence whose partners in another satisfy a predicate.

```
(remove '(3 1 4 1 5 9) 1)                ; => (3 4 5 9)
(remove '(3 1 4 1 5 9) 1 count: 1)       ; => (3 4 1 5 9)
(remove "banana" #\a)                    ; => "bnn"
(remove! (list 3 1 4 1 5 9) 1)           ; => (3 4 5 9)
(choose even? '(3 1 4 1 5 9))            ; => (4)
(choose even? #(1 2 3 4))                ; => #(2 4)
(choose-by even? (range from: 1) '(a b c d e f g h i))
; => (b d f h)
```

9.6.2 Sets

`intersection` answers the elements of the first sequence that are also in the second; `union` answers the elements of both, those of the second only if the first lacks them; `remove-duplicates` answers a sequence without repeated elements, keeping the first of each. The test of `intersection` and `union` is called with an element of the first sequence and one of the second.

```
(intersection '(john paul george ringo)
              '(richard george edward charles john))
; => (john george)
(union '(butter flour sugar salt eggs)
       '(eggs butter mushrooms onions salt))
; => (butter flour sugar salt eggs mushrooms onions)
(remove-duplicates '(spam eggs spam sausage spam spam spam))
; => (spam eggs sausage)
(remove-duplicates '(1 1.0 2) test: =)   ; => (1 2)
```

9.6.3 Copying and joining

`copy-sequence` answers a new sequence of the elements from `start:` (default 0) up to `end:` (default the size); bounds outside the sequence are an error, as for `fill!`. `concatenate` joins sequences into a new one of the first one's `class-for-copy`; `concatenate-as` takes the class. `replace-subsequence!` overwrites the elements of a sequence from `start:` on with those of another, which must fit.

```
(define hamlet '(to be or not to be))
(id? hamlet (copy-sequence hamlet))          ; => #f
(copy-sequence hamlet start: 2 end: 4)       ; => (or not)
(copy-sequence "hello" start: 1)             ; => "ello"
(concatenate "low-" "calorie")               ; => "low-calorie"
(concatenate '(0 1 2) '(3 4 5) '(6 7 8))     ; => (0 1 2 3 4 5 6 7 8)
(concatenate #(1) '(2))                      ; => #(1 2)
(concatenate-as <string> '(#\n #\o #\n) '(#\f #\a #\t))   ; => "nonfat"
(concatenate-as <vector> '(0 1 2) '(3 4 5))  ; => #(0 1 2 3 4 5)
(define phrase (copy-sequence "I hate oatmeal."))
(replace-subsequence! phrase "like" start: 2)  ; => "I like oatmeal."
```

(`phrase` is a copy because a string literal is a constant; see section 9.9.)

9.6.4 Reversing and sorting

`reverse` answers the elements in the opposite order. `sort` answers the elements in ascending order by `test:`, which defaults to `<`. Both sorts are stable, keeping equal elements in their original order, whether or not `stable:` is given. `reverse!` and `sort!` reuse the storage of a mutable sequence: they put its elements in the new order and answer the sequence itself, except that `reverse!` of a list relinks its pairs and answers the new first pair, so the variable that held the list then holds its last pair. On a range they answer what `reverse` and `sort` do.

```
(define x '(bim bam boom))
(reverse x)                  ; => (boom bam bim)
x                            ; => (bim bam boom)
(reverse "abc")              ; => "cba"
(reverse! (list 1 2 3))      ; => (3 2 1)
(define v (vector 3 1 2))
(sort! v)                    ; => #(1 2 3)
v                            ; => #(1 2 3)
(define nums '(3 1 4 1 5 9))
(sort nums)                  ; => (1 1 3 4 5 9)
nums                         ; => (3 1 4 1 5 9)
(sort nums test: >)          ; => (9 5 4 3 1 1)
(sort #(3 1 2))              ; => #(1 2 3)
(sort "hello")               ; => "ehllo"
(sort '("pear" "apple" "fig"))           ; => ("apple" "fig" "pear")
(sort '((b . 2) (a . 1) (c . 2) (d . 1))
      test: (method (x y) (< (cdr x) (cdr y))))
; => ((a . 1) (d . 1) (b . 2) (c . 2))
```

9.6.5 Particular elements and searching

`first`, `second`, `third` and `last` answer those elements, and with `set!` replace them. Asking for an element that is not there is an error, unless a `default:` is given, which is then the answer. `first`, `second` and `third` call `element` with the index (and `default:`), so their errors are `element`'s.

```
(first '(a b c))              ; => a
(third '(a b c))              ; => c
(last '(emperor of china))    ; => china
(last #(1 2 3))               ; => 3
(second '(a) default: 'none)  ; => none
(last '() default: 0)         ; => 0
(define l (list 1 2 3))
(set! (first l) 'one)
(set! (last l) 'three)
l                             ; => (one 2 three)
```

`(subsequence-position` *`big pattern`*`)` answers the index at which *pattern* first occurs in *big*, or the `count:`th occurrence, or `#f`:

```
(subsequence-position "Ralph Waldo Emerson" "Waldo")   ; => 6
(subsequence-position "banana" "an" count: 2)          ; => 3
(subsequence-position "banana" "x")                    ; => #f
(subsequence-position '(a b c b c) '(b c))             ; => 1
```

9.7 Lists and pairs

A list is either the empty list, `'()`, the only instance of `<empty-list>`, or a `<pair>`, whose *car* is an element and whose *cdr* is the rest of the list. The classes are sealed. The empty list is not `#f`: it is a true value.

```
(cons 1 2)                  ; => (1 . 2)
(cons 1 '(2 3))             ; => (1 2 3)
(list 1 2 3)                ; => (1 2 3)
(list* 1 2 3 '(4 5 6))      ; => (1 2 3 4 5 6)      the last argument is the tail
(list* 1 2)                 ; => (1 . 2)
(car '(4 5 6))              ; => 4
(cdr '(4 5 6))              ; => (5 6)
(car '())                   ; => ()
(cdr '())                   ; => ()
(object-class '())          ; => <empty-list>
(object-class '(1))         ; => <pair>
```

`(setter car)` and `(setter cdr)` change a pair. The twelve functions `caar` through `cdddr` compose `car` and `cdr` as their names spell (`(caddr x)` is `(car (cdr (cdr x)))`), and each has a setter:

```
(define x (list 4 5 6))
(set! (car x) 9)
(set! (cdr (cdr x)) '(a b))
x                           ; => (9 5 a b)
(cadr '(1 2 3))             ; => 2
```

```
(caddr '(1 2 3))           ; => 3
(cddr '(1 2 3))            ; => (3)
(define y (list 1 2 3))
(set! (cadr y) 'two)
y                          ; => (1 two 3)
```

`append` joins lists; its result shares the last list. `(find-pair` *`item list`*`)` finds the first element of a list of pairs whose car is the item (by `test:`):

```
(append '(1 2) '(3) '(4 5))              ; => (1 2 3 4 5)
(find-pair 'b '((a . 1) (b . 2)))        ; => (b . 2)
(find-pair 'z '((a . 1)))                ; => #f
```

`(make <list> size:` *`n`* `fill:` *`x`*`)` makes a list of n copies of x (default `#f`). `add!` on a list is `cons` and `remove!` may splice the list. The `size` of a circular list is `#f`:

```
(define circle (list 1 2 3))
(set! (cdr (cddr circle)) circle)
(size circle)    ; => #f
```

Important: Do not let the Common Lisp implementation's read-eval-print loop print a circular list, as it would by showing the value of the `set!` above typed on its own: it prints forever. Wrap such a form in `(begin ... #f)`.

The 1993 design note that renamed `cons`, `car` and `cdr` to `pair`, `head` and `tail` and removed `list*`, `append` and the `c...r` functions is not followed: both implementations have the 1992 names only.

9.8 Vectors and arrays

A vector is a one-dimensional array with constant-time access by index. `#(...)` writes a constant vector, `vector` makes one of its arguments, and `make` makes one of a given size:

```
(vector 1 2 3)                                  ; => #(1 2 3)
(make <vector> size: 3)                         ; => #(#f #f #f)
(make <simple-object-vector> size: 2 fill: 'x)  ; => #(x x)
(as <vector> '(1 2))                            ; => #(1 2)
(as <list> #(1 2))                              ; => (1 2)
```

A `<simple-object-vector>` holds any objects and cannot change size. Its elements are read and set by `element`, or by the functions of arrays below.

An `<array>` has any number of dimensions; its keys are sequences of indices. `make` takes `dimensions:` (required) and `fill:` (default `#f`). `aref` reads the element at indices given as separate arguments, `(setter aref)` sets it, and `dimensions` answers the dimensions as a list. A vector is an array of one dimension: `make` of `<array>` with one dimension, and `as` of `<array>`, answer a `<simple-object-vector>`. An array of any other number of dimensions is an instance of an internal subclass of `<array>`, `<%general-array>`, which no program can name; it prints as `{<array> ...}`. `aref` of a vector takes one index and finds it as `element` does.

```
(define a (make <array> dimensions: '(2 3) fill: 0))
(set! (aref a 1 2) 'x)
(aref a 1 2)            ; => x
(element a '(1 2))      ; => x
```

```
(dimensions a)          ; => (2 3)
(size a)                ; => 6
(aref #(7 8 9) 1)       ; => 8
(dimensions #(1 2 3))   ; => (3)
(define vv (vector 7 8 9))
(set! (aref vv 1) 5)
vv                      ; => #(7 5 9)
```

An array is iterated over in row-major order. A wrong number of indices, or an index out of bounds, is an error. `map` over an array answers an array with the same dimensions:

```
(define b (map list a))
(dimensions b)          ; => (2 3)
(aref b 1 2)            ; => (x)
```

9.9 Strings

A string is a mutable sequence of characters. A `<byte-string>` holds characters with codes below 256, a `<unicode-string>` any characters. `"..."` writes a constant string; `make` of `<string>` or `<byte-string>` takes `size:` and `fill:` (default `#\space`).

```
(make <string> size: 3 fill: #\z)   ; => "zzz"
(object-class "abc")                ; => <byte-string>
(as <list> "abc")                   ; => (#\a #\b #\c)
(as <string> '(#\h #\i))            ; => "hi"
(size "abc")                        ; => 3
```

A string literal with a character above code 255 is a `<unicode-string>`, and a byte string cannot hold such a character (storing one is an error). `make` and `as` of `<unicode-string>` make one from any characters, and the new sequences of `copy-sequence`, `reverse`, `add`, `as-lowercase` and the like keep it a `<unicode-string>`.

Important: A string literal is a constant. Each literal in a program is its own object, so `(id? "abc" "abc")` is `#f`; but a literal is one object however often it is evaluated, so a method whose body is `"lit"` answers the same string each time. Do not change a literal: copy it first with `copy-sequence`.

Strings compare lexicographically by their characters, a prefix being less than the longer string; `=` compares their characters. The case functions of characters extend to strings: `as-uppercase` and `as-lowercase` answer new strings, and `as-uppercase!` and `as-lowercase!` change the string they are given.

```
(< "apple" "banana")        ; => #t
(< "app" "apple")           ; => #t
(< "Zebra" "apple")         ; => #t      Z (90) is less than a (97)
(= "abc" "abc")             ; => #t
(as-uppercase "Van Gogh")   ; => "VAN GOGH"
(as-lowercase "Van Gogh")   ; => "van gogh"
(define s (copy-sequence "Van Gogh"))
(as-uppercase! s)
s                           ; => "VAN GOGH"
(sort '("pear" "Apple" "fig")
      test: (method (a b) (< (as-lowercase a) (as-lowercase b))))
```

```
; => ("Apple" "fig" "pear")
```

9.10 Stretchy vectors

A `<stretchy-vector>` is a vector that grows and shrinks. `add!` adds an element at its end and answers the same vector; `remove!` removes elements from the vector itself. Its size can be set: a larger size adds `#f` elements and a smaller one drops elements from the end.

```
(define sv (make <stretchy-vector>))
(add! sv 'a)
(add! sv 'b)
(add! sv 'c)
(size sv)           ; => 3
(element sv 1)      ; => b
(set! (size sv) 5)
(as <list> sv)      ; => (a b c #f #f)
(set! (size sv) 2)
(as <list> sv)      ; => (a b)
(remove! sv 'a)
(as <list> sv)      ; => (b)
```

`make` takes `size:` and `fill:`. Setting an element beyond the end is an error in both implementations; use `add!`, or set the size first.

9.11 Deques

A `<deque>`, a double-ended queue, adds and removes elements at either end in constant time. `push` adds at the front and `push-last` at the back, each answering the deque; `pop` and `pop-last` remove and answer the first or the last element, and it is an error to pop an empty deque. `add!` is `push`, and `remove!` removes from the deque itself.

```
(define d (make <deque>))
(push d 2)
(push d 1)
(push-last d 3)
(add! d 0)
(as <list> d)       ; => (0 1 2 3)
(pop d)             ; => 0
(pop-last d)        ; => 3
(as <list> d)       ; => (1 2)
(remove! d 2)
(as <list> d)       ; => (1)
```

A deque prints as `{<deque> 0 1 2 3}`. `make` takes `size:` and `fill:`. A deque's size can be set, as a stretchy vector's can: a smaller size drops elements from the end, and a larger one adds `#f` elements there.

9.12 Ranges

A `<range>` is an arithmetic progression, computed rather than stored, and possibly unbounded. `range` makes one from keywords: `from:` (default 0) is the first value and `by:` (default 1)

the step; `up-to:` is an exclusive bound, `through:` an inclusive one, and `size:` a number of elements. With none of the last three the range is unbounded. A negative `by:` makes a range that runs downward, with the bounds below it. With `by:` 0 every element is `from:`, so `up-to:` or `through:` leaves the range unbounded if `from:` is within the bound and empty if not; `size:` still limits it. `make` of `<range>` takes the same keywords.

```
(as <list> (range from: 1 up-to: 4))              ; => (1 2 3)
(as <list> (range from: 1 through: 4))            ; => (1 2 3 4)
(as <list> (range from: 10 through: 1 by: -3))    ; => (10 7 4 1)
(as <list> (range from: 0 by: 1/2 up-to: 2))      ; => (0 1/2 1 3/2)
(as <list> (make <range> from: 2 size: 3))        ; => (2 3 4)
(size (range from: 1))                            ; => #f
(element (range from: 10 by: 5) 100)              ; => 510
```

Several functions treat ranges specially so that they finish even on an unbounded one. `member?` computes its answer; `copy-sequence` and `reverse` answer ranges (an unbounded range cannot be reversed); `intersection` of two ranges is a range; and `=` compares two ranges by their values:

```
(member? 1000000 (range))                    ; => #t
(member? 6 (range from: 1 by: 2))            ; => #f
(as <list> (copy-sequence (range from: 5) start: 2 end: 5))  ; => (7 8 9)
(as <list> (intersection (range from: 0 by: 2)
                         (range from: 0 by: 3 up-to: 20)))   ; => (0 6 12 18)
(= (range from: 1 up-to: 4) (range from: 1 through: 3))      ; => #t
(= (range from: 1 up-to: 4) '(1 2 3))                        ; => #t
```

A range is not mutable; its `class-for-copy` is `<list>`, so `map`, `choose`, `add` and `concatenate` over a range answer lists. `add` and `concatenate` signal an error for an unbounded range:

```
(map (curry * 2) (range from: 1 through: 3))   ; => (2 4 6)
(choose odd? (range from: 1 through: 6))       ; => (1 3 5)
(add (range from: 1 through: 3) 0)             ; => (0 1 2 3)
(concatenate (range from: 1 size: 2) '(3))     ; => (1 2 3)
```

A range prints as `{range 1 by 1 size 3}`.

9.13 Tables

A `<table>` maps keys of any kind to elements. Keys are compared with `=` and hashed with `=hash`, so two strings with the same characters are the same key, and so are `1` and `1.0`. Setting the element of a new key adds it; `remove-key!` removes one (and does nothing if the key is absent).

```
(define t (make <table>))
(set! (element t 'one) 1)
(set! (element t "two") 2)
(element t 'one)                      ; => 1
(element t (copy-sequence "two"))     ; => 2
(element t 'three default: 0)         ; => 0
(size t)                              ; => 2
(remove-key! t 'one)
(key-sequence t)                      ; => ("two")
```

Asking for a missing key without a `default:` is an error. A table is unstable under iteration:

its elements come in an order of the implementation's choosing, which may change as it grows. A table prints as `{table "two" => 2}`.

9.14 Equality: = and id?

Dylan has two equalities. `id?` answers whether two objects are the same object, indistinguishable by any program; `=` answers whether they are equal in value, by the generic function `binary=`:

- numbers are `=` when they have the same mathematical value (chapter 8);
- collections are `=` when they have the same keys and the elements under each key are `=`; for sequences, the same elements in the same positions, whatever the classes of the sequences;
- pairs, including those of dotted lists, are `=` when their cars and their cdrs are;
- anything else is `=` only when it is `id?`.

```
(= '(1 2) (list 1 2))                  ; => #t
(id? '(1 2) (list 1 2))                ; => #f
(= #(1 2) '(1 2))                      ; => #t
(= "abc" #(#\a #\b #\c))               ; => #t
(= (list 1 (vector 2)) (list 1.0 (vector 2.0)))   ; => #t
(id? 'a 'a)                            ; => #t
(id? #\a #\a)                          ; => #t
```

The functions that search take a `test:` defaulting to `id?`, so pass `test: =` to find a value equal to another but not the same object.

To give a class of your own a notion of equality, add a method to `binary=`. If its instances are to be table keys, also add one to `=hash`, which must answer the same integer for any two objects that are `=`:

```
(define-class <point> (<object>)
  (x init-keyword: x:)
  (y init-keyword: y:))

(define-method binary= ((a <point>) (b <point>))
  (and (= (x a) (x b)) (= (y a) (y b))))

(define-method =hash ((p <point>))
  (+ (* 31 (=hash (x p))) (=hash (y p))))

(define p1 (make <point> x: 1 y: 2))
(define p2 (make <point> x: 1 y: 2))
(id? p1 p2)                            ; => #f
(= p1 p2)                              ; => #t
(member? p2 (list p1))                 ; => #f
(member? p2 (list p1) test: =)         ; => a list holding p1, a true value
(define names (make <table>))
(set! (element names p1) "home")
(element names p2)                     ; => "home"
```

Chapter 10

Functional Operations

Functions in Dylan are objects: they can be passed as arguments, stored in variables and collections, and returned as results (chapter 6). The functions in this chapter make new functions out of existing ones. They come from chapter 17 of the 1992 manual (*DIRM* chapter 17), with `identity` from chapter 18 and `apply` from chapter 8. Combined with the collection functions of chapter 9, they let many loops be written as a single expression.

Each function here answers a new function that accepts any number of arguments (as many as the functions it was made from need). In both implementations they are ordinary closures; the examples show their results as comments, and all of them give the same results in the compiler to C and the Common Lisp implementation.

10.1 compose

`(compose` *f g* `...)` answers a function that calls the last function on its arguments, then each function before it on the previous result, from right to left. With one argument, `compose` answers that argument. Thus `cadr` could be written `(compose car cdr)`:

```
((compose car cdr) '(1 2 3))              ; => 2
((compose (curry * 2) (curry + 1)) 5)     ; => 12     (5 + 1) * 2
(map (compose - abs) '(-1 2 -3))          ; => (-1 -2 -3)
```

Only the first value of each inner call is passed on: `((compose list floor) 7/2)` is `(3)`, though `floor` answers two values.

This is the manual's example: the distance of a point from the origin, built from three small functions.

```
(define-method sum ((numbers <sequence>))
  (reduce1 + numbers))
(define-method square ((x <number>)) (* x x))
(define-method square-all ((coords <sequence>))
  (map square coords))

(define distance (compose sqrt sum square-all))
(distance '(3 4 5))      ; => 7.0710678118654755
```

10.2 complement, disjoin and conjoin

`(complement` *`predicate`*`)` answers a function that answers `#t` where the predicate answers `#f`, and `#f` otherwise:

```
(map (complement even?) '(1 2 3))          ; => (#t #f #t)
((complement id?) 'a 'b)                   ; => #t
((complement member?) 5 '(1 2))            ; => #t
(define not-empty? (complement empty?))
(choose not-empty? '("a" "" "bc"))         ; => ("a" "bc")
```

`(disjoin` *`p q ...`*`)` answers the *disjunction* of its predicates: a function that calls them in order on its arguments and answers the first true value any of them returns, without calling the rest, or `#f` if none does. It is like an `or` of calls.

```
(define small-or-even? (disjoin (curry > 3) even?))
(choose small-or-even? '(1 2 3 4 5 6 7))           ; => (1 2 4 6)
((disjoin (method (x) #f) (method (x) (* x 10))) 4) ; => 40
(choose (disjoin zero? (curry < 10)) '(0 5 11 3 20)) ; => (0 11 20)
```

`(conjoin` *`p q ...`*`)` answers the *conjunction*: a function that calls the predicates in order and answers `#f` as soon as one does, or else the value of the last. It is like an `and` of calls.

```
(define positive-integer? (conjoin (rcurry instance? <integer>) positive?))
(map positive-integer? '(5 -5 5.0))                    ; => (#t #f #f)
((conjoin (rcurry instance? <integer>) (method (x) (* x 2))) 4)   ; => 8
(find-key '(3 8 12 5) (conjoin even? (curry < 10)))    ; => 2
```

10.3 curry and rcurry

`(curry` *`function arg ...`*`)` answers a function that calls *function* with the given arguments followed by its own. `rcurry`, "right curry", puts the given arguments after its own instead:

```
((curry list 1 2) 3 4)             ; => (1 2 3 4)
((rcurry list 1 2) 3 4)            ; => (3 4 1 2)
(map (curry + 1) '(3 4 5))         ; => (4 5 6)
(map (rcurry - 1) '(5 6 7))        ; => (4 5 6)
```

So `(curry id? 'x)` tests for identity with the symbol `x`; `(curry > 6)` is true of numbers less than 6, and `(rcurry > 6)` of numbers greater than 6:

```
((curry id? 'x) 'x)                ; => #t
((curry > 6) 3)                    ; => #t
((rcurry > 6) 3)                   ; => #f
((curry concatenate "set-") "size" "!")    ; => "set-size!"
(define yuppify (rcurry concatenate ", ayup"))
(yuppify "I'm from New Hampsha")   ; => "I'm from New Hampsha, ayup"
```

`rcurry` is the natural way to fix the second argument of a two-argument function, as in `(rcurry instance? <integer>)`, a predicate for integers, or `(rcurry element 0)`, which takes the first element of a sequence:

```
(map (rcurry element 0) '("abc" "xyz"))    ; => (#\a #\x)
```

10.4 always and identity

`(always` *`object`*`)` answers a function that ignores any arguments and answers the object:

```
((always 1) 'x 'y 'z)       ; => 1
((always #t) #f #f)         ; => #t
((always 'k))               ; => k
(map (always 0) '(a b c))   ; => (0 0 0)
```

`(identity` *`object`*`)` answers its argument. It is useful where a function is required and nothing should be done, such as the start of a chain of compositions:

```
(identity 5)                ; => 5
(map identity '(1 2))       ; => (1 2)
(define add1 (curry + 1))
((reduce (method (f g) (compose f g)) identity (list add1 add1 add1)) 0)
; => 3
```

10.5 apply

`(apply` *`function arg ... sequence`*`)` calls the function with the given arguments followed by the elements of the sequence, which may be any sequence (*DIRM* chapter 8):

```
(apply + 1 2 '(3 4))                    ; => 10
(apply max '(3 1 4))                    ; => 4
(apply list '())                        ; => ()
(apply + #(1 2 3))                      ; => 6
(apply + 1 (range from: 1 through: 3))  ; => 7
(apply (curry + 100) '(1 2))            ; => 103
(apply concatenate '("a" "b" "c"))      ; => "abc"
```

Since `map` takes any number of collections, applying it to a list of lists transposes them:

```
(apply map list '((1 2 3) (a b c)))     ; => ((1 a) (2 b) (3 c))
```

10.6 Idioms

These operations are at their best as the arguments of `map`, `choose`, `reduce`, `any?`, `every?` and `find-key`, where they replace small anonymous methods.

Filtering and transforming. Choose the odd numbers from 1 to 10, double them, and add them up:

```
(reduce + 0 (map (curry * 2) (choose odd? (range from: 1 through: 10))))
; => 50
```

Negating a test. Keep the lists that do not contain 2. Because `member?` takes the value first, `curry` fixes it:

```
(choose (complement (curry member? 2)) '((1 2) (3 4) (2 5)))
; => ((3 4))
```

Testing membership in a fixed set. `rcurry` fixes the collection instead. On a list `member?` answers the tail where the element was found, which `any?` passes on as its true value:

```
(any? (rcurry member? '(a b)) '(x y b))     ; => (b)
```

Counting. Count the odd numbers greater than 3:

```
(define count-if (method (pred seq) (size (choose pred seq))))
(count-if (conjoin odd? (curry < 3)) '(1 2 3 4 5 6 7))   ; => 2
```

Comparing by a key. Sort strings without regard to case, and find the length of the longest:

```
(sort '("pear" "Apple" "fig")
      test: (method (a b) (< (as-lowercase a) (as-lowercase b))))
; => ("Apple" "fig" "pear")
(reduce1 max (map size '("pear" "banana" "fig")))    ; => 6
```

Checking several classes. `curry` fixes the object and leaves the class to vary:

```
(every? (curry instance? 3) (list <integer> <rational> <number>))   ; => #t
```

Chapter 11

Conditions

A *condition* is an object that represents an exceptional situation: an error, a warning, or a request to recover in some particular way. A program *signals* a condition when the situation arises, and the condition system finds a *handler* for it, a function that some caller established in advance, and calls it. This chapter follows chapter 13 of the 1992 manual (*DIRM* chapter 13), which both implementations provide complete; it also describes what each implementation does with a condition no handler takes, and the errors the implementations themselves signal. As in chapter 8, a class in a result is written by its name alone.

11.1 How the condition system works

Dylan's conditions are objects, instances of subclasses of `<condition>`, and a handler is chosen by the condition's class: a handler for `<error>` applies to every error, one for `<type-error>` only to type errors. Handlers are established dynamically, for the duration of a form's evaluation, and the most recently established applicable handler is tried first.

Signalling is a function call. When a condition is signalled, the signaller is still active and the handler runs "on top of" it. The handler then does one of three things:

- It **exits** non-locally, abandoning the computation that signalled, to a point established outside it. This is the common case, and `handler-case` does it for you.
- It **returns** values, which become the values of `signal` in the signaller. This is allowed only for conditions whose *recovery protocol* says what the values mean, such as warnings.
- It **declines**, by calling the next handler, letting an older handler decide.

If every handler declines, or there are none, the condition's *default handler* runs. For a warning it prints the message and returns; for an error it stops the program, or in the Common Lisp implementation enters the debugger.

Recovery can also be formal. A *restart* is a condition that a handler signals to ask for a particular recovery, and a *restart handler*, established by the code that knows how to recover, handles it. Section 11.7 describes restarts.

11.2 The condition classes

The predefined condition classes are these:

- <condition>
  - <serious-condition>
    - <error>
      - <simple-error>
      - <type-error>
  - <warning>
    - <simple-warning>
  - <restart>
    - <simple-restart>
    - <abort>

<condition> The class of all conditions. Its default handler returns `#f`, so a plain condition nobody handles is ignored.

<serious-condition> Conditions that cannot safely be ignored. Its default handler is the debugger of last resort (section 11.10).

<error> Serious conditions that represent something wrong with the program, as distinct from its environment; a handler for `<error>` does not catch other serious conditions.

<simple-error> An error that is just a message: a format string and arguments. They are given by the init-keywords `format-string:` and `format-arguments:`, and the functions `condition-format-string` and `condition-format-arguments` read them. `error` makes one from a string.

<type-error> An error saying a value was not of the expected type. Its init-keywords are `value:` and `type:`, read by `type-error-value` and `type-error-expected-type`. `check-type` signals one, and so do the implementations when a typed slot or variable is given a value of the wrong type.

<warning> Conditions worth reporting but not stopping for. The default handler prints the message and returns `#f`.

<simple-warning> A warning that is a format string and arguments, like `<simple-error>`; `signal` makes one from a string.

<restart> Requests to recover. The default handler signals an error saying no handler was established for the restart. The init-keyword `condition:` is accepted, so that a restart can name the condition it recovers from.

<simple-restart> The restart `cerror` and `break` establish handlers for.

<abort> The restart that asks to abandon the current command and return to something like a command loop.

Conditions are made with `make` like any object. A condition prints as its message:

```
(print (make <simple-error> format-string: "bad value: ~s"
                            format-arguments: '(42)))
; prints bad value: 42
```

11.3 Signalling

`(error` *`condition`*`)` signals a condition and never returns. `(error` *`string argument ...`*`)` makes a `<simple-error>` from a format string and arguments and signals it. This is the usual way to report an error:

```
(define-method withdraw (balance amount)
  (if (> amount balance)
      (error "cannot withdraw ~a from a balance of ~a" amount balance)
      (- balance amount)))
```

If a handler for the error returns instead of exiting, `error` does not return either: it reports the error as unhandled (section 11.10).

`(signal` *`condition`*`)` signals a condition and answers whatever values the handler returns. `(signal` *`string argument ...`*`)` makes a `<simple-warning>` and signals it. Any other first argument, or an argument after a condition, is an error, here and for `error`, `cerror` and `break`: `(signal 42)` reports `signal: not a condition or a string: 42`. With no handler, a warning is printed and `signal` answers `#f`:

```
(signal "disk is ~a% full" 91)
; prints a warning:  disk is 91% full
; => #f
```

The compiler to C prints the warning on standard error as `Dylan warning: disk is 91% full`; the Common Lisp implementation uses Common Lisp's `warn`, which prints `WARNING: disk is 91% full`.

The format strings of `error`, `signal`, `cerror` and `break` understand the directives `~a` and `~d` (the argument as `display` prints it), `~s` (as `print` prints it), `~%` (a new line) and `~~` (a tilde). Other directives are copied as they are, and arguments no directive uses are printed after the message, separated by spaces.

`(cerror` *`restart-description condition-or-string argument ...`*`)` is a *continuable error*. It signals the error as `error` does, but first establishes a handler for `<simple-restart>` described by *restart-description*. If a handler (or, in the Common Lisp implementation, a person at the debugger) signals a `<simple-restart>`, `cerror` returns `#f` and the program carries on with the correction the description promised:

```
(define-method careful-sqrt (x)
  (if (negative? x)
      (begin (cerror "Use the absolute value" "negative argument ~a" x)
             (sqrt (abs x)))
      (sqrt x)))

(handler-bind (<error> (method (c next) (signal (make <simple-restart>))))
  (careful-sqrt -16))                          ; => 4.0
```

`(check-type` *`value type`*`)` answers the value if it is an instance of the type, and signals a `<type-error>` if not:

```
(check-type 5 <integer>)        ; => 5
(handler-case (check-type "abc" <integer>)
  ((<type-error> condition: c)
   (list (type-error-value c) (type-error-expected-type c))))
; => ("abc" <integer>)
```

The type error is signalled with `error`, so a handler may not return from it: one that does causes the error "a handler returned from error".

`(abort)` signals an `<abort>` with `error`; it never returns. A handler for `<abort>` decides what abandoning means:

```
(handler-case (begin (abort) 'not-reached)
  ((<abort>) 'aborted))         ; => aborted
```

`(break)` or `(break` ***`string argument ...`***`)` is a breakpoint. In the Common Lisp implementation it enters the debugger with the message, offering a restart that continues; in the compiler to C, which has no debugger, it prints `Dylan break:` and the message on standard error and continues. Either way it then answers `#f`.

11.4 Handling with handler-case

`handler-case` is the usual way to handle conditions: it evaluates a form, and if a condition of one of the listed classes is signalled during it, abandons the form and evaluates the matching clause instead.

```
(handler-case protected-form
  ((type option ...) form ...)
  ...)
```

If the protected form finishes, `handler-case` answers its values. If a condition is signalled that is an instance of a clause's *type* (a class, or a singleton), the protected form is exited and that clause's forms are evaluated, their last value being the answer, or `#f` if there are none. The clauses are tried in order, so put the more specific classes first. The options are:

`condition:` ***`name`*** binds the variable *name* to the condition in the clause's forms;

`test:` ***`function`*** a predicate the condition must also satisfy for the clause to apply;

`description:` ***`string`*** a description of the handler, which introspection (`do-handlers`) can see.

```
(handler-case (error "bad value: ~s" 42)
  ((<simple-error> condition: c)
   (list (condition-format-string c) (condition-format-arguments c))))
; => ("bad value: ~s" (42))

(handler-case (error "code ~a" 7)
  ((<simple-error> test: (method (c) (= (first (condition-format-arguments c)) 8)))
   'eight)
  ((<error>) 'other))
; => other

(handler-case (error "x")
  ((<error>) 'first)
  ((<simple-error>) 'second))
; => first

(handler-case (values 1 2)
```

```
  ((<error>) 'oops))
; => 1 and 2
```

The clause's forms run after the exit, in the dynamic environment of the `handler-case` itself, so its handlers are no longer in force there. A singleton type handles one particular condition object:

```
(define c1 (make <simple-error> format-string: "one"))
(define c2 (make <simple-error> format-string: "two"))
(define-method try (c)
  (handler-case (error c)
    (((singleton c1)) 'got-c1)
    ((<simple-error> condition: e) (list 'other (condition-format-string e)))))
(list (try c1) (try c2))     ; => (got-c1 (other "two"))
```

11.5 Handling with handler-bind

`handler-bind` is the primitive form, and both implementations have it. It establishes one handler, a function, while its body is evaluated, and answers the body's values:

```
(handler-bind (type function option ...)
  form ...)
```

The type, function and options are evaluated before the body; the options are `test:` and `description:`, as for `handler-case`. When an applicable condition is signalled, the function is called with two arguments, the condition and a *next-handler* function, without exiting anything. It may exit non-locally (with `bind-exit`, chapter 5); it may return values, which `signal` then answers; or it may decline by calling the next-handler with no arguments, which tries the next older applicable handler.

```
(handler-bind (<warning> (method (c next) (print 'outer) 'outer-value))
  (handler-bind (<warning> (method (c next) (print 'inner-declines) (next)))
    (signal "low battery")))
; prints inner-declines, then outer
; => outer-value
```

There is no "condition wall": while a handler runs, every handler established before it is still in force, including those established inside the signalling code. That is what lets a handler signal a restart that the signaller handles (section 11.7). It also means that a handler which signals a condition of its own class, or causes an error it also handles, can recurse until the stack is exhausted.

11.6 Default handlers

When no dynamic handler handles a condition, `signal` calls `default-handler`, a generic function, on it. Its predefined methods are:

Method on	Does
`<condition>`	answers `#f`
`<serious-condition>`	reports the condition as unhandled
`<warning>`	prints the message, answers `#f`
`<restart>`	signals an error: no handler was established for the restart

A condition class of your own inherits the method of its superclass. A condition that is neither serious nor a warning is simply ignored when unhandled:

```
(define-class <note> (<condition>)
  (text init-keyword: text:))

(signal (make <note> text: "hello"))          ; => #f
(handler-case (signal (make <note> text: "hi"))
  ((<note> condition: n) (text n)))           ; => "hi"
```

A program may call `default-handler` itself, and may add methods to it for its own classes.

11.7 Restarts

A restart is a condition that asks for a particular way of recovering. The code that knows how to recover establishes a handler for a restart class, usually with `handler-case`, around the code that may fail; a handler for the original condition, which knows which recovery it wants, signals the restart. Values a recovery needs travel in the restart's slots.

This program parses digits. When a character is not a digit it signals an error, but offers a `<use-value>` restart that supplies a value instead; the caller's handler chooses that recovery:

```
(define-class <use-value> (<restart>)
  (value init-keyword: value:))

(define-method parse-digit ((c <character>))
  (if (and (>= c #\0) (<= c #\9))
      (- (as <integer> c) (as <integer> #\0))
      (handler-case (error "not a digit: ~s" c)
        ((<use-value> condition: r) (value r)))))

(handler-bind (<simple-error>
               (method (c next) (signal (make <use-value> value: -1))))
  (map parse-digit '(#\4 #\x #\2)))
; => (4 -1 2)
```

The handler for `<simple-error>` runs while `parse-digit`'s `handler-case` is still in force, so its `signal` reaches the restart handler, which exits from the `handler-case` with the value.

`cerror` establishes a `<simple-restart>` handler, as shown in section 11.3; signalling a `<simple-restart>` where none is established is an error. The `condition:` init-keyword lets a restart be tied to one condition, so that a `test:` on its handler can accept only restarts meant for that condition (*DIRM* chapter 13 has an example).

11.8 Introspection

`(do-handlers` *`function`*`)` calls the function on each handler in force, the most recent first, with four arguments: the handler's type, test, function and description. An absent test is given as a function that answers `#t`, and an absent description as `#f`.

```
(handler-bind (<warning> (method (c n) #f) description: "ignore warnings")
  (handler-bind (<error> (method (c n) #f))
    (do-handlers (method (type test function description)
                   (print (list type description))))))
; prints (<error> #f), then (<warning> "ignore warnings")
```

At the Common Lisp implementation's prompt the list ends with the loop's own handler, an `<abort>` described as `"Return to the read-eval-print loop."`

Four generic functions let an interactive debugger ask whether a condition may be handled by returning (`return-allowed?`), what returning would mean (`return-description`), and the user for the contents of a restart (`restart-query`) or for values to return (`return-query`). For any condition `return-allowed?` and `return-description` answer `#f`, but for a warning, whose recovery protocol allows returning *DIRM* chapter 13, `return-allowed?` answers `#t` and `return-description` the string `"Return; the value is ignored."`. `restart-query` does nothing. The compiler to C, having no debugger, has `return-query` answer `#f`; the Common Lisp implementation's reads values typed by the user.

11.9 Defining your own conditions

Define a condition class as a subclass of the predefined class closest in meaning: `<error>` for a program error, `<warning>` for something worth reporting, `<restart>` for a recovery. Slots carry whatever a handler needs to know.

The simplest way to give a condition a message that both implementations report is to subclass `<simple-error>` (or `<simple-warning>`) and pass `format-string:` and `format-arguments:` when making it:

```
(define-class <bad-reading> (<simple-error>)
  (sensor init-keyword: sensor:))

(define-method report-bad (sensor value)
  (error (make <bad-reading> sensor: sensor
               format-string: "sensor ~a read ~a"
               format-arguments: (list sensor value))))

(handler-case (report-bad 'north 9999)
  ((<bad-reading> condition: c) (list (sensor c) c)))
; => (north sensor north read 9999)
```

Unhandled, it is reported as `sensor south read -1`. A direct subclass of `<error>` with no message is reported by its class only: in the compiler to C as `Dylan error: {instance of <oops>}`, even if the class has a `print-object` method (chapter 13); that method is used when the program prints the condition itself.

11.10 Unhandled conditions

What happens to a serious condition that no handler takes, the "debugger of last resort" of the manual, is where the implementations differ most.

11.10.1 In the Common Lisp implementation

An unhandled error enters the Common Lisp debugger, which shows the message and offers restarts: Dylan's restart handlers in force, then restarts that return to the Dylan prompt, then Common Lisp's own. Type the number of a restart to choose it. Here a `cerror` offers its restart:

```
? (define-method checked (n)
    (when (> n 10)
      (cerror "Use 10 instead" "~a is too big" n)
      (set! n 10))
    n)

;Value: checked

? (checked 99)

debugger invoked on a DYLAN::DYLAN-CONDITION in thread
#<THREAD tid=712328 "main thread" RUNNING {1200030003}>:
  Error: 99 is too big

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [DYLAN-RESTART] Use 10 instead
  1:                 Return to the read-eval-print loop.
  2: [DYLAN-REP    ] Return to the Dylan read-eval-print loop.
  3: [ABORT        ] Exit debugger, returning to top level.

((LAMBDA NIL :IN DYLAN::IMPLEMENTATION-SPECIFIC/INDUCE-ERROR))
0] 0

;Value: 10
```

Choosing `Return to the read-eval-print loop` abandons the form and prints `;Aborted`; `DYLAN-REP` abandons it without the message. `break` offers a `CONTINUE` restart, which makes `break` answer `#f`.

An error signalled by Common Lisp itself (a Lisp type error inside a built-in function, a floating-point overflow) is offered first to Dylan's handlers as a `<simple-error>` whose message begins `Lisp condition:`. If none takes it, it reaches the debugger as the Lisp error it is, and the debugger offers `DYLAN-REP`, which returns to the Dylan prompt, and Common Lisp's own restarts. In a program compiled to Lisp with `dylan:compile-file` and loaded into the runtime, an unhandled error likewise goes to the Common Lisp debugger (or, when SBCL runs with its debugger disabled, ends SBCL with a backtrace). Chapter 14 describes the debugger further.

11.10.2 In the compiler to C

A compiled program has no debugger. An unhandled serious condition in the main thread prints `Dylan error:` and the condition's message on standard error and ends the program with exit status 1. Anything the program had written to standard output is flushed first.

```
$ cat range.dyl
(print "before")
(error "the value ~s is out of range" 42)
(print "after")
$ bin/dylanc range.dyl
$ ./range
"before"
Dylan error: the value 42 is out of range
$ echo $?
1
```

The cleanup forms of `unwind-protect` that are in force are *not* run when a program stops this way. Some other cases:

Program	Reported
`(check-type "x" <integer>)`	`Dylan error: "x" is not an instance of {the class <integer>}`
a handler returns from `(error "nope")`	`Dylan error: a handler returned from error: nope`
`(abort)` with no handler	`Dylan error: no handler was established for the restart {instance of <abort>}`

A condition a thread other than the main one does not handle ends that thread instead, as chapter 16 describes.

11.11 Errors the implementations signal

Errors the implementations detect themselves are signalled as Dylan conditions, so programs can handle them like any other. They are `<type-error>`s when a value of the wrong type is stored in a typed slot or variable, or passed to a method whose parameter requires another type, and `<simple-error>`s otherwise. Both implementations word them alike:

Cause	Message
no method applies to the arguments	`no applicable method for f`
wrong number of arguments	`wrong number of arguments (2)`
a keyword the function does not take	`keyword argument not accepted: b:`
calling something not a function	`not a function: 5`
integer overflow	`integer overflow (integers are 63 bits)`
division by zero	`division by zero`
a typed slot or variable given a wrong value	`"s" is not an instance of {the class <integer>}` (a `<type-error>`)
reading an uninitialized slot	`the slot s is not initialized`
a variable with no value	`the variable x@dylan-user has no value`
`car` of a non-list	`not a list: 5`
an element that does not exist	`element: there is no element at the index 5`
`make` of an abstract class	`cannot make an instance of this class: {the class <integer>}`

The compiler also warns, when compiling, about a variable that is not defined anywhere (`warning: x is not defined`); the program is still built, and signals the error above if the reference is evaluated.

In the Common Lisp implementation an error that arises inside Common Lisp itself is reflected as a Dylan `<simple-error>`, which handlers see as they see any other: the host's errors for a call with the wrong number of arguments, a value of the wrong type (`not a list: 5` for `(car 5)`) and division by zero are worded as above, and any other has a message beginning `Lisp condition:` followed by Common Lisp's own description.

```
(define-method f ((x <integer>)) x)

(handler-case (f "s")
  ((<error> condition: c) c))
; C:    no applicable method for f
; Lisp: generic-dispatch -- no applicable methods {the generic function f} ("s")

(define-class <account> (<object>)
  (balance type: <integer> init-value: 0))

(handler-case (set! (balance (make <account>)) "lots")
  ((<type-error> condition: c) (type-error-value c)))
; => "lots"
```

Chapter 12

Modules

A small program can live comfortably in one namespace: every class, generic function and variable it defines sits side by side, and every name means the same thing everywhere. A larger program soon outgrows that. Two parts of it want to use the same name for different things, or one part wants to keep its helpers private so that nobody else comes to depend on them. Dylan's answer is the *module*.

This chapter explains what a module is, how the `define-module` form declares one, how a source file says which module its code belongs to, and how each implementation handles programs spread over several modules and several files. It ends with a complete multi-file program.

12.1 What a module is

A module is a mapping from names to *module variables*. It is not a container for classes or functions as such: Dylan modules deal only in variables. Since classes and functions are ordinary values stored in variables, controlling who can see a variable controls who can use the class or function in it. If you export the variable `<point>`, other modules can make points; if you do not, the class is private *DIRM* chapter 5.

A module gets its variables in two ways.

- It may *own* a variable. A definition compiled in the module creates the variable there if the name does not already refer to one. The defining forms are `define`, `define-class`, `define-method` and `define-generic-function`.
- It may *import* a variable that another module exports, by *using* that module. An imported variable is the very same variable as in the module that owns it, not a copy: an assignment made through one module is seen through every other.

Each variable is owned by exactly one module, but it may be visible in many. Within one module a name refers to at most one variable; it is an error to import two different variables under the same name, or to import a name and also define it.

Only *exported* variables can be imported. A module exports a variable it owns by naming it in an `export` or `create` clause, and it can pass on variables it imported with the `export:` option of a `use` clause.

Modules affect only module variables. A local variable made by `bind`, by a method's parameters or by any other binding form is looked up lexically, as always, and hides a module variable of the same name.

12.2 The predefined modules

Two modules exist before any program runs.

dylan owns every predefined variable of the language: `+`, `make`, `<object>`, `map` and the rest (Appendix B lists them all). It exports them all. It cannot be redefined. In the compiler to C the library itself is written in Dylan, in the module `dylan`, and it also owns a few internal variables, whose names begin with `%`, which it does not export.

dylan-user uses `dylan` and owns nothing else to begin with. Code goes here unless something says otherwise, which is why the examples in the rest of this book never mention modules at all.

12.3 Declaring a module

The 1992 manual says that modules are "manipulated by the programming environment" and gives no syntax for declaring them. Apple's later design note 32 supplied one, in the infix syntax. Both implementations accept it in prefix form:

```
(define-module module-name clause ...)
```

Each *clause* is a `use` clause, an `export` clause or a `create` clause. The module name is a symbol. Module names live in a namespace of their own, separate from variables, so `define-module` creates no variable, and a module may share its name with a variable. `define-module` is a defining form: it belongs at top level or inside a top-level `begin`. The compiler to C reports one inside a method ("define-module must be at top level"); the interactive implementation does not check.

12.3.1 `use` clauses

A `use` clause imports variables from another module:

```
(use used-module
     import:  all | (name-or-rename ...)
     exclude: (name ...)
     prefix:  "string"
     rename:  ((old-name => new-name) ...)
     export:  all | (name ...))
```

All the options are optional. With none, every variable the used module exports is imported under its own name, and nothing is exported again. The options work as follows.

import: Which variables to import. The default, `all`, imports everything the used module exports. A list imports only the names listed. An entry of the form `(old => new)` imports `old` under the name `new`; it is shorthand for listing `old` and adding the same pair to `rename:`.

exclude: Names not to import. It is allowed only when `import:` is `all`.

prefix: A string put in front of each imported name. With `prefix: "gfx$"` the used module's `draw-line` is `gfx$draw-line` here. Names given in `rename:` do not get the prefix.

rename: Pairs `(old => new)`. Each `old` is imported, whatever `import:` says, under the name `new`.

export: Variables imported by this clause that the module being defined exports again, named by their names in this module; `all` re-exports everything the clause imports. The default is to re-export nothing.

Renaming and excluding work on symbols, not on single variables. A name `foo` and the setter name `(setter foo)` travel together: if you import a slot getter `x` under the name `px`, its setter comes with it, and `(set! (px p) 3)` works in the importing module.

A module may use the same module in more than one clause; it gets the union of what the clauses import. Each clause's options apply to that clause alone.

12.3.2 `export` and `create` clauses

```
(export name ...)
(create name ...)
```

An `export` clause names variables that the module owns, which a definition in the module will create, and makes them visible to modules that use it. A `create` clause names variables that the module owns and exports but does *not* define: they are to be defined by a module that uses this one. That is how one module can publish an interface whose implementation lives in another. It is an error to name in either clause a variable the module imports. A name may appear more than once.

12.3.3 Rules

- The uses must not form a cycle: if `a` uses `b`, `b` may not use `a`, directly or through other modules.
- A module cannot use itself, and `dylan` cannot be redefined.
- A module may not define a variable it imports, with one exception: it may define a variable that the owning module named in a `create` clause. Adding a *method* to an imported generic function with `define-method` is not defining the variable; it is the normal way one module extends another's generic functions, and it is always allowed.

12.4 An example of each option

Design note 32 gives a set of four modules that between them use every option. Written in prefix form and cut down a little, they are:

```
(define-module graphics
  (use dylan)
  (create draw-line erase-line skew-line))

(define-module lines
  (use dylan)
  (use graphics import: (draw-line erase-line skew-line)))

(define-module rects
  (use dylan)
```

```
  (use graphics prefix: "g$" exclude: (skew-line)))

(define-module gx
  (use dylan export: all)
  (use graphics rename: ((skew-line => warp-line)) export: all))
```

- `graphics` owns and exports three variables and defines none of them.
- `lines` imports all three and supplies their definitions, which `create` allows.
- `rects` sees `g$draw-line` and `g$erase-line`, and not `skew-line` under any name.
- `gx` sees `draw-line`, `erase-line` and `warp-line`, and re-exports them together with everything in `dylan`. A module that says only `(use gx)` therefore gets the whole language plus the three graphics operations.

Given these declarations, the file for `lines` might read

```
module: lines

(define-method draw-line (from to)
  (display "draw ") (display from) (display "-") (display to) (newline))
(define-method erase-line (from to)
  (display "erase ") (display from) (display "-") (display to) (newline))
(define-method skew-line (line)
  (display "skew ") (display line) (newline))
```

a file in `rects` calls `(g$draw-line 1 2)`, and a file in a module that uses `gx` calls `(warp-line 'l1)`, which runs the `skew-line` method above and prints `skew l1`. Both implementations run this program as shown.

12.5 Which module code belongs to

Every expression is compiled in some module, and that module decides what each free name in it means. Design note 33 says how a source file names its module: by a *file header*, a set of lines of the form `keyword: value` at the very top of the file, ended by a blank line.

```
module: shapes
author: A. Programmer
        Example Software, Inc.
version: 1.2

(define-class <shape> (<object>))
...
```

The rules for the header are these.

- Each keyword starts at the beginning of a line. Keywords are not case-sensitive, so `Module:` is the same as `module:`.
- A line that begins with white space continues the value of the line before it.
- The first blank line ends the header. It is required: without it the header runs on into the code, and both implementations silently skip every line up to the first blank one.

- `module:` names the module. The other keywords (`author:`, `copyright:`, `version:` and any others) are read and ignored by both implementations.

A file without a header, one whose first line is not of the form `keyword: value`, is compiled in `dylan-user`. Design note 33 makes `module:` required; both implementations instead treat its absence as `module: dylan-user`, so that a one-file program needs no header at all.

A `define-module` form is itself compiled in some module, normally `dylan-user`, but the names in its clauses are not looked up there: they name variables of the module being declared or of the modules it uses. It is usual to put all the `define-module` forms of a program in one file with no header.

12.6 Modules in the compiler to C

The compiler to C compiles a whole program at once, so it sees every module before it compiles any code. Give `dylanc` all the files of the program:

```
$ bin/dylanc -o app modules.dyl shapes.dyl app.dyl
```

It first finds every top-level `define-module` form in every file, then defines each module after the modules it uses. The order of the `define-module` forms, and of the files that contain them, therefore does not matter. Then it compiles each file's forms in the module its header names. The top-level forms of all the files run in the order the files were given on the command line, and within each file from top to bottom.

Mistakes in modules are reported when compiling, with the file and line:

```
e4.dyl:2:1: error: define-module: modules m1 use each other
e5.dyl:1:1: error: unknown module nosuch
e7.dyl:1:1: error: define-module: cannot export car, which is imported
e8b.dyl:5:5: error: define of f: the variable is imported from module a
```

A name that is used in a module but defined nowhere it can see draws a warning, not an error, and an error only if the program actually reaches it:

```
e1b.dyl:1:8: warning: g is not defined (module b)
$ ./e1
Dylan error: the variable g@b has no value
```

As the message shows, the compiler writes a variable as *name*`@`*module* when the module matters. Here `g` was defined in module `a` but not exported, so the `g` in module `b` is a different, undefined variable.

The generated C is organized by module. With `-k` (keep the C), a program `app` with code in the modules `shapes` and `app` leaves `app.c` (data, dispatch and `main`), `app.h`, `app-shapes.c` and `app-app.c`, one file of functions for each module that has any, and `app-dylan.c`, the library's own functions. Chapter 15 describes these files and the compiler's other options.

12.7 Modules in the interactive implementation

The Common Lisp implementation compiles one form at a time, so a module must be declared before any code that uses it is compiled, and a module must be declared after the modules it uses:

```
? (define-module m1 (use dylan) (use m2))

debugger invoked on a DYLAN::DYLAN-CONDITION ...
  Error: unknown module m2
```

Declaring the modules in order is enough. Redefining a module with `define-module` replaces its clauses, and every module that uses it is brought up to date at once, as *DIRM* chapter 5 asks: variables newly exported become visible immediately in the modules that import them.

12.7.1 At the read-eval-print loop

The loop starts in `dylan-user`. Typing `(dylan:in-module` *name*`)` at the `?` prompt switches the module that later input is compiled in; from Lisp, the function `(dylan:in-module "`*name*`")`, given the name as a string, does the same. The loop answers with `;Module:` and the new module's name.

```
? (define-module counter (use dylan) (export next-count))
;Value: counter

? (dylan:in-module counter)
;Module: counter

? (define count 0)
;Value: count

? (define-method next-count () (set! count (+ count 1)) count)
;Value: next-count

? (dylan:in-module dylan-user)
;Module: dylan-user

? (define-module client (use dylan) (use counter))
;Value: client

? (dylan:in-module client)
;Module: client

? (next-count)
;Value: 1

? (next-count)
;Value: 2
```

Module `client` can call `next-count` but cannot see `count`, which `counter` keeps to itself. (Chapter 14 describes the loop in full.)

12.7.2 Files

`dylan:compile-file` reads a file's header and compiles the file in the module it names. Names are resolved to variables as the file is compiled, so the compiled Lisp needs no module information to run, and a compiled `define-module` form defines its module again when the file is loaded. To compile a program of several files, compile them in one Lisp session in the order a reader would need them, the file of `define-module` forms first, and then load the results in the same order:

```
$ sbcl --non-interactive --eval '(load "src/load-compiler.lisp")' \
    --eval '(dylan:compile-file "modules.dyl" "modules.lisp")' \
    --eval '(dylan:compile-file "shapes.dyl" "shapes.lisp")' \
    --eval '(dylan:compile-file "app.dyl" "app.lisp")'
$ sbcl --non-interactive --eval '(load "src/load-runtime.lisp")' \
    --eval '(load "modules.lisp")' --eval '(load "shapes.lisp")' \
    --eval '(load "app.lisp")'
```

Compiling `shapes.dyl` in a session that has not seen `modules.dyl` fails with "unknown module shapes". When the Lisp files are loaded, SBCL may print `WARNING: redefining ...` lines for the small accessor functions that each file declares for the module variables it shares with the others; they are harmless.

12.8 Read-only variables

DIRM Chapter 5 makes some module variables *read-only*: the variable always holds the same object, though that object may itself be changed. `define-class` and `define-generic-function` create read-only variables, so a class or generic function cannot be replaced by assignment. Variables made by `define` (or by `define-method` alone) are ordinary and can be assigned with `set!`.

The two implementations check this at different times. The compiler to C refuses the program:

```
e3.dyl:2:1: error: <p> holds a class or generic function and cannot be assigned
```

The interactive implementation signals an error when the assignment is run:

```
? (define-class <p> (<object>))
;Value: <p>

? (set! <p> 3)

debugger invoked on a DYLAN::DYLAN-CONDITION ...
  Error: variable is read-only <p>
```

Methods can still be added to a read-only generic function: the variable does not change, only the function in it.

12.9 A complete multi-file program

This program has three files. The first declares the modules; it has no header, so it is compiled in `dylan-user`.

```
;;; modules.dyl
(define-module shapes
  (use dylan)
  (export <shape> <circle> <square> area describe-shape))

(define-module app
  (use dylan)
  (use shapes import: (<circle> <square> (describe-shape => show))))
```

Module `shapes` exports five variables. Module `app` imports only two classes and one function, and calls the function `show`. It does not import `area` or `<shape>`.

```
;;; shapes.dyl
module: shapes
author: A. Programmer

(define-class <shape> (<object>))
(define-class <circle> (<shape>)
  (radius init-keyword: radius:))
(define-class <square> (<shape>)
  (side init-keyword: side:))

(define-generic-function area (shape))
(define-method area ((c <circle>)) (* 3 (radius c) (radius c)))
(define-method area ((s <square>)) (* (side s) (side s)))

(define-method describe-shape ((s <shape>))
  (display "a shape of area ")
  (display (area s))
  (newline))
```

The slot getters `radius` and `side` are owned by `shapes` but not exported, so no other module can read the slots directly.

```
;;; app.dyl
module: app

(show (make <circle> radius: 2))
(show (make <square> side: 3))
```

With the compiler to C:

```
$ bin/dylanc -o app modules.dyl shapes.dyl app.dyl
$ ./app
a shape of area 12
a shape of area 9
```

With the interactive implementation, compile the three files and load them as shown in Section 12.7; the output is the same.

If `app.dyl` also called `(area (make <square> side: 1))`, the compiler to C would warn that `area` is not defined in module `app`, and the program would stop with an error on reaching the call, because `app` did not import it.

A program meant to run in both implementations must keep to the names both provide; Appendix B marks the ones that only one of them has.

Chapter 13

Printing and Streams

The 1992 manual says remarkably little about output. Its examples print values with `print`, its chapter on conditions passes streams to functions and calls `format`, but it defines no printing functions and no streams at all. Both implementations fill the gap in the same way: with `print`, `display`, `write`, `write-line`, `newline` and `pp`; with streams and `format`; and with a generic function, `print-object`, through which a program decides how its own objects print. A program prints exactly the same in either implementation.

This chapter describes the printing functions first, then how each kind of object prints, then `print-object`, streams and `format`.

13.1 Two ways to print an object

Every printing function prints an object in one of two styles.

- *Escaped* output, the style of `print` and `write`, is meant to be read by a programmer, and where possible by the Dylan reader. Strings appear in double quotes with `"` and `\` escaped by a backslash, and characters appear as `#\a` or `#\space`.
- *Plain* output, the style of `display`, is meant for the person running the program. Strings and characters appear as themselves, with no quotes or `#\`.

The two styles differ only for strings and characters, and for collections and objects that contain them. A number, a symbol or a list of numbers looks the same either way.

13.2 The printing functions

Each of these answers its argument (`newline` answers `#f`), so a `print` can be wrapped around an expression without changing its value. Each takes a `stream:` keyword argument naming the stream to write to, which defaults to `*standard-output*` (Section 13.5).

`(print` *`object`* **`#key stream verbose)`** prints the object escaped, then a newline. It is a generic function, so a program can add methods to it, as *DIRM* chapter 13 does; `verbose:` is accepted and ignored.

`(write-line` *`object`* **`#key stream)`** does the same as `print`.

(pp *object* #key stream) *pretty-prints* the object: as `print` does, but laid out over lines as code (Section 13.2.1).

(write *object* #key stream) prints the object escaped, and nothing else.

(display *object* #key stream) prints the object plain, and nothing else: no newline, no space.

(newline #key stream) ends the line.

```
(print "a \"quoted\" string")     ; prints "a \"quoted\" string"
(display "a \"quoted\" string")   ; prints a "quoted" string
(print #\a)                       ; prints #\a
(display #\a)                     ; prints a
(display 42) (newline)            ; prints 42 and ends the line
(print 1) (display "x") (newline) ; prints 1 and x on two lines
```

13.2.1 Pretty-printing

`pp` writes what `print` writes, but breaks it over lines of at most 79 columns where it can. An object whose printed form fits on what is left of the line is written as `print` writes it. Otherwise a list that begins with a symbol has, after the symbol, as many of its elements as fit on the line, and each further element on a line of its own, indented two columns past the opening parenthesis; any other list has each element on a line of its own, one column past the parenthesis; and a vector has them two columns past the `#`.

```
(pp '(define-method foo ((x <integer>))
       (bind ((y (* x x)) (z (+ x x x x x x x x x x x x x x x x x x x x)))
         (list y z y z y z y z y z y z y z y z y z))))
```

prints

```
(define-method foo ((x <integer>))
  (bind ((y (* x x)) (z (+ x x x x x x x x x x x x x x x x x x x x)))
    (list y z y z y z y z y z y z y z y z y z)))
```

13.3 How objects print

The table below shows how each kind of object prints with `print`. Objects that cannot be read back print inside braces. Classes, generic functions, methods and quoted lists print as the 1992 manual's examples show them (*DIRM* chapters 3, 8, 10 and 11); the manual shows no printed form for the other objects in braces, so theirs are this system's own.

Object	Prints as
`42`, `-7`	`42`, `-7`
`3/4`	`3/4`
`1.5`, `9.0`, `0.1`	`1.5`, `9.0`, `0.1`
`1.0e20`, `(sqrt 2)`	`1.0e20`, `1.4142135623730951`
`#t`, `#f`, `'()`	`#t`, `#f`, `()`

Object	Prints as
`'foo`, `'Hello`	`foo`, `Hello`
`foo:`	`foo:`
`'(1 (2 3) . 4)`	`(1 (2 3) . 4)`
`''x`	`(quote x)`
`(vector 1 "two" #\3)`	`#(1 "two" #\3)`
a stretchy vector of 1 and 2	`{<stretchy-vector> 1 2}`
a deque of 1 and 2	`{<deque> 1 2}`
`(range from: 0 up-to: 5)`	`{range 0 by 1 size 5}`
`(range from: 0)`	`{range 0 by 1}`
a table mapping `a` to 1	`{table a => 1}`
a 2-by-2 array of zeros	`{<array> 0 0 0 0}`
the class `<point>`	`{the class <point>}`
an instance of `<point>`	`{instance of <point>}`
a generic function	`{the generic function print}`
`(make <generic-function> required: 1)`	`{an anonymous generic function}`
a method of the generic function `double`	`{a method on double}`
a predefined method, such as `+`	`{method +}`
any other method, such as `(method (x) x)`	`{an anonymous method}`
`(singleton 3)`	`{singleton 3}`
a slot descriptor	`{slot x}`, named by its getter
a stream	`{stream}`
a `<simple-error>`	its message, `bad thing`

Some details:

- **Floats** print in the shortest form that reads back as the same number, with at least one digit after the point (`9.0`, never `9.`). A float is written without an exponent when its magnitude is at least 10^{-5} and below 10^{16}, so `123456789.0` prints as `123456789.0` and `1e-7` as `1.0e-7` (Section 8.12).
- **Ratios** print as *numerator*`/`*denominator* in lowest terms; `(/ 6 4)` prints as `3/2`.
- **Symbols** print as they were first spelled. Dylan symbols are not case-sensitive, but the reader remembers the spelling a symbol was first entered in, so `'Hello` prints as `Hello` *DIRM* chapter 15. **Keywords** print with their colon.
- **Characters** with names print escaped by name: `#\space`, `#\newline`, `#\tab`, `#\return`, `#\page`, `#\null`, `#\rubout`, `#\backspace`, `#\escape`.
- **Lists and vectors** print their elements in the same style as the whole: `(display (list "a" #\b))` prints `(a b)`.
- **Conditions** print as their message, the format string filled in with the format arguments, as *DIRM* chapter 13 suggests. A `<type-error>` prints as "*value* is not an instance of *type*", where the type is printed as a class prints: `3 is not an instance of {the class <string>}`.

13.4 Customizing printing with `print-object`

Every printing function, `format` included, works by calling one generic function:

```
(print-object object stream escape?)
```

`print-object` writes *object* to *stream*; *escape?* is `#t` for escaped output (`print`, `write`, `~s`) and `#f` for plain output (`display`, `~a`). The library defines a method for each of its own classes, and a default method on `<object>` that prints `{instance of class}`. To make the instances of your own class print differently, add a method:

```
(define-class <point> (<object>)
  (x init-keyword: x: init-value: 0)
  (y init-keyword: y: init-value: 0))

(define-method print-object ((p <point>) stream escape?)
  (format stream "{point ~s, ~s}" (x p) (y p)))

(define p (make <point> x: 3 y: 4))
(print p)                       ; prints {point 3, 4}
(print (list p p))              ; prints ({point 3, 4} {point 3, 4})
(format #t "p is ~a~%" p)       ; prints p is {point 3, 4}
```

The method is used wherever a point is printed, including inside lists, vectors and other collections, and by `format`. Write the output to the *stream* argument, never to `*standard-output*` directly, or printing into a string will not work.

A method can honor the *escape?* argument, so that `print` shows something a programmer can recognize and `display` shows what a user should see:

```
(define-class <name> (<object>) (text init-keyword: text:))

(define-method print-object ((n <name>) stream escape?)
  (if escape?
      (format stream "#<name ~s>" (text n))
      (display (text n) stream: stream)))

(define n (make <name> text: "Ada"))
(print n)                       ; prints #<name "Ada">
(display n)                     ; prints Ada
(format #t "~a / ~s~%" n n)     ; prints Ada / #<name "Ada">
```

Methods are inherited and can call `next-method` like any other:

```
(define-class <temp> (<object>) (deg init-keyword: deg:))
(define-method print-object ((t <temp>) stream escape?)
  (display (deg t) stream: stream)
  (display " degrees" stream: stream))

(define-class <hot> (<temp>))
(define-method print-object ((t <hot>) stream escape?)
  (display "hot! " stream: stream)
  (next-method))

(print (make <hot> deg: 99))    ; prints hot! 99 degrees
```

`print-object` is an extension: the 1992 manual has no such function.

13.5 Streams

Both implementations provide a small set of streams, enough for the uses the 1992 manual makes of them. They are an extension: the manual's printing functions take a `stream:` argument, and its examples call `format`, but it defines no streams. There is no input: a program cannot read from a stream or a file.

`<stream>` The class of all streams.

`*standard-output*` A stream that writes to the program's standard output.

`*standard-error*` A stream that writes to the program's standard error.

`<string-stream>` A stream that collects what is written to it, in memory. You make one by calling `make`: `(make <string-stream>)`.

`(stream-contents` ***`string-stream`*****`)`** Answers a new string of everything written to the stream so far. The stream keeps its contents; calling `stream-contents` again answers them again, with anything written since.

Every printing function takes a `stream:` keyword argument naming the stream to write to. It defaults to `*standard-output*`, and `#t` (or `#f`) also means `*standard-output*`.

```
(define s (make <string-stream>))
(print 42 stream: s)
(display "hello" stream: s)
(write #\x stream: s)
(display (stream-contents s))
;; prints  42
;;         hello#\x

(display "warning: disk nearly full" stream: *standard-error*)
(newline stream: *standard-error*)
```

`<stream>` cannot usefully be subclassed by a program. The operation that puts characters on a stream is internal to the library, so printing to an instance of your own subclass of `<stream>` is an error.

13.6 `format`

```
(format stream control-string argument ...)
```

`format` writes *control-string* to *stream*, replacing each *directive*, a tilde and a letter, by output for the next argument. Its directives are a small subset of Common Lisp's, the ones the 1992 manual's examples use:

Directive	Output
`~a`	the next argument, plain, as `display` prints it
`~s`	the next argument, escaped, as `print` and `write` print it
`~d`	the next argument, plain (the same as `~a`)
`~%`	a newline
`~&`	a newline (the same as `~%`)
`~~`	a tilde

The letters may be upper or lower case. Any other tilde sequence is copied to the output unchanged. There are no field widths, no padding and no number bases: `~d` prints an integer exactly as `display` would, and `~&` always starts a new line, even at the beginning of one.

The *stream* argument is a stream, or one of two booleans:

- `#t` writes to `*standard-output*`, and `format` answers `#f`.
- `#f` writes to a new string, and `format` answers the string.

```
(format #t "~a + ~a = ~a~%" 1 2 (+ 1 2))    ; prints 1 + 2 = 3
(format #f "~a + ~a = ~a" 1 2 (+ 1 2))      ; => "1 + 2 = 3"
(format #f "~A ~S ~D ~z" "x" "x" 5)         ; => "x \"x\" 5 ~z"
(format *standard-error* "to stderr~%")
```

Too few arguments for the directives is an error ("`format: too few arguments for "~a and ~a"`"); arguments left over after the last directive are ignored.

13.7 Common Lisp's format

When you need more than Dylan's `format` offers, the interactive implementation can reach Common Lisp's own `format` with `lisp-procedure` (Section 14.6). It has every Common Lisp directive, field widths included. Pass `#t` for standard output, or the empty list, which is Lisp's `NIL`, to get a string back:

```
? (define cl-format (lisp-procedure 'format))
;Value: cl-format

? (cl-format #t "~5,2f|~8d|~r~%" 3.14159 42 12)
 3.14|      42|twelve
;Value: ()

? (cl-format '() "~a-~a" 1 2)
;Value: "1-2"
```

Important: Common Lisp's `format` prints Dylan values as Common Lisp sees them, not as Dylan's printer does: `#t` comes out as `T`, a keyword `foo:` as `FOO`, and `#f`, which is not a Lisp value at all, as `#f` only because the implementation teaches Lisp to print it so. Use it for numbers and strings. Do not pass `#f` as the destination: to Common Lisp it is an ordinary true object, not `NIL`.

Part III

The Implementations

Chapter 14

The Interactive Implementation

The interactive implementation runs Dylan inside a Common Lisp system, SBCL. You type a Dylan expression, it is compiled to Common Lisp at once, the Lisp compiles that to machine code and runs it, and the value is printed. Nothing about the program is fixed in advance: classes can gain slots, generic functions can gain and lose methods, and definitions can be replaced while the program runs, all as the 1992 manual describes. That makes it the natural place to learn the language, to try out an idea, and to follow the DIRM's examples.

This chapter describes how the implementation is built, how to load and use it, how Dylan values look to Lisp and how to call Lisp from Dylan, how it is made fast, and how it differs from the compiler to C.

14.1 How it works

The implementation began in 1992 as Thomas, a Dylan system written in Scheme at Digital Equipment Corporation's Cambridge Research Laboratory. This project translated it into Common Lisp and has since corrected it against the manual and rebuilt much of it for speed. Every source file in `src/` still names the Thomas file it grew from.

An expression goes through four stages.

1. **Reading.** Common Lisp's reader cannot read Dylan: a colon is a package marker to it, and it knows nothing of `#t` and `#f`. A small reader of Dylan's own (in `compat.lisp`) turns text into lists, symbols, numbers and strings. A Dylan symbol becomes a Lisp symbol in the `DYLAN` package; a keyword `foo:` becomes the Lisp keyword `:FOO`.

2. **Compiling to Lisp.** The compiler (`compiler.lisp` and the `comp-*.lisp` files) turns each Dylan form into a Lisp form: `begin` into `progn`, `set!` into `setq`, a method into a `lambda`, a call into a call through the runtime's dispatcher. Each module variable becomes a Lisp special variable with a name of its own, `!MODULE/X/` for a variable `x` of `dylan-user` and `!MODULE/X@M/` for one owned by module `m`.

3. **Compiling to machine code.** The Lisp form is handed to SBCL's own compiler, which turns it into native code.

4. **Running.** The code runs against the runtime library, the `runtime-*.lisp` files, which implement classes, generic dispatch, collections, numbers and conditions.

The files of `src/`, in the order they are loaded, do these jobs:

File	Role
`package.lisp`	the `DYLAN` package and its exported entry points
`globals.lisp`	declares the runtime's global variables, which nearly every file refers to before the file that defines them is loaded
`compat.lisp`	the Dylan reader and printer, weak tables, escapes, sorting and the interface to Common Lisp's condition system
`support.lisp`	small utilities shared by the compiler and runtime
`comp-util.lisp`	the compiler's utilities: names, variables, parameter lists
`comp-module.lisp`	modules, `define-module` and the `module:` file header (Chapter 12)
`compiler.lisp`	the heart of the compiler, the table of special forms, the predefined names, `compile-file` and `compile-to-file`
`comp-method.lisp`	`method`, `define-method`, `define-generic-function`
`comp-class.lisp`	`define-class` and `define-slot`
`comp-sf.lisp`	the other special forms: `bind`, `for`, `select` and the rest
`comp-exc.lisp`	`handler-bind`, `handler-case`, `bind-exit`, `unwind-protect`
`class.lisp`	classes, slots, instances and `make`
`generic.lisp`	generic functions and method dispatch, with its caches
`class-structure.lisp`	the predefined class heterarchy
`runtime-top.lisp`, `runtime-internal.lisp`	common parameter lists and the procedures compiled code calls
`runtime-methods.lisp`, `runtime-functions.lisp`	the predefined methods and functions
`runtime-extensions.lisp`	`lisp-variable`, `lisp-procedure`
`runtime-bitstrings.lisp`	`logand` and the other bit operations
`runtime-collections*.lisp`	collections: the iteration protocol, the generic operations, and one file for each kind (lists, vectors, strings, deques, ranges, tables, arrays)
`runtime-exceptions.lisp`	conditions, handlers and restarts
`runtime-printing.lisp`	streams, `print-object`, `format` and the other printing functions (Chapter 13)
`runtime-extended-float.lisp`	`<extended-float>` arithmetic (Chapter 8)
`runtime-fast-paths.lisp`	turns on the guarded fast paths (Section 14.7)
`rep.lisp`	the read-eval-print loop
`load-dylan.lisp`, `load-compiler.lisp`, `load-runtime.lisp`	loaders
`build-dylan.lisp`	saves the executable `bin/dylan`
`full-test.lisp`	the regression test driver

`src/README` describes the translation from Scheme in more detail.

14.2 Loading it

The usual way in is the executable `bin/dylan`. Running `make` in the repository's top directory builds it: `src/build-dylan.lisp` loads the whole system into SBCL and saves it, with SBCL, as one executable that starts in the read-eval-print loop.

```
$ make
$ bin/dylan

Entering Dylan read-eval-print-loop.
Leave by typing "(exit)"

?
```

The executable needs nothing else from the repository, so it may be copied or linked anywhere. It does not see later changes to `src/`; run `make` again to rebuild it. On Windows, without `make`, run `sbcl --script src/build-dylan.lisp`, which writes `bin\dylan.exe`.

The other way is to start a plain SBCL in the repository's top directory and load the system into it, which is how `make` itself begins:

```
$ sbcl
* (load "src/load-dylan.lisp")
;Dylan loaded.  (dylan:rep) starts a read-eval-print loop.
```

The first time, this compiles every source file into a *fasl* (Lisp's compiled-file format), which takes a little while. The fasls are kept in `src/fasl/`*`lisp-version`*`/`, for example `src/fasl/SBCL-2-6-3/`, one directory for each Lisp and version since their fasls are incompatible. Later loads use them directly. When a source file changes, it is recompiled together with every file loaded after it, since those may depend on its macros and inline functions.

Three loaders exist, for the three ways of using the system that Thomas offered:

`load-dylan.lisp` the interactive environment: the read-eval-print loop.

`load-compiler.lisp` the compiler to Lisp: `dylan:compile-file` and `dylan:compile-to-file`.

`load-runtime.lisp` what compiled programs need to run.

In this version the three load the same thing, the whole system; `load-compiler.lisp` and `load-runtime.lisp` simply load `load-dylan.lisp`. Use whichever names your purpose. A loader can be given on the command line as well:

```
$ sbcl --eval '(load "src/load-dylan.lisp")' --eval '(dylan:rep)'
```

14.3 The read-eval-print loop

`bin/dylan` starts the loop at once; in a Lisp with the system loaded, `(dylan:rep)` starts it. It prompts with `?`, reads one Dylan form, compiles and runs it, and prints the value after `;Value:`. A form may span several lines; the loop reads until the form is complete.

```
* (dylan:rep)

Entering Dylan read-eval-print-loop.
```

```
Leave by typing "(exit)"

? (define-method double ((n <number>)) (+ n n))

;Value: double

? (double 21)

;Value: 42

? (values 1 2 3)

;Value[1]: 1
;Value[2]: 2
;Value[3]: 3

? (values)


? dylan:done
DYLAN::|DYLAN:DONE|
*
```

A form that returns several values shows each on its own `;Value[`*n*`]:` line; one that returns none shows nothing. Values are printed as `print` prints them (Chapter 13), and output the form itself writes appears before the `;Value:` line. A definition answers the name it defined.

Typing `dylan:done`, or ending the input (Ctrl-D), leaves the loop. In `bin/dylan` that ends the program. In a Lisp you loaded the system into, it returns to Lisp's `*` prompt; the Lisp prints the symbol the loop returned. Everything the loop defined remains: call `(dylan:rep)` again and carry on where you left off. `dylan:done` may also be typed in parentheses, as `(dylan:done)`.

Typing `(exit)` quits altogether: it ends `bin/dylan`, and in a Lisp you loaded the system into, it ends the Lisp as well. It is a command of the loop, like `dylan:done`, not a Dylan function; a variable named `exit` is still evaluated when typed bare, and once your program has given `exit` a value, `(exit)` calls it instead. Typing `(dylan:in-module` *`name`*`)` at the `?` prompt changes the module later input is compiled in (Section 12.7).

Redefining things is normal at the loop. A second `define` of a variable replaces its value with a warning, a second `define-method` with the same specializers replaces the method, and a second `define-class` redefines the class in place:

```
? (define v 1)
;Value: v

? (define v 2)
WARNING: redefining variable v
;Value: v

? (define-method describe ((x <integer>)) "an integer")
;Value: describe

? (define-method describe ((x <integer>)) "a whole number")
;Value: describe

? (describe 3)
```

```
;Value: "a whole number"
```

From Lisp, `(dylan:empty-environment!)` makes the loop forget the module variables it has defined, and their values: a variable defined earlier is unbound afterwards, and using it is an error. It is rarely needed.

14.3.1 Errors and the Lisp debugger

An error that the Dylan program does not handle itself reaches SBCL's debugger. The debugger lists the *restarts* available, and the Dylan restarts in effect appear among them. For an error Dylan signals, the loop provides "Return to the read-eval-print loop", which abandons the form, prints `;Aborted` and gives you a new `?` prompt; for every error, whatever signalled it, it also provides `DYLAN-REP`, "Return to the Dylan read-eval-print loop", which does the same without the `;Aborted`:

```
? (define x 10)
;Value: x

? (error "Oops: ~a" x)

debugger invoked on a DYLAN::DYLAN-CONDITION in thread
#<THREAD tid=690943 "main thread" RUNNING {1200030003}>:
  Error: Oops: 10

Type HELP for debugger help, or (SB-EXT:EXIT) to exit from SBCL.

restarts (invokable by number or by possibly-abbreviated name):
  0: [DYLAN-RESTART] Return to the read-eval-print loop.
  1: [DYLAN-REP    ] Return to the Dylan read-eval-print loop.
  2: [ABORT        ] Exit debugger, returning to top level.

((LAMBDA NIL :IN DYLAN::IMPLEMENTATION-SPECIFIC/INDUCE-ERROR))
0] 0
;Aborted

? x
;Value: 10
```

While in the debugger you can use all of SBCL's debugger commands, such as `backtrace`; the frames show the Lisp the Dylan was compiled to.

A restart the program itself establishes is offered too. `cerror` makes one whose description is its first argument; choosing it makes `cerror` return `#f` and the program carry on:

```
? (cerror "Use 0 instead" "bad value ~a" 7)

debugger invoked on a DYLAN::DYLAN-CONDITION ...
  Error: bad value 7
...
restarts (invokable by number or by possibly-abbreviated name):
  0: [DYLAN-RESTART] Use 0 instead
  1:                 Return to the read-eval-print loop.
  2: [DYLAN-REP    ] Return to the Dylan read-eval-print loop.
  3: [ABORT        ] Exit debugger, returning to top level.
```

Calling `(abort)` from Dylan, or signaling any `<abort>`, returns straight to the prompt with `;Aborted`, as *DIRM* chapter 13 describes. `(break "look at ~a" 42)` enters the debugger with the message `Dylan breakpoint: look at 42`; its `CONTINUE` restart makes `break` return `#f`.

Note: Some errors are detected by Common Lisp itself rather than by the Dylan runtime, for example `(car 5)`, which reaches Lisp's `car` with a non-list. Such an error arrives in the debugger as a Lisp `TYPE-ERROR`, and the restarts offered are `DYLAN-REP` and Lisp's own:

```
restarts (invokable by number or by possibly-abbreviated name):
  0: [DYLAN-REP] Return to the Dylan read-eval-print loop.
  1: [ABORT    ] Exit debugger, returning to top level.
```

Choose `DYLAN-REP` to carry on at the `?` prompt. (In `bin/dylan`, `ABORT` also returns to the loop. In a Lisp you loaded the system into, it lands at the Lisp `*` prompt; nothing is lost: type `(dylan:rep)` to go back in.) A Dylan handler still catches these errors as instances of `<error>`: `(handler-case (car 5) ((<error>) 'caught))` answers `caught`.

14.3.2 Running a file at the loop

The loop reads from standard input, so `bin/dylan < fact.dyl` runs a file, printing each value as if it had been typed; at the end of the file the program exits. An unhandled error ends it there, with `Dylan error:` and its message on the standard error and status 1, and `break` prints its message and carries on, as in a compiled program, since the debugger would take the rest of the file for its commands. The loop has no command to load a file of Dylan. The regression test driver can do it: `dylan:test` reads a file form by form, echoes each form, runs it, and prints its value, or the message of an error, and goes on to the next form.

```
$ cd src
$ sbcl --eval '(load "load-dylan.lisp")' --eval '(load "full-test.lisp")' \
       --eval '(dylan:test "/path/to/fact.dyl")'

(define-method fact ((n <integer>))
 (if (= n 0)
     1
     (* n (fact (- n 1)))))
;Value: fact

(print (fact 30))
265252859812191058636308480000000
;Value: 265252859812191058636308480000000

(car 3)
;Error: not a list: 3
```

The definitions remain afterwards, so `(dylan:rep)` can follow in the same session. Note that `full-test.lisp` must be loaded after `load-dylan.lisp`.

14.4 Compiling Dylan to Lisp files

The compiler can also write its output to a file of Lisp instead of running it, so that a program can be compiled once and loaded later without being compiled again.

(dylan:compile-file *input output*) compiles the file of Dylan *input* into the file of Lisp *output*, in the module the input's `module:` header names, or `dylan-user`.

(dylan:compile-to-file *output form* ...) compiles Dylan forms, already read, into *output*.

`dylan:compile-file` shadows Common Lisp's function of the same name: inside the `DYLAN` package, `compile-file` compiles Dylan. The forms given to `dylan:compile-to-file` must be Dylan forms whose symbols are in the `DYLAN` package, which is what the Dylan reader `dylan::dylan-read` produces; Lisp forms typed at the `*` prompt would have their symbols in the wrong package.

Here is a complete round trip. The program:

```
;;; hello.dyl
(define-method greet ((name <string>))
  (display "Hello, ")
  (display name)
  (display "!")
  (newline))
(greet "world")
```

is compiled in one Lisp session and run in another:

```
$ sbcl --non-interactive --eval '(load "src/load-compiler.lisp")' \
       --eval '(dylan:compile-file "hello.dyl" "hello.lisp")'
$ sbcl --non-interactive --eval '(load "src/load-runtime.lisp")' \
       --eval '(load "hello.lisp")'
Hello, world!
```

The output file is ordinary Common Lisp. It begins with the Dylan it came from, as comments, and then the compiled forms:

```
(in-package #:dylan)

(defvar !module/greet/ '<<the-unassigned-value>>)
...
(dylan%catch-all-conditions
 (lambda ()
   (progn
    (progn
     (cond
      ((dylan%eq? !module/greet/ '<<the-unassigned-value>>)
       (setq !module/greet/ (dylan%create-generic-function 'greet 1 nil nil)))
      ...)
     (dylan%add-method !module/greet/ ...)
     'greet)
    (dylan%call nil (dylan%free-variable-ref !module/greet/ 'greet) "world"))))
```

`compile-to-file` is the way to compile forms you build in Lisp:

```
(dylan:compile-to-file "two.lisp"
  (dylan::dylan-read (make-string-input-stream "(define x 41)"))
  (dylan::dylan-read (make-string-input-stream "(print (+ x 1))")))
```

Loading `two.lisp` prints `42`. A program of several files is compiled file by file in one session and loaded in the same order (Section 12.7).

14.5 Dylan values as Lisp objects

Most Dylan values are the corresponding Lisp values, which is what makes calling between the two languages easy.

Dylan	Common Lisp
integers	integers, fixnums or bignums: there is no limit on size
ratios	ratios
floats	`double-float`s
complex numbers	complex numbers; `(make-rectangular 1 2)` is `#c(1 2)`
characters	characters
strings	Lisp strings
`<simple-object-vector>`	simple vectors
lists	lists; the empty list is `NIL`
`#t`	`T`
`#f`	a distinct object of its own, the value of `DYLAN::DYLAN%FALSE`
symbols	symbols in the `DYLAN` package
keywords	Lisp keywords: `foo:` is `:FOO`
functions	Lisp functions, which take two hidden leading arguments
classes, instances	structures of the runtime's own
tables, stretchy vectors, ranges, deques	Dylan instances

`#f` and `NIL`. Thomas ran on a Scheme in which false and the empty list were the same object. The 1992 manual makes them different: `#f` is the only false value, and the empty list is true *DIRM* chapters 3 and 8. This implementation follows the manual. The empty list is Lisp's `NIL`, because Dylan lists are Lisp lists, so `#f` has to be something else, an object created for the purpose. `#t` is Lisp's `T`. The consequence for Lisp code is that a Lisp "false" (`NIL`) is a Dylan empty list and therefore *true* in Dylan, and Dylan's `#f` is a non-`NIL` object and therefore *true* in Lisp.

Symbols. The Dylan reader interns each name in the `DYLAN` package, folded to upper case as Lisp's reader would, so Dylan's `foo` is the Lisp symbol `DYLAN::FOO`. (The names `t` and `nil` are kept in lower case, so as not to be Lisp's `T` and `NIL`.) Because the `DYLAN` package uses `COMMON-LISP`, a Dylan symbol whose name is that of a Common Lisp function or variable *is* the Common Lisp symbol: Dylan's `'expt` is `CL:EXPT`. The exceptions are the few names the package shadows: `add-method`, `adjoin`, `compile-file`, `find-method`, `last`, `reduce`, `set-difference` and `union`. The spelling a symbol was first typed in is remembered for printing.

14.6 Calling Common Lisp from Dylan

Two functions, extensions carried over from Thomas, reach into the host Lisp.

`(lisp-variable` ***symbol***`)` answers the value of the Lisp global variable named by *symbol*.

(lisp-procedure *symbol*) answers a Dylan function that calls the Lisp function named by *symbol* (or, if the symbol has no function, the function that is the symbol's value). The Dylan function accepts any number of arguments; the Lisp function reports the wrong number.

```
? (lisp-variable 'most-positive-fixnum)
;Value: 4611686018427387903

? (lisp-variable 'pi)
;Value: 3.141592653589793

? ((lisp-procedure 'expt) 2 100)
;Value: 1267650600228229401496703205376

? ((lisp-procedure 'string-upcase) "hello")
;Value: "HELLO"

? (define cl-sort (lisp-procedure 'sort))
;Value: cl-sort

? (cl-sort (list 3 1 2) '<)
;Value: (1 2 3)
```

Functions of your own written in Lisp can be called the same way. Define them in the `DYLAN` package, so that the Dylan symbol names them:

```
$ sbcl --eval '(load "src/load-dylan.lisp")' \
       --eval '(defun dylan::square (x) (* x x))' --eval '(dylan:rep)'

? (define square (lisp-procedure 'square))
;Value: square

? (square 12)
;Value: 144

? (square 1 2)

debugger invoked on a DYLAN::DYLAN-CONDITION ...
  Error: invalid number of arguments: 2
```

Keep these rules in mind:

- **Lisp's false is Dylan's true.** A Lisp predicate answers `NIL` for false, which Dylan sees as the empty list, a true value:

  ```
  (if ((lisp-procedure 'evenp) 3) 'yes 'no)    ; => yes
  ```

  Test the result against the empty list instead:

  ```
  (define lisp-true? (method (x) (not (id? x '()))))
  (lisp-true? ((lisp-procedure 'evenp) 3))      ; => #f
  ```

  Likewise, do not pass `#f` to a Lisp function that expects `NIL`; pass `'()`.

- **Dylan functions cannot be passed to Lisp.** A Dylan function, including one made by `lisp-procedure`, expects two hidden arguments that Lisp does not supply, so handing one to a Lisp function such as `mapcar` fails with "invalid number of arguments". Pass a symbol naming a Lisp function, as with `'<` above.
- **Lisp values print as Dylan sees them.** A Lisp value that has no Dylan meaning, such as a package or a hash table, prints in Lisp's notation.

14.7 How it is made fast

Thomas's authors did not aim at speed, and a direct translation of it was slow. The Common Lisp implementation has since been rebuilt around a few techniques, all of which keep the language's full dynamism.

- **Dispatch caches.** Each generic function caches, for each combination of argument classes it has seen, the sorted list of applicable methods, along with the keyword checks those methods need. Adding or removing a method, or changing a class's superclasses, invalidates the cache. A generic function of one, two or three required arguments calls the cached method directly without building an argument list, and calls with several applicable methods (`next-method` chains) and with keyword arguments go through the cache too. Singleton specializers are found through a hash table.
- **Direct calls.** A call compiles to a direct call of the function, without wrapping each argument in a closure as Thomas did. A method whose specializers are all `<object>` is called as its compiled body, without an argument-checking wrapper.
- **Guarded fast paths.** Calls of `+`, `-`, `*`, `<`, `>`, `<=`, `>=`, `=`, `modulo`, `remainder`, `element` and `(setter element)` compile inline when the arguments are Lisp numbers or characters, or a simple vector or string and an index. `for-each`, `map`, `reduce` and their kin walk lists, simple vectors and ranges directly. Each fast path is guarded by a flag that is true only while no method a program has added could apply to those arguments; add a method to `binary+` that could apply to integers, and the inline path for `+` turns itself off, so the language's meaning never changes.
- **Direct slot access.** Slots the runtime's own classes keep privately are read straight from the instance, and each class caches its slots' required init keywords for `make`.

`generic.lisp` holds the dispatcher and its caches; `runtime-fast-paths.lisp` turns the fast paths on once every predefined method exists. `bench/run.lisp` measures the implementation against the same programs written in plain Common Lisp; Chapter 18 gives the results.

14.8 The book's examples and the regression test

The file `dylan-examples.dyl` at the top of the repository holds the examples of the 1992 manual, extracted and edited into runnable form. `dylan:test-examples` runs it with `dylan:test` (Section 14.3.2):

```
$ cd src
$ sbcl --eval '(load "load-dylan.lisp")' \
       --eval '(load "full-test.lisp")' \
       --eval '(dylan:test-examples)'
```

The transcript it prints is compared with `src/reference/MIT.output`, the transcript Thomas produced from the same file on MIT Scheme in 1992 (the transcripts from Gambit and Scheme->C are kept beside it). The two agree on every value except a handful of places where the manual and Thomas disagree and the manual is followed: `#f` and `()` are different objects, floats print as `9.0` rather than `9.`, `(sqrt 25)` is `5.0`, `make` given a keyword that nothing accepts is an error (the book's `<displaced-view>` example does this, so it and the forms after it fail), and `map-into` passes only the source elements. `src/README` and Appendix C list them.

14.9 Differences from the compiler to C

The two implementations accept the same language, but each goes beyond the other in places.

What only the interactive implementation does. It is fully dynamic, as the 1992 manual allows:

- `add-method`, `remove-method`, `add-slot`, `remove-slot` and `freeze-methods` work. A program can add a slot to a class that already has instances:

```
? (define-class <sq> (<object>) (side init-keyword: side:))
;Value: <sq>

? (define s (make <sq> side: 2))
;Value: s

? (define-generic-function color (object))
;Value: color

? (add-slot <sq> getter: color init-value: 'red)
;Value: {slot color}

? (color s)
;Value: red
```

- New code can be compiled and run at any time, and any definition can be replaced (Section 14.3).
- Integers have no limit on size, and complex numbers exist: `(* 99999999999 99999999999)` is `9999999999800000000001`, and `(make-rectangular 1 2)` is `#c(1 2)`. The compiler to C has 63-bit integers, where overflow is an error, and no complex numbers.
- `lisp-variable` and `lisp-procedure` reach the host Lisp.

What only the compiler to C has. Threads (Chapter 16): `make-thread` and the rest, locks, condition variables, atomic integers, thread variables and `dynamic-bind` do not exist in the Common Lisp implementation. Appendix B marks every name that only one implementation provides.

What behaves differently.

- Modules must be declared before they are used, in order; the compiler to C sees the whole program first (Chapter 12).

- Errors are reported when the code runs; the compiler to C reports many of them, such as an assignment to a read-only variable, when it compiles.

14.10 Limitations

- There is no way to load a file of Dylan source at the read-eval-print loop itself; use `dylan:test` or compile the file with `dylan:compile-file` and load the result.
- Thomas's MIT Scheme restart example, written against MIT Scheme's condition system, has no counterpart; it would have to be rewritten against Common Lisp's.

Chapter 15

The Compiler to C

The second implementation in this system is a compiler. It reads a whole Dylan program, translates it into C, and runs a C compiler to make a native executable. The executable needs nothing from SBCL at run time: it is an ordinary program linked with a small runtime library written in C, with a garbage collector — the Memory Pool System, or by choice the Boehm-Demers-Weiser collector (Section 15.8) — and with the system's thread library.

This chapter is about using the compiler: the `bin/dylanc` command and every one of its options, the files it writes, the cache it keeps, the kinds of build it can make, how to call it from Common Lisp, what a compiled program does when something goes wrong, and what a program must not do because it is compiled whole and in advance. How the compiler works inside is the subject of Chapter 17, and how to make compiled programs fast is the subject of Chapter 18.

15.1 A first build

A program is one or more files of Dylan source, conventionally with the extension `.dyl`. Here is the smallest useful one:

```
(define-method greet ((name <string>))
  (format #t "Hello, ~a!~%" name))
(greet "world")
```

Compile it and run the result:

```
$ bin/dylanc hello.dyl
$ ./hello
Hello, world!
```

`bin/dylanc` is a short shell script that runs `compiler/dylanc.lisp` under `sbcl --script`, so SBCL must be on your `PATH`; the script can be run from any directory, and can be linked into a directory on your `PATH`. The executable is named after the first file, less its `.dyl`, and is written beside that file: `bin/dylanc src/hello.dyl` makes `src/hello`.

The top-level forms of the program run in order, from the first file to the last, when the executable starts. There is no `main` function to write: the program *is* its top-level forms, as it would be if you typed them into the interactive implementation's read-eval-print loop. A top-level form's value is not printed; use `print`, `format` and the other printing functions (Chapter 13) to produce output. When the last top-level form has run, the program exits with status 0.

Several files. Give several files to make one program from them all:

```
$ bin/dylanc -o app utilities.dyl shapes.dyl main.dyl
```

The files are read in the order given, and their top-level forms run in that order. A file whose first line is a `module:` header (design note 33; see Chapter 12) is compiled in the module it names; any other file is compiled in `dylan-user`. Every `define-module` form in every file is known before any other form is compiled, so a module may be defined after the files that use it.

What a first build costs. The compiler is SBCL code, compiled into `compiler/fasl/` the first time it is used, and the runtime library is compiled by `make` the first time a build needs it. After that, a small program builds in about a second and a half on a current machine. The first build with a new combination of C compiler and options takes several seconds longer, because the library's C must be compiled for it (Section 15.4).

15.2 The options

`bin/dylanc -h` lists the options:

```
Usage: dylanc [options] file.dyl ...

Compile the Dylan files, in order, into one executable.

Options:
  -o FILE          the executable (default: the first file's name, less .dyl)
  -O0 -O1 -O2 -O3  how much the C compiler optimizes (default -O2)
  -Os              a small executable: optimized for size, unused code dropped
                   (the library compiled with the program, as --whole)
  --whole          compile the library with the program, not apart (the
                   library's C, compiled apart, is compiled once and kept)
  --lto            optimize the program and the runtime library together when
                   linking (link-time optimization)
  -g, --debug      for debugging: no optimization, debugging information, and
                   every inline operation checking its arguments when run
  -k, --keep-c     keep the C: FILE.c, FILE-MODULE.c for each module, FILE.h
  -C, --c-only     write the C only (FILE.c and the rest); do not compile it
  -E, --expand     write each use of the program's macros and its expansion;
                   no C
  --trampoline     tail calls by trampolines, for C compilers without musttail
  --cc CC          the C compiler (default: $CC, or cc)
  --no-lines       no #line directives pointing to the Dylan source
  --sanitize KIND  for testing: a debug build with the sanitizers, KIND
                   address (AddressSanitizer and UndefinedBehaviorSanitizer)
                   or thread (ThreadSanitizer)
  --gc NAME        the garbage collector: mps (the default), the Memory Pool
                   System, generational and moving; or boehm, the installed
                   Boehm-Demers-Weiser collector (libgc)
  --gc-stress      for testing: a collection every DYLAN_GC_STRESS (default
                   100) allocations
  -h, --help       this
```

Options and file names may be mixed in any order. Each option is described below, with the occasions for using it.

`-o` *file* Names the executable. Without it the executable is named after the first file. The C files, when they are kept, are named after the executable: `-o build/app` writes `build/app.c` and the rest.

`-O0`, `-O1`, `-O2`, `-O3` How hard the C compiler optimizes. The default, `-O2`, is right for almost every use. `-O0` compiles fastest, which matters only for large programs while you are developing them; measure before assuming that `-O3` helps, since it makes larger code. An ordinary build is also compiled with `-g`, so a debugger can show the Dylan source even in an optimized program (see Section 15.6). The Dylan compiler's own optimizations (Chapter 17) are always on; these options affect only the C compiler.

`-Os` Makes a small executable. The library is compiled with the program, and only the parts of it the program can reach are kept (*tree shaking*); the C is compiled with `-Os`, each function and datum in its own section, the linker drops the sections nothing uses, and the executable is stripped of its symbols. Use it for programs you ship or copy elsewhere, when size matters more than build time. Section 15.5 shows what it gains.

`--whole` Compiles the library with the program, as `-Os` does, and shakes it the same way, but optimizes for speed as usual. Normally the library is compiled apart from the program, once, and kept (Section 15.4); with `--whole` its C is compiled afresh with every build. The result is smaller than an ordinary build, and a build takes about twice as long. It is mostly a check that the separately compiled library and the shaken one agree, which the test suites make (Appendix C).

`--lto` Link-time optimization: the C compiler optimizes the program and the C runtime library together when it links them, so that small runtime functions can be inlined into the program. The runtime library is built for this the first time it is asked for, with `-O2 -g -flto`, in `runtime/build/lto-cc/`. The effect on speed is usually small; try it on your own program and measure. It is ignored by `-g` and by the testing builds below.

`-g`, `--debug` A build for debugging: the C is compiled with `-O0 -g -DDY_DEBUG`, and the compiler trusts none of the types it infers, so every operation it would have done inline tests its arguments when the program runs. Section 15.6 has more.

`-k`, `--keep-c` Keeps the C files, which are otherwise deleted once the executable is made. Use it to read what the compiler wrote (Section 17.13), or to see what a debugger is showing you.

`-C`, `--c-only` Writes the C files and stops: no C compiler is run and no executable is made. Use it to compile the C elsewhere, with a build system of your own, or on a machine without SBCL. Section 15.3 shows the command that compiles the result by hand.

`-E`, `--expand` Writes each use of the program's own macros, with the place it occurs and what it expands into, and stops; no C is written. Use it to see what your macros do (Chapter 16).

`--trampoline` Makes tail calls with trampolines instead of the `musttail` attribute. Every Dylan call in tail position must run in constant stack space, and C does not promise that; GCC 15 and later and Clang 13 and later do, through `musttail`. With an older or different C compiler, `--trampoline` makes each tail call return to a loop that makes the call, which any C11 compiler can do. Programs built this way are correct but slower: a call-heavy benchmark (`tak`) took about 1.7 times as long. Without this option, a C

compiler that lacks `musttail` stops with the error "This C compiler cannot guarantee tail calls (musttail): define DY_TRAMPOLINE."

`--cc` ***cc*** The C compiler to run, such as `gcc` or `clang`, or a full path. Without it, the compiler is the value of the `CC` environment variable, or `cc` if that is not set. It is worth trying both GCC and Clang on a program whose speed matters.

`--no-lines` Leaves out the `#line` directives that point the C back to the Dylan source (Section 15.3). C compilers' messages, and debuggers, then refer to the C files themselves. Use it when you want to step through the generated C rather than the Dylan.

`--gc` ***name*** The garbage collector the program is built with: `mps`, the Memory Pool System, the default, which collects by generations and moves objects; or `boehm`, the Boehm-Demers-Weiser collector, which must be installed. See Section 15.8. The environment variable `DYLAN_GC=boehm` makes `boehm` the default.

`--sanitize` ***kind*** For testing the compiler and runtime: a debug build with *kind* `address` (AddressSanitizer and UndefinedBehaviorSanitizer) or `thread` (ThreadSanitizer). See Section 15.7.

`--gc-stress` For testing: a garbage collection every few allocations. See Section 15.7.

`-h, --help` Prints the summary above and exits with status 0.

When options conflict, the more specific build wins: a testing build (`--sanitize`, `--gc-stress`) uses its own C flags whatever else is given; otherwise `-g` wins over `-Os`, and `-Os` over `-O`*n*.

15.3 The files the compiler writes

For a program whose executable is *name*, the compiler writes these C files:

***name*`.c`** The program's static data — every class, symbol, keyword and constant, laid out as the runtime expects — with the object system's functions (the dispatch function of each generic function, slot accessors, and the makers `make` calls), and `main`.

***name*`-module.c`** The functions written in each module, one file for each module. A program written in `dylan-user` has ***name*`-dylan-user.c`**; a program with modules `counting` and `app` has ***name*`-counting.c`** and ***name*`-app.c`** too.

***name*`-dylan.c`** The library's functions: the module `dylan`, written in Dylan in `runtime/dylan/` and compiled with every program. This is the largest file by far, about a megabyte.

***name*`.h`** What the files share: a declaration of every object ***name*`.c`** defines and a prototype of every function.

The C is linked with `runtime/build/libdylan.a`, the runtime library compiled from `runtime/c/` with the collector in it, and with `-lm -lpthread` (with `--gc boehm`, the library is `runtime/build/boehm-`*cc*`/libdylan.a` and `-lgc` is linked too). With `-k` the files stay after the build:

```
$ bin/dylanc -k hello.dyl
$ ls hello*
hello  hello.c  hello.dyl  hello-dylan.c  hello-dylan-user.c  hello.h
```

`#line` directives. The generated C carries `#line` directives that name the Dylan file and line each statement came from, and after each function a directive back to the C file itself. A C compiler's warnings, a debugger's breakpoints and stack traces, and a profiler's reports therefore name places in your Dylan source. Here is the start of the function `hello-dylan-user.c` holds for `greet`:

```
D fn1190_greet(D self, D a0, D a1, D a2, D a3, intptr_t argc) {
    (void)self; (void)a0; (void)a1; (void)a2; (void)a3; (void)argc;
    D v105609, v105610, v105611;
#line 1 "hello.dyl"
    if (DY_UNLIKELY(argc != 1)) dy_wrong_argument_count(self, argc);
    v105609 = a0;
#line 2 "hello.dyl"
    ...
```

Compiling the C yourself. With `-C`, the compiler writes the C and stops. The files can then be compiled with the C compiler's options and the runtime library's directory:

```
$ bin/dylanc -C hello.dyl
$ cc -O2 -std=c11 -pthread -DDY_MPS \
     -I$DYLAN/runtime/c -I$DYLAN/runtime/mps/code \
     hello.c hello-dylan.c hello-dylan-user.c \
     $DYLAN/runtime/build/libdylan.a -lm -lpthread -o hello
$ ./hello
Hello, world!
```

Here `$DYLAN` stands for the root of the source tree. The runtime library `libdylan.a` must exist; `make -C runtime` makes it, and any build with `dylanc` makes it too. Add `-DDY_TRAMPOLINE` to the C compiler's options, and link `libdylan-trampoline.a` instead, if your C compiler lacks `musttail`. For the Boehm collector, leave out `-DDY_MPS` and the MPS include directory, link `runtime/build/boehm-cc/libdylan.a` (made by any `dylanc --gc boehm` build), and add `-lgc`. The C is C11 with POSIX threads; `runtime/c/dylan.h` checks what it assumes (64-bit pointers, arithmetic right shifts) with `_Static_assert`, and checks that the runtime it is compiled against is the version the compiler wrote for.

15.4 The library's cache

Every program carries the whole Dylan library, about a megabyte of C. To avoid compiling that with every build, the compiler writes the library's C so that it does not depend on the program: *name*`-dylan.c` is the same text for almost every program. The driver compiles it once for each C compiler and set of options and keeps the object in `runtime/build/library/`, named for a digest (MD5) of the C text, the C compiler's name and `--version` output, the options, and the runtime headers `dylan.h` and `primitive.h`. A later build that would compile the same text the same way uses the object instead. This halves the time of a small build.

The newest 40 objects are kept, with the C each was compiled from; older ones are deleted. The cache is safe to delete at any time (`rm -r runtime/build/library`); it is rebuilt as needed. Builds running at the same time share it safely.

A program can change the library's C — for example by defining a method that replaces one of the library's own. Then the digest differs and the library is simply compiled for that build. `-Os` and `--whole` do not use the cache at all: they compile the library with the program, keeping only what the program can reach.

15.5 Size

These are the sizes of the `greet` program above, built with GCC 16 on x86-64 Linux, as reported by `size` (the code, the initialized data and the uninitialized data, excluding the shared C and collector libraries):

Build	text + data (KB)	file size (KB)
default (`-O2`)	463	2765
`--whole`	307	1798
`--lto`	334	2374
`-Os`	221	231

The file sizes of all but the `-Os` build include debugging information. Hello world built `-Os` is about 220 KB of code and data; most of it is library code that a call of a generic function such as `print-object`, `make` or `as` could reach. Chapter 18 says more about size.

15.6 Debug builds

A program built with `-g` differs from an ordinary one in three ways:

- The C is compiled with `-O0 -g`, so a debugger can show every variable and step through every line.
- The compiler trusts none of the classes it has inferred. Where an ordinary build adds two integers with no test because it has proved them integers, a debug build tests them anyway. A debug build therefore catches a mistake in the compiler's reasoning as an error, rather than as a crash or a wrong answer.
- `DY_DEBUG` is defined, which makes the runtime's accessors check the representation of each value they take apart: that what is read as a pair *is* a pair, and so on. A violation stops the program with a message naming what was expected.

None of these change what a correct program does: a debug build reports the same Dylan errors, in the same way, as an ordinary one. Debug builds are for finding bugs in the compiler or runtime, and for using a debugger. Because of the `#line` directives, a debugger works in Dylan terms — breakpoints can be set at lines of the Dylan file. Given this `sum.dyl`,

```
(define-method sum-to ((n <integer>))
  (for ((i 0 (+ i 1))
        (s 0 (+ s i)))
       ((> i n) s)))
(print (sum-to 100))
```

a debug build stops where it is asked to:

```
$ bin/dylanc -g -o sum sum.dyl
$ gdb -batch -ex "break sum.dyl:3" -ex run sum
...
Thread 1 "sum" hit Breakpoint 1, fn1190_sum_to (self=4692864, a0=201,
    a1=0, a2=0, a3=0, argc=1) at /home/me/sum.dyl:3
3               (s 0 (+ s i)))
```

The names of the C functions are those the compiler gave the Dylan functions (`fn1190_sum_to` is the method `sum-to`), and the values are Dylan values in their machine representation: `a0=201` is the integer 100, stored shifted left one bit with the low bit set. Chapter 17 explains the representation.

15.7 Testing builds

Two options build programs for testing the compiler and runtime rather than for use. Each builds its own variant of the runtime library, the first time it is needed, in `runtime/build/`*`variant`*`-cc/`, with the same C flags as the program.

`--sanitize address` Compiles with `-O1 -g -fno-omit-frame-pointer -fsanitize=address,undefined -DDY_DEBUG`, as a debug build: AddressSanitizer reports any read or write outside an object, and UndefinedBehaviorSanitizer any undefined behavior in C, such as a signed overflow or a misaligned access.

`--sanitize thread` Compiles with `-O1 -g -fsanitize=thread -DDY_DEBUG`: ThreadSanitizer reports data races between threads.

`--gc-stress` Compiles with `-O2 -g -DDY_GC_STRESS`: the program forces a garbage collection every n allocations, where n is the value of the environment variable `DYLAN_GC_STRESS`, or 100. A pointer the collector cannot see — a bug in the compiler or runtime — shows up quickly as a crash or a wrong answer. Such a program runs very slowly.

```
$ bin/dylanc --gc-stress -o hgc hello.dyl
$ DYLAN_GC_STRESS=10 ./hgc
Hello, world!
```

One of AddressSanitizer's checks is turned off by the runtime itself: its detection of a frame used after its function returned, which keeps the frames of functions apart from the stack. The collector finds the references a program holds by scanning its stack, and would not see those frames; an object referred to only from one would be freed, or moved, while in use. The runtime supplies the option `detect_stack_use_after_return=0` as the sanitizer's default, so if you set `ASAN_OPTIONS` yourself, leave that one off.

The test suites use these builds (Appendix C). An ordinary Dylan program has no need of them, but a program that misbehaves in a way you cannot explain is worth running under them: what they find is a bug to report.

15.8 The garbage collector

A compiled program's memory is managed by one of two collectors, chosen when it is built.

`--gc mps` (the default) The Memory Pool System (MPS), Ravenbrook Limited's collector, written in the first place for Harlequin's Dylan. Its source is in the tree (`runtime/mps/`, under its own BSD license) and is compiled into the runtime library; nothing need be installed. It is *generational* and *moving*: new objects go in a nursery, a collection of the nursery copies the few that survive out of it and leaves the rest behind without visiting them, and the older generations are collected rarely. The stacks and the program's static data are scanned conservatively, so an object they point to stays in place; the heap itself is scanned exactly. Each thread allocates by bumping a pointer in its own region, without locking.

`--gc boehm` The Boehm-Demers-Weiser collector, `libgc`, which must be installed on the system (Section 2.1). It is *conservative*: it treats any word that looks like a pointer as one, so it needs no knowledge of the program's objects, and objects never move. It collects by marking and sweeping the whole heap, in parallel on two threads, and allocates from per-thread free lists. It runs on more platforms than the MPS, and is the choice where the MPS has no port.

The MPS is the faster. A program that makes many short-lived objects — lists built and discarded, boxed floats, strings, conditions — gains most, two to three times on such programs, since a collection costs in proportion to what survives, not to what was allocated; a program that keeps a large heap alive gains from the old generations being collected rarely; and a program that allocates little sees no difference. The price is memory: a program that allocates heavily uses about twice as much under the MPS, whose nursery is 128 MB and whose heap may double between collections of everything, and Section 18.6 tells how to trade that back. The two collectors are the same to the Dylan program, with one visible exception: under the MPS an object's identity hash is a number given it the first time it is hashed, not its address, since the address changes.

Every object of a program must be built for the same collector, and the runtime library for each is kept apart: the MPS one is `runtime/build/libdylan.a`, and the Boehm one is in `runtime/build/boehm-cc/`, with the variant and `--trampoline` in the name as for testing builds (Section 15.7). The testing builds work with either collector. Under the MPS, `--gc-stress` forces a collection of every generation, and a ThreadSanitizer build uses the MPS without its memory barrier, whose signal handling and the sanitizer's would deadlock, so its collections there are not incremental. Setting the environment variable `DYLAN_GC=boehm` makes the Boehm collector the default, which is how the test suites (Appendix C) are run with it. The MPS's behavior can be tuned by environment variables, in Section 18.6. It has been built here on Linux on x86-64 and macOS on Apple silicon, and has ports to FreeBSD, to Windows with Visual C, and to Linux on ARM64.

15.9 Calling the compiler from Lisp

The compiler is a Common Lisp program, and it can be called from a Lisp session. Load `compiler/load.lisp`, which loads the interactive implementation's reader and the compiler, compiling any source newer than its fasl; the package is `DYLAN-COMPILER`, nicknamed `DYLANC`. Two functions do the work:

`(dylanc:build` *`files executable`* `&key ...)` Compiles the list of Dylan *files* to C and on to the *executable*. The keyword arguments correspond to the options of `dylanc`: `:optimize`

(a string such as `"-O3"`, default `"-O2"`), `:size t` (`-Os`), `:whole t`, `:lto t`, `:debug t`, `:trampoline t`, `:cc` (default `"cc"`; the `CC` environment variable is not consulted here), `:variant` (`:address`, `:thread` or `:gc-stress`), `:gc` (`:mps` or `:boehm`), and `:c-file`, the name of the main C file (default *executable*`.c`). It answers `T` if the executable was made and `NIL` if not. Diagnostics are written to `*error-output*`. Unlike `dylanc`, it leaves the C files in place.

`(dylanc:compile-files` *`files c-file`*`)` Writes the C only, as `-C` does. It answers the list of C files written, or `NIL` after reporting errors.

```
(load "compiler/load.lisp")
(dylanc:build '("hello.dyl") "hello")              ; => T
(dylanc:build '("hello.dyl") "hello-small" :size t) ; => T
(dylanc:compile-files '("hello.dyl") "cf.c")
;; => (#P"cf.c" #P"cf-dylan.c" #P"cf-dylan-user.c")
```

Relative file names are taken relative to the Lisp's current directory (`*default-pathname-defaults*`).

15.10 Errors when compiling

The compiler reports problems as *diagnostics*, one to a line, on the standard error stream, in the form *file*`:`*line*`:`*column*`:` *kind*`:` *message*, where *kind* is `error` or `warning`:

```
$ cat r1.dyl
(print (undefined-thing 3))
$ bin/dylanc r1.dyl
r1.dyl:1:8: warning: undefined-thing is not defined (module dylan-user)
```

A **warning** does not stop the build. The commonest is a variable that is referred to but defined nowhere in the program; the program is built, and fails only if the reference is reached (see Section 15.11). Treat such a warning as an error unless you know why it is there: it is usually a misspelling.

An **error** stops the build: no executable is made and `dylanc` exits with status 1. Errors found while reading and converting the source — an unbalanced parenthesis, a malformed special form, an integer too large — are all reported in one run; the compiler abandons the top-level form that has the error and goes on to the next. Errors found later, when the program is analysed as a whole, stop at the first. For example:

```
$ cat err2.dyl
(define f (method (x) (+ x 1))
(print (f 1))
$ bin/dylanc err2.dyl
err2.dyl:2:14: error: End of file inside a Dylan list.
```

The position of a reading error is where the reader stopped, which for a missing closing parenthesis is the end of the file; the unclosed list began earlier.

Exit status. `dylanc` exits with status 0 when it has made what was asked; 1 when the program had errors or the C compiler failed ("`dylanc: the C compiler failed.`"); and 2

for a bad command line — an unknown option, an option missing its value, no files, or a file that does not exist — after printing the usage summary.

15.11 Errors when running

A compiled program signals conditions as Chapter 11 describes, and handlers established with `handler-case` or `handler-bind` see them as they would in the interactive implementation. The runtime's own errors — a wrong number of arguments, no applicable method, a type error, integer overflow, division by zero, an unbound variable — are signalled as Dylan conditions too, `<simple-error>` or `<type-error>`, and can be handled.

There is no debugger to fall into when a condition is not handled. Instead, the default handler (`default-handler`) acts as follows:

- An unhandled **serious condition** — any `<error>`, among others — writes `Dylan error:` and the condition's message to the standard error stream, and the program exits with status 1. The standard output is flushed first, so everything printed before the error appears.
- An unhandled **warning** writes `Dylan warning:` and its message to the standard error stream, and `signal` returns `#f`; the program continues.
- `break` writes `Dylan break:` and its message, and returns `#f`.
- `restart-query` and `return-query` do nothing.

```
(define f (method (x) (+ x 1)))
(print (f 1))
(print (f "one"))
(print 'never)
```

```
$ bin/dylanc err1.dyl && ./err1; echo "status $?"
2
Dylan error: no applicable method for binary+
status 1
```

The message names the generic function `binary+`: a call of `+` with two arguments calls the library's binary addition directly (Chapter 17). In a thread other than the main one, an unhandled serious condition ends only that thread; see Chapter 16.

Important: A recursion that is *not* in tail position uses C stack, and a program that recurses too deeply — millions of frames — overflows the stack and is killed by the operating system with a segmentation fault, not a Dylan error. Output it had printed but not yet flushed is lost. Calls in tail position never do this: they run in constant space however deep they go. Write deep recursions as tail calls, or raise the stack limit with `ulimit -s`.

15.12 What a compiled program cannot do

The compiler compiles a *static program*: everything the program defines is known when it is compiled, and its classes and generic functions do not change while it runs. That is what lets

the compiler lay out every class, compile every generic function's dispatch in advance, and call methods directly. The 1992 manual describes a dynamic language in which a running program may add methods and slots and make new classes, and the interactive implementation allows all of that. A compiled program does not. The restrictions, and how each shows itself, are these.

Changing classes and generic functions. `add-method`, `remove-method`, `add-slot`, `remove-slot` and `freeze-methods` cannot be used. Any use of them — even one that would never run — is a compile error:

```
$ cat s.dyl
(define-generic-function g (x))
(add-method g (method ((x <integer>)) x))
$ bin/dylanc s.dyl
s.dyl:2:1: error: add-method cannot be used: a program's classes and
generic functions are fixed when it is compiled
```

(The message is on one line; it is broken here to fit the page.) Define methods with `define-method` instead, which adds them when the program is compiled. A class cannot be defined twice ("`<a> is defined twice (classes cannot be redefined)`").

Specializers and superclasses known when compiling. Each specializer of a method made by `define-method`, and each superclass of a class, must be something the compiler can find: a class named directly, a variable defined as a constant class, `(singleton` *x*`)`, or `(make <singleton> object:` *x*`)` of a constant. A variable whose value is computed when the program runs is a compile error:

```
(define c (if (> 2 1) <integer> <string>))
(define-method m ((x c)) x)
; error: a specializer must be a class or a singleton known at compile time
```

while this is accepted, because `c` is defined as a class:

```
(define c <integer>)
(define-method m ((x c)) 'yes)
(print (m 3))        ; prints yes
```

The owner of a slot added with `define-slot` may be any singleton, but a class owner must be known when compiling.

Making classes and generic functions when running. `(make <class> ...)` is an error when it runs ("cannot make an instance of this class"). `(make <generic-function> ...)` answers a generic function with no methods; since no method can be added, every call of it signals "no applicable method". `seal` works as the manual says, and `make-read-only` answers its argument.

Numbers. Integers are 63 bits: from -2^{62} to $2^{62}-1$, that is from `-4611686018427387904` to `4611686018427387903`. There are no bignums. An integer literal outside that range is a compile error; an operation whose result falls outside it signals an error when it runs:

```
(print (* 4611686018427387903 2))
; Dylan error: integer overflow (integers are 63 bits)
```

The same overflow rule applies to the numerator and denominator of a ratio. `<complex>` exists, as the abstract superclass of `<real>`, but has no instances, and a complex literal such as `#c(1 2)` is a compile error. The interactive implementation has both bignums and complex numbers (Chapter 8).

No evaluation at run time. A compiled program cannot evaluate or load code while it runs: there is no `eval` and no `load`. Any code it runs must be in its source when it is compiled. (Macros, which run at compile time, can compute code; see Chapter 16.)

Undefined variables. A reference to a variable that nothing defines is only a warning when compiling, as shown in Section 15.10. If the reference is reached, it is an error when the program runs:

```
$ ./r1
Dylan error: the variable undefined-thing@dylan-user has no value
```

Appendix A collects these restrictions with every other difference from the 1992 manual.

Chapter 16

Macros and Threads

The 1992 manual defines a language without macros and without threads. This system adds both: macros in both implementations, threads in the compiler to C alone. They are *extensions*: a program that uses them is not a program of the 1992 language. Everything in this chapter applies to programs compiled with `dylanc`; the section on macros applies to the interactive Common Lisp implementation too, with the one difference Section 16.1.4 describes. (That implementation has extensions of its own, described in Chapter 14.)

16.1 Macros

A *macro* is a function that runs while the program is being compiled. It receives the forms of a call, unevaluated, and answers a new form, its *expansion*, which the compiler compiles in place of the call. Macros let you add new syntax to the language: a new looping form, a definition form that writes several definitions, a form that evaluates its argument in a special way.

The macros here are Common Lisp's kind. If you know `defmacro`, you know them already: the parameters are bound to the argument forms, the body builds the expansion, usually with backquote, and nothing protects the expansion's names from the names around the call except the care of the macro's writer and `gensym`.

The manual calls `and`, `or`, `cond`, `case`, `for` and the other syntax of Chapter 5 macros. In this system those are built into both implementations, not written with `define-macro`, and the program cannot redefine them: a `define-macro` of one of their names is accepted without complaint, but a call of the name still means the built-in form.

16.1.1 `define-macro`

`(define-macro` *name parameter-list form ...*`)`

at top level makes *name*, a variable of the current module, a macro. From then on, a call `(`*name argument ...*`)` is replaced, while compiling, by the value of the *form*s, run with the parameters bound to the argument forms. The parameter list is a method's: required parameters, `#rest` and `#key` are all allowed.

```
(define-macro my-unless (test #rest body)
  `(if ,test #f (begin ,@body)))

(my-unless (= 1 2)
  (print 'one)
```

```
    (print 'two))
(print (my-unless #t 'never))
```

prints

```
one
two
#f
```

Here `test` is bound to the form `(= 1 2)` and `body` to the list of the two `print` forms; the backquote template builds an `if` form from them. Backquote follows the usual Lisp rules (`,` inserts a value, `,@` splices a list, and backquotes may nest); see Appendix A.

Because the argument forms arrive unevaluated, a macro can use a form both as data and as code:

```
(define-macro show (form)
  `(format #t "~s => ~s~%" ',form ,form))

(show (+ 1 2))
(show (reverse '(a b c)))
```

prints

```
(+ 1 2) => 3
(reverse (quote (a b c))) => (c b a)
```

A macro's body is ordinary Dylan, and it may compute its expansion any way it likes; it need not use backquote, and its result need not even be a list. Keyword parameters work as in a method:

```
(define-macro scaled (#key (scale 1) value)
  (* scale value))
(print (list (scaled value: 5) (scaled scale: 3 value: 5)))
; prints (5 15)
```

Here the multiplication happens when the program is compiled; the expansions are the constants `5` and `15`.

Macros may expand into uses of other macros, or of themselves; the implementation expands again until no macro call is left. An expansion may also be a definition, or a `begin` of several, at top level. A macro that never stops expanding is reported ("one never stops?").

16.1.2 Rules

- `define-macro` must be at top level. Inside a method it is an error ("define-macro must be at top level").
- A macro must be defined before its first use in the program. A call that comes earlier is compiled as a call of a variable that is not defined, and fails when it runs; the compiler warns of it.
- A name cannot be both a macro and a variable. `(define m 2)` after `(define-macro m () 1)`, or a use of `m` other than as the operator of a call, is an error ("m is a macro"). In the interactive implementation a `define-macro` may replace a macro, as a `define` may replace a variable.

- A call of a macro must be a proper list, and must supply arguments its parameter list accepts ("macro m: wrong number of arguments").

- An error signalled by a macro's body is reported at the call, with the macro's name: `(define-macro m (a) (error "no good: ~s" a))` and a call `(m 1)` report "`macro m: no good: 1`".

16.1.3 Names in expansions: `gensym` and capture

The names in an expansion mean what they mean *where the macro is used*. This is simple and predictable, but it means that the expansion and the code around it can interfere. A macro that binds a variable of its own risks capturing a variable of the same name at the call:

```
(define-macro bad-swap! (a b)
  `(bind ((tmp ,a)) (set! ,a ,b) (set! ,b tmp)))

(bind ((tmp 1) (other 2))
  (bad-swap! tmp other)
  (print (list tmp other)))
; prints (1 2): nothing was swapped
```

The expansion binds a new `tmp`, and `(set! tmp other)` then assigns that one, not the caller's. The cure is `(gensym)`, which answers a fresh symbol that no program can write, for the expansion to bind:

```
(define-macro swap! (a b)
  (bind ((tmp (gensym "tmp")))
    `(bind ((,tmp ,a))
       (set! ,a ,b)
       (set! ,b ,tmp))))

(define tmp 1)
(define other 2)
(swap! tmp other)
(print (list tmp other))      ; prints (2 1)
```

`gensym` takes an optional string, the start of the new name, and appends a number. The name only *looks* like one a program could write: `dylanc -E` shows it as `tmp1`, but it is not the symbol `tmp1`, and nothing in the program can refer to it.

Capture can happen the other way too: an expansion that calls a function by name gets whatever that name means at the call. If the caller has a local variable of that name, the expansion uses it:

```
(define-macro pair-of (x) `(list ,x ,x))
(print (pair-of 1))                    ; prints (1 1)
(bind ((list vector))
  (print (pair-of 2)))                 ; prints #(2 2)
```

There is no protection against this short of not binding, near a macro's use, the names its expansions call. The built-in syntax — `for`, `case`, `with-lock` and the rest — is not exposed to either kind of capture: its expansions refer to the library's variables directly and make their temporaries with names no program can write.

A local variable or parameter with the name of a macro hides the macro, as it would hide a

module variable:

```
(define-macro double (x) `(* 2 ,x))
(print (double 4))                                  ; prints 8
(define apply-it (method (double) (double 4)))
(print (apply-it (method (n) (+ n 100))))           ; prints 104
```

16.1.4 The compile-time environment and `define-for-macros`

Macro bodies run in the interactive Common Lisp implementation of Dylan (Chapter 14), which the compiler is loaded with. Under the compiler they run in a *compile-time environment* of their own, separate from the program being compiled. It holds the whole library of the interactive implementation, `gensym`, and whatever `define-for-macros` has defined — but nothing the program itself defines. A macro body that calls one of the program's functions fails:

```
(define helper (method (x) (list x x)))
(define-macro use-helper (x) (helper x))
(print (use-helper 3))
; error: macro use-helper: unbound variable helper
```

Functions and variables for macro bodies to use are defined with

```
(define-for-macros form ...)
```

at top level, which runs the *form*s in the compile-time environment while compiling, and leaves nothing in the compiled program. A variable defined there keeps its value from one expansion to the next. Here a helper builds the names of accessors for a macro that defines a class:

```
(define-for-macros
  (define accessor-name
    (method (class-name slot)
      (as <symbol>
          (concatenate (as <string> class-name) "-" (as <string> slot))))))

(define-macro define-record (name #rest slots)
  `(define-class ,name (<object>)
     ,@(map (method (s)
              `(,(accessor-name name s)
                init-keyword: ,(as <keyword> (as <string> s))))
            slots)))

(define-record point x y)
(bind ((p (make point x: 3 y: 4)))
  (print (list (point-x p) (point-y p))))   ; prints (3 4)
```

Two consequences of running macro bodies in the interactive implementation are worth knowing. First, under the compiler, `gensym` and anything `define-for-macros` defines exist only at compile time; a compiled program cannot call `gensym`. Second, where the two implementations differ, a macro body sees the interactive one: it has bignums, for instance (Appendix A).

The interactive implementation itself has no separate environment: it compiles and runs a program a form at a time, and a macro's body runs, while the forms after its definition are compiled, in the same world as the program. Everything defined so far is visible to it, the program's own functions included, and what `define-for-macros` defines is as visible to

the program as any other definition; the forms of a `define-for-macros` run once, as they are compiled, and are left out of the compiled code, as under the compiler. (`dylan:compile-file` compiles a whole file before any of it runs, so there, as under the compiler, a macro body can use only what `define-for-macros` has defined.) A program meant to run both ways should keep to what works under the compiler: helpers for macros in `define-for-macros`, and none of them called by the program itself.

16.1.5 Macros and modules

A macro is a variable of the module it is defined in, and it is exported, imported, renamed and excluded exactly as a variable is (Chapter 12). Here the module `macros` defines and exports `repeat`, and `user` uses it:

```
; macmods.dyl
(define-module macros (use dylan) (export repeat))
(define-module user (use dylan) (use macros))


; macdef.dyl
module: macros

(define-macro repeat (n #rest body)
  (bind ((i (gensym)))
    `(dotimes (,i ,n) ,@body)))


; macuse.dyl
module: user

(repeat 3 (print 'hi))


$ bin/dylanc -o macs macmods.dyl macdef.dyl macuse.dyl
$ ./macs
hi
hi
hi
```

(The first lines of `macdef.dyl` and `macuse.dyl` are the `module:` headers; the comments above them are not part of the files.) The names in an expansion are looked up in the module where the macro is *used*, so `dotimes` must mean the same there as in the macro's module — here both use `dylan`.

16.1.6 Seeing expansions: `dylanc -E`

`dylanc -E` writes each use of the program's own macros, where it occurs, and its expansion, and makes no C. It is the way to debug a macro:

```
$ bin/dylanc -E m1.dyl
m1.dyl:4:1: (my-unless (= 1 2) (print 'one) (print 'two))
  => (if (= 1 2) #f (begin (print 'one) (print 'two)))
m1.dyl:7:8: (my-unless #t 'never)
  => (if #t #f (begin 'never))
m1.dyl:12:1: (show (+ 1 2))
  => (format #t "~s => ~s~%" '(+ 1 2) (+ 1 2))
```

```
m1.dyl:13:1: (show (reverse '(a b c)))
  => (format #t "~s => ~s~%" '(reverse '(a b c)) (reverse '(a b c)))
```

Only the program's macros are shown, not the built-in syntax. Each step of a macro that expands into another macro is shown separately. The interactive implementation has no equivalent; there, a macro's expansion can be seen by calling its body's logic as an ordinary function on quoted forms.

16.2 Threads

A compiled program can run several *threads* at once. Each Dylan thread is a POSIX thread, so threads really do run in parallel on a machine with several processors. The garbage collector is built for threads and collects while they all allocate. Threads share the program's module variables and every object they can reach; each has its own stack, its own handlers and exits, and its own values of thread variables (Section 16.2.7).

The threads API is small. Table 16.1 lists it; the sections after it explain each part.

Form	What it does
`(make-thread` *`function`*`)`	start a thread calling *function*, which takes no arguments; answer the `<thread>`
`(join-thread` *`thread`*`)`	wait for *thread* to end; answer its function's values
`(current-thread)`	the `<thread>` running
`(thread-yield)`	let other threads run; answer `#f`
`(make <lock>)`, `(make <recursive-lock>)`	a new lock
`(acquire-lock` *`lock`*`)`, `(release-lock` *`lock`*`)`	hold and let go of a lock
`(with-lock (`*`lock`*`)` *`form ...`*`)`	the forms, holding the lock
`(make <condition-variable>)`	a new condition variable
`(wait-condition-variable` *`cv lock`*`)`	release the lock, wait to be notified, acquire it again
`(notify-condition-variable` *`cv`*`)`	wake one waiting thread
`(notify-all-condition-variable` *`cv`*`)`	wake every waiting thread
`(make <atomic-integer> value:` *`n`*`)`	an integer cell for atomic updates (*n* defaults to 0)
`(atomic-value` *`a`*`)`	its value
`(atomic-add!` *`a n`*`)`	add *n*; answer the new value
`(atomic-compare-and-swap!` *`a old new`*`)`	if it holds *old*, store *new* and answer `#t`; else `#f`
`(define-thread-variable` *`name value`*`)`	a module variable with a value for each thread
`(dynamic-bind ((`*`name value`*`) ...)` *`form ...`*`)`	the forms, with thread variables given new values in this thread
`<thread-error>`, `thread-error-thread`, `thread-error-condition`	what `join-thread` signals when a thread ended with an unhandled condition

Table 16.1: The threads API.

16.2.1 Making and joining threads

`(make-thread` *`function`*`)` starts a new thread that calls *function* with no arguments, and answers at once with the `<thread>` object. `(join-thread` *`thread`*`)` waits until that thread's

function returns, and answers the values it returned. A thread may be joined more than once, and by any thread but itself; each join answers the same values.

This program sums the integers below a million in four threads, each summing a quarter, and adds the four results:

```
(define sum-range
  (method (from below)
    (for ((i from (+ i 1))
          (s 0 (+ s i)))
         ((= i below) s))))

(define workers
  (map (method (k)
         (make-thread
           (method () (sum-range (* k 250000) (* (+ k 1) 250000)))))
       '(0 1 2 3)))

(print (reduce + 0 (map join-thread workers)))
; prints 499999500000
```

The threads run in whatever order the operating system chooses, but the program's output does not depend on it: each thread computes alone, and the main thread waits for all four before it adds. Write threaded programs this way where you can — threads that share nothing while they work and are joined to collect their results.

A thread's function may return several values, and `join-thread` answers them all:

```
(define t (make-thread (method () (values 1 2 3))))
(print (bind ((a b c (join-thread t))) (list a b c)))   ; prints (1 2 3)
```

`(current-thread)` answers the thread that calls it. In the main thread it answers a `<thread>` too, but the main thread cannot be joined. `(thread-yield)` asks the operating system to run other threads before this one continues; it answers `#f`. A thread prints as `{thread}`.

16.2.2 When the program ends

The program ends when the main thread has run the last top-level form, *whatever its other threads are doing*. It does not wait for them; they simply stop. This program prints `main-is-done` and exits with status 0, although its other thread would loop forever:

```
(define spinner
  (make-thread
    (method ()
      (bind-methods ((spin () (thread-yield) (spin)))
        (spin)))))
(print 'main-is-done)
```

Join every thread whose work must finish. Equally, an unhandled error in the main thread ends the whole program at once, with the other threads.

16.2.3 Unhandled conditions in threads

A serious condition that a thread does not handle ends *that thread only*. The program goes on. The condition is kept with the thread, and `join-thread` on it signals a `<thread-error>` — a

subclass of `<error>` — in the joining thread. `thread-error-thread` answers the thread that ended, and `thread-error-condition` the condition that ended it:

```
(define worker (make-thread (method () (error "disk ~a is full" 3))))

(handler-case (join-thread worker)
  ((<thread-error> condition: e)
   (bind ((original (thread-error-condition e)))
     (format #t "worker failed: ~a ~s~%"
             (condition-format-string original)
             (condition-format-arguments original))
     (print (id? (thread-error-thread e) worker)))))
```

prints

```
worker failed: disk ~a is full (3)
#t
```

If the `<thread-error>` is not handled either, it ends the joining thread in turn — in the main thread, the program — with a message that carries the original one:

```
Dylan error: a thread ended with an unhandled condition: disk 3 is full
```

A thread that is never joined ends silently when a condition is not handled in it. Warnings are not serious conditions: an unhandled warning in a thread is reported on the standard error stream, as in the main thread, and the thread goes on.

16.2.4 Locks

Threads that change shared data must take turns, or their changes will be lost. A *lock* lets one thread at a time hold it. `(make <lock>)` makes one; `(acquire-lock` *`lock`*`)` waits until no other thread holds it and then holds it; `(release-lock` *`lock`*`)` lets it go. Usually you will use

```
(with-lock (lock) form ...)
```

which holds the lock while the forms run and releases it however they are left — by returning, by a non-local exit, or by an error — and answers the forms' values. Three threads each adding 100 000 to a shared counter:

```
(define counter 0)
(define counter-lock (make <lock>))
(define bump
  (method ()
    (dotimes (i 100000)
      (with-lock (counter-lock)
        (set! counter (+ counter 1))))))
(define threads (list (make-thread bump) (make-thread bump)
                      (make-thread bump)))
(for-each ((t threads)) () (join-thread t))
(print counter)                                  ; prints 300000
```

Without the lock, two threads can read the same old value of `counter` and both store one more than it, and the total comes out short.

A `<lock>` checks its use. Acquiring a `<lock>` the thread already holds would wait forever, so it is an error instead ("acquire-lock: the thread holds the lock already"), and so is releasing a lock the

thread does not hold ("release-lock: the thread does not hold the lock"). A `<recursive-lock>` — a subclass of `<lock>` — may be acquired again by the thread holding it, and is released when it has been released as many times:

```
(define r (make <recursive-lock>))
(print (with-lock (r) (with-lock (r) 'nested)))   ; prints nested
```

Neither kind of lock takes init-keywords.

16.2.5 Condition variables

A thread that must wait until another thread has done something — put an item in a queue, say — waits on a *condition variable*. `(wait-condition-variable cv lock)`, called while holding *lock*, which must be a `<lock>` (not a `<recursive-lock>`), releases the lock and waits; when another thread calls `(notify-condition-variable cv)`, one waiting thread wakes, acquires the lock again, and returns. `notify-all-condition-variable` wakes every waiting thread.

A wait can also end without a notification, and another thread may get to the data first. So always wait in a loop that tests what is awaited, with the lock held. Here a consumer thread takes ten numbers from a queue that the main thread fills:

```
(define lock (make <lock>))
(define not-empty (make <condition-variable>))
(define queue (make <deque>))

(define put!
  (method (item)
    (with-lock (lock)
      (push-last queue item)
      (notify-condition-variable not-empty))))

(define take!
  (method ()
    (with-lock (lock)
      (while (empty? queue)
        (wait-condition-variable not-empty lock))
      (pop queue))))

(define consumer
  (make-thread
    (method ()
      (for ((n 0 (+ n 1))
            (total 0 (+ total (take!))))
           ((= n 10) total)))))

(dotimes (i 10) (put! (* i i)))
(print (join-thread consumer))                  ; prints 285
```

However the two threads interleave, the consumer takes the ten squares and answers their sum.

16.2.6 Atomic integers

For a counter or a flag, a lock is more than is needed. An `<atomic-integer>` holds an integer that threads can update without one. `(make <atomic-integer> value: n)` makes one holding *n* (0 if `value:` is not given); `atomic-value` reads it; `(atomic-add! a n)` adds *n* and answers the new value, as one indivisible step; and `(atomic-compare-and-swap! a old new)` stores *new* only if *a* still holds *old*, answering `#t` if it did and `#f` if not.

```
(define hits (make <atomic-integer>))
(define bump-atomically
  (method () (dotimes (i 100000) (atomic-add! hits 1))))
(do join-thread (list (make-thread bump-atomically)
                      (make-thread bump-atomically)))
(print (atomic-value hits))                          ; prints 200000
(print (atomic-compare-and-swap! hits 200000 0))     ; prints #t
(print (atomic-compare-and-swap! hits 200000 0))     ; prints #f
(print (atomic-add! hits 5))                         ; prints 5
```

The values must be integers, and an addition that overflows the 63-bit range signals an error, as ordinary arithmetic does.

16.2.7 Thread variables and `dynamic-bind`

```
(define-thread-variable name value)
```

at top level defines a module variable that has a separate value in each thread. Reading it, and assigning it with `set!`, affect only the current thread's value. The main thread starts with *value*, and so does every thread made after the definition has run — a new thread does *not* inherit its creator's current value.

```
(dynamic-bind ((name value) ...)  form ...)
```

gives each thread variable *name* the new *value* in the current thread while the forms run, and restores the old value however they are left. This is how such variables are normally used: as settings that a computation and everything it calls can see, without passing them as arguments, and without one thread's settings disturbing another's.

```
(define-thread-variable *indent* 0)

(define show
  (method (text)
    (format #t "~a~a~%" (make <string> size: *indent* fill: #\space) text)))

(define nested
  (method (depth)
    (show depth)
    (when (< depth 2)
      (dynamic-bind ((*indent* (+ *indent* 2)))
        (nested (+ depth 1))))))

(nested 0)
(print *indent*)
(print (join-thread (make-thread (method () (set! *indent* 99) *indent*))))
(print *indent*)
```

prints

```
0
  1
    2
0
99
0
```

The other thread's `set!` changed only its own value.

16.2.8 What threads may share

Assigning a module variable or a slot is a single store of one machine word, so a thread never sees a value half written. Beyond that, data that threads share and change must be protected with locks. In particular the library's collections — tables, stretchy vectors, deques and the rest — do no locking of their own: two threads adding to one table at once can leave it in a wrong state. The runtime's own shared state, such as the table of symbols, is locked internally, and since the program's classes and generic functions never change, method dispatch needs no locks at all.

Chapter 17

Inside the Compiler

This chapter describes how the compiler to C works: the stages a program passes through, how Dylan values look in memory, how functions are called and how tail calls are kept proper, how generic functions dispatch, how exits, cleanups and conditions are built, and how the garbage collector and threads fit in. You do not need any of it to use the compiler. It helps when you want to know why one way of writing a program is faster than another (Chapter 18), when you read the C the compiler writes, or when you want to work on the compiler itself. The design, phase by phase, is also recorded in `CompilerPlan.md` at the top of the source tree.

17.1 The stages

The compiler is about 9 000 lines of Common Lisp in `compiler/`. It is a *whole-program compiler*: it sees the entire program, with the library, before it writes any C, and since nothing changes once the program runs, everything it learns about the program is true for good. A build passes through these stages:

1. **Reading** (`front/source.lisp`). Each file is read with the reader of the interactive implementation (`src/compat.lisp`), the only code the two implementations share besides the macros' run time. The reader records the file, line and column where every list begins, so that every later diagnostic, and every `#line` directive, can name a place in the source. A file's `module:` header (design note 33) says which module its forms belong to.

2. **Modules** (`front/modules.lisp`). Every `define-module` form in the program is found first, and the modules are defined, each after the modules it uses. Then every name in the program is resolved to a *module variable*: one object per variable, owned by one module, however many modules import it and under whatever names. From here on names are gone, and nothing can be confused by renaming or shadowing.

3. **Macros and core forms** (`front/convert.lisp`, `front/macros.lisp`). Each top-level form is converted to *core Dylan*, about a dozen kinds of node: constants, variable references and assignments, `if`, sequences, `bind` (with multiple values), mutually recursive local methods, methods, calls, calls of runtime primitives, `bind-exit` and `unwind-protect`, and the definitions. Every other special form and built-in macro — `cond`, `case`, `for`, `for-each`, `handler-case`, backquote, `with-lock` and the rest — is rewritten into these. A loop becomes a local method that calls itself in tail position. The program's own macros are expanded here too, by running their bodies in the interactive implementation (Chapter 16).

4. **The IR** (`ir/anf.lisp`, `ir/ir.lisp`). Core Dylan is converted to an intermediate representation in *A-normal form*: every intermediate value is given a name, and every operand of a call or primitive is a constant or a variable. Every variable is bound exactly once (the IR is *alpha-converted*), so a variable is a unique object that analyses can annotate. Operands are evaluated left to right, as in the source.

5. **Analysis** (`ir/analyze.lisp`). For each variable: whether it is assigned, whether a function other than the one binding it uses it (it is *captured*), and how often it is referred to. For each local function: where it is called, and whether it *escapes* — is used other than by being called. For each exit procedure of a `bind-exit`: whether it escapes its function. And for each call: whether it is in *tail position*, that is, whether its value is the value of its function.

6. **Control lowering** (`ir/control.lisp`). A `bind-exit` whose exit is only called directly within its own function stays, and will become a jump. Every other `bind-exit`, and every `unwind-protect`, is rewritten as a call of a runtime function, with the body moved into a new closure (Section 17.7).

7. **Contification** (`ir/contify.lisp`). A local function that never escapes and whose every call is a jump — a call in tail position from the function that defines it, or from another such function — is not compiled as a function at all, but as a labelled block of the function that contains it, its calls becoming assignments to its parameters and a `goto`. This is how loops become C loops.

8. **Type inference** (`backend/types.lisp`, Section 17.12) finds a class for as many variables as it can, and decides which `<double-float>` variables can be plain C `double`s.

9. **Calls** (`backend/calls.lisp`, Section 17.12). Each call whose function is known is resolved: to a method, to a generic function's dispatch function, to a runtime primitive, or to inline C.

10. **Escape analysis** (in `backend/codegen.lisp`) finds `#rest` lists, closures and instances that cannot outlive the function that makes them, and puts them on the C stack instead of the heap.

11. **Code generation** (`backend/codegen.lisp`, `backend/objects.lisp`, `backend/cdata.lisp`). Every remaining function becomes a C function; every class, symbol and constant becomes static C data; every generic function gets a dispatch function.

12. **Output** (`backend/output.lisp`). The C is split into *`name`*`.c`, one file per module, and *`name`*`.h` (Section 15.3), and `driver.lisp` runs the C compiler.

When the library is compiled with the program (`-Os`, `--whole`), tree shaking (`backend/shake.lisp`, Section 17.11) runs between control lowering and code generation.

The library itself is Dylan, in module `dylan`, in the files of `runtime/dylan/` read in name order: conditions, numbers, characters, collections, sequences, lists, vectors, strings, stretchy vectors, deques, ranges, tables, arrays, printing and threads. It is compiled with every program, by the same stages, on top of the `%` primitives written in C in `runtime/c/library.c` and the other files of `runtime/c/`.

17.2 How values are represented

Every Dylan value is one 64-bit machine word, the C type `D` (`runtime/c/dylan.h`). The low bits of the word say what it is:

Low bits	Value
`...xx1`	an `<integer>`: the value shifted left one bit, so 63 bits signed
`...010`	a `<character>`: the Unicode code point shifted left three bits
`...000`	a pointer to an object, 8-byte aligned

Integers and characters are *immediate*: they need no memory of their own, and allocating one costs nothing. The integer n is the word $2n + 1$; that is why a debugger shows the integer 100 as 201. Adding two tagged integers $2a + 1$ and $2b + 1$ is one machine addition and a subtraction of 1, with the overflow checked by the C compiler's `__builtin_add_overflow`; an overflow signals an error rather than wrapping.

Every object a pointer points to begins with a *header* word: a pointer to its class's *wrapper*. The wrapper holds what the runtime needs to know about the class — the class object, the class's numeric id, the instance size, how instances are represented (an ordinary instance, a pair, a byte string, a double float, and so on), and the set of the class's ancestors. Pairs are a header and two words; strings a header, a length and the bytes; a `<double-float>` a header and a C `double`; an instance of a class the program defines a header and one word per slot.

`#t`, `#f` and `()` are three static objects, compared by address. `#t` and `#f` are instances of an internal class `<boolean>`, which the manual does not name and the `dylan` module does not export.

Class ids. Since the program is static, the compiler knows every class. It numbers them in a preorder walk of the tree that each class makes with its *first* superclass, so that a class's descendants in that tree have the ids from its own up to a limit. Where every subclass of a class lies in that range, `(instance? x C)` is a range check on `x`'s class id: one subtraction and one comparison. Where multiple inheritance puts a subclass outside it, the test reads a bit in the ancestor set every wrapper carries. The classes, their wrappers and the symbols are all static C data emitted by the compiler, so a program does no work at startup to build its class hierarchy.

Memory. Objects are allocated by the garbage collector (Section 17.8), through the functions `dy_alloc` and its kin in `dylan.h`, which hide which collector it is: those that can hold pointers apart from those that cannot — strings, float boxes, ratios — so that the collector does not scan the latter.

17.3 The calling convention

Every Dylan function has a *uniform entry*, a C function of one prototype:

```
D f(D self, D a0, D a1, D a2, D a3, intptr_t argc);
```

`self` is the function object being called — a closure, a generic function, or a method — through which a closure reaches its free variables. The first four arguments travel in `a0` to `a3`, in machine registers; `argc` says how many there are. Arguments after the fourth go in a per-thread

argument area, which the callee reads before it calls anything else. Every function checks `argc` on entry, so calling a function with the wrong number of arguments is an error, not a crash.

One prototype for every function is what makes proper tail calls possible (Section 17.4). It costs a little: a function of two arguments is still passed six. So a function that is called from places the compiler can see, other than in tail position, also gets a *specialized entry*, named with `_spec`, with exactly its own parameters: `double`s where the arguments are unboxed floats, and one parameter per keyword, filled in when compiling from the keywords the call passes (`DY_UNSUPPLIED` for those it does not). A call through a specialized entry passes no count, checks no arity and parses no keywords when it runs.

Multiple values. A function returns its first value as its C return value. When it returns some other number of values, it stores the count and all the values in a per-thread values buffer. A caller that wants all the values sets the count to 1 before the call; a function that returns one value leaves the buffer alone unless it has itself made a call that may have changed the count. A caller that wants only the first value — the common case — reads the C return value and ignores the buffer.

`next-method`. A method that calls `next-method` receives its next method through the thread: the dispatch function (or the method before it) stores it there, and the method reads it on entry. The chain of methods for each combination of argument classes is known when compiling, so the next-method objects are static data, one for each tail of each chain. Methods that do not use `next-method` ignore it, and a method whose `next-method` is only called, with no arguments, reuses its own arguments rather than capturing them in a closure.

17.4 Proper tail calls

Dylan requires that a call in tail position not use stack: a loop written as a recursion must run in constant space, however many times it goes round. C promises nothing of the kind. The compiler meets the requirement in three ways.

1. A function's tail call of itself, and calls among local methods that have been contified, become `goto`s within one C function. This covers almost every loop: `for`, `while`, `dotimes` and `for-each` all expand into local methods calling themselves in tail position.

2. Every other tail call — through a module variable, a closure passed as an argument, a generic function, `next-method`, `apply` — is a C tail call marked `__attribute__((musttail))`, which GCC 15 and later and Clang 13 and later *guarantee* to compile as a jump. The attribute requires caller and callee to have the same prototype, which the uniform entry provides. The generated C writes it with the macro `DY_TAIL_CALL`.

3. With `--trampoline`, a tail call instead stores the function and its arguments in the thread and returns a marker; the nearest caller that is not making a tail call runs a loop that makes the pending call until a real value comes back. Any C11 compiler can compile this.

A specialized entry cannot make a `musttail` call, because its prototype differs from its callee's. When a function makes tail calls, either its uniform entry holds the only body, or its specialized entry gets a second copy of the body whose tail calls are ordinary calls. That is safe:

a specialized entry is only ever entered by a call that is not a tail call, so each holds at most one extra C frame.

Calls inside an `unwind-protect`, inside a `bind-exit` whose exit escapes, and inside a handler's extent are not in tail position: their frame must remain until they return, as the language's semantics already imply.

17.5 Closures

Closures are *flat*: a closure is a function object holding its C entry and a copy of the value of each free variable it uses. A variable that is both captured by a closure and assigned lives instead in a one-word heap *cell*, which every closure that uses it shares, so all of them see each assignment. A function with no free variables — every method defined at top level, for instance — is a static object and costs nothing to make. A local method that is only called, and never escapes, is not made into a closure at all: it is contified or called directly.

17.6 Generic dispatch

Because the program is static, the compiler knows every method of every generic function. For each generic function it computes, at compile time, which methods apply to each combination of argument classes and singletons, sorts each such set by specificity, and writes a C *dispatch function*: a decision tree that tests the arguments — singletons first, by comparing pointers, then a `switch` on the class id — and ends at a leaf for each distinct sorted list of methods. A leaf checks keyword arguments, if the methods take any, sets the thread's next method, and tail-calls the first method. A call that no method fits reaches `dy_no_applicable_method`, which signals the error.

Here is the dispatch function for a generic function `area` with one method on `<square>` and one on `<circle>`:

```
D dy_gf_area_51a90f52_dispatch(D self, D a0, D a1, D a2, D a3,
                               intptr_t argc) {
    (void)a0; (void)a1; (void)a2; (void)a3;
    if (DY_UNLIKELY(argc != 1)) dy_wrong_argument_count(self, argc);
    D x0 = a0; (void)x0;
    switch (dy_wrapper_of(x0)->id) {
    case 61:
        goto leaf_0;
    case 62:
        goto leaf_1;
    default: goto no_method;
    }
leaf_0:
    dy_thread->next_method = DY_FALSE;
    DY_TAIL_CALL_DIRECT(fn1197_area, DY_OBJ(&dy_method_326),
                        a0, a1, a2, a3, argc);
leaf_1:
    dy_thread->next_method = DY_FALSE;
    DY_TAIL_CALL_DIRECT(fn1199_area, DY_OBJ(&dy_method_327),
                        a0, a1, a2, a3, argc);
no_method:
    dy_no_applicable_method(self);
```

```
}
```

(Three long lines have been broken to fit the page.)

There are no caches and no locks: the tree is fixed, and threads share it freely.

Methods are ordered by the rule of the manual's chapter 10: arguments compared left to right, a singleton before any class, a class before its superclasses. The manual leaves open the order of two classes neither of which is a subclass of the other; this compiler, like CLOS and like the interactive implementation, puts first the one that comes earlier in the *argument's* class precedence list. Two methods that remain unordered are ambiguous, and calling them is an error.

Better still is not to dispatch at all. Where type inference shows that only one method can apply, the call goes straight to that method, or the method is done inline (Section 17.12). A generic function with a single method, specialized on `<object>` for every argument, is never dispatched.

Classes and `make`. For each class that is instantiated, the compiler writes a *maker*, which allocates the instance, fills each slot from its init-keyword or its default, checks the init-keywords, and calls `initialize` only if the program defines methods on it. A `make` of a class the program defines, where only the library's `make` method on `<class>` applies, calls the maker directly. Slot offsets are fixed where every possible class of the receiver agrees on them; otherwise accessors look the offset up in a small table in the wrapper.

17.7 Exits, cleanups and conditions

A `bind-exit` whose exit procedure is only called directly, within the function that binds it (or within local methods contified into it), compiles to a jump: calling the exit assigns the values and does a `goto` past the body. This is the common case — an early return from a loop — and it costs nothing.

Every other `bind-exit`, and every `unwind-protect`, needs a *frame* in the thread's *dynamic chain*, the list of active exits, cleanups and handlers, innermost first. The frames live on the C stack of three runtime functions in `runtime/c/control.c`: `dy_call_with_exit`, `dy_unwind_protect` and `dy_handler_bind`. The compiler moves the body into a closure and passes it to one of these:

```
(unwind-protect p c)  =>  (%primitive unwind-protect (method () p)
                                                     (method () c))
(bind-exit (x) b)     =>  (%primitive call-with-exit (method (x) b))
```

`dy_call_with_exit` pushes an exit frame, calls `_setjmp`, and calls the body with an exit procedure. Calling that exit procedure stores its arguments as the values to return, runs the cleanups of every frame between it and the target, innermost first, and `longjmp`s to the target frame. An exit called after its `bind-exit` has returned is detected — each frame carries a serial number — and signalled as an error. Because only these runtime functions call `setjmp`, generated code never does, and can make tail calls everywhere.

Conditions. Handlers are frames in the same chain. `signal` walks the chain for a handler whose type matches and whose test, if any, is true, and calls it with the condition and a `next-handler` that continues the search further out; if none applies it calls `default-handler`. The condition classes and `signal`, `error`, `cerror`, `check-type`, `abort`, `break` and `default-handler` are

written in Dylan, in `runtime/dylan/01-conditions.dyl`; only the frame operations and the transfer of control are C. When the runtime detects an error of its own — a wrong argument count, a type error, an overflow — it calls back into that Dylan code, so the error is an ordinary `<simple-error>` or `<type-error>` that handlers can catch.

17.8 The garbage collector

Memory is managed by the Memory Pool System in `runtime/mps/`, compiled as one C file into the runtime library, which is built with `DY_MPS`. It is a moving collector, so it must know every object's layout: `runtime/c/gc-mps.c` gives it an *object format* — for each kind of object, its size, the words of it that are references, and how to leave a forwarding pointer or padding where an object was — and the *roots*: the program's static data (found from the linker's symbols on Linux, and from the Mach-O segments on macOS) and each thread's stack and registers, scanned conservatively, and each thread's state and thread variables, allocated outside the heap and registered as a root. A tagged integer or character on a stack that happens to look like a pointer into the heap can only keep an object alive, and in place, longer than necessary; it can do no harm.

Objects go in five *pools*: AMC, generational and moving, for objects that may hold references; AMCZ for those that do not (strings, floats, ratios), which the collector never scans; AMS, which does not move, for `<thread>` objects, since a thread's state points to its own; LO, which is neither moved nor protected, for objects holding a mutex or condition variable the kernel uses; and AWL, weak, for the table of identity hashes. Each thread has its own *allocation points* in AMC and AMCZ, from which it allocates by bumping a pointer; the slow path, when the point's buffer is empty, enters the collector. An object's fields are filled in before it is committed to the collector, which may move it or ask for it to be allocated again. Because the collector moves objects, a Dylan object's identity hash cannot be its address: `%object-hash` gives each object a number the first time it is asked, kept in a table with weak keys, rebuilt when the collector reports that keys have moved. The collector's behavior can be tuned with environment variables (Chapter 18).

The Boehm-Demers-Weiser collector. With `dylanc --gc boehm` (Section 15.8) the runtime is built without `DY_MPS` and memory is managed instead by `libgc`, built for threads. It is a *conservative* collector: it treats any word on a stack, in a register or in a scanned object that looks like a pointer into the heap as one, so it needs no object format. Strings and float boxes are allocated atomic so that their contents are not mistaken for pointers. Each thread's state — the values buffer, the argument area, the dynamic chain — is allocated uncollectable, so the collector scans it and sees pointers held there. The collector does mark-and-sweep collection, in parallel on two threads, and allocates small objects from thread-local free lists without locking.

The functions `dy_alloc` and `dy_alloc_init` in `dylan.h` allocate with either collector.

17.9 Threads

Each thread registers itself and its stack with the collector as it starts, and makes its allocation points (under the Boehm collector, `dylan.h` defines `GC_THREADS` and includes `gc.h` before `pthread.h`, so `pthread_create` goes through the collector, which registers each thread's stack). Each thread has one `_Thread_local` pointer, `dy_thread`, to its state: the values buffer, the

argument area, the dynamic chain, its next method, its pending tail call in trampoline mode, its `<thread>` object, and its values of the program's thread variables.

A thread starts by setting up that state and an exit frame at the top of its dynamic chain, then calls its function. An unhandled serious condition reaches `default-handler`, which in a thread other than the main one records the condition in the `<thread>` and unwinds to that top frame, ending the thread; `join-thread`, written in Dylan in `runtime/dylan/20-threads.dyl`, finds it there and signals a `<thread-error>`. Threads are detached when made; `join-thread` waits on a condition variable in the `<thread>`. A `<lock>` is an error-checking POSIX mutex, a `<recursive-lock>` a recursive one, and an `<atomic-integer>` a C11 `_Atomic` integer.

17.10 Separate compilation of the library

Compiling the library's megabyte of C with every program would double the time of a small build. So, by default, the compiler compiles the library *as if it did not know the program.* Every function the `dylan` module exports, or that the compiler's expansions name, may be called from anywhere with anything: no argument types and no specialized entries are derived from the program's calls, and nothing of the library is dropped. The C names of the library's constants, variables and generic functions are derived from what they are, not from the order in which the compiler met them, and its C temporaries are numbered within each function. *name*`-dylan.c` declares only what it refers to. The result is that the library's C is the same text for almost every program, and the driver compiles it once per C compiler and option set and caches the object (Section 15.4).

What is lost is specialization of library code to the program, and the dropping of unused library code. Calls *from* the program *into* the library lose little, since the compiler still inlines the library's arithmetic, `element`, `size` and iteration on known collections at the call site.

17.11 Tree shaking

With `-Os` or `--whole` the library is compiled with the program, and `backend/shake.lisp` keeps only what the program can reach. The program's own top-level forms are all kept, as are the library's top-level forms that are not definitions and every `define-class` (classes are cheap static data). A library definition of a variable, generic function or method is kept when something kept refers to its variable. A method is kept, besides, only if each class its specializers name could have instances — a class the runtime itself makes instances of, or one that something kept uses as a value (makes, say, or answers). Naming a class as a specializer or a superclass does not count.

Shaking is by name and class, not by data flow, and so it keeps more than a program needs: a call of a generic function such as `print-object`, `make` or `as` keeps every method that could apply to a class with instances. That is why hello world is still about 220 KB when built `-Os`.

17.12 Type inference and inlining

The type of a variable, for the compiler, is a class all its values are instances of, or nothing known. Types come from:

- constants;

- the result classes of the operations done inline — an integer addition answers an integer, a float multiplication a `<double-float>`;
- `make` of a class whose maker the call goes to;
- the getters of slots declared with a `type:`;
- the type a `bind` declares, as in `(bind (((x <integer>) init)) ...)`, which is checked when the binding is made;
- the specializers of a method defined with `define-method`, which dispatch has already checked;
- for local functions, global functions and sole methods whose every call is known — a loop, most often — the classes of the arguments at those calls, and the classes of what the functions return.

These are found together, starting from nothing, until none changes. The types are also *flow-sensitive* where a test proves more: in the branch an `(instance? x C)` test chooses, `x` is known to be a `C`.

Calls whose function is known are then resolved (`calls.lisp`). A call of `+`, `-`, `*`, `/`, `<`, `>` or `=` with two arguments calls the binary generic function (`binary+` and so on) directly. The library's methods for integer and float arithmetic and comparison, for `size`, `element` and the iteration protocol on lists, simple object vectors and byte strings, and `id?`, `not`, `car`, `cdr`, `instance?` and slot getters and setters are done *inline* — when the static program shows that no other method could apply to such arguments. The manual's generic functions that the runtime writes in C — `instance?`, `object-class`, `subclass?` and the other reflective functions of *DIRM* chapters 10 and 11 — each have one library method, which calls a `%` primitive; while the program adds no method of its own to one, its calls are calls of the primitive, and `instance?` of a known class is done inline. Each inline case is guarded by a test of the arguments' classes, and the test is left out where their types are known. A float operation with one argument known to be a `<double-float>` answers a `<double-float>` (float contagion), so its result's type is known even if the other argument's is not.

Unboxing. A `<double-float>` variable that no closure holds, and every use of which wants a C `double`, is *unboxed*: it is a C `double` local, and computing it allocates nothing. A float variable that is returned, stored in a slot or passed to an unknown function needs its box, and is allocated on the heap each time it changes. Integers need no unboxing; they are immediate.

Escape analysis. A `#rest` list that is only walked, through variables no closure holds, is built on the C stack (up to eight pairs); keyword arguments are parsed where they were passed, with no list at all. A closure, or an instance whose class's maker `make` calls (and which has no `initialize` methods), is made on the stack of the function that makes it when nothing can reach it after that function returns: it is only tested, called, given to a slot accessor sure to apply, or passed to a known function whose parameter is used the same way. Constants propagate through `bind`s, integer arithmetic on constants is done when compiling, an `if` on a constant keeps one branch, and unused pure bindings are dropped.

In a debug build (`-g`) none of the inferred types is used, so every inline operation tests its arguments.

17.13 Reading the generated C

`dylanc -k` keeps the C. Here is a small program, `sum.dyl`:

```
(define-method sum-to ((n <integer>))
  (for ((i 0 (+ i 1))
        (s 0 (+ s i)))
       ((> i n) s)))
(print (sum-to 100))
```

and the C function the compiler writes for `sum-to`, in `sum-dylan-user.c`:

```
D fn1190_sum_to(D self, D a0, D a1, D a2, D a3, intptr_t argc) {
    (void)self; (void)a0; (void)a1; (void)a2; (void)a3; (void)argc;
    D v105609, v105610, v105612, v105613, v105614, v105615, v105616,
      v105617, v105618, v105619;
    D t1;
    D t2;
    D t3;
    D t4;
#line 1 "sum.dyl"
    if (DY_UNLIKELY(argc != 1)) dy_wrong_argument_count(self, argc);
    v105609 = a0;
#line 2 "sum.dyl"
    t1 = DY_INT(0);
    t2 = DY_INT(0);
    v105612 = t1;
    v105613 = t2;
    goto for_1;
    for_1:;
#line 4 "sum.dyl"
    v105615 = DY_BOOL((intptr_t)(v105609) < (intptr_t)(v105612));
#line 2 "sum.dyl"
    if (v105615 != DY_FALSE) {
        dy_thread->mv_count = 1;
        return v105613;
    } else {
        v105617 = dy_int_add(v105612, DY_INT(1));
#line 3 "sum.dyl"
        v105619 = dy_int_add(v105613, v105612);
#line 2 "sum.dyl"
        t3 = v105617;
        t4 = v105619;
        v105612 = t3;
        v105613 = t4;
        goto for_1;
    }
}
```

(one line of declarations has been broken to fit the page). Reading it from the top:

- The function has the uniform entry. Its name, `fn1190_sum_to`, is a number and the Dylan name; every variable `v`*n* is one IR variable, and every `t`*n* a temporary.
- It checks that it was given one argument; the method is the only method of `sum-to`, and

its dispatch function has already checked that the argument is an `<integer>`.

- The `for` loop has become a label, `for_1`, and a `goto`. The step forms are evaluated into temporaries before any loop variable is assigned, because `for` steps its variables in parallel.
- `(> i n)` is a single C comparison of the tagged words, with no test of their classes: `n` is an integer by its specializer, and `i` by type inference, since it starts at 0 and is only ever incremented. `(> i n)` became `(< n i)`, and on tagged integers $2a + 1 < 2b + 1$ exactly when $a < b$.
- `(+ i 1)` and `(+ s i)` are `dy_int_add`, an inline function in `dylan.h` that adds the tagged words and signals an error on overflow.
- Returning one value sets the values count to 1 first, because a value count might be left over from a call made earlier in the function.
- The `#line` directives name the Dylan line each statement came from.

If `sum-to` were instead written as `(define sum-to (method ((n <integer>)) ...))`, `n` would not be known to be an integer inside it: a method made by `method` and stored in a variable is called through the variable, and must check its own argument. The comparison would then be preceded by a test, `if (DY_IS_INT(v105609))`, with a call of the generic function `binary<` for the other case. Small differences like this are the subject of the next chapter.

Chapter 18

Performance

Dylan was designed to be both dynamic and efficient, and the compiler to C aims at its original targets: small and fast. This chapter reports how fast compiled programs are on a set of benchmarks, explains what makes a Dylan program fast or slow under each implementation, and shows how to measure a program and tune the garbage collector. Most of the advice follows from how the compiler works (Chapter 17); the measurements here show how much each point matters.

Note: Every time in this chapter was measured on one machine: an Intel Core i9-14900KF running Fedora Linux, with SBCL 2.6.3 and GCC 16.2.1. Times depend on the processor, the C compiler, the collector's version and the load on the machine, and vary by several percent from run to run. Treat them as an indication of proportions, not as absolute figures, and run the benchmarks on your own machine for numbers that apply to you.

18.1 The benchmarks

The directory `bench/` holds eleven small programs, each written three times: in Dylan (`benchmarks.dyl`), in plain Common Lisp (`reference.lisp`) and in plain C (`reference.c`). Each answers a value, and the three versions must agree.

Name	What it does
`table`	insert into and look up in a `<table>`
`sort`	sort a list of pseudo-random integers
`string`	build and compare strings
`deque`	push and pop at both ends of a `<deque>`
`stretchy`	grow a `<stretchy-vector>` and walk it
`range`	iterate over ranges
`exit`	leave a computation early with `bind-exit`
`condition`	signal errors and handle them with `handler-case`
`next-method`	call down a chain of methods with `next-method`
`tak`	the Takeuchi function: integer arithmetic and calls
`float`	points of a grid inside the Mandelbrot set: floating point

Two scripts run them. Each prints, for every benchmark, the best of three runs.

```
$ sbcl --script bench/run-c.lisp          # compiled Dylan vs C and Lisp
$ sbcl --script bench/run.lisp            # interactive Dylan vs Lisp
$ sbcl --script bench/run.lisp table tak  # just these
```

Their builds go in `bench/build/`. A line marked `MISMATCH` means the versions answered differently, which would be a bug.

At their usual sizes several benchmarks finish in a few milliseconds, too quickly to compare reliably. The environment variable `BENCH_SCALE`, if set to an integer, multiplies every size, in both scripts and in `reference.c`:

```
$ BENCH_SCALE=10 sbcl --script bench/run-c.lisp
```

The file `SpeedComparison.md` at the top of the repository puts the three ways of running the benchmarks — plain SBCL, the Common Lisp implementation and the compiler to C — side by side, with hand-written C, at `BENCH_SCALE=10`, and discusses the results. The tables below were run at the usual sizes.

18.1.1 The compiler to C

`bench/run-c.lisp` compiles the Dylan with `dylanc` (at `-O2`), compiles `reference.c` with `cc -O2`, and loads `reference.lisp` into SBCL. A run gave:

benchmark	Dylan->C ms	C ms	D/C	Lisp ms	D/Lisp
table	13	6	2.17	8.0	1.63
sort	29	13	2.23	26.0	1.12
string	11	2	5.50	7.0	1.57
deque	6	2	3.00	2.0	3.00
stretchy	3	0	-	2.0	1.50
range	5	0	-	2.0	2.50
exit	1	3	0.33	1.0	1.00
condition	4	0	-	1.0	4.00
next-method	5	2	2.50	8.0	0.62
tak	22	17	1.29	35.0	0.63
float	3	2	1.50	55.0	0.05

The columns are the compiled Dylan's time, the C's, their ratio, plain SBCL's time and the ratio to it. A ratio is shown as `-` when the other time is under a millisecond and the ratio would mean nothing.

Read it this way. On calls and integer arithmetic (`tak`) and on floating point (`float`), compiled Dylan takes 1.3 to 1.5 times as long as hand-written C. On the collections it is two to three times the C's time, and on strings about five times. Non-local exits (`exit`) are *faster* than the C, because the Dylan exit becomes a jump where the C uses `setjmp`. Against plain SBCL, compiled Dylan is faster on calls, `next-method` and floats, and slower on the collection benchmarks by up to three times. The very short benchmarks (`stretchy`, `range`, `condition`) take a few milliseconds where the C takes under one, so their ratios vary most from run to run; in other runs the `string` ratio was 6.5 and the `table` ratio 1.6.

18.1.2 The interactive implementation

`bench/run.lisp` compiles `benchmarks.dyl` with the interactive implementation's `dylan:compile-file`, compiles the resulting Lisp with SBCL, and compares it with `reference.lisp`:

benchmark	size	dylan	lisp	ratio
table	200000	0.076s	0.007s	10.9x
sort	100000	0.083s	0.025s	3.3x

```
string          200000     0.050s     0.005s    10.0x
deque           300000     0.029s     0.002s    14.5x
stretchy        300000     0.026s     0.002s    13.0x
range          1000000     0.048s     0.003s    16.0x
exit            300000     0.027s     0.001s    27.0x
condition        30000     0.010s     0.000s    10.0x
next-method     300000     0.039s     0.008s     4.9x
tak                300     0.469s     0.036s    13.0x
float              300     0.089s     0.054s     1.6x
```

The interactive implementation is typically ten to fifteen times slower than plain Lisp, and so ten to twenty times slower than the compiler to C. That is the price of a language whose classes and methods can change while it runs: every call of a generic function must allow for methods added since the last call. The interactive implementation is for developing and exploring; the compiler is for running.

18.2 Writing fast Dylan for the compiler

The compiler is fastest where it knows the classes of the values it works with. Then it calls methods directly, does arithmetic and collection access inline without tests, and keeps floats unboxed. Each point below says how to give it that knowledge, with a measurement. The programs are small, and each was timed as a whole, run as a separate executable built at `-O2`.

18.2.1 Prefer `define-method` to `define` of a `method`

How a function is defined matters more than you might expect. These four versions of `tak`, computing `(tak 30 22 12)`, differ only in their first lines:

Definition	Time
`(define tak (method (x y z) ...))`	0.04 s
`(define-method tak (x y z) ...)`	0.04 s
`(define-method tak ((x <integer>) (y <integer>) (z <integer>)) ...)`	0.08 s
`(define tak (method ((x <integer>) (y <integer>) (z <integer>)) ...))`	0.30 s

- An unspecialized `define-method` with only one method is not dispatched at all, and a `define` of a `method` with no specializers is called directly; both are as fast as the program can be.
- Specializing the only method on `<integer>` makes every call go through the dispatch function, which tests the argument classes. Inside the method the arguments are then known to be integers, which helps a method that does much work with them, but here the method is tiny and the dispatch doubles its cost.
- A `method` with specializers stored with `define` is the slowest: it is a closure that checks its own arguments on every call, is called through the variable, and does not pass on to the compiler the knowledge its specializers give.

So: specialize methods when you need dispatch, or when the method does enough with its arguments to profit from knowing their classes; do not specialize a lone method merely to document its argument types; and define functions with `define-method`, or with `define` of an unspecialized `method`.

18.2.2 Let the compiler know classes: declarations in `bind`

A value fetched from a collection, or returned by a function the compiler cannot see into, has no known class. Every operation on it is then preceded by tests of its class, and a float result must be boxed because the compiler cannot be sure it is a float. A type declaration in `bind` (*DIRM* chapter 4) tells the compiler the class; it is checked once, when the binding is made. Summing squares of the floats in a vector of 1000 elements, 100 000 times:

```
(define-method sum-squares (v)
  (bind ((s 0.0))
    (for-each ((x v)) ()
      (set! s (+ s (* x x))))
    (> s 0.0)))                                    ; 1.06 s

(define-method sum-squares (v)
  (bind ((s 0.0))
    (for-each ((x v)) ()
      (bind (((y <double-float>) x))
        (set! s (+ s (* y y)))))
    (> s 0.0)))                                    ; 0.20 s
```

In the first version `(* x x)` tests whether `x` is an integer or a float and allocates a box for the product; in the second it is one machine multiplication, and nothing is allocated. Specializing a method on the class of its argument has the same effect inside the method; so does a `type:` on a slot, for the values its getter answers.

Collections are no different. A `for-each` over a parameter of unknown class tests, on each step, whether it is walking a list or a vector; over one known to be a `<simple-object-vector>` it is a plain indexed loop. Summing a vector of 1000 integers 100 000 times took 0.16 s with an unspecialized parameter and 0.10 s with the parameter specialized on `<simple-object-vector>`.

18.2.3 Keep floats unboxed

A `<double-float>` is a heap object, so a float computation that creates a new one on every step spends most of its time allocating. The compiler avoids this by keeping a float variable as a C `double` when *every* use of it wants a `double` (Section 17.12). Returning the variable, storing it, or passing it to a function the compiler cannot see into each need a box, and force one on every step. Compare (100 million steps each):

```
(define-method norm-sum (scale n)
  (for ((i 0 (+ i 1))
        (s 0.0 (+ s (* scale i))))
       ((= i n) s)))                    ; returns s itself: 0.87 s

(define-method norm-sum (scale n)
  (for ((i 0 (+ i 1))
        (s 0.0 (+ s (* scale i))))
       ((= i n) (> s 1.0)))             ; only compares s: 0.04 s
```

The difference is not the work but the boxes. A hot loop should keep its float accumulators in variables used only arithmetically, and box the result once, after the loop, by passing it to a function or returning it from an enclosing one.

Use float contagion, not `as`. Mixing an integer into float arithmetic converts it automatically (*DIRM* chapter 14), and the compiler does the conversion inline. An explicit `(as <double-float> i)` calls the generic function `as` instead. In the loop above, writing `(* scale (as <double-float> i))` instead of `(* scale i)` made it two and a half times slower (2.2 s against 0.87 s).

18.2.4 Typed slots

A slot declared with `type: <double-float>` lets the compiler know the class of what its getter answers, so arithmetic on it needs no test. The value stored in the slot is still a boxed float, so the gain is modest when the program writes the slot often: stepping a particle's position 100 million times took 1.18 s with untyped slots and 1.05 s with typed ones.

18.2.5 Local exits are free; escaping ones are not

A `bind-exit` whose exit procedure is called only directly within the function that binds it — an early return from a loop — compiles to a jump. One whose exit is passed to another function needs a frame in the dynamic chain, `setjmp`, `longjmp`, and a closure (Section 17.7). Searching a ten-element vector 30 million times, returning early:

```
(define-method find-first (v (x <integer>))
  (bind-exit (return)
    (for-each ((e v)) ()
      (when (= e x) (return e)))
    #f))                                              ; 0.08 s

(define-method check (e x return) (when (= e x) (return e)))
(define-method find-first (v (x <integer>))
  (bind-exit (return)
    (for-each ((e v)) ()
      (check e x return))
    #f))                                              ; 1.48 s
```

The second is nearly twenty times slower. Keep the calls of an exit procedure inside the function that binds it where you can. The same applies to `unwind-protect` (and so to `with-lock` and `dynamic-bind`, which use it), whose cleanup always needs a frame, and to `handler-case`: a handled error costs a few hundred nanoseconds.

18.2.6 First-class calls

A call of a function held in a variable or passed as an argument goes through the function's uniform entry, with no inlining and no specialized entry. In a hot loop it costs more than a direct call or inline code, but often less than you might fear: summing squares of a vector with `(reduce (method (acc x) (+ acc (* x x))) 0 v)` took 0.19 s where the equivalent `for-each` loop took 0.16 s. It matters most when the function called is small and could otherwise have been inlined. `apply` with a long list, and `#rest` lists that escape, allocate.

18.2.7 `#rest` and keywords

A `#rest` list that is only walked — not stored, returned, or captured by a closure — is built on the C stack, for up to eight elements, and costs no allocation. Keyword arguments to a function

the compiler can see are matched when compiling, with no list built at all. So these are cheap in the common cases; a `#rest` list that is stored or returned is allocated on the heap.

18.2.8 What the static program gives

Everything in a compiled program is effectively sealed: the compiler knows every class and every method. That is what allows it to do `+` on integers inline, or `element` on a vector: it can see that no method but the library's could apply to such arguments. If your program defines methods on the library's generic functions for the library's own classes — a method on `binary+` for two integers, say, or on `element` for `<simple-object-vector>` — the inline code for those classes is dropped, and every such call in the program goes through dispatch. (With a method on `binary+` for two integers, `+` of two integers calls it, as it must; the inline cases for floats remain.) Methods for your own classes, or for combinations the library does not handle inline, do not have this effect.

18.3 Writing fast Dylan for the interactive implementation

The interactive implementation compiles Dylan to Common Lisp, which SBCL compiles to machine code. Generic functions dispatch through caches; calls of known functions are made directly; arithmetic on numbers, `element` on simple vectors and strings, and iteration over simple vectors, lists and ranges are done inline, behind a test of the arguments' classes; and slots are read directly. Two things defeat the fast paths:

- Each inline operation is guarded by a flag that is true only while the generic functions behind it have no methods but the predefined ones that could apply to those arguments. Adding a method to `+`, `element`, `<` and the like for a built-in class turns the fast path off for the rest of the session.
- Redefining things as the program runs (adding methods, adding slots) invalidates dispatch caches, which are then rebuilt on the next calls.

Beyond that, the advice for the compiler applies here too, in smaller measure: local exits are cheaper than escaping ones, and code that works on known collection classes is faster. For a program whose speed matters, develop it interactively and compile it with `dylanc`.

18.4 Size

By default the library is compiled apart from the program and linked whole (Section 15.4), which makes builds fast but executables large: hello world is about 460 KB of code and data. With `-Os`, the library is compiled with the program and shaken (Section 17.11), and hello world is about 220 KB, stripped; Section 15.5 compares the builds. The original goal was under 100 KB. The shaking is by name and class, so a call of a generic function such as `print-object`, `make` or `as` keeps every method that could apply to a class with instances, and those reach much of the library; getting further needs an analysis of which classes flow to which calls, which the compiler does not yet do.

A program built `-Os` may also run slower than one built `-O2`, as C optimized for size often does: the vector-summing loop of Section 18.2 took 0.27 s built `-Os` against 0.16 s, though `tak` and the float loop ran at nearly the same speed.

18.5 Measuring

An ordinary build is compiled with `-g`, so the usual tools for C programs work on compiled Dylan, and its `#line` directives make them name Dylan source lines. On Linux, `perf` is the simplest:

```
$ bin/dylanc takp.dyl
$ perf stat -e task-clock ./takp
15
             78.05 msec task-clock:u
       0.078615932 seconds time elapsed
$ perf record ./takp
$ perf report --stdio
    50.04%  takp     takp                  [.] fn1190_tak
    49.68%  takp     takp                  [.] dy_gf_tak_e10c5970_dispatch
...
```

The functions are named as the compiler names them (Section 17.13): `fn1190_tak` is a method `tak`; `dy_gf_tak_e10c5970_dispatch` is the dispatch function of the generic function `tak`. Here, with the method specialized on `<integer>`, half the time goes to dispatch — the case of the second table above. Other names to recognize: `_spec` marks a specialized entry, `dy_maker_`*class* a class's maker, and `GC_`*name* the collector; time in the collector's functions is time spent allocating and collecting. `perf annotate` shows the hot instructions with the Dylan lines they came from.

For timing inside a program, the `dylan` module exports `(current-milliseconds)`, an extension in both implementations that answers a millisecond count from a fixed but arbitrary moment, as the benchmark driver uses it.

18.6 Tuning the garbage collector

The collector is configured by environment variables, which it reads when the program starts; nothing in Dylan needs to change. The Memory Pool System, the default collector (Section 15.8), reads these:

`DYLAN_MPS_NURSERY` The nursery's size in kilobytes, 131 072 (128 MB) by default. New objects go here, and it is collected when full: a larger nursery collects less often, and more of what it holds has died by then. A much smaller one collects too often, each time protecting again the older objects the program is writing into and taking a fault on each page it then writes; with 32 MB, programs that build and keep writing into a large table or vector ran slower than under the Boehm collector, and with 128 MB faster. The nursery is also the floor of the memory a program that allocates heavily uses: a program that must run in little memory can set, say, 32 768.

`DYLAN_MPS_EXTEND` How much memory, in bytes, a pool takes from the system at a time: a megabyte by default. The collector's own default, which is small, costs a system call for each.

`DYLAN_MPS_SPARE` The fraction of the memory the collector holds that it may keep unused rather than return to the system, between 0 and 1; the default is 1, keeping everything. Memory returned must be faulted in afresh when the nursery next fills, which for an allocation-heavy program took longer than the collecting: with 0.75, the MPS's own

default, the benchmarks of Section 18.1 took 15 to 20 percent longer and used a quarter to a half less memory. A program's memory under the MPS is otherwise about twice what it is under the Boehm collector, since the collector lets the heap double between collections of everything.

`DYLAN_MPS_PAUSE` The longest the collector should work at once, in seconds, before giving the program a turn; the collector's default is 0.1. The MPS collects incrementally, protecting the memory it has yet to scan; a shorter pause means more interruptions and more protection faults.

The Boehm-Demers-Weiser collector. A program built with `--gc boehm` reads the Boehm collector's variables instead. These were checked with that collector (version 8.2.6), with a program that builds and discards 200 lists of 100 000 elements. `GC_PRINT_STATS=1` makes the collector report each collection on the standard error, which is how the counts below were made.

`GC_INITIAL_HEAP_SIZE` The heap's starting size, in bytes or with a suffix such as `64M` or `1G`. The heap starts small and grows; a larger start means fewer collections. The test program made 202 collections by default and 10 with `64M`. Fewer collections are not automatically faster: here the larger heap made the run slower (0.24 s against 0.18 s), because touching fresh memory costs more than the small collections it saved. It helps programs that keep much live data.

`GC_FREE_SPACE_DIVISOR` How eagerly the heap grows rather than collecting; the default is 3, and smaller values grow the heap more and collect less. With 1, the test program made 101 collections instead of 202 and used about 13 MB instead of 7 MB.

`GC_MARKERS` How many threads mark in parallel during a collection. The runtime sets two, since more spend longer waiting on each other than marking; `GC_MARKERS=1` turns parallel marking off, and a larger number is allowed.

`GC_PRINT_STATS` Set to 1 to report each collection.

```
$ GC_PRINT_STATS=1 ./alloc 2>&1 | grep -c 'Marking for collection'
202
$ GC_INITIAL_HEAP_SIZE=64M GC_PRINT_STATS=1 ./alloc 2>&1 |
      grep -c 'Marking for collection'
10
```

The collectors' own documentation lists further variables. As with the compiler's options, measure your program before and after: the right settings depend on how much it allocates and how much it keeps.

Appendix A

Differences from the 1992 Manual

The standard for both implementations is the 1992 manual, *Dylan: an object-oriented dynamic language* (*DIRM*). Where an implementation departs from it — by necessity, by a decision where the manual is silent or contradicts itself, or by adding something — this appendix says so. It collects in one place what the other chapters mention as they go. Each entry says which implementation it concerns: *both*, the *interactive* Common Lisp implementation, or the *compiler* to C.

A.1 Different behavior, in both implementations

`!key`, `!rest` and `!next`. These are read as `#key`, `#rest` and `#next`, as Thomas read them, because Thomas's programs — the book's examples among them — use them. So they cannot be used as variable names. Every other name the manual allows is allowed. `((method (a !key (b 2)) (list a b)) 1 b: 5)` ⇒ `(1 5)`.

Modules. The 1992 manual describes modules but leaves the way they are declared to the programming environment. Both implementations declare them with `define-module`, as design note 32 describes, written in prefix form, and name a file's module with a `module:` header, as design note 33 describes. Chapter 12 has the details.

Backquote. Chapter 3 of the manual defers backquote to "a separate section of this manual" that the manual does not contain. Both implementations follow the usual Lisp rules for `` ` ``, `,` and `,@`, nesting included: `` `(1 ,(+ 1 1) ,@(list 3 4)) `` ⇒ `(1 2 3 4)`.

`map-into`. The function is passed only the elements of the source collections, as chapter 12's prose and design note 27 say. The chapter's one example, `(map-into x + '(1 2 3 4))` with `x` holding `(100 100 200 200)`, shows `(101 102 203 204)`, which would require passing the target's elements too; here it answers `(1 2 3 4)`.

Irrational functions. `sqrt`, `exp`, `log`, the trigonometric functions and their kin answer floats for exact arguments, as in Common Lisp, which the manual cites for them, and as the manual's examples show: `(sqrt 25)` ⇒ `5.0`, `(exp 0)` ⇒ `1.0`. Rationals are otherwise exact: `(/ 6 4)` ⇒ `3/2`.

Method specificity among unrelated classes. The manual orders applicable methods by comparing their specializers, but leaves open the order of two classes neither of which is a

subclass of the other. Both implementations follow CLOS: the class that comes first in the *argument's* class precedence list is more specific.

```
(define-class <a> (<object>))
(define-class <b> (<object>))
(define-class <c> (<a> <b>))
(define-method which ((x <a>)) 'a)
(define-method which ((x <b>)) 'b)
(which (make <c>))                ; => a
```

A.2 Additions in both implementations

Printing. `display`, `newline`, `print`, `write-line`, `write` and `pp`, and the generic function `print-object`, to which programs add methods to control how their objects print (Chapter 13). The manual uses `print` in its examples without defining it.

Streams. `<stream>`, `*standard-output*`, `*standard-error*`, `<string-stream>` with `stream-contents`, and `format`; every printing function takes `stream:`. The manual uses streams and `format` but does not define them. There is no input.

Macros. `define-macro`, `define-for-macros` and `gensym` (Chapter 16). Under the compiler, macro bodies run in a compile-time environment apart from the program; in the interactive implementation, in the program's own.

Timing. `current-milliseconds`, a clock in milliseconds from a fixed but arbitrary moment.

A.3 The interactive implementation

Lisp access. `lisp-variable` and `lisp-procedure` reach Common Lisp values and functions from Dylan. Lisp's `NIL` arrives as the empty list, not `#f` (Chapter 14).

`#t` and `#f`. Their class, as `object-class` answers it, is `<object>`; the manual names no class for them. (The compiler differs; see below.)

No threads. The interactive implementation has none of the compiler's threads (Chapter 16).

A.4 The compiler to C

A.4.1 What a static program cannot do

The compiler compiles whole, static programs, whose classes and generic functions are fixed when they are compiled (Section 15.12). The manual allows a running program to change them; a compiled program cannot:

- `add-method`, `remove-method`, `add-slot`, `remove-slot` and `freeze-methods` are compile errors, wherever they appear.
- Each specializer of a method made with `define-method`, and each superclass of a class, must be known when compiling: a class, a constant variable holding one, `(singleton` *`x`*`)`, or `(make <singleton> object:` *`x`*`)` of a constant. A `define-slot` owner may be any singleton, but a class owner must be known when compiling.

- A class cannot be defined twice.
- `(make <class> ...)` is an error when it runs. A `<generic-function>` made by `make` has no methods, and none can be added.
- There is no evaluation or loading of code at run time.

A.4.2 Numbers

- **Integers are 63 bits**, from -2^{62} to $2^{62}-1$. There are no bignums. A literal out of range is a compile error; an arithmetic result out of range signals "integer overflow (integers are 63 bits)" rather than giving a wrong answer. The numerator and denominator of a ratio have the same limit. The interactive implementation has bignums: `(* 4611686018427387904 2)` ⇒ `9223372036854775808` there, while in a compiled program even the literal is an error.
- **No complex numbers.** `<complex>` exists, abstract, as the superclass of `<real>` (so `(instance? 3 <complex>)` ⇒ `#t`), but it has no instances. A complex literal is a compile error, and `make-rectangular` with a non-zero imaginary part is an error. A function whose result would be complex signals an error in both implementations: `(sqrt -4.0)` signals "sqrt: the result would be complex".
- `<single-float>` is a C `float`; `<double-float>` and `<extended-float>` are both a C `double`. Floats print as the shortest decimal that reads back: `21.0`, and `1.5s0` for a single float.

A.4.3 Conditions without a debugger

There is no interactive debugger. An unhandled serious condition prints `Dylan error:` and its message on the standard error and ends the program with status 1 — or, in a thread other than the main one, ends that thread (Chapter 16). An unhandled warning prints `Dylan warning:` and its message and continues. `break` prints its message and returns `#f`. `restart-query` and `return-query` do nothing.

A.4.4 Representation

`#t` and `#f` are instances of an internal class, `<boolean>`, which `object-class` answers but which the `dylan` module does not export (the manual names no class for them). `seal` works as the manual says, and `make-read-only` answers its argument, a class, unchanged, since a compiled program's classes are already fixed.

A.4.5 Additions

All of these are extensions; the 1992 manual has none of them.

- **Seeing macro expansions**: `dylanc -E` (Chapter 16). The macros themselves are in both implementations (Section A.2).
- **Threads**: `make-thread`, `join-thread`, `current-thread`, `thread-yield`; `<lock>`, `<recursive-lock>`, `acquire-lock`, `release-lock` and `with-lock`; `<condition-variable>` with `wait-condition-variable`, `notify-condition-variable`

and `notify-all-condition-variable`; `<atomic-integer>` with `atomic-value`, `atomic-add!` and `atomic-compare-and-swap!`; `define-thread-variable` and `dynamic-bind`; `<thread-error>` with `thread-error-thread` and `thread-error-condition` (Chapter 16).

A.5 Where the two implementations disagree

Besides the differences above, the implementations disagree where the manual leaves a choice open or where the host shows through. A portable program depends on none of these; each is described where it arises.

- Assigning a read-only module variable is a compile error in the compiler to C and an error when it runs in the interactive implementation, which also needs modules declared before use (Sections 12.8 and 12.7).
- A compiled program that an unhandled error stops reports it on the standard error, `Dylan error:` and the message; the interactive implementation enters its debugger with the same message (Section 11.11).

Appendix B

Library Reference

This appendix lists every name the module `dylan` exports, in either implementation, and the special forms of the language. Each entry gives the name with its parameters, as a call would be written, and a one-line description. The full definitions are in the DIRM, cited by chapter at the head of each section, and in the chapters of this book.

In a signature, `#rest` and `#key` introduce rest and keyword parameters as in a method's parameter list (Chapter 6). A name in angle brackets is a class. Where a function answers several values, the description says so.

The narrow last column says which implementations provide the name.

- An empty column means both do.
- C means only the compiler to C provides it.
- L means only the interactive (Common Lisp) implementation provides it.

Every entry was checked by referring to it in a small program compiled with `bin/dylanc` and by evaluating it at the interactive implementation's read-eval-print loop. A name marked L is either unknown to the compiler to C or rejected by it when compiling: the compiler reports `add-method`, `remove-method`, `add-slot`, `remove-slot` and `freeze-methods` as impossible in a static program, and `<rectangular-complex>` and `(setter direct-superclasses)` as not supported.

B.1 Objects, equality and comparison

The root class and the operations every object supports.

Name	Description
`<object>`	The class of all objects; every class has it as a superclass.
`(id? object1 object2 #rest objects)`	True if the arguments are all the same object.
`(= object1 object2 #rest objects)`	True if the arguments are all equal, as `binary=` decides.
`(/= object1 object2)`	True if the arguments are not equal.
`(binary= object1 object2)`	The generic equality test on which `=` is built; add methods to it to define equality for a class.

Name	Description	
`(=hash object)`	A hash code for `object` consistent with `=`: equal objects answer equal codes.	
`(< object1 object2 #rest objects)`	True if each argument is less than the next, as `binary<` decides.	
`(> object1 object2 #rest objects)`	True if each argument is greater than the next.	
`(<= object1 object2 #rest objects)`	True if each argument is less than or equal to the next.	
`(>= object1 object2 #rest objects)`	True if each argument is greater than or equal to the next.	
`(binary< object1 object2)`	The generic ordering on which `<`, `>`, `<=` and `>=` are built; methods exist for reals, characters and strings.	
`(not object)`	`#t` if `object` is `#f`, otherwise `#f`.	
`(identity object)`	Answers `object`.	

B.2 Classes, instances and slots

Making objects and examining classes (*DIRM* chapter 11). The compiler to C fixes a program's classes when it compiles it, so it rejects `add-slot` and `remove-slot` (Chapter 15).

Name	Description	
`<class>`	The class of classes.	
`<singleton>`	The class of singleton specializers, each matching exactly one object.	
`(singleton object)`	A singleton specializer for `object`.	
`<slot-descriptor>`	The class of the objects that describe slots.	
`(make class #rest key-value-pairs)`	Makes and initializes a new instance of `class`, passing it the init arguments.	
`(initialize instance #rest init-arguments)`	Called by `make` on each new instance; add methods to it to finish initializing instances of a class.	
`(instance? object type)`	True if `object` is a general instance of `type` (a class or singleton).	
`(subclass? class1 class2)`	True if `class1` is `class2` or one of its subclasses.	
`(object-class object)`	The class of which `object` is a direct instance.	
`(all-superclasses class)`	The class and all its superclasses, in precedence order.	
`(direct-superclasses class)`	The class's direct superclasses.	
`((setter direct-superclasses) class superclasses)`	Changes the class's direct superclasses.	L
`(direct-subclasses class)`	The class's direct subclasses.	
`(as class object)`	Converts `object` to an instance of `class`, for example a list to a vector or a character to an integer.	
`(shallow-copy object)`	A new object like `object`, sharing its contents.	
`(class-for-copy object)`	The class that `map`, `copy-sequence` and the like use for a copy of `object`.	
`(seal class)`	Makes the class read-only and forbids new subclasses of it; answers it.	

Name	Description	
`(make-read-only object)`	Makes a class or generic function read-only, so that its slots or methods cannot change; answers it.	
`(slot-descriptors class)`	The descriptors of the class's slots.	
`(add-slot owner #key getter setter type init-value init-function init-keyword allocation)`	Adds a slot to a class (or singleton) that may already have instances.	L
`(remove-slot owner slot-descriptor)`	Removes a slot.	L
`(slot-descriptor instance getter)`	The descriptor of the slot of `instance` that `getter` reads.	
`(slot-getter slot-descriptor)`	The slot's getter, or `#f`.	
`(slot-setter slot-descriptor)`	The slot's setter, or `#f`.	
`(slot-type slot-descriptor)`	The slot's type.	
`(slot-allocation slot-descriptor)`	The slot's allocation, a symbol such as `instance` or `class`.	
`(init-value slot-descriptor)`	The slot's init value, and a second value saying whether it has one.	
`(init-function slot-descriptor)`	The slot's init function, or `#f`.	
`(init-keyword slot-descriptor)`	The slot's init keyword, or `#f`.	
`(slot-value instance slot-descriptor)`	The value of the slot in `instance`.	
`((setter slot-value) instance slot-descriptor new-value)`	Sets the slot's value.	
`(slot-initialized? instance getter)`	True if the slot `getter` reads has a value in `instance`.	

B.3 Functions and generic functions

Calling functions and examining and changing generic functions (*DIRM* chapter 10). The compiler to C rejects `add-method`, `remove-method` and `freeze-methods`, and a `<generic-function>` it makes with `make` has no methods.

Name	Description	
`<function>`	The class of all functions.	
`<generic-function>`	The class of generic functions.	
`<method>`	The class of methods.	
`(apply function #rest arguments)`	Calls `function`; the last argument is a sequence of further arguments.	
`(values #rest objects)`	Answers its arguments as multiple values.	
`(add-method generic-function method)`	Adds a method to a generic function, replacing one with the same specializers; answers the method and the one replaced, or `#f`.	L
`(remove-method generic-function method)`	Removes a method from a generic function.	L
`(freeze-methods generic-function #rest specializers)`	Forbids changes to the methods applicable to arguments of the given classes.	L
`(generic-function-methods generic-function)`	The generic function's methods.	
`(method-specializers method)`	The specializers of the method's required parameters.	

Name	Description
`(function-arguments function)`	Three values: the number of required arguments, whether there is a `#rest` argument, and the keywords accepted.
`(applicable-method? function #rest sample-arguments)`	True if `function` could be called with such arguments.
`(sorted-applicable-methods generic-function #rest sample-arguments)`	The methods applicable to such arguments, most specific first, and those whose order is ambiguous.
`(find-method generic-function specializers)`	The method with exactly these specializers, or `#f`.

B.4 Functional operations

Functions that make functions (*DIRM* chapter 17).

Name	Description
`(compose function1 #rest functions)`	A function that applies the functions from right to left.
`(complement predicate)`	A function that answers `#t` where `predicate` answers `#f`, and `#f` otherwise.
`(disjoin predicate1 #rest predicates)`	A function true when any of the predicates is true of its arguments.
`(conjoin predicate1 #rest predicates)`	A function true when all of the predicates are true of its arguments.
`(curry function #rest arguments)`	A function that calls `function` with these arguments first, then its own.
`(rcurry function #rest arguments)`	A function that calls `function` with its own arguments first, then these.
`(always object)`	A function that ignores its arguments and answers `object`.

B.5 Numbers

The number classes and arithmetic (*DIRM* chapter 14). In the compiler to C integers are 63 bits, overflow is an error, and there are no complex numbers: `<complex>` has no instances, `make-rectangular` and `make-polar` signal an error unless the result is real. The interactive implementation has Common Lisp's unlimited integers and complex numbers. In both, a function whose result would be complex for a real argument, such as `(sqrt -4)`, signals an error.

Name	Description	
`<number>`	The class of all numbers.	
`<complex>`	Complex numbers, the superclass of `<real>`.	
`<rectangular-complex>`	Complex numbers with a nonzero imaginary part.	L
`<real>`	Real numbers.	
`<float>`	Floating-point numbers.	
`<single-float>`	Single-precision floats.	

Name	Description
`<double-float>`	Double-precision floats, the class of float literals such as `1.5`.
`<extended-float>`	Extended-precision floats.
`<rational>`	Exact rational numbers.
`<ratio>`	Ratios that are not integers, such as `3/4`.
`<integer>`	Integers.
`(odd? integer)`	True if `integer` is odd.
`(even? integer)`	True if `integer` is even.
`(zero? number)`	True if `number` is zero.
`(positive? real)`	True if `real` is greater than zero.
`(negative? real)`	True if `real` is less than zero.
`(integral? number)`	True if `number` has an integer value.
`(+ #rest numbers)`	The sum; `(+)` is `0`.
`(* #rest numbers)`	The product; `(*)` is `1`.
`(- number #rest numbers)`	The difference, or the negation of a single argument.
`(/ number #rest numbers)`	The quotient, or the reciprocal of a single argument. Integers divide exactly, giving a ratio when needed.
`(binary+ number1 number2)`	The generic addition `+` is built on; similarly the next five.
`(binary- number1 number2)`	Generic subtraction.
`(binary* number1 number2)`	Generic multiplication.
`(binary/ number1 number2)`	Generic division.
`(unary- number)`	Generic negation.
`(unary/ number)`	Generic reciprocal.
`(floor real)`	The greatest integer not above `real`, and the remainder.
`(ceiling real)`	The least integer not below `real`, and the remainder.
`(round real)`	The nearest integer (to even on a tie), and the remainder.
`(truncate real)`	The integer part, toward zero, and the remainder.
`(floor/ real1 real2)`	The quotient rounded down, and the remainder.
`(ceiling/ real1 real2)`	The quotient rounded up, and the remainder.
`(round/ real1 real2)`	The quotient rounded to nearest, and the remainder.
`(truncate/ real1 real2)`	The quotient truncated toward zero, and the remainder.
`(modulo real1 real2)`	The remainder of `floor/`, with the sign of `real2`.
`(remainder real1 real2)`	The remainder of `truncate/`, with the sign of `real1`.
`(abs number)`	The absolute value.
`(magnitude number)`	The magnitude (the absolute value of a real).
`(min real #rest reals)`	The smallest argument.
`(max real #rest reals)`	The largest argument.
`(gcd #rest integers)`	The greatest common divisor.
`(lcm #rest integers)`	The least common multiple.
`(binary-gcd integer1 integer2)`	The generic function `gcd` is built on.
`(binary-lcm integer1 integer2)`	The generic function `lcm` is built on.
`(numerator rational)`	The numerator, in lowest terms.

Name	Description
`(denominator rational)`	The denominator, in lowest terms.
`(rationalize number)`	A simple rational close to `number`.
`(sqrt number)`	The square root, a float (`(sqrt 25)` is `5.0`).
`(exp number)`	*e* raised to the power `number`.
`(log number)`	The natural logarithm.
`(expt number1 number2)`	`number1` raised to the power `number2`; exact for an integer power of an exact number.
`(sin number) (cos number) (tan number)`	The trigonometric functions, in radians.
`(asin number) (acos number) (atan number)`	Their inverses.
`(atan2 number1 number2)`	The arc tangent of `number1/number2`, using both signs to choose the quadrant.
`(sinh number) (cosh number) (tanh number)`	The hyperbolic functions.
`(asinh number) (acosh number) (atanh number)`	Their inverses.
`(make-rectangular real imaginary)`	The complex number with these parts.
`(make-polar magnitude angle)`	The complex number with this magnitude and angle.
`(real-part number)`	The real part.
`(imag-part number)`	The imaginary part (`0` for a real).
`(angle number)`	The angle of a complex number in radians.
`(logior #rest integers)`	Bitwise inclusive or.
`(logxor #rest integers)`	Bitwise exclusive or.
`(logand #rest integers)`	Bitwise and.
`(logeqv #rest integers)`	Bitwise equivalence (exclusive nor).
`(lognot integer)`	Bitwise complement.
`(lognand integer1 integer2)`	The complement of `logand`.
`(lognor integer1 integer2)`	The complement of `logior`.
`(logandc1 integer1 integer2)`	The and of the complement of `integer1` with `integer2`.
`(logandc2 integer1 integer2)`	The and of `integer1` with the complement of `integer2`.
`(logorc1 integer1 integer2)`	The or of the complement of `integer1` with `integer2`.
`(logorc2 integer1 integer2)`	The or of `integer1` with the complement of `integer2`.
`(logbit? index integer)`	True if bit `index` of `integer` is 1.
`(ash integer count)`	`integer` shifted left `count` bits (right if `count` is negative).

B.6 Characters, strings and symbols

Characters (*DIRM* chapter 16), symbols and keywords (*DIRM* chapter 15), and the case conversions. The string classes are collections, listed with them below.

Name	Description
`<character>`	The class of characters.
`<symbol>`	The class of symbols.

Name	Description
`<keyword>`	The class of keywords such as `foo:`.
`(as-uppercase object)`	The upper-case version of a character, or a new upper-case string.
`(as-lowercase object)`	The lower-case version of a character, or a new lower-case string.
`(as-uppercase! string)`	Converts `string` to upper case in place.
`(as-lowercase! string)`	Converts `string` to lower case in place.

B.7 Collection classes

The classes of the collection library (*DIRM* chapter 12).

Name	Description
`<collection>`	All collections.
`<explicit-key-collection>`	Collections whose keys are not simply indices, such as tables.
`<mutable-collection>`	Collections whose elements can be changed.
`<mutable-explicit-key-collection>`	Mutable explicit-key collections.
`<sequence>`	Collections whose keys are the integers from 0.
`<mutable-sequence>`	Sequences whose elements can be changed.
`<array>`	Multi-dimensional arrays; make one with `dimensions:`.
`<vector>`	One-dimensional arrays.
`<simple-object-vector>`	The vectors `vector` makes; they hold any object.
`<stretchy-vector>`	Vectors that grow as elements are added.
`<string>`	Strings, the superclass of the two below.
`<byte-string>`	Strings of 8-bit characters; string literals are byte strings.
`<unicode-string>`	Strings of any characters.
`<deque>`	Double-ended queues.
`<list>`	Lists: the empty list and pairs.
`<pair>`	Pairs (cons cells).
`<empty-list>`	The class of the empty list, `()`.
`<range>`	Arithmetic sequences, possibly unbounded.
`<table>`	Hash tables, keyed by any object, compared with `=`.

B.8 Collection operations

Operations on every collection, and the iteration protocol that a new collection class implements to take part in them.

Name	Description
`(element collection key #key default)`	The element at `key`; `default` if there is none (an error if no default is given).
`((setter element) collection key new-value)`	Sets the element at `key`. Usually written `(set! (element c k) v)`.

Name	Description
`(size collection)`	The number of elements, or `#f` for an unbounded collection.
`((setter size) sequence new-size)`	Changes the size of a stretchy sequence.
`(empty? collection)`	True if the collection has no elements.
`(do procedure collection #rest collections)`	Calls `procedure` on the elements, in parallel across the collections; answers `#f`.
`(map procedure collection #rest collections)`	A new collection of the results of `procedure` on the elements.
`(map-as class procedure collection #rest collections)`	As `map`, but the result is an instance of `class`.
`(map-into collection procedure source #rest sources)`	Stores the results of `procedure` on the sources' elements into `collection`; answers it. (Only the sources' elements are passed; see Appendix A.)
`(any? procedure collection #rest collections)`	The first true result of `procedure` on the elements, or `#f`.
`(every? procedure collection #rest collections)`	True if `procedure` is true of every element.
`(reduce procedure initial-value collection)`	Combines the elements with `procedure`, starting from `initial-value`.
`(reduce1 procedure collection)`	As `reduce`, starting from the first element.
`(member? value collection #key test)`	True if `value` is an element (compared with `test`, default `id?`).
`(find-key collection procedure #key skip failure)`	The key of the first element (after skipping `skip` matches) for which `procedure` is true; `failure` (default `#f`) if none.
`(replace-elements! collection predicate new-value-function #key count)`	Replaces each element satisfying `predicate` with `new-value-function` of it, at most `count` times.
`(fill! collection value #key start end)`	Stores `value` into every element (of a sequence, from `start` to `end`).
`(key-sequence collection)`	A sequence of the collection's keys.
`(initial-state collection)`	The state of the first element, or `#f` if there is none.
`(next-state collection state)`	The state of the next element, or `#f` at the end.
`(current-element collection state)`	The element at `state`.
`((setter current-element) collection state new-value)`	Replaces the element at `state`.
`(current-key collection state)`	The key of the element at `state`.
`(final-state collection)`	The state of the last element of a sequence, or `#f`.
`(previous-state collection state)`	The state of the element before `state`, or `#f`.
`(copy-state collection state)`	A copy of `state` that can be advanced independently.

B.9 Sequence operations

Operations on sequences. Unless `test:` is given, elements are compared with `id?`.

Name	Description
`(add sequence new-element)`	A new sequence with `new-element` added (at the front of a list).

Name	Description
`(add! sequence new-element)`	Adds `new-element`, altering `sequence` if it can; answers the result.
`(add-new sequence new-element #key test)`	As `add`, unless the element is already present.
`(add-new! sequence new-element #key test)`	As `add!`, unless the element is already present.
`(remove sequence value #key test count)`	A new sequence without the elements equal to `value` (at most `count` of them).
`(remove! sequence value #key test count)`	As `remove`, altering `sequence` if it can.
`(choose predicate sequence)`	A new sequence of the elements satisfying `predicate`.
`(choose-by predicate test-sequence value-sequence)`	The elements of `value-sequence` whose corresponding elements of `test-sequence` satisfy `predicate`.
`(intersection sequence1 sequence2 #key test)`	The elements of `sequence1` that are also in `sequence2`.
`(union sequence1 sequence2 #key test)`	The elements of either sequence, without duplicates between them.
`(remove-duplicates sequence #key test)`	A new sequence with each element only once.
`(remove-duplicates! sequence #key test)`	As `remove-duplicates`, altering `sequence` if it can.
`(copy-sequence sequence #key start end)`	A new sequence of the elements from `start` to `end`.
`(concatenate sequence #rest sequences)`	A new sequence of all the elements, of the first sequence's kind.
`(concatenate-as class sequence #rest sequences)`	As `concatenate`, but an instance of `class`.
`(replace-subsequence! sequence insert-sequence #key start)`	Overwrites elements of `sequence`, from `start`, with those of `insert-sequence`.
`(reverse sequence)`	A new sequence with the elements in reverse order.
`(reverse! sequence)`	As `reverse`, reusing `sequence` if it can.
`(sort sequence #key test stable)`	A new sequence sorted by `test` (default `<`).
`(sort! sequence #key test stable)`	As `sort`, reusing `sequence` if it can.
`(first sequence #key default)` `(second sequence #key default)` `(third sequence #key default)`	The first, second and third elements, or `default` if there is none.
`(last sequence #key default)`	The last element, or `default` if the sequence is empty.
`((setter first) sequence value)` and likewise `second, third, last`	Set the first, second, third and last elements.
`(subsequence-position big pattern #key test count)`	The index at which `pattern` occurs in `big` (the `count`th occurrence), or `#f`.

B.10 Lists

Pairs and lists.

Name	Description
`(cons car cdr)`	A new pair.
`(list #rest objects)`	A new list of the arguments.
`(list* #rest objects)`	A new list of the arguments, the last of which is its tail.
`(car list)` `(cdr list)`	The first element, and the rest, of a pair; both are `()` for `()`.
`(caar list)` ... `(cdddr list)`	The compositions of `car` and `cdr` to three levels: `caar`, `cadr`, `cdar`, `cddr`, `caaar`, `caadr`, `cadar`, `caddr`, `cdaar`, `cdadr`, `cddar`, `cdddr`.
`((setter car) pair value)` and likewise for the others	Store into a pair; every `c...r` above has a setter.
`(append list #rest lists)`	A new list of the elements of all the lists.
`(find-pair item list #key test)`	The first pair in `list` whose `car` matches `item`, or `#f`.

B.11 Vectors, arrays, deques, ranges and tables

Operations specific to particular collections.

Name	Description
`(vector #rest objects)`	A new simple object vector of the arguments.
`(aref array #rest indices)`	The element of `array` at the indices.
`((setter aref) array #rest indices-and-new-value)`	Sets an element of an array.
`(dimensions array)`	A sequence of the array's dimensions.
`(push deque new-value)`	Adds `new-value` at the front of the deque.
`(pop deque)`	Removes and answers the first element.
`(push-last deque new-value)`	Adds `new-value` at the end of the deque.
`(pop-last deque)`	Removes and answers the last element.
`(range #key from up-to through by size)`	A range from `from` (default 0) by `by` (default 1), stopping before `up-to`, at `through`, or after `size` elements; unbounded if none is given.
`(remove-key! table key)`	Removes `key` and its element from the table.

B.12 Conditions

Signaling and handling conditions (*DIRM* chapter 13).

Name	Description
`<condition>`	The class of all conditions.
`<serious-condition>`	Conditions that must be dealt with.
`<error>`	Errors.
`<simple-error>`	Errors with a message; init arguments `format-string:` and `format-arguments:`.
`<type-error>`	A value not of the expected type; init arguments `value:` and `type:`.
`<warning>`	Warnings; their default handler prints them and returns `#f`.

Name	Description
`<simple-warning>`	Warnings with a message, like `<simple-error>`.
`<restart>`	The class of restarts.
`<simple-restart>`	A plain restart.
`<abort>`	The restart that abandons the current operation.
`(signal condition #rest arguments)`	Signals a condition (or a `<simple-warning>` made from a format string and arguments); answers what a handler returns.
`(error condition #rest arguments)`	Signals an error (or a `<simple-error>` made from a format string and arguments); never returns.
`(cerror restart-description condition #rest arguments)`	As `error`, but with a restart that makes `cerror` return `#f`.
`(break #rest arguments)`	Enters the debugger with a message made like `signal`'s. The compiler to C, which has no debugger, prints the message and continues.
`(check-type value type)`	Signals a `<type-error>` unless `value` is an instance of `type`; otherwise answers `value`.
`(abort)`	Signals an `<abort>`.
`(default-handler condition)`	Called when no handler takes a condition: a serious condition enters the debugger (in the compiler to C, is reported and stops the program or thread), a warning is printed, and other conditions answer `#f`.
`(do-handlers function)`	Calls `function` with the type, test, function and description of each active handler.
`(restart-query restart)`	Asks the user for what a restart needs (in the compiler to C, does nothing and answers `#f`).
`(return-query condition)`	Asks the user for values to return from `signal` (in the compiler to C, answers `#f`).
`(return-allowed? condition)`	True if a handler may return from this condition.
`(return-description condition)`	A description of what returning from the condition means.
`(condition-format-string condition)`	The format string of a simple error, warning or restart.
`(condition-format-arguments condition)`	Its format arguments.
`(type-error-value type-error)`	The value that had the wrong type.
`(type-error-expected-type type-error)`	The type it should have had.

B.13 Printing and streams

Output (Chapter 13). The 1992 manual defines none of these; both implementations have all of them.

Name	Description
`(print object #key stream verbose)`	Prints `object` escaped on a line of its own; answers it.
`(write-line object #key stream)`	The same as `print`.
`(display object #key stream)`	Prints `object` plain; answers it.

Name	Description
`(newline #key stream)`	Ends the line; answers `#f`.
`(pp object #key stream)`	Prints `object` as `print` does, laid out over lines as code.
`(write object #key stream)`	Prints `object` escaped, with no newline; answers it.
`(print-object object stream escape?)`	The generic function all printing goes through; add methods to change how a class prints.
`(format stream control-string #rest arguments)`	Formatted output: `~a`, `~s`, `~d`, `~%`, `~&`, `~~`. A stream of `#t` is standard output; of `#f`, answer a string.
`<stream>`	The class of streams.
`<string-stream>`	Streams that collect their output in memory.
`*standard-output*`	The stream to standard output.
`*standard-error*`	The stream to standard error.
`(stream-contents string-stream)`	A string of everything written to the stream.

B.14 Threads

An extension of the compiler to C (Chapter 16). The special forms `with-lock`, `define-thread-variable` and `dynamic-bind` are in Section B.16.

Name	Description	
`<thread>`	The class of threads.	C
`(make-thread function)`	Starts a thread that calls `function` with no arguments; answers the thread.	C
`(join-thread thread)`	Waits for the thread to end and answers its function's values; signals a `<thread-error>` if the thread ended with an unhandled condition.	C
`(current-thread)`	The thread that calls it.	C
`(thread-yield)`	Lets other threads run.	C
`<thread-error>`	The error `join-thread` signals for a thread that ended with an unhandled condition.	C
`(thread-error-thread thread-error)`	The thread that failed.	C
`(thread-error-condition thread-error)`	The condition it did not handle.	C
`<lock>`	Mutual-exclusion locks; `(make <lock>)`.	C
`<recursive-lock>`	Locks the holding thread may acquire again.	C
`(acquire-lock lock)`	Waits for and takes the lock.	C
`(release-lock lock)`	Releases the lock.	C
`<condition-variable>`	Condition variables; `(make <condition-variable>)`.	C
`(wait-condition-variable condition-variable lock)`	Releases `lock`, waits for a notification, and takes `lock` again.	C
`(notify-condition-variable condition-variable)`	Wakes one waiting thread.	C
`(notify-all-condition-variable condition-variable)`	Wakes all waiting threads.	C
`<atomic-integer>`	Integers updated atomically; `(make <atomic-integer> value: n)`.	C
`(atomic-value atomic-integer)`	The current value.	C

Name	Description	
`(atomic-add! atomic-integer n)`	Adds `n` atomically and answers the new value.	C
`(atomic-compare-and-swap! atomic-integer old new)`	If the value is `old`, makes it `new` and answers `#t`; otherwise answers `#f`.	C

B.15 Other extensions

Extensions of the library beyond the manual's, in both implementations unless marked.

Name	Description	
`(current-milliseconds)`	The time in milliseconds from a fixed but arbitrary moment, for timing.	
`(gensym #rest prefix)`	A symbol no program can write, for macros (Chapter 16); the optional string begins its name. In the interactive implementation, and to macro bodies under the compiler.	L
`(lisp-variable symbol)`	The value of the Common Lisp variable named `symbol` (Section 14.6).	L
`(lisp-procedure symbol)`	A Dylan function that calls the Common Lisp function named `symbol`.	L

B.16 Special forms and macros

These forms are not functions: each has its own rules for which of its parts are evaluated. They are listed by the shape of a use. The three marked C are extensions of the compiler to C (Chapter 16); `define-macro` and `define-for-macros`, with which programs define macros of their own, are extensions too, in both implementations.

Name	Description
`(quote object)`	The object itself, unevaluated; also written `'object`.
`(begin form ...)`	Evaluates the forms in order; answers the last one's values.
`(if test consequent alternative)`	Evaluates `consequent` if `test` is true, otherwise `alternative`.
`(when test form ...) (unless test form ...)`	Evaluates the forms if `test` is true (false).
`(cond (test form ...) ...(else: form ...))`	Evaluates the forms of the first clause whose test is true.
`(case target ((key ...) form ...) ...(else: form ...))`	Chooses the clause whose keys include the target (compared with `id?`).
`(select target test ((match ...) form ...) ...(else: form ...))`	As `case`, but the matches are evaluated and compared with the function `test`, such as `=` or `instance?`.
`(and form ...) (or form ...)`	Logical and and or, evaluating only as many forms as needed.
`(bind ((var ...init) ...) form ...)`	Binds local variables (several to the values of one form if more than one is named) and evaluates the forms.

Name	Description	
`(set! place new-value)`	Assigns a variable, or calls the setter of `place`: `(set! (f x) v)` is `((setter f) x v)`.	
`(setter name)`	The setter function of `name`, `(setter name)`.	
`(method parameter-list form ...)`	An anonymous method.	
`(bind-methods ((name parameter-list form ...) ...) form ...)`	Local methods that may call each other.	
`(for ((var init step) ...) (end-test result ...) form ...)`	General iteration.	
`(for-each ((var collection) ...) (end-test result ...) form ...)`	Iterates over collections in parallel.	
`(dotimes (var count result) form ...)`	Evaluates the forms with `var` from 0 below `count`.	
`(while test form ...) (until test form ...)`	Repeats the forms while (until) `test` is true; answers `#f`.	
`(bind-exit (var) form ...)`	Binds `var` to an exit procedure that returns from the form.	
`(unwind-protect protected-form cleanup-form ...)`	Evaluates the cleanup forms however `protected-form` is left.	
`(handler-bind (type function #key test description) form ...)`	Evaluates the forms with a condition handler in effect.	
`(handler-case form ((type #key condition test description) form ...) ...)`	Evaluates `form`; if a condition of a listed type is signaled, exits and evaluates that clause.	
`(define name value)`	Defines a module variable.	
`(define-class name (superclass ...) slot-spec ...)`	Defines a class and its slot accessors; the variable is read-only.	
`(define-generic-function name parameter-list)`	Defines a generic function; the variable is read-only.	
`(define-method name parameter-list form ...)`	Adds a method to a generic function, creating it if need be.	
`(define-slot owner keyword value ...)`	Adds a slot to a class or singleton.	
`(define-module name clause ...)`	Declares a module (Chapter 12).	
`` `template ,form ,@form ``	Backquote: a list built from a template, as in Common Lisp.	
`(define-macro name parameter-list form ...)`	Defines a macro (Chapter 16).	
`(define-for-macros form ...)`	Forms run only while compiling, for macros to use.	
`(define-thread-variable name value)`	Defines a variable each thread can rebind.	C
`(dynamic-bind ((name value) ...) form ...)`	Gives thread variables new values in this thread while the forms run.	C
`(with-lock (lock) form ...)`	Holds the lock while the forms run.	C

Appendix C

Testing

Both implementations come with tests. This appendix describes each suite: what it tests, how to run it, how long it takes, how to run a single case, and how to add one. All commands are run from the root of the source tree.

Note: The expected results of the tests come from the 1992 manual, not from either implementation. Where the manual gives an example with its result, the test expects that result; where it specifies behavior without an example, the test is written from the specification and cites the chapter and section. Neither implementation is taken as the reference for the other.

C.1 The suites at a glance

Command	What it tests
`sbcl -script tests/front-end/run.lisp`	the compiler's front end: reading, modules, conversion to core forms, diagnostics
`sbcl -script tests/ir/run.lisp`	the IR and its analysis
`sbcl -script tests/codegen/run.lisp`	the compiler's features, phase by phase: programs compiled, run, and their output compared
`sbcl -script tests/conformance/run.lisp`	the manual's examples and specifications, chapter by chapter
`sbcl -script tests/stress/run.lisp`	10^8 tail calls; many threads at once
`sh tests/driver/run.sh`	the command line, `bin/dylanc`
`make -C runtime test`	the C runtime's data representation
`(dylan:test-examples)`	the interactive implementation, on the book's examples
`sbcl -script tests/interpreter/run.lisp`	the compiler's test programs, run in the interactive implementation

Each command prints one line per failure, beginning `FAIL`, and a summary; the suites of the compiler exit with status 0 when everything passed and 1 otherwise.

C.2 The front end and the IR

`tests/front-end/cases.lisp` holds 55 cases. Each is a list

```
(name "dylan source" expected)
```

where *expected* is either the core forms the source converts to, as the compiler's `unparse` writes them, one per line, or `(:diagnostics "text" ...)`: the diagnostics the source must produce, in order, each given as a string it must contain. For example:

```
(no-complex
 "#c(1 2)"
 (:diagnostics "error: complex numbers are not supported"))
```

After the cases, the runner converts `dylan-examples.dyl`, the book's examples, which must produce no diagnostic but the one error that its page 68 example is meant to cause.

`tests/ir/cases.lisp` holds 18 cases of the form

```
(name "dylan source" "expected IR" check ...)
```

The expected IR is the A-normal form of each top-level form; each *check* names a variable or function and properties it must have — whether a variable is captured or assigned, how many references it has, whether a function escapes, where it is called from and which of those calls are tail calls, and so on (the file's header lists them). The IR of `dylan-examples.dyl` must then pass the checker of the IR's invariants.

Neither suite runs a C compiler. Given the argument `print`, each shows what every case converts to, which is how to write the expected text of a new case:

```
$ sbcl --script tests/front-end/run.lisp print
```

C.3 Programs compiled and run

Three suites — `tests/codegen`, `tests/conformance` and `tests/stress` — consist of whole Dylan programs that are compiled, built, run, and whose output is compared with what the test expects. They share one runner, `tests/codegen/runner.lisp`.

`tests/codegen/cases*.lisp` 108 programs testing the compiler's features phase by phase: `cases.lisp` (calls, tail calls, closures, `#rest` and `#key`, multiple values, run-time errors), `cases-control.lisp` (exits, cleanups, conditions), `cases-objects.lisp` (classes, slots, dispatch, `next-method`, reflection), `cases-library.lisp`, `cases-threads.lisp`, `cases-macros.lisp` and `cases-optimize.lisp` (type inference, inlining, unboxing, escape analysis).

`tests/conformance/ch*.lisp` 237 programs, one file for each chapter of the manual from 2 to 18: the manual's examples with the results it gives, and its specifications, each test citing its section. What a static program cannot do is not tested here; it is listed in Appendix A.

`tests/stress/cases-*.lisp` 11 programs that take minutes: 10^8 tail calls through unknown functions, generic functions, closures, `next-method` and `apply`, each in a 256 KB stack; and many threads allocating, contending for a bounded buffer, interning symbols, signalling conditions, binding thread variables and making threads.

C.3.1 The case format

A file of cases holds one list of cases, each

```
(name "dylan source" "expected standard output"
      [:error "text"] [:compile-error "text"] [:compiler-only t])
```

The program is the source string; the strings follow Lisp's rules, so a double quote inside is written `\"` and a backslash `\\`. The output must be exactly the expected text, newline for newline. With `:error`, the program must instead stop with status 1 and an error message on the standard error containing the text — after writing the expected output. With `:compile-error`, the compiler must report an error containing the text, and the program is not run. `:compiler-only t` marks a case that tests what the interactive implementation does not do by design — 63-bit integers, a static program's restrictions, the compiler's own diagnostics, threads — which `tests/interpreter/run.lisp` (Section C.6) skips. From `cases-threads.lisp`:

```
(thread-values
 "(define t1 (make-thread (method () (values 1 2))))
  (bind ((a b (join-thread t1))) (print (list a b)))
  (print (join-thread t1))
  (define t2 (make-thread (method () (current-thread))))
  (print (list (id? (join-thread t2) t2) (id? (current-thread) t2)))
  (print (instance? t2 <thread>))"
 "(1 2)
1
(#t #f)
#t
")
```

and from `cases-macros.lisp`, a case that must fail to compile:

```
(macro-not-top-level
 "(define f (method () (define-macro m () 1)))"
 ""
 :compile-error "define-macro must be at top level")
```

Threads run in any order, so a test of threads must print only what does not depend on the order. Every test program runs with a 256 KB stack (`ulimit -s 256`), so a call in tail position that was not compiled as a jump overflows it quickly and the test fails.

C.3.2 Modes

Each program is compiled to C once and then built and run in each *configuration* of a *mode*, named on the command line:

`all` (the default) GCC and Clang, each with `musttail` and with trampolines: four builds of each program, the first at `-O2` and the others at `-O0`.

`quick` The first of those only: GCC, `musttail`, `-O2`.

`whole` As `quick`, but with the library compiled with each program and tree shaken (`--whole`), not compiled apart.

`gc-stress` GCC, with `musttail` and with trampolines, each forcing a collection every 1000 allocations (`--gc-stress` with `DYLAN_GC_STRESS=1000`).

`address` AddressSanitizer and UndefinedBehaviorSanitizer, GCC and Clang, in debug builds.

`thread` ThreadSanitizer, GCC, in a debug build.

Every mode builds with the default collector, the Memory Pool System. With the environment variable `DYLAN_GC=boehm` set, it builds with the Boehm-Demers-Weiser collector instead (Section 15.8); the compiler and runtime should pass every suite with either.

```
$ sbcl --script tests/codegen/run.lisp quick
$ sbcl --script tests/conformance/run.lisp            # all
$ sbcl --script tests/stress/run.lisp thread
```

The builds go in `runtime/build/tests/` (a subdirectory named for the mode, except in `all`). The sanitizer and stress modes build their own variants of the runtime library the first time they need them.

C.3.3 Running some of the cases

Further arguments name the case files to run, and the cases: any argument ending in `.lisp` is a file to read instead of the suite's own, and any other (besides the mode) is the name of a case to run, whichever file holds it.

```
$ sbcl --script tests/codegen/run.lisp quick thread-values
$ sbcl --script tests/codegen/run.lisp quick tests/codegen/cases-macros.lisp
$ sbcl --script tests/conformance/run.lisp quick ch08-true-false
```

The summary line counts the programs, the configurations and the failures, as in `1 program in 1 configuration, 0 failures`.

C.3.4 How long they take

The programs are built and run one after another, so the time grows with the number of programs and configurations. A build takes one to two seconds once the compiler and the library's object are cached (Section 15.4); a configuration whose C compiler and options are new must first compile the library, which takes some seconds more. On the 32-thread machine of Chapter 18, the conformance suite in mode `all` — 237 programs in four configurations — took about a quarter of an hour; `quick` needs a fraction of that. The `address`, `thread` and `gc-stress` modes are much slower, because their programs run slowly, and the stress suite takes minutes in any mode. The front-end and IR suites run no C compiler and are quick. Use `quick` while working, and `all` and the other modes before committing a change to the compiler or runtime.

C.4 The command line

`sh tests/driver/run.sh` tests `bin/dylanc` itself, in a temporary directory: the default executable name and `-o`; that the C is removed unless `-k` is given, and what `-k` keeps; the `#line` directives and `--no-lines`; `-C`, and compiling its C by hand; each `-O` level, `-Os` and `-g`; `--trampoline`, `--cc`, `CC`, `--lto` with GCC and with Clang, and `--whole`; the library's object cache, and that two programs' library C is the same text; a program of several modules, with a file for each; a macro exported from its module, and `-E`; run-time errors and their exit status; and the exit status of compile errors and of bad command lines. It prints `FAIL` and a description for each check that fails, and a count of checks and failures.

C.5 The runtime

`make -C runtime test` builds and runs `runtime/build/test-runtime`, a C program that tests the runtime's data representation without any Dylan program: the built-in classes as the compiler emits them (which `tests/runtime/generate.lisp` writes): immediate values, whether each class is a subclass of each other, how constants print, integer arithmetic at the edges of the 63-bit range and its errors, other numbers, strings and symbols, the collector, and threads. It runs in moments once built.

C.6 The interactive implementation

The interactive implementation is tested on the examples from the Dylan book, `dylan-examples.dyl`:

```
$ cd src
$ sbcl --eval '(load "load-dylan.lisp")' \
       --eval '(load "full-test.lisp")' \
       --eval '(dylan:test-examples)'
```

`test-examples` reads the file form by form, echoes each form, evaluates it, and prints its values — or, for a form that signals a condition, `;Error:` and the condition's message, after which it goes on with the next form. It does not compare the results itself. The expected results are in `src/reference/`, the transcripts Thomas produced from the same file in 1992 (`MIT.output` among them); compare the output with them. Where they and the manual disagree, the manual decides; the few remaining differences are listed in `src/README`. Run it from the `src` directory, as shown, since `test-examples` finds the examples at `../dylan-examples.dyl`; give it another file name as its argument to run other examples the same way.

The programs the compiler is tested on run in the interactive implementation too, by `tests/interpreter/run.lisp`, which is how the two implementations are kept in agreement. It takes files of cases in the form of Section C.3.1, and test names, as the compiler's runners do; by default all of `tests/codegen/cases*.lisp`, and `tests/interpreter/cases.lisp`, programs the manual allows that the compiler to C rejects for what it does not support yet. Each program runs in `dylan-user`, as a compiled program does, or in the module its `module:` header names, and what it defined is forgotten before the next; its output must be the expected output, and the error it signals, if the case expects one, must have a message containing the expected text — whether the compiler would find the error compiling or running makes no difference here, and the `Dylan error:` of a compiled program's report is not expected. Cases marked `:compiler-only` are skipped and counted:

```
$ sbcl --script tests/interpreter/run.lisp
97 programs, 0 failures, 14 compiler-only skipped
$ sbcl --script tests/interpreter/run.lisp tests/codegen/cases-control.lisp
```

Every program not so marked must pass in both implementations.

C.7 Adding a test

1. Choose the suite. A feature of the compiler goes in the `tests/codegen/cases-*.lisp` file for its area; an example or specification from the manual goes in the `tests/conformance/ch`*nn*`.lisp` of its chapter, with a comment citing the section and

saying whether the expected result is the manual's example or follows from its text; a test that takes long goes in `tests/stress/`.

2. Write the case in the format of Section C.3.1: a unique name, the program, and the output the manual or the specification requires — not merely what the implementation prints today. Print every result with `print` or `format`, and, for threads, print only what does not depend on their order.

3. Run it alone in `quick` mode until it passes, then in the default mode, and in `gc-stress`, `address` or `thread` if it exercises memory or threads:

   ```
   $ sbcl --script tests/codegen/run.lisp quick my-new-case
   $ sbcl --script tests/codegen/run.lisp my-new-case
   ```

4. For the front end or the IR, add a case to its `cases.lisp`, using the runner's `print` argument to see what the source converts to, and checking that by hand against the manual before making it the expected text.

A test that fails because the implementation is wrong is a bug to fix in the implementation, not in the test; a test is changed only when it misread the manual.

Index

`!key`, 207
*, 94, 215
standard-error, 222
`*standard-error*`, 147
standard-output, 222
`*standard-output*`, 147
+, 94, 215
-, 94, 215
/, 94, 215
/=, 211
<, 95, 212
<=, 212
<abort>, 126, 221
<array>, 116, 217
<atomic-integer>, 222
<byte-string>, 117, 217
<character>, 101, 216
<class>, 212
<collection>, 105, 217
<complex>, 91, 214
<condition-variable>, 222
<condition>, 126, 220
<deque>, 118, 217
<double-float>, 91, 215
<empty-list>, 115, 217
<error>, 126, 220
<explicit-key-collection>, 105, 217
<extended-float>, 91, 215
<float>, 91, 214
<function>, 213
`<function>`, 65
<generic-function>, 213
<integer>, 91, 215
<keyword>, 102, 217
`<keyword>`, 47
<list>, 115, 217
<lock>, 222
<method>, 213
<mutable-collection>, 105, 217
<mutable-explicit-key-collection>, 105, 217
<mutable-sequence>, 105, 217
<number>, 91, 214
<object>, 211
<pair>, 115, 217
<range>, 118, 217
<ratio>, 91, 215
<rational>, 91, 215
<real>, 91, 214
<rectangular-complex>, 91, 214
<recursive-lock>, 222
<restart>, 126, 221
<sequence>, 105, 217
<serious-condition>, 126, 220
<simple-error>, 126, 220
<simple-object-vector>, 116, 217
<simple-restart>, 126, 130, 221
<simple-warning>, 126, 221
<single-float>, 91, 214
<singleton>, 212
`<singleton>`, 72
<slot-descriptor>, 212
<stream>, 222
`<stream>`, 147
<stretchy-vector>, 118, 217
<string-stream>, 222
`<string-stream>`, 147
<string>, 117, 217
<symbol>, 102, 216
`<symbol>`, 46
<table>, 119, 217
<thread-error>, 222
<thread>, 222
<type-error>, 126, 220
<unicode-string>, 117, 217
<vector>, 116, 217
<warning>, 126, 220
=, 95, 120, 211
=hash, 120, 212
>, 212
>=, 212

#key, 21
#line directives, 167
#rest, 21

A-normal form, 188
abort, 128, 221
abs, 97, 215
acos, 216
acosh, 216
acquire-lock, 222
add, 113, 218
add!, 113, 219
add-method, 213
add-new, 113, 219
add-new!, 113, 219
add-slot, 213
aligned, 110
all-superclasses, 212
`all-superclasses`, 85
always, 123, 214
and, 23, 223
`and`, 57
angle, 100, 216
anonymous method, 66
any?, 111, 218
append, 116, 220
applicable, 65
applicable-method?, 214
`applicable-method?`, 75
apply, 123, 213
`apply`, 74
aref, 116, 220
argument area, 190
arithmetic, 94
as, 212
    characters, 101
    numbers, 101
as-lowercase, 102, 117, 217
as-lowercase!, 117, 217
as-uppercase, 102, 117, 217
as-uppercase!, 117, 217
ash, 99, 216
asin, 216
asinh, 216
atan, 216
atan2, 216
atanh, 216
atomic-add!, 223
atomic-compare-and-swap!, 223
`<atomic-integer>`, 184
atomic-value, 222

backquote, 48, 207, 224
begin, 223
`begin`, 56
`BENCH_SCALE`, 200
benchmarks, 199
binary*, 94, 215
binary+, 94, 215
binary-, 94, 215
binary-gcd, 97, 215
binary-lcm, 97, 215
binary/, 94, 215
binary<, 96, 212
binary=, 96, 120, 211
bind, 20, 223
`bind`, 54
    type declarations, 202
bind-exit, 33, 224
`bind-exit`, 62
    cost, 203
    implementation, 192
bind-methods, 224
`bind-methods`, 61
body, 54, 66
Boehm collector, 169, 193
    tuning, 206
break, 128, 221
`build-dylan.lisp`, 153

C files, 166
    reading, 196
caar, 220
call, 18
calling convention, 189
car, 115, 220
case, 23, 223
`case`, 58
case of symbols, 46
cdddr, 220
cdr, 115, 220
ceiling, 96, 215
ceiling/, 97, 215
cell, 191
cerror, 127, 221
character, 45, 101

comparison, 102
check-type, 127, 221
choose, 32, 113, 219
choose-by, 113, 219
class, 72
class ids, 189
class precedence list, 73, 80
class-for-copy, 107, 212
classes, 28
closure, 4, 22, 66
closures, 191
collection, 105
defining, 108
collection classes, 105
collection functions, 110
comment, 44
comparison, 95
`dylan:compile-file`, 156
`dylan:compile-to-file`, 156
compiler
stages, 187
complement, 32, 122, 214
complex numbers, 93
compose, 32, 121, 214
concatenate, 114, 219
concatenate-as, 114, 219
cond, 23, 223
`cond`, 57
condition, 4, 33, 125
defining, 131
condition classes, 125
condition variable, 183
condition-format-arguments, 126, 221
condition-format-string, 126, 221
`<condition-variable>`, 183
conditions, 33
implementation, 192
congruent, 70
conjoin, 122, 214
conjunction, 122
cons, 115, 220
contagion, 95
continuable error, 127
copy-sequence, 114, 219
copy-state, 107, 218
cos, 216
cosh, 216
create clause, 137
current-element, 107, 218
setter, 107
current-key, 107, 218
current-milliseconds, 223
current-thread, 222
curry, 32, 122, 214

debugger, 132, 155
debugging, 168
default handler, 125
default-handler, 129, 221
define, 224
`define`, 53
define-class, 28, 224
`define-class`, 79
define-for-macros, 224
`define-for-macros`, 178
define-generic-function, 224
`define-generic-function`, 71
define-macro, 224
`define-macro`, 175
define-method, 21, 224
`define-method`, 70
define-module, 136, 224
define-slot, 224
`define-slot`, 84
define-thread-variable, 224
`define-thread-variable`, 184
denominator, 100, 216
deques, 26
diagnostics, 171
dimensions, 116, 220
direct-subclasses, 212
direct-superclasses, 212
directive, 147
DIRM, 3
disjoin, 122, 214
disjunction, 122
dispatch, 191
dispatch function, 191
display, 23, 144, 221
division, 96
division by zero, 94
do, 110, 218
do-handlers, 131, 221
dotimes, 24, 224
`dotimes`, 60
Dylan

1992 definition, 3
dylan, 10, 153
dylan module, 136
dylan-examples.dyl, 160
dylan-user module, 136
dylan:compile-file, 12
dylan:rep, 11
dylanc, 5, 38
first program, 13
modules, 139
dylanc, 163
-E, 179
-o, 165
options, 164
dylanc package, 170
dynamic chain, 192
dynamic extent, 62
dynamic-bind, 224
dynamic-bind, 184

element, 106, 217
elements, 105
dylan:empty-environment!, 155
empty?, 106, 218
equality, 120
error, 33, 127, 221
escape (printing), 143
escape analysis, 195
Escaped, 143
even?, 96, 215
every?, 111, 218
exclude: (module option), 136
(exit), 154
exit procedure, 33, 62
exit status
of dylanc, 171
exp, 98, 216
expansion, 175
explicit-key collection, 105
export clause, 137
export: (module option), 137
exported, 135
expressions, 18
expt, 98, 216

#f, 47
#f, 158
fasl, 153
fasl cache, 153
fasl files, 10
file header, 138
fill!, 112, 218
filtered slot, 83
final-state, 107, 218
find-key, 112, 218
find-method, 214
find-method, 75
find-pair, 116, 220
first, 115, 219
float, 92
printing, 101
float contagion, 203
floating-point number, 44
floor, 96, 215
floor/, 97, 215
for, 24, 224
for, 59
for-each, 24, 224
for-each, 59
format, 147, 222
format directives, 127
forms, 18
frame, 192
freeze-methods, 213
function call, 49
function-arguments, 214
function-arguments, 75

garbage collection, 193
choosing, 166, 169
tuning, 205
gcd, 97, 215
general instance, 79
generic function, 4, 30, 65
generic functions, 4
dispatch, 191
generic-function-methods, 213
gensym, 223
gensym, 177
getter, 28, 80

handler, 125
handler-bind, 129, 224
handler-case, 34, 128, 224
header, 51, 189

id?, 95, 120, 211

identity, 123, 212
idioms, 123
if, 22, 223
`if`, 56
imag-part, 100, 216
immediate, 189
implementations
    choosing, 4
import, 135
import: (module option), 136
`dylan:in-module`, 140
init-function, 213
init-keyword, 213
init-value, 213
initial-state, 107, 218
initialize, 29, 212
`initialize`, 83
instance, 28
instance?, 212
`instance?`, 85
integer, 44
    range, 92
integral?, 96, 215
interactive implementation, 4, 151
    architecture, 151
    modules, 139
    performance, 160
    representation, 158
    versus compiler to C, 161
intersection, 113, 219
iteration protocol, 107
iteration state, 107

join-thread, 222
`join-thread`, 180

key, 105, 106
`#key`, 48
key-sequence, 106, 218
keyword, 18, 47, 102
keyword parameters, 21, 68

last, 115, 219
lcm, 97, 215
lexical variable, 135
Lexical variables, 49
library cache, 167, 194
library reference, 211
license, 7
link-time optimization, 165
lisp-procedure, 148, 158, 223
lisp-variable, 158, 223
list, 115, 220
list*, 115, 220
lists, 25
`load-dylan.lisp`, 11, 153
local variables, 20
lock, 182
`<lock>`, 182
log, 98, 216
logand, 99, 216
logandc1, 99, 216
logandc2, 99, 216
logbit?, 99, 216
logeqv, 99, 216
logical operations, 99
logior, 99, 216
lognand, 99, 216
lognor, 99, 216
lognot, 99, 216
logorc1, 99, 216
logorc2, 99, 216
logxor, 99, 216

macro, 175
macros, 175
    hygiene, 177
magnitude, 97, 215
make, 28, 212
`make`, 83
make-polar, 216
make-read-only, 213
`make-read-only`, 87
make-rectangular, 216
make-thread, 222
`make-thread`, 180
maker, 192
map, 32, 110, 218
map-as, 110, 218
map-into, 111, 218
`map-into`, 207
max, 98, 215
member?, 112, 218
Memory Pool System, 169, 193
    tuning, 205
method, 21, 30, 65, 224
`method`, 66

method dispatch, 65
method-specializers, 213
min, 98, 215
module, 135
module variable, 20, 135
Module variables, 49
module variables, 135
module: (file header), 138
modules, 207
modulo, 97, 215
multimethods, 73
multiple dispatch, 4, 30
multiple values, 61, 190
musttail, 190
mutable collection, 105

NAME-MODULE.c, 139
natural order, 108
negative?, 96, 215
newline, 23, 144, 222
`#next`, 48
next method, 69
next-handler, 129
next-method, 31
`next-method`, 74
next-state, 107, 218
non-local exit, 62
not, 23, 212
`not`, 56
notify-all-condition-variable, 222
notify-condition-variable, 222
number classes, 91
number literals, 93
numbers, 19
numerator, 100, 215

object-class, 212
`object-class`, 85
odd?, 96, 215
operator, 49
or, 23, 223
`or`, 57
overflow, 92
own, 135

pairs, 25
`perf`, 205
Plain, 143
pop, 118, 220
pop-last, 118, 220
positive?, 96, 215
pp, 144, 222
predicates
    numeric, 96
prefix syntax, 3, 18
prefix: (module option), 136
pretty-prints, 144
previous-state, 107, 218
print, 23, 143, 221
print-object, 146, 222
printing, 143
profiling, 205
proper tail calls, 77
push, 118, 220
push-last, 118, 220

quote, 223
`quote`, 47

range, 118, 220
ranges, 26
ratio, 44, 92
rational canonicalization, 92
rational contagion for comparisons, 95
rationalize, 100, 216
rcurry, 122, 214
read-eval-print loop, 10, 153
read-only, 141
read-only variable, 141
reader, 44
real-part, 100, 216
recovery protocol, 125
`<recursive-lock>`, 182
reduce, 32, 111, 218
reduce1, 111, 218
regression test, 160
release-lock, 222
remainder, 97, 215
remove, 113, 219
remove!, 113, 219
remove-duplicates, 113, 219
remove-duplicates!, 113, 219
remove-key!, 119, 220
remove-method, 213
remove-slot, 213
rename: (module option), 137
`dylan:rep`, 153

replace-elements!, 112, 218
replace-subsequence!, 114, 219
representation of values, 189
requirements, 9
`#rest`, 48
restart, 125, 130, 155
restart handler, 125
restart-query, 131, 221
restarts, 35, 155
return-allowed?, 131, 221
return-description, 131, 221
return-query, 131, 221
reverse, 114, 219
reverse!, 114, 219
round, 96, 215
round/, 97, 215
runtime errors, 133

seal, 212
`seal`, 87
second, 115, 219
select, 223
`select`, 58
separate compilation, 194
sequence, 105, 108
sequence functions, 112
sequences, 25
set!, 224
`set!`, 55
setter, 26, 80, 224
setter variable, 46
shallow-copy, 212
signal, 34, 125, 127, 221
signals, 33, 125
sin, 216
singleton, 4, 31, 72, 212
`singleton`, 72
sinh, 216
size, 106, 218
size builds, 165, 204
slot, 28
slot descriptor, 85
slot-allocation, 213
slot-descriptor, 213
slot-descriptors, 213
slot-getter, 213
slot-initialized?, 213
`slot-initialized?`, 82
slot-setter, 213
slot-type, 213
slot-value, 213
slots, 4, 79
sort, 27, 114, 219
sort!, 114, 219
sorted-applicable-methods, 214
`sorted-applicable-methods`, 75
special form, 223
special forms, 18
specialized entry, 190
specializer, 21, 65
specificity, 207
sqrt, 98, 216
stable, 108
static program, 172
stream, 147
stream-contents, 147, 222
string, 45, 117
strings, 26
subclass?, 212
`subclass?`, 85
subsequence-position, 115, 219
symbol, 18, 46, 102
    case, 102
    comparison, 102
syntax form, 49
syntax operator, 49

`#t`, 47
table, 119
tables, 27
tagged words, 189
tail calls, 190
tail position, 77
tan, 216
tanh, 216
`dylan:test`, 156
test cases, 226
test modes, 227
`test-examples`, 229
`tests/interpreter/run.lisp`, 229
third, 115, 219
Thomas, 5
`<thread-error>`, 181
thread-error-condition, 222
thread-error-thread, 222
thread-yield, 222

threads, 180
    implementation, 193
    program exit, 181
top-level form, 50
trampoline, 165
tree shaking, 165, 194
trigonometric functions, 98
truncate, 96, 215
truncate/, 97, 215
type inference, 194
type-error-expected-type, 126, 221
type-error-value, 126, 221

unary-, 94, 215
unary/, 94, 215
unboxing, 195, 202
unhandled condition, 132
unhandled conditions, 172
uniform entry, 189
uninitialized, 81
union, 113, 219
unless, 22, 223
`unless`, 57
until, 24, 224
`until`, 60
unwind-protect, 33, 224
`unwind-protect`, 63
    implementation, 192
use clause, 136
using, 135

values, 21, 213
`values`, 61
variable clause, 59
variables, 20
vector, 116, 220
vectors, 25

wait-condition-variable, 222
warnings, 34
when, 22, 223
`when`, 57
while, 24, 224
`while`, 60
white space, 44
whole-program compiler, 187
with-lock, 224
`with-lock`, 182
wrapper, 189
write, 144, 222
write-line, 143, 221

zero?, 96, 215